

Unity®

PERFORMANCE TUNING TIPS 2026

Written By
CyberAgent Game & Entertainment Division



CyberAgent

Unityパフォーマンスチューニングティップス2026

株式会社サイバーエージェント | ゲーム・エンターテイメント事業部 著



Unity®

PERFORMANCE TUNING TIPS 2026

Written By
CyberAgent Game & Entertainment Division

CyberAgent®

Unityパフォーマンスチューニングティップス2026

株式会社サイバーエージェント　ゲーム・エンターテイメント事業部 著

はじめに

2022年の刊行以来、多くの反響をいただいた前作『Unity パフォーマンスチューニング バイブル』[*1]では、誰でもゼロからパフォーマンスチューニングを体系的に身につけられるよう、原因を切り分けるワークフローと、その前提となる基礎知識に加え、グラフィックスからスクリプティングまで幅広い分野のチューニング実例を解説しました。

2026年の今も、前作で述べた内容はそのまま参考になる部分が多くあります。一方で、刊行から4年を経た現在、Unity 6世代の登場から、AIによる開発体験そのものの変化まで、幅広い領域で大きな変化が生まれています。そこで一度立ち止まってパフォーマンスチューニングの現在地を確かめるべく、社内の有識者を集め、本書『Unity パフォーマンスチューニング ティップス 2026』をまとめました。

本書では、Render Graph、GraphicsStateCollection、Android向けの Link Time Optimizationなど Unity 6 で新しく選べるようになった機能から、開発の現場ですぐに試せる実践的なトピックまで、幅広い主題を実測値とともに取り上げます。

チューニングの判断は、最終的にはプロジェクトごとの測定結果に依存します。本書が示すのは、どこを見て、何と比べ、どこで打ち切るかという判断の材料です。手に取って頂いた方にとって、少しでも開発の手助けになれば幸いです。

本書について

本書はスマートフォン向けアプリを前提とした内容になります。一部の解説は他プラットフォームに当てはまらないものもあるかもしれませんがご留意ください。

検証にはUnity 6系（6000.x）を使用していますが、使用したバージョンは章ごとに異なります。とくに断りがない場合は、各章に記載したバージョンを参照してください。Unity 6以降で追加された機能を扱う章を除けば、それより前のバージョンにも適用できる内容が多く含まれています。また、本書に掲載した数値はいずれも各章に記載した測定環境での実測値または試算です。環境が変われば値も傾向も変わるため、数値そのものではなく、何がどう効いたのかを読み取って頂ければと思います。

[*1] https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningBible

本書の構成

本書は4部21章で構成されています。それぞれの章は独立しているため、必要な箇所のみ読み進めていくのもよいでしょう。巻頭の「課題別の逆引きガイド」には、代表的な症状と、それを扱っている章をまとめました。すでに課題が見えている場合は、そちらから引くのが近道です。「序章」では、前作からの数年間でUnityとモバイル環境に何が起きたのかを整理します。

第1部「グラフィックス最適化」では、Render Graphによるタイルメモリの節約、UnsafePassの使いどころ、FinalBlitPassの省略、シェーダーの軽量化とウォームアップなど、描画まわりを取り上げます。

第2部「スクリプティング・アニメーション最適化」では、キャラクターアニメーションのチューニング、VATによるボーン負荷の軽減、ParticleSystemのスパイク、非同期処理のライフタイム管理、UIシステムの使い分けを取り上げます。

第3部「アセット・設定・ビルド最適化」では、AssetBundleのライフタイム管理、アセット肥大化の調査、端末に応じた描画設定の切り替え、Code StrippingとLTO、iOSのメモリ上限を取り上げます。

第4部「計測と自動化」では、iOS標準の計測機能、デイリービルドの実機プロファイリングを自動化する構成、そして本書の知見をAIコーディングエージェントのスキルへ変換する試みを取り上げます。

GitHub

本書のリポジトリ[*2]を公開します。随時、加筆修正を行う予定です。またPRやIssueを用いて修正点の指摘や追記の提案が可能です。よろしければご利用ください。

[*2] https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026/

目次

はじめに 2

課題別の逆引きガイド 6

まずは計測から 7

容量 8

メモリ 9

CPU 10

GPU 12

序章: Unityの進化とチューニングの現在地 14

第1部　グラフィックス最適化 17

2026年のUnityグラフィックス動向 18

第1章　Unity 6 Render Graph タイルメモリ最適化 22

第2章　RenderGraphのUnsafePassを使うべき場面 56

第3章　URPのFinalBlitPassを省略する方法 68

第4章　モバイル向けシェーダー最適化：超越関数の計測と軽量化 78

第5章　Renderer Shader User Value を活用したグラフィックス最適化 120

第6章　GraphicsStateCollectionによるシェーダーウォームアップ 128

第2部　スクリプティング・アニメーション最適化 145

2026年のスクリプティング・アニメーション動向 146

第1章 Unity 3D キャラクターアニメーションのチューニング 150

第2章 ResetControllerStateでAnimatorプーリングを安全に行う 184

第3章 VATによるボーンアニメーション負荷軽減 188

第4章 ParticleSystemで長い時間Simulateをすると巨大なスパイクが発生する 198

第5章 UniTaskとキャンセル処理のライフタイム管理 204

第6章 UIシステムの使い分け – IMGUI / uGUI / UI Toolkit 214

第3部 アセット・設定・ビルド最適化 221

2026年のアセット・設定・ビルド動向 222

第1章 AssetBundleのライフタイム管理 232

第2章 運用フェーズのアセット肥大化とProject Auditorによる調査 238

第3章 端末に応じた描画設定の切り替え 248

第4章 Unity Managed Code Stripping 実践ガイド 252

第5章 UnityのLTO設定とAndroid・iOSでの性能比較 282

第6章 iOSのメモリ上限と仮想アドレス空間 306

第4部 計測と自動化 321

第1章 Unity で活用する iOS 標準のパフォーマンス計測機能 322

第2章 デイリービルドの実機プロファイリングを自動化する 342

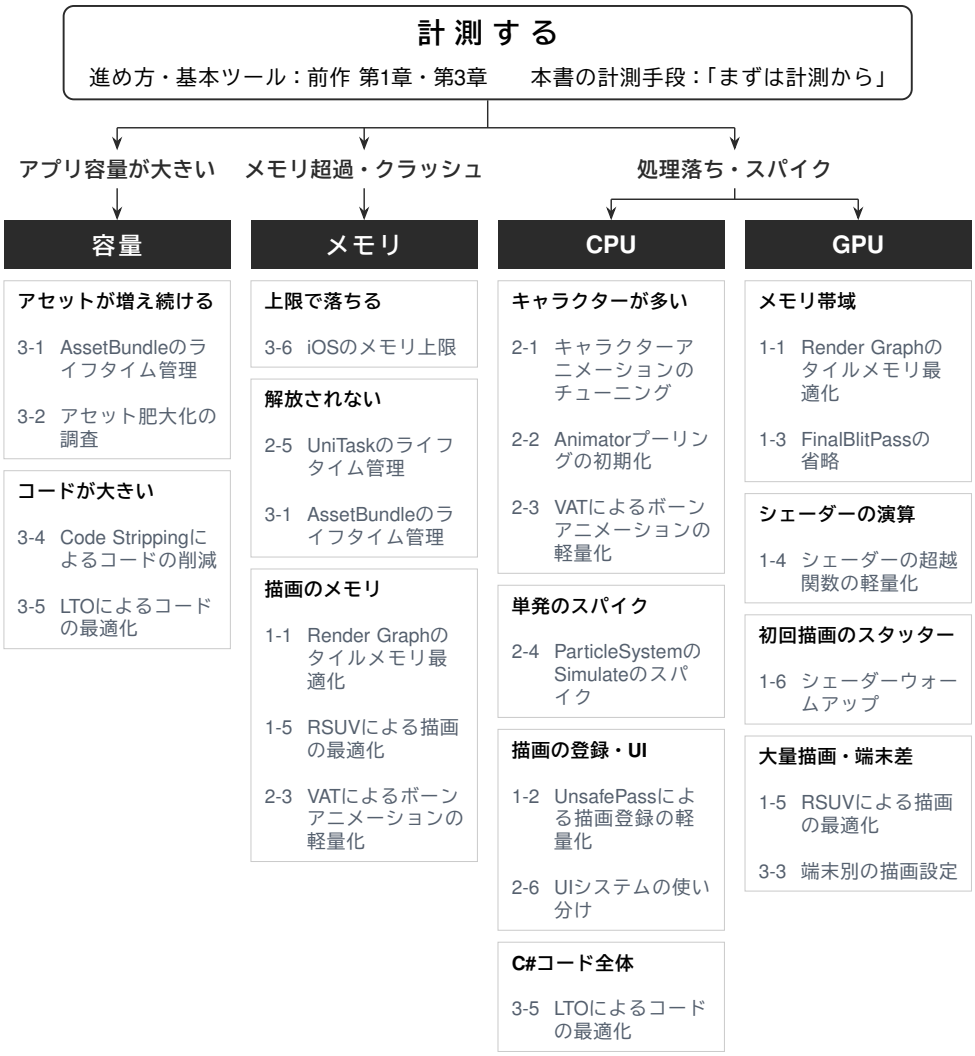
第3章 チューニングバイブルをAIコーディングエージェントのスキルにする 354

おわりに 366

著者紹介 368

課題別の逆引きガイド

症状ごとに、まず見るべき値と、それを扱っている章を並べています。最初に計測の進め方を確認し、そのあと容量、メモリ、CPU、GPUの順に課題から引けるようにしました。



まずは計測から

どこが遅いのかを決めてから手を付けます。推測ではなく計測で原因を特定し、修正の前後で結果を比較するという進め方と、メモリ超過・処理落ちの切り分け方は、前作『Unity パフォーマンスチューニング バイブル』の第1章「パフォーマンスチューニングを始めよう」で解説しました。Unity ProfilerやFrame Debugger、RenderDocなどの基本的な使い方も第3章「プロファイリングツール」にまとめています。

ここでは、前作で扱っていない計測手段と、既存のツールで新たに確認できるようになった項目を紹介します。iOS向けであれば、**Metal Performance HUD**と**Performance Trace**がDevelopment Buildフラグに依存せずに使えるため、計測処理をまだ組み込めていない段階でも、リリースに近い構成のままでも負荷の当たりを付けられます。
（**第4部1章「Unityで活用するiOS標準のパフォーマンス計測機能」**）

描画側は、計測の前にパイプラインの状態を確認しておきます。**Frame Debugger**はイベント名の**(RP n:m)**表記でパスがマージされているかを示し、**Render Graph Viewer**は各アタッチメントのロード/ストアアクションとMemorylessの状態に加えて、マージされなかった理由（Pass Break Reasoning）まで表示します。Unityの表示とドライバに渡っている値を突き合わせたいときは、**RenderDoc**でVulkanのキャプチャを取り、loadOp / storeOpをAPIレベルで確認します。
（**第1部1章「Unity 6 Render Graphタイルメモリ最適化」**）

計測そのものを毎日回す場合は、デバイスファームで借りた実機をアプリ内のリモート操作で画面遷移させ、メモリスナップショットとフィルタ済みのProfilerキャプチャをビルドごとに送る構成があります。端末上でフレームを絞り込む仕組みや閾値の決め方、長い巡回を端末の容量に収める工夫も併せて扱っています。
（**第4部2章「デイリービルドの実機プロファイリングを自動化する」**）

計測ではなくレビューの段階で効かせたい場合は、本書の知見そのものをAIコーディングエージェントの評価ルールへ変換する方法があります。前作の77ルールに本書からの86ルールを統合し、163ルールとしてスキル化した事例を紹介しています。
（**第4部3章「チューニングバイブルをAIコーディングエージェントのスキルにする」**）

容量

アプリ容量は、個々のアセットを絞る前に、増え方の原因を分類したほうが早く止まります。圧縮設定の誤り、解像度の過剰、余分な付随データ、形式の選択ミスの4つはImport時に判定できますが、まとめ方の不一致とバンドル間の重複は他のアセットとの関係で決まるため、ビルド前の検査が必要です。とくに重複は目に見えません。Addressablesに登録していない共有アセットは参照元のバンドルごとに複製されるため、4MBのテクスチャを2つのPrefabが参照しているだけで合計10MBになり、専用グループへ登録すれば6MBまで下がります。
（**第3部2章「運用フェーズのアセット肥大化とProject Auditorによる調査」**）

コード側にも削る余地があります。Managed Stripping LevelをMinimalからHighに上げると、終了処理のスクリプトを1つ置いただけのシンプルなシーンでも、マネージドアセンブリが約6.7%減りました。
（**第3部4章「Unity Managed Code Stripping実践ガイド」**）

そのうえで、どのアセットがいつまで必要かを設計として残します。ロードと解放の窓口を基盤へ集約し、ライフタイムの分類をAssetBundleの粒度とあわせて決めておくと、寿命の違うアセットが同じバンドルに混ざる状態を避けられます。
（**第3部1章「AssetBundleのライフタイム管理」**）

メモリ

iOSでは、Xcodeの設定を1つ有効にするだけで、アプリに適用されるメモリ上限が上がることがあります。提供された実機観測では、**Increased Memory Limit**によって4GB端末で約240MB、6GB端末で約1GBの余裕が増えました。ただし3GB端末では有効と無効で差がありませんでした。footprintに余裕があるのに確保が失敗する場合は、仮想アドレス空間側を広げる**Extended Virtual Addressing**が候補になります。
（**第3部6章「iOSのメモリ上限と仮想アドレス空間」**）

常駐量そのものを減らす側では、ロードしたアセットの解放漏れを最初に疑います。
（**第3部1章「AssetBundleのライフタイム管理」**）

非同期処理では、CancellationTokenSourceのDispose漏れがリンク元からの参照を残し、不要なオブジェクトが解放されない形で現れます。
（**第2部5章「UniTaskとキャンセル処理のライフタイム管理」**）

描画側のメモリは構造で決まります。アニメーションするオブジェクトを16体表示したときのGPUメモリの増分は、FBX方式が +411.8MB、Skin方式が +38.2MBで、VAT方式はほぼ増えませんでした。
（**第2部3章「VATによるボーンアニメーション負荷軽減」**）

マテリアルの複製もメモリを押し上げます。10000個のRendererにそれぞれマテリアルを持たせると15.79MB増えましたが、RSUVで値を渡せば増分は1.66KBで済みました。
（**第1部5章「Renderer Shader User Valueを活用したグラフィックス最適化」**）

ロードもストアもしないアタッチメントをメモリーレスにすれば、メインメモリ上に実体を持たずに済みます。
（**第1部1章「Unity 6 Render Graphタイルメモリ最適化」**）

ビルド設定では、Managed Code Strippingによって未使用コードをそもそもメモリに乗せずに済みます。
（**第3部4章「Unity Managed Code Stripping実践ガイド」**）

CPU

キャラクターを多数出す場面では、Transform階層の扱いが効きます。Optimize Game Objectsで階層をフラット化すると、300体の構成でメインスレッドの**Mesh Skinning.Update**が6.23msから約1.5msへ減り、スキニング行列の計算は全スレッド合計36.6msが完全に消えました。Playable Graphの手動更新はグラフ単位でしか並列化されないため、600体を1グラフにまとめると8.396msが4.500msになります。IKはメインスレッドで解かずにAnimation Jobへ移すと、300体が64回ずつ解く構成で9.094msが2.795msになりました。効いているのはBurstではなく、実行スレッドを移したことです。

遠景のキャラクターは、Nフレームに1回だけ更新します。**Animator.Update()**を個別に呼ぶより、**Animator.enabled**の切り替えや位相ごとの手動グラフでUnityの一括更新に戻したほうが安く、600体を2フレームに1回にすると5.037msが3.082msになりました。更新する体をフレームごとにずらしておけば、4フレームに1回の更新でもp95が9.470msから2.707msに収まり、スパイクになりません。
（**第2部1章「Unity 3Dキャラクターアニメーションのチューニング」**）

ボーンの更新そのものを事前計算へ置き換えるVATでは、16体表示時のCPUメインスレッドが154.3msから12.6msになりました。インスタンシングによって描画コマンドが1フレームあたり約21万から99に減り、SRPBatcher.Flushの76.17msがほぼ消えたことが効いています。
（**第2部3章「VATによるボーンアニメーション負荷軽減」**）

プールから戻ってきたAnimatorの初期化も、この設計に含めて考えます。
（**第2部2章「ResetControllerStateでAnimatorプーリングを安全に行う」**）

単発の大きなスパイクは、別の探し方をします。Noiseモジュールを有効にしたParticle Systemに**ParticleSystem.Simulate(20f, false, false)**のような大きな秒数を渡すと、NoiseJobが約2000回呼ばれ、約50msのスパイクが発生しました。
（**第2部4章「ParticleSystemで長い時間Simulateをすると巨大なスパイクが発生する」**）

描画の登録コストもCPU負荷です。マージできないPassチェーンを1つのUnsafePassにまとめると、RasterRenderPass 1つあたり約1.8µ sの固定コストを削減できます。100パスのピンポンBlurでは、1080.7µ sが898.5µ sになりました。
（**第1部2章「RenderGraphのUnsafePassを使うべき場面」**）

大量描画では、マテリアルを複製せずにオブジェクトごとの値を渡せるRSUVが効きます。iPhone 12でSphere 10000個のパラメータを毎フレーム変える計測では、Rendererごとのマテリアルが8.1FPSだったのに対し、RSUVとGPU Resident Drawerの併用では60.0FPSを保ちました。
（**第1部5章「Renderer Shader User Valueを活用したグラフィックス最適化」**）

UIでは、要素を1つ変えるとCanvas全体のメッシュが作り直されるuGUIに対し、UI Toolkitは変化した部分だけを更新します。
（**第2部6章「UIシステムの使い分け − IMGUI / uGUI / UI Toolkit」**）

C# コード全体に効く手段としてはLTOがあります。AndroidではReleaseビルドに**-flto=thin**を加えるだけで、ベンチマークの直接呼び出しが21.8%短縮し、Master（-O2）へ切り替えたときの5.9%を大きく上回りました。実タイトルの定常画面ではUnityMainが3.21%短縮でした。
（**第3部5章「UnityのLTO設定とAndroid・iOSでの性能比較」**）

GPU

モバイルでは、GPUの演算性能より先にメモリ帯域がボトルネックになります。1080pのRGBA8を1回ストアして次のパスでロードするだけで往復約16MB、60FPSなら約950MB/sを消費します。これを削る手段が、ロード/ストアアクションの明示、パスのマージ、フレームバッファフェッチ、メモリーレスの4つで、いずれもリソースの使い方をRender Graphへ正しく伝えることが条件になります。

たとえば全画素を必ず上書きするパスは、**AccessFlags.WriteAll**で宣言するだけで、1080pなら約8MB/フレームのロードがなくなります。ディファードのG-Bufferをフレームバッファフェッチで読めば、3枚分のストアだけで約24MB/フレーム、60FPSで約1.4GB/sの帯域を削れます。MSAAはStoreではなくResolveにすると、タイルメモリ上で解決され、4サンプル分のバッファをメモリーレスにできます。
（**第1部1章「Unity 6 Render Graphタイルメモリ最適化」**）

最後に1回だけ走るフルスクリーンのBlitも、高解像度では無視できないコストです。AfterRenderingPostProcessing以降にカスタムパスを注入するとURPのFinalBlitPassがほぼ必ず追加されますが、Unity 6.3からは最後のカメラの最終パスで**SwitchActiveTexturesToBackbuffer()**を呼べば、直接BackBufferへ出力して省略できます。カスタムFXAAの例では、Unity 6.0で2回必要だったBlitが1回になり、中間テクスチャも不要になりました。
（**第1部3章「URPのFinalBlitPassを省略する方法」**）

シェーダーの中身では超越関数が目立ちます。sinは乗算と**abs()**だけの放物線近似に置き換えられ、10回実行時にiPhone Xで約2.5ms、Galaxy S9で約1.3msのGPU時間を削減しました。tanの近似は10回実行時にGPU時間が本来の80%（iPhone 15 Pro）や50%（iPhone 14）になりました。ただし実行回数が少ないうちはレジスタープレッシャーによるオーバーヘッドが目立ちます。
（**第1部4章「モバイル向けシェーダー最適化：超越関数の計測と軽量化」**）

初回描画時のスタッターは、GPU側のコンパイルが原因です。MetalやVulkanでは1回のPSO生成に数十ミリ秒かかることがあり、GraphicsStateCollectionで任意のタイミングに済ませます。**WarmUpProgressively()**で1フレームあたりの件数を指定すれば、ウォームアップ自体のスパイクも避けられます。導入後もスタッターが残る場合は、Shader Strippingによるバリアントの欠落やメタデータの不一致を疑います。iOSでは、シェーダーを1つのバンドルにまとめて二重に読み込まれるのを防ぎ、SubPassを使うシェーダーはロード画面で一度描画してPSOを作っておきます。
（**第1部6章「GraphicsStateCollectionによるシェーダーウォームアップ」**）

それでも収まらない場合は、端末に応じて描画設定そのものを切り替えます。**deviceModel**、**graphicsDeviceName**、**systemMemorySize**で端末を判定し、Quality Levelを切り替えるか、RenderPipelineAssetのrenderScaleや影のカスケード数を直接操作します。自作のRendererFeatureにも、一時テクスチャのフォーマットや解像度を下げる、Blitの数を減らす、簡易なシェーダーに置き換えるといった切り替え口を用意しておくと、同じ仕組みで負荷を調整できます。
（**第3部3章「端末に応じた描画設定の切り替え」**）

序章: Unityの進化とチューニングの現在地

前作『Unity パフォーマンスチューニング バイブル』から数年が経ち、Unityにも、端末にも、チューニングの進め方にも変化がありました。この序章では、まずエンジンと端末の変化を大づかみに振り返り、そのうえで2026年時点でチューニングがどこへ向かっているのかを述べます。

変わった前提、変わらない前提

Unityでいちばん大きく動いたのは描画です。Unity 6でURPにRender Graphが入り、最適化の作業は、コマンドを自分で並べ替えることから、リソースの使い方をエンジンへ正しく宣言し、その結果を確かめることへと重心が移りました。シェーダーのウォームアップや、オブジェクトごとのパラメーターの渡し方にも、Unity 6世代で新しい選択肢が加わっています。C# 側はUnity 6のうちは大きく変わらないものの、Unity 7ではエディタと一部のデスクトッププレイヤーにCoreCLRが導入され、GCの仕組みも大きく変わります。一方、モバイルは当面IL2CPPとBoehm GCのままなので、これまでのメモリ最適化の考え方は引き続き有効です。ビルドと配布のまわりでは、コードを書き換えずに容量・メモリ・速度のバランスを動かせる設定が増えました。

端末も大きく変わりました。GPUのGeekbenchスコア[*3]で比べると、前作を執筆していた2022年のハイエンドであるiPhone 14 Pro（A16）は18,691でしたが、2026年のiPhone 18 ProやiPhone Duoが載せるA20 Proは55,120と、約3倍になっています。Android側でも、Galaxy S22（Snapdragon 8 Gen 1、Adreno 730）の6,179に対し、Galaxy S26 Ultra（Snapdragon 8 Elite Gen 5 for Galaxy、Adreno 840）は27,501と、4倍を超えています。

[*3] Geekbench Browser `https://browser.geekbench.com/`

ただし、ローエンドはさほど変わっていません。国内で2022年に発売されたGalaxy A23 5Gは、Dimensity 700（Mali-G57 MC2）とメモリ4GBの構成でした。2025年に発売された同じシリーズのGalaxy A25 5Gも、SoCはDimensity 6100+に替わったものの、GPUは同じMali-G57 MC2でスコアは1,400、メモリも4GBのままです。Galaxy S26 Ultraとは約20倍の開きがあります。

iPhoneでも、2022年の時点では、iPhone SE（第2世代）をはじめ、メモリ3GB以下の端末がまだ広く使われていました。2026年にはiPhone 14以降のメモリ6GB以上の端末が主流になりましたが、すでに2022年の時点で、普段使いには十分な性能の端末が行き渡っていたため、新しい端末が出ても買い替えは進みにくくなっています。2025年初めまで販売されていたiPhone SE（第3世代）やiPhone 13などの4GBの端末は今も多く残っており、3GBのiPhone SE（第2世代）もまだ一部で使われています。

その結果、どの端末までをターゲットにするかという、Androidで長く悩まされてきた問題が、iOSでも無視できなくなりました。上はメモリ12GBのiPhone 18 Proから、下は3GBのiPhone SE（第2世代、2020年発売）まで、同じタイトルが対応すべき端末の幅は広がっています。ゲームのジャンルに合わせて、端末の性能に応じて描画表現や品質設定を切り替える工夫が、これまで以上に求められています（第3部3章「端末に応じた描画設定の切り替え」）。

とはいえ、解こうとしている問題そのものは、前作の頃から大きくは変わっていません。モバイルGPUのメモリ帯域、大量のオブジェクトを描くときのCPU負荷、シェーダーコンパイルによるスパイク、アセットの解放漏れ、端末ごとのメモリ上限。新しい機能の多くは、こうした以前からの問題に対する新しい道具です。そのため本書は、Unity 6以降でしか使えない機能だけを扱っているわけではありません。キャラクターアニメーションの負荷対策、シェーダーの超越関数の近似、AssetBundleの寿命管理、iOSのメモリ上限の引き上げなど、それより前のバージョンのプロジェクトにもそのまま持ち込める内容も多く含んでいます。

第1部から第3部の冒頭では、その領域で2026年に何が変わり、各章で何を扱うのかをあらためて整理しています。症状から章を探したい場合は、巻頭の「課題別の逆引きガイド」を参照してください。

チューニングの現在地 – 計測と自動化

一方、開発現場の側で2026年に最も大きな関心事になっているのはAIです。AIコーディングエージェントは実装やレビューの一部を任せる道具として日常的に使われるようになり、パフォーマンスチューニングもその例外ではありません。本書の第4部3章「チューニングバイブルをAIコーディングエージェントのスキルにする」では、前作と本書の知見をエージェントの評価ルールへ変換する試みを扱っています。

ただしチューニングには、AIに任せる以前の前提があります。計測です。どこが遅いのかを測らずに手を入れても、速くなったのかどうかを判断できません。そのため業界全体としてはまず、計測の自動化に向けた取り組みが進んでいます。ビルドごとに決まった手順で実機の数値を取り、過去の測定と比べて劣化を自動で検知することで、人手でProfilerを開く回数を減らし、気付くのが遅れないようにする方向です。

筆者らが所属するサイバーエージェント ゲーム・エンターテイメント事業部でも、自動化の取り組みとして、AIエージェントからUnityアプリをリアルタイムに観察・操作するためのランタイムブリッジを整備しています。これをデバイスファームと組み合わせ、CIから「機種を確保してアプリを入れ、決めた手順で操作し、計測して結果を引き戻す」までをエージェントに任せる構成です。計測には実機のアプリに組み込んだランタイムのプロファイラを使い、取得したデータをサーバーへ送って、ブラウザから過去の測定と並べて確認したり、閾値を超えた変化を通知したりできるようにしています。サーバーへ送ったデータは後からAIエージェントが取得し、ボトルネックの診断から修正までを行います（第4部2章「デイリービルドの実機プロファイリングを自動化する」）。

もちろん、これで人の判断が要らなくなるわけではありません。何を見て、何と比べ、どこで打ち切るかという基準は、依然として人が持っておく必要があります。自動化できるのは、その基準がすでにある部分だけです。本書が実測値とともに判断の材料を並べているのは、そのためでもあります。

Unity®

PERFORMANCE TUNING TIPS 2026

CHAPTER 01

第1部

グラフィックス最適化

CyberAgent Smartphone Games & Entertainment

2026年のUnityグラフィックス動向

Render Graphの正式リリース

Unity 6が正式にリリースされて、この文章の執筆時点で約2年近くの時間が経過しました。Unity 6リリースの大きな目玉の一つとして、Render Graphの登場がありました。Render Graphは、Frostbite（DICE）が2017年に発表したFrame GraphやUnreal EngineのRDG（Render Dependency Graph）のように、現在のモダンなレンダリングエンジンには多く取り入れられている、1フレームのリソース依存関係を宣言的に記述し、実行前にグラフ全体を最適化する考え方に基づいた仕組みです。

リソース依存性を記述することによって、レンダーテクスチャの使いまわしや不要なレンダーテクスチャの破棄、適切なメモリのロードストアの自動設定など様々な恩恵を受けることができます。

UnityではHDRPに先行して導入されていましたが、URPには導入されておらず、ついにUnity 6に合わせて待望のリリースとなりました。

本書の第1部1章「Unity 6 Render Graphタイルメモリ最適化」では、Render Graphのパワーを最大限引き出すために必要なTBDRアーキテクチャのGPUとRender Graphの関連性について紐解いていきます。

続く第1部2章「RenderGraphのUnsafePassを使うべき場面」ではRender Graphの恩恵が弱くなるUnsafe Passについて解説されています。Render Graphは通常Raster Render PassやCompute Passを利用することで、Render Graphによる最適化の恩恵を受けることができます。多くの場面でRaster Render PassやCompute Passを選ぶのが最善の選択になります。その中であえて、Unsafe Passを選ぶべき場面についてご紹介します。パフォーマンス向上のため、移植のため、Unsafeでしか実現できない機能を実装するために、適切にUnsafe Passを選ぶケースについて紹介します。

Render Graphの登場ですべてが変わったのか

このようにRender Graphの登場によって、レンダリングシステムは大きく変わりました。Render Graphは自動的に最適化してくれますが、仕組みを理解せずに使っていると、最適化の恩恵を最大限生かすことはできません。新しいレンダリングシステムを理解し、ハンドリングする責任は我々開発者にあります。

さて、一方でRender Graphの登場によって、これまでの最適化の概念が全て覆ったのかというとそうではありません。Render Graphはグラフィックス領域では高いレイヤーの話です。よりハードウェアに近い、低いレイヤーに向けると昔ながらの知識は依然として有効です。例えば、シェーダーではsin、cos、tan、powなどの超越関数の演算処理はコストが高いという話があります。このような話は現代のモバイル環境であっても同様に通じます。第1部4章「モバイル向けシェーダー最適化：超越関数の計測と軽量化」では、現代のモバイル環境において超越関数を利用したときの計測結果の一例を示します。そのうえで、超越関数の軽量化として、いくつかの近似関数の紹介を行います。私自身も演算性能が向上した、現代のモバイル環境でも近似関数は必要なのか、という考えを持ったことがあります。そのような疑問の一つの答えを示してくれます。第1部5章「Renderer Shader User Valueを活用したグラフィックス最適化」と第1部6章「GraphicsStateCollectionによるシェーダーウォームアップ」で紹介される機能も、Unity 6以前から残っている問題を解決するために追加された機能です。

大量オブジェクト描画の負荷

Renderer Shader User Value（RSUV）はUnity 6.3から追加された新しい機能です。この機能が解決しようとしている問題は、昔からある大量のオブジェクトを描画するときのCPU負荷、特にSet Pass Callの負荷です。この問題を解決する手段として、SRP BatcherやGPUインスタンシングなどがあります。

しかし、SRP Batcherには「オブジェクトごとに見た目を少しだけ変えたい」という場面で制約がありました。SRP BatcherはMaterialPropertyBlockを使ったレンダラーごとのプロパティ変更に対応していないため、SRP Batcherを維持したまま個別のパラメーターを渡すには、カスタマイズしたいレンダラーの数だけMaterialを複製する必要がありました。わずかなパラメーターの違いのためにMaterialを増やすのは、メモリ的にも管理面でも望ましくありません。

RSUVはこの問題に対するより軽量な選択肢です。RSUVを利用すると、レンダラーごとに32bit符号なし整数を1つ持たせ、シェーダー側から参照できます。この値の設定はバッチングを壊さないため、SRP Batcherを維持したままパラメーターを変更できます。渡せるのは32bitの値1つという制限はありますが、色やインデックス、フラグなどをビットパッキングして詰め込むことで、多くの用途をカバーできます。

シェーダーコンパイル時のスパイク

GraphicsStateCollectionによるシェーダーウォームアップが解決するのは、シェーダーコンパイル時のスパイクの問題です。GraphicsStateCollectionはUnity 6でExperimental（実験的機能）として追加されて、Unity 6.5で正式な機能としてリリースされました。

以前からあるShaderVariantCollection.WarmUpなどのウォームアップAPIはVulkan/Metalなどでは期待通り機能していませんでした。これはVulkan/Metalなどのシェーダーウォームアップの仕組みを知ると分かることですが、旧来のAPIには正しくウォームアップするために必要なパラメーターを渡すことができませんでした。そのため、Vulkan/Metalではウォームアップを完全に行うことはできませんでした。そこで、開発者はゲームプレイ中のスパイクを回避するため、画面暗転中に裏で描画するなどのワークアラウンド的なウォームアップを行う必要がありました。

GraphicsStateCollectionはこの問題を解決する新しいAPIです。GraphicsStateCollectionはシェーダーウォームアップに必要な頂点レイアウトなどの様々な情報をアプリ実行時に集めるための仕組みです。これらの情報を収集することで、完璧にシェーダーをウォームアップすることができます。しかし、GraphicsStateCollectionには情報の収集漏れなど、いくつかの新たな問題があります。これは、GraphicsStateCollectionの問題というよりは、UnityのShader Variant Stripに起因するものや、Vulkan/Metalなどの現在のモダンなグラフィックスAPIの仕様に起因しているものなど様々です。

第1部に向けて

Render Graphの登場でレンダリングパイプラインを構築するシステムは大きく変わりました。リソース依存性を定義することで不要なリソースを破棄でき、タイルベースGPUとの相性も高い新しいレンダリングシステムの登場です。

しかし、高いレイヤーのレンダリングシステムが一新されたからといって、よりハードに近いレイヤーの仕組みが変わったわけではありません。Render Graph、Graphics StateCollection、RSUVが解決しようとしている問題は以前から抱えている問題です。

第1部では、Unity 6で導入された新しい機能、普遍的なもの、第1部3章「URPのFinal BlitPassを省略する方法」のように、明日からすぐに使えるTips的なものも紹介しています。

これらの内容が皆様の開発の一助になれば幸いです。

第1章

Unity 6 Render Graph タイルメモリ最適化

清原 隆行 Kiyohara Takayuki

株式会社サイバーエージェント SGEコア技術本部

X @nezumimusume

1.0.1　はじめに

本章では、Unity 6のRender Graphを利用して、モバイルGPU（タイルベースレンダリングアーキテクチャ）のタイルメモリを活用し、GPU時間・メモリ帯域・メモリ使用量を削減する方法を解説します。モバイルのゲーム開発では、GPUの演算性能よりも先にメモリ帯域がボトルネックになることが多くあります。また、メモリ帯域の消費は消費電力と発熱に直結するため、フレームレートだけでなく端末の発熱やバッテリー消費にも影響してきます。本章では次の4つをテーマとして解説していきます。それぞれに同名の節をあてています。

1. タイルメモリのロード/ストア
2. Render Pass/Subpass
3. フレームバッファフェッチ
4. メモリーレス

1.1　タイルベースレンダリングの基礎

この節では、なぜモバイルGPUがタイルメモリを持っているのかについて解説します。

1.1.1　モバイルGPUとメモリ帯域

現代のGPUは、演算速度とメモリの読み書き速度の差がどんどん大きくなっており、メモリアクセスが大きなボトルネックになっています。GPUコアの演算性能（FLOPS）は世代ごとに大きく伸びる一方で、メモリ帯域の伸びはそれに追いついていません。PCのディスクリートGPUは、この問題をGDDRのようなグラフィックス専用の広帯域メモリを搭載することで軽減しています。しかし、モバイルSoCではそうはいきません。

項目	PC（ディスクリートGPU）	モバイルSoC
グラフィックスメモリ	専用（GDDR6等、数百GB/s）	なし（LPDDRをCPU/GPU/ISP等で共有）
帯域の規模感	数百GB/s〜1TB/s	数十GB/s（かつ共有）
電力の制約	数百W級の電源と冷却	数W、パッシブ冷却のみ

モバイルGPUには専用のグラフィックスメモリがなく、CPUと同じLPDDRメモリを共有しています。しかも共有相手はCPUだけでなく、カメラのISPや機械学習アクセラレータなどSoC上のあらゆるブロックです。帯域そのものが細いうえに、取り合いになります。さらに、メインメモリ（DRAM）へのアクセスはSoCの中でも特に電力を消費する操作です。帯域を使えば使うほど消費電力が増え、発熱し、サーマルスロットリングによってCPU/GPUのクロックが下がり、フレームレートが落ちます。つまりモバイルでは、

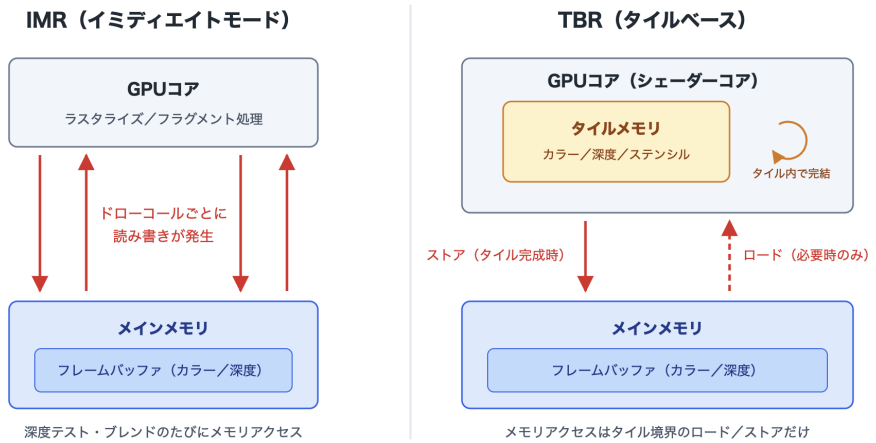
```
メモリ帯域の消費 → 消費電力の増加 → 発熱 → スロットリング → 性能低下
```

という流れが起きます。最初の数分は60FPSで動くのに、しばらく遊んでいるとフレームレートが落ちてくるという場合、この帯域と発熱が原因になっていることがよくあります。この制約の中で絵を描き切るために生まれたのが、タイルベースレンダリング（TBR: Tile-Based Rendering）アーキテクチャのGPUです。

1.1.2　IMRとTBR

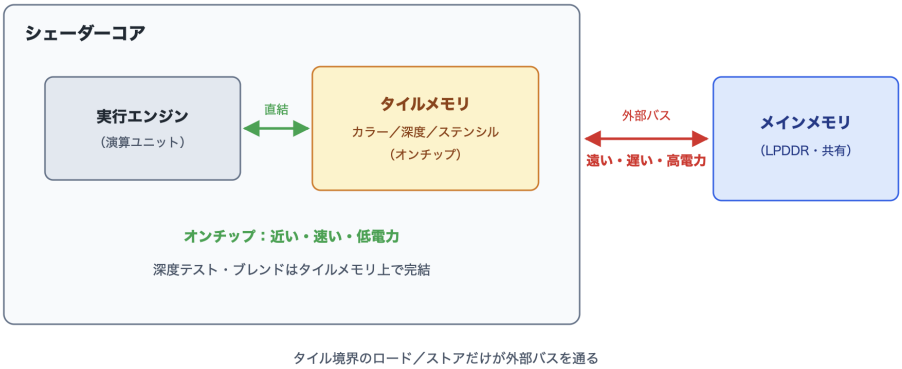
TBRを理解するために、まずPCのGPUで主流のイミディエイトモードレンダリング（IMR: Immediate Mode Rendering）と比較してみましょう。IMRは、ドローコールで投入されたプリミティブをその場でラスタライズし、フラグメントシェーダーを実行し、結果をメモリ上のレンダーターゲットに書き込んでいく方式です。深度テストや

ブレンディングのたびにレンダーターゲットへの読み書きが発生しますが、PCのGPUは広帯域なグラフィックスメモリと大容量キャッシュでこれを受け止めています。一方TBRは、画面を小さなタイルに分割し、タイル1枚ずつレンダリングを完結させる方式です。各シェーダーコアにはタイルメモリと呼ばれる小さなオンチップメモリが搭載されており、タイル内の描画（カラー、深度、ステンシルの読み書き、ブレンディング）はすべてこのタイルメモリ上で行われます。メインメモリへのアクセスは、原則としてタイルの描画開始時（ロード）と終了時（ストア）の2回だけになります。



▲図1.1　IMRとTBRの比較

次の図はTBRのGPUにおけるシェーダーコアの概念図です。シェーダーコアにタイルメモリが直結しています。オンチップかつ物理的距離が近いため、タイルメモリへのアクセスはメインメモリへのアクセスと比べて桁違いに高速かつ低消費電力です。実際のGPUでもこの構造は確認でき、たとえばARM Mali-G71の公式資料のシェーダーコア設計図には、タイルメモリがシェーダーコアに直結している様子が示されています。



▲図1.2　シェーダーコアとタイルメモリ（概念図）

このように、IMRでは最もメモリアクセスが激しかった深度テストとブレンディングが、TBRではタイルメモリ内で完結します。

1.1.3　TBRの2フェーズ

TBRのGPUは、1フレームの描画を大きく2つのフェーズに分けて実行します。

ビニングフェーズ（ジオメトリフェーズ）

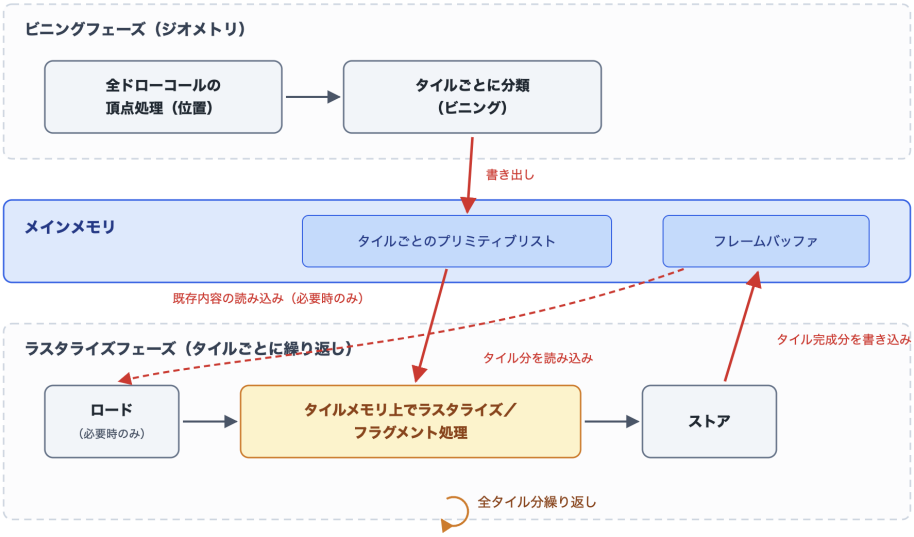
まず、フレーム内の全ドローコールの頂点シェーダー（の位置計算部分）を実行し、各プリミティブがどのタイルにかかるかを分類（ビニング）します。分類結果は「タイルごとのプリミティブリスト」としてメインメモリに書き出されます。ここで注意したいのは、ビニングフェーズ自体もメインメモリへの帯域を消費するという点です。頂点データが多いほどプリミティブリストの読み書きは増えます。TBRのGPUでハイポリゴンなシーンが不利になりやすいのはこのためです。

ラスタライズフェーズ（レンダリングフェーズ）

続いて、タイルを1枚ずつ処理していきます。各タイルについて、

1. 必要であればレンダーターゲットの既存内容をタイルメモリにロード
2. そのタイルにかかるプリミティブをラスタライズし、フラグメントシェーダーを実行（読み書きはすべてタイルメモリ上）
3. 完成したタイルの内容をメインメモリにストア

という手順で描画が進みます。この手順1と3が、本章のテーマであるロード/ストアです。詳しくは次節「タイルメモリのロード/ストア」で扱いますが、タイルの描画そのものよりも、タイル境界でのメインメモリとのやり取りの方が高くつく、というのがTBRのコスト構造です。



▲図1.3　ビニングフェーズとラスタライズフェーズ

ベンダーごとの呼称と特性

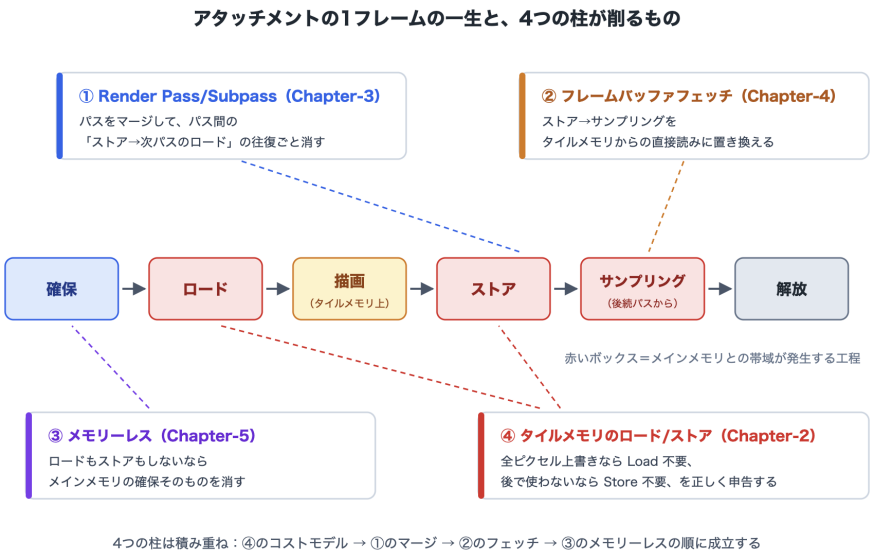
TBRの基本構造は共通ですが、GPUベンダーごとに呼称や細部の実装が異なります。本章では以降、総称としてタイルメモリ／ビニングという言葉を使います。

ベンダー（GPU）	フェーズ/機構の呼称	特記事項
Qualcomm Adreno	Binning / GMEM	GMEMと呼ぶオンチップメモリを分割して使う。描画内容によってIMR的動作へ切り替わるモードを持つ
ARM Mali	Tiler / Tile Memory	タイルは16×16ピクセルが基本

ベンダー（GPU）	フェーズ/機構の呼称	特記事項
Imagination（PowerVR/IMG）	TBDR	隠面除去（HSR）をハードウェアで行うTBDR。Pixel 10のIMG DXTもこの系譜
Apple GPU	Tile Memory	PowerVR系譜のTBDR。Metalからタイルメモリを明示的に扱える

1.1.4　本章で扱う4つのテーマ

ここまでで、タイル境界のロード/ストアがコストになることが分かりました。では、そのコストはどうやって削ればいいのでしょうか。本章で扱う4つのテーマは、いずれもこの問いに対する手段です。



▲図1.4　アタッチメントの一生と4つのテーマの関係

あるレンダーターゲット（アタッチメント）の1フレームのライフサイクルを考えてみます。

確保 → ロード → 描画 → ストア →（後続パスでテクスチャとして）サンプリング → 解放

このライフサイクルのどこを削るかで、4つのテーマは次のように整理できます。

テーマ	削るもの	考え方	節
ロード/ストア	不要なロードと不要なストア	「全ピクセル上書きするならロード不要」「後で使わないならストア不要」を明示する	「タイルメモリのロード/ストア」
Render Pass/Subpass	パスとパスの間のストア→ロード往復	複数のパスを1つのRender Passに統合（マージ）し、中間結果をタイルメモリに留める	「Render Pass / Subpass」
フレームバッファフェッチ	ストア→サンプリングの往復	前のサブパスの結果をタイルメモリから直接読む	「フレームバッファフェッチ」
メモリーレス	メインメモリの確保そのもの	ロードもストアもしないなら、メインメモリ上の実体は不要	「メモリーレス」

この4つは独立したテクニックではありません。パスがマージされて初めてフレームバッファフェッチが使え、ロードもストアも消えて初めてメモリーレスが成立します。本章の節構成も、コストモデル（ロード/ストア）→ 仕組み（Subpass）→ 活用（フェッチ）→ 仕上げ（メモリーレス）というこの依存関係に沿っています。また、Render Graphはパスごとのリソース利用情報（何を読み、何に書くか）を集めてからレンダリングの実行計画を立てるシステムです。そのため、パスのマージ、ロード/ストアアクションの決定、メモリーレス化といった最適化を自動的に行えます。本章で解説するのは、これらの最適化が働くように、リソースの使い方を正しくRender Graphに伝える方法だと言えます。

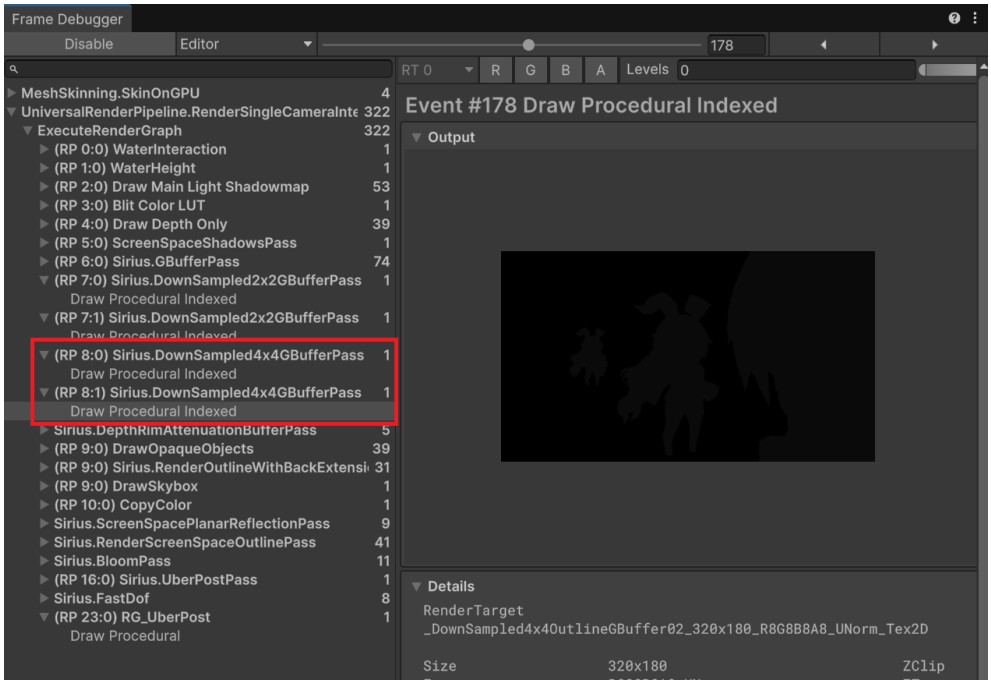
1.1.5 検証ツールの使い方

本章では、パイプラインの状態をツールで確認してから計測する、という流れを繰り返します。使うのは主に次の3つです。

1.1.6 Frame Debugger

UnityのFrame Debugger（Window > Analysis > Frame Debugger）は、パスの構造を確認する基本ツールです。本章で特に見るのはイベント名の`(RP n:m)`表記です。`RP 4:0`、`RP 4:1`のように、同じRender Pass番号（n）にサブパス番号（m）が連番で並んで

いれば、それらのパスは1つのNative Render Passにマージされています。なお、パスのマージ（Native Render Pass）が機能するのはVulkan/Metalなどの環境です。エディタのグラフィックスAPIがD3D等になっているとマージを観察できないため、エディタで確認する場合はGraphics APIをVulkanに変更するか、実機ビルドに対してFrame Debuggerをアタッチしてください。



▲図1.5　Frame Debuggerでのマージ状態

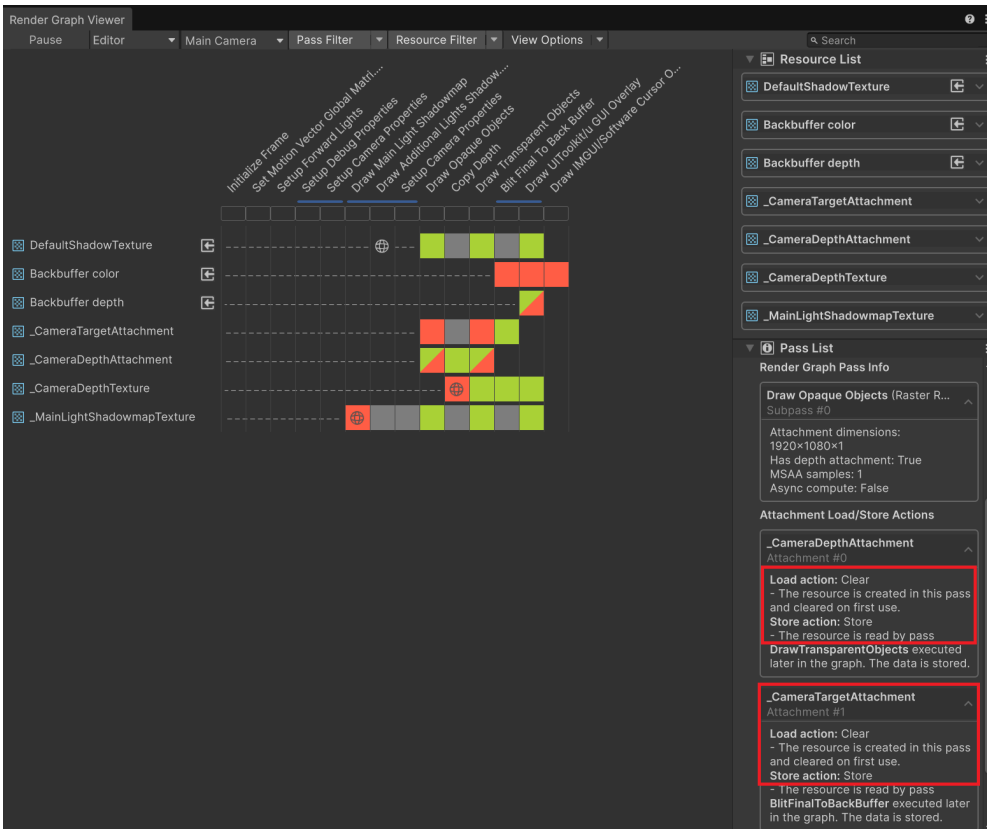
1.1.7　Render Graph Viewer

Render Graph Viewer（Window > Analysis > Render Graph Viewer）は、Render Graphが構築した実行計画そのものを可視化するツールです。

- パスの一覧と、各パスがアクセスするリソースのタイムライン（どのリソースがいつ生成され、いつ破棄されるか）
- アタッチメントごとのLoad Action / Store Action、そしてMemorylessの状態の表示。本章ではこの表示を使って最適化の結果を確認していきます

- パスがマージされているかどうかの表示
- なぜマージされなかったのか（Pass Break Reasoning）の表示

4つ目のPass Break Reasoningは、「マージされるはずのパスがマージされない」ときに最初に確認する情報です。「Render Pass / Subpass」の節以降で繰り返し使います。



▲図1.6　Render Graph Viewerでのマージ状態とロード/ストアアクションの確認

1.1.8　RenderDoc

RenderDocでVulkanのキャプチャを取ると、各Render Passのアタッチメントに指定された`loadOp`/`storeOp`をAPIレベルで確認できます。Render Graph Viewerの表示をもう一段掘りたいときや、Unityの表示とドライバに渡っている値を突き合わせたいときに使います。

1.2　タイルメモリのロード/ストア

この節では、本章全体の土台となるロード/ストアアクションについて解説します。

1.2.1　ロード/ストアアクションとは

TBRのGPUはタイルの描画開始時にレンダーターゲットの既存内容をタイルメモリへ読み込み（ロード）、描画終了時にタイルメモリの内容をメインメモリへ書き出します（ストア）。ただし、ロードとストアは常に必要なわけではありません。グラフィックスAPIには、アタッチメントごとに「開始時にどうするか」「終了時にどうするか」を指定するためのアクションが用意されています。

ロードアクション

アクション	動作	帯域コスト
Load	メインメモリの既存内容をタイルメモリへ読み込む	高い（解像度×フォーマット分の読み込み）
Clear	タイルメモリを指定色でクリアする	ほぼゼロ（タイルメモリ内の操作のみ）
DontCare	何もしない（タイルメモリの初期値は不定）	ゼロ

ストアアクション

アクション	動作	帯域コスト
Store	タイルメモリの内容をメインメモリへ書き出す	高い（解像度×フォーマット分の書き込み）
Resolve	MSAAのサンプルを解決した結果だけを書き出す	中（解決後の1サンプル分のみ）
DontCare	何も書き出さない（内容は破棄される）	ゼロ

つまり、帯域を消費するのはLoadとStore（およびResolve）だけです。逆に言えば、Clear/DontCareで開始してDontCareで終了できるアタッチメントは、メインメモリと一切やり取りしません。この状態が後で扱うメモリーレスの前提条件になります。ロード/ストアを最適化するには、各アタッチメントについて次の2点を確認していくことになります。

1. 開始時、以前の内容は必要か？　全ピクセルを上書きするパスなら不要（Dont Care）です。クリアから始めるならClear、以前の描画結果に重ね描きするときだけLoadになります。
2. 終了時、この内容を後で使うか？　後続のパスや次フレームで使うならStore、このRender Pass内で使い終わるならDontCareです。

1.2.2　帯域コストの試算

ロード/ストア1回のコストは単純な掛け算で概算できます。

```
1回のロード（またはストア）のデータ量 = 横解像度 × 縦解像度 × 1ピクセルのバイト数
```

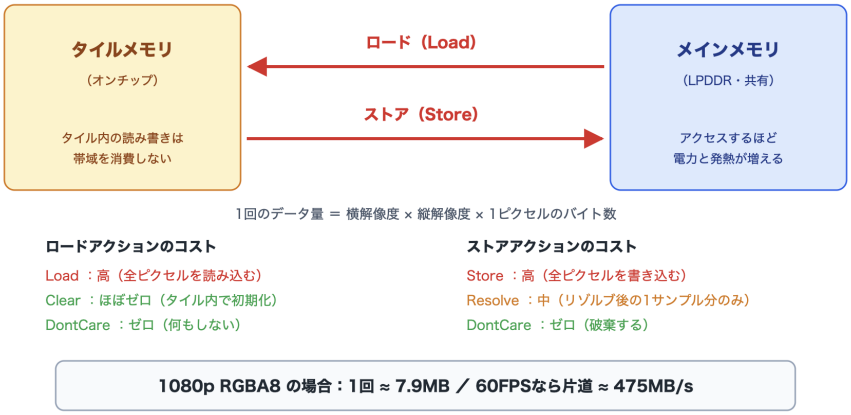
1080p（1920×1080）での代表的なフォーマットの値を示します。

フォーマット	1ピクセル	1回のロード/ストア	60FPS時の帯域
R8G8B8A8 (RGBA8)	4バイト	約7.9MB	約475MB/s
B10G11R11_Float	4バイト	約7.9MB	約475MB/s
R16G16B16A16_Float	8バイト	約15.9MB	約950MB/s
D24S8 / D32（深度）	4バイト	約7.9MB	約475MB/s

たとえば「カメラカラー（RGBA8）をストアし、次のパスでロードする」だけで往復約16MB、60FPSなら約950MB/sの帯域を消費します。モバイルSoCの実効帯域が数十GB/sで、しかもCPUと取り合いであることを考えると、無視できる量ではありません。さらに次の点にも注意してください。

- MSAAは倍率がかかります。4xMSAAのカラーバッファをResolveせずにStoreすると、データ量は4倍になります（MSAAとストアの関係は「メモリーレス」の節で扱います）。
- ロードとストアは両方数えます。パスをまたぐたびに「前のパスのStore＋次のパスのLoad」が対で発生します。
- ストアした結果を後続パスでサンプリングすれば、（キャッシュに乗らない分は）さらに読み込みが発生します。

開発中のゲームのレンダリングパイプラインについて、1フレームで何MBがタイルメモリとメインメモリの間を往復しているかを一度調べることをおすすめします。Render Graph Viewerで各パスのアタッチメントとアクションを確認して上の表を掛け算するだけで、どのパスから削るべきかの優先順位が見えてきます。



▲図1.7　タイルメモリ⇔メインメモリのロード/ストアと帯域コスト

1.2.3　AccessFlagsによるリソース利用の申告

　では、このロード/ストアアクションをRender Graphではどう指定するのでしょうか。実は、Render Graphではロード/ストアアクションを直接指定しません。パスを追加するときにビルダーへリソースの使い方を申告し、Render Graphがその情報からアクションを決定します。このとき申告に使うのがAccessFlagsです。

▼リスト1.1　AccessFlagsによるリソース申告

```
public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Example", out var
    passData))
    {
        // 出力アタッチメントを申告する。
        // AccessFlags.WriteAll = このパスは全ピクセルを書き込む(以前の内容は不要)
        builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
        ...
    }
}
```

`AccessFlags`の主な値と意味は次の通りです。

AccessFlags	意味	アクションへの影響
Read	以前の内容を読む	ロードアクションがLoadになる
Write	書き込む（一部のピクセルかもしれない）	以前の内容の保全が必要になり、ロードがLoadになり得る
ReadWrite	読み書き両方	ロードがLoadになる
Discard	以前の内容は破棄してよい	ロードをDontCareにできる
WriteAll	全ピクセルを書き込む（Write + Discard）	ロードをDontCareにできる

ここで特に注意したいのがWriteとWriteAllの違いです。単なる`Write`は「一部のピクセルにしか書かないかもしれない」という申告なので、Render Graphは書かれなかったピクセルの内容を保全するためにロードアクションをLoadにせざるを得ません。全画面Blitのように必ず全ピクセルを上書きするパスでは`WriteAll`を指定してください。1080pのRGBA8であれば、この指定の違いだけで約8MB/フレームのロードが発生するかどうかが変わります。一方、ストアアクションを決めるのは、そのリソースが後続のパスで使われるかどうかです。これは開発者が指定するのではなく、Render Graphがグラフ全体を見て判断します。次の項でそのルールを見ていきましょう。

1.2.4 Render Graphがアクションを決定するルール

Render Graphは、記録されたすべてのパスのリソース利用情報からNative Render Passを構築する際に、各アタッチメントのロード/ストアアクションを決定します。判断ルールを整理すると次のようになります。

ロードアクションの決定

条件	決定されるアクション
AccessFlags.WriteAll（またはDiscard）で申告された	DontCare
リソースの最初の利用で、クリアが要求されている	Clear
以前の内容への読み込み/部分書き込みがある	Load

ストアアクションの決定

条件	決定されるアクション
後続のどのパスからも利用されない	DontCare
後続のパスからUseTexture（テクスチャとして利用）される	Store
後続のパスからSetInputAttachment（入力アタッチメント）としてのみ利用され、同一Native Render Pass内にマージされた	DontCare（タイルメモリ内で受け渡し）
グローバルテクスチャとして設定されている	Store（後続の利用がRender Graphから見えないため保守的にストア）
MSAAでリゾルブ先がある	Resolve

このルールから、実務上の指針が2つ導けます。

- グローバルテクスチャ化は最後の手段にする。`SetGlobalTextureAfterPass`等でグローバル化されたテクスチャは、Render Graphから利用状況を追跡できなくなるため、常にストアされ、メモリーレス化の対象からも外れます。とりあえずグローバルにしておく、という書き方をすると最適化が効かなくなってしまいます。
- `UseTexture`と`SetInputAttachment`は帯域の面ではまったくの別物です。同じ「前のパスの結果を読む」でも、前者はストア→サンプリングの往復が発生し、後者はタイルメモリ内で完結し得ます（「フレームバッファフェッチ」の節で詳しく扱います）。

インポートしたテクスチャの扱い

`RTHandle`など、アプリケーション側で管理しているテクスチャを`renderGraph.ImportTexture`でインポートした場合、Render Graphはそのテクスチャがグラフの外でどう使われるかを知りません。そのため、デフォルトでは保守的に扱われます（初回利用でもロードされ、最後の利用後もストアされる）。インポート時に`ImportResourceParams`を渡すことで、この保守的な挙動を緩和できます。

▼リスト1.2　インポート時のパラメータ設定

```
var importParams = new ImportResourceParams
{
    clearOnFirstUse = true,   // 初回利用時にクリアする（ロードをClearにできる）
    discardOnLastUse = true,  // 最後の利用後は破棄してよい（ストアをDontCareにできる）
};
var handle = renderGraph.ImportTexture(_rtHandle, importParams);
```

インポートしたテクスチャが「毎フレーム作り直すので以前の内容は不要」なのであれば、`clearOnFirstUse`/`discardOnLastUse`を設定してください。なお、Render Graph管理のテクスチャ（`renderGraph.CreateTexture`で作成したもの）であれば、これらはすべて自動で判断されます。ライフサイクル管理が本当に必要なテクスチャ以外は、Render Graph管理に任せるのが基本です。

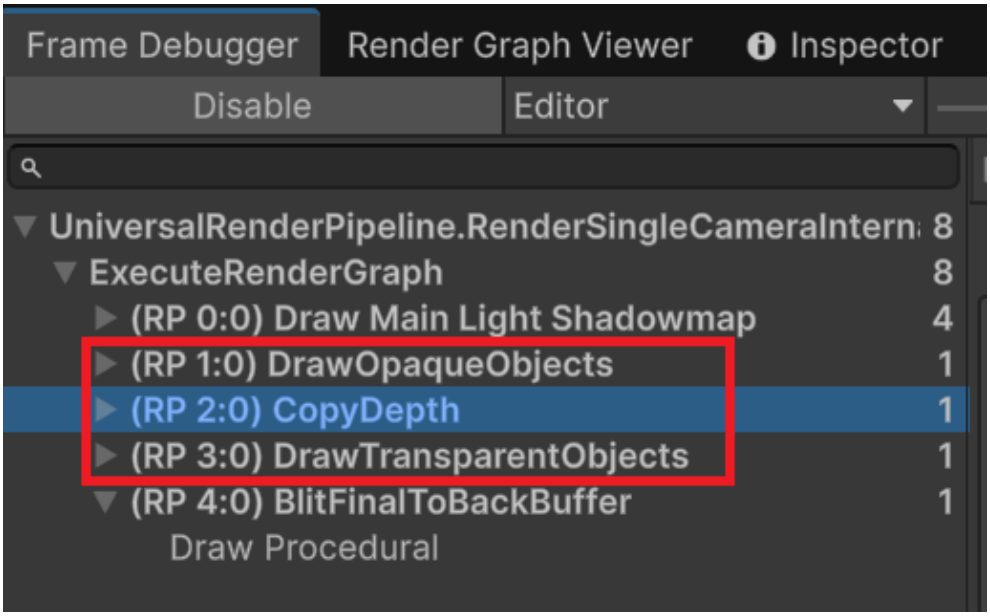
ロード/ストアアクションを読み解く

ルールが分かったところで、実際のフレームを読み解いてみましょう。URPの標準的なフォワードレンダリングのフレームをRender Graph Viewerで見ると、たとえば次のような構造が観察できます。

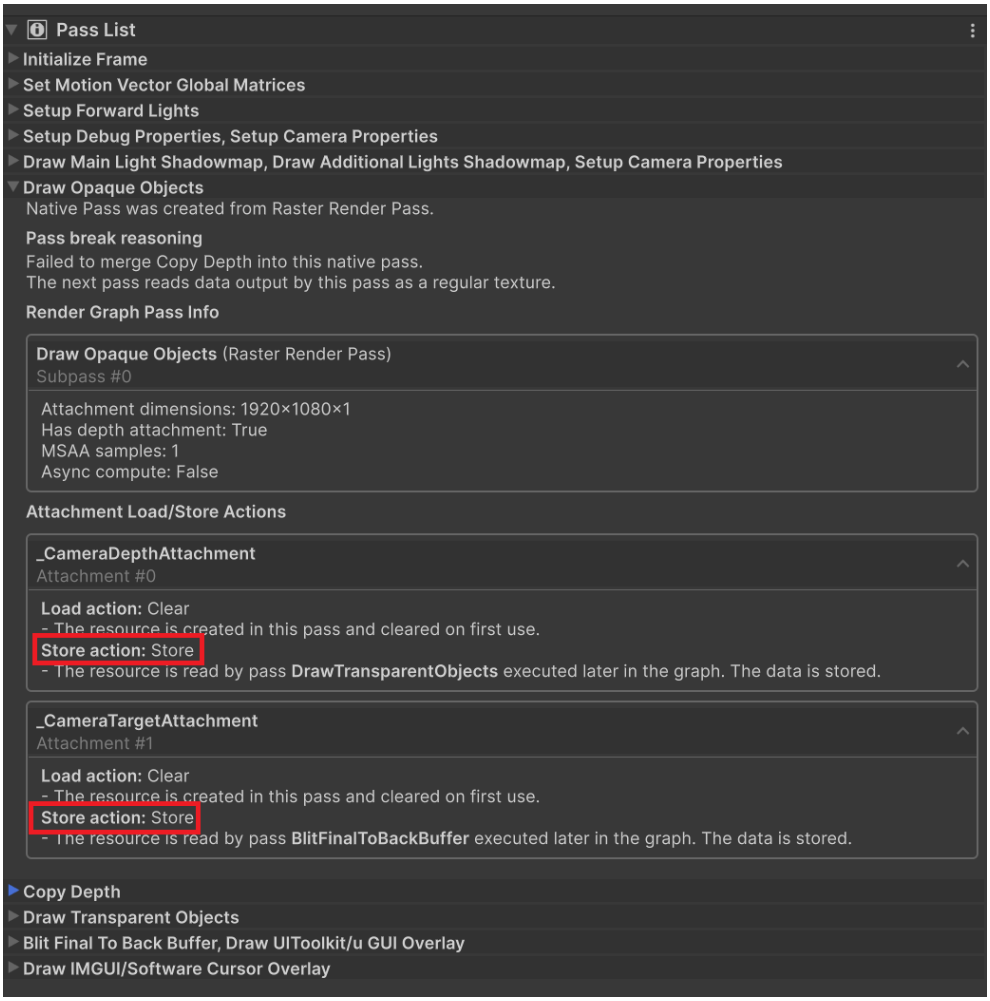
パス	アタッチメント	Load	Store
MainLightShadow	シャドウマップ（深度）	Clear	Store
DrawOpaqueObjects	カメラカラー	Clear	（後続次第）
カメラ深度	Clear	（後続次第）	
DrawTransparentObjects	カメラカラー	（マージ時は継続）	Store
FinalBlit	バックバッファ	DontCare	Store

読み解きのポイントは次の通りです。

- シャドウマップは「描いて、後でサンプリングする」ものなので、Store以外になりようがありません。これは削れないストアです。
- 不透明→半透明が1つのNative Render Passにマージされていれば、その間のカメラカラーのストア/ロードは発生しません。逆に、間に何かが挟まってマージが切れると、そこに往復が生まれます。
- カメラ深度のストアアクションにも注目してください。Depth Textureをオンにするとカメラ深度がStoreされるようになります。シェーダーやポストプロセスが`_CameraDepthTexture`を要求していないのに設定だけオンになっている、というのは実プロジェクトでよくある無駄です。



▲図1.8　Copy DepthによってDraw Opaque ObjectsとDraw Transparent Objectsが分断



▲図1.9　分断されているためDraw Opaque Objects終了時にStore

このように、各アタッチメントのStoreに正当な理由（後続の利用）があるかを1つずつ確認していくのが、タイルメモリ最適化の基本の流れです。

1.2.5　AccessFlagsによる削減例

最後に、`AccessFlags`の指定だけでアクションが変わることを実例で確認します。全画面Blitを行うパスで、出力アタッチメントの申告を`Write`にした場合と`WriteAll`にした場合を比較します。

▼リスト1.3　AccessFlagsのWriteとWriteAllの比較

```
// パターンA: Write(全ピクセル書き込みを申告しない)
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.Write);

// パターンB: WriteAll(全ピクセル書き込みを申告する)
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
```

描画結果はどちらも同じです。しかしRender Graph Viewerで確認すると、パターンAではロードアクションがLoadに、パターンBではDontCareになります。全画面Blitなので、ロードされた内容は全ピクセル上書きされて捨てられます。つまりパターンAのロードは無駄になっています。

1.3　Render Pass / Subpass

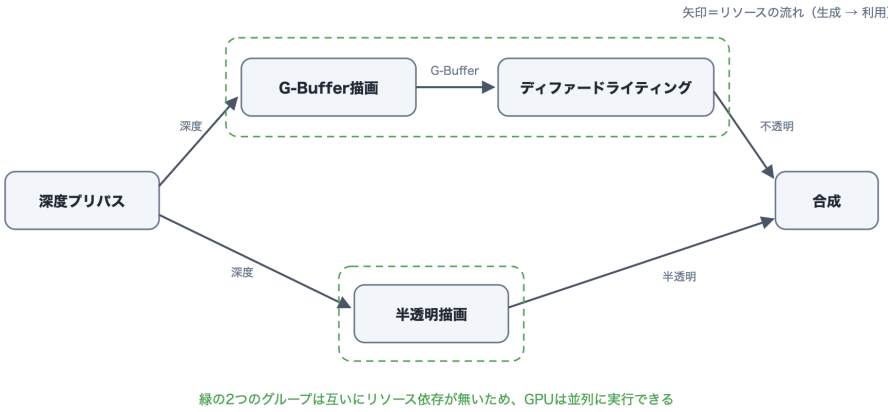
ここまでで、パスとパスの間のストア→ロード往復がコストになることが分かりました。この節では、そのコストを消すための仕組みであるRender Pass/Subpassと、Render Graphによるパスマージについて解説します。

1.3.1　リソース依存性とDAG

ゲームの絵を完成させるまでの1フレームの工程をレンダリングパイプライン、それを構成する一つ一つの処理をレンダリングパスと呼びます。各パスは「どのリソースを読み、どのリソースへ書くか」というリソース依存性を持っています。たとえばディファードレンダリングを含む典型的なジオメトリ描画は、次のような依存関係を持ちます。

1. 深度プリパスで深度バッファを作成
2. G-Bufferに不透明オブジェクトを描画（深度バッファを使用）
3. 不透明バッファにディファードライティング（G-Bufferと深度バッファを使用）
4. 半透明バッファに半透明オブジェクトを描画（深度バッファを使用）
5. 不透明バッファと半透明バッファを合成してフレームバッファに描画

パスをノード、リソース依存性をエッジとして描くと、このパイプラインは有向非巡回グラフ（DAG: Directed Acyclic Graph）になります。4番のパスは2番・3番の出力を使っていないため、依存性さえGPUに伝われば並列実行が可能です。



▲図1.10　レンダリングパスのDAGと並列化

Render Graphという名前の「Graph」はこのDAGを指しています。Render Graphは、パスのリソース依存性を集めてDAGを構築し、それをもとに実行計画を立てるシステムです。

1.3.2　VulkanのRenderPass/Subpass

リソース依存性をGPUに伝える仕組みとして、VulkanにはRenderPass/Subpassという概念があります。Render Graphが最終的に構築するのはこのVulkanのRenderPassなので、少しだけAPIレベルの話をします。VulkanのRenderPassは次の要素で構成されます。

- アタッチメント：RenderPassで使うレンダーターゲットの宣言。各アタッチメントには`loadOp`（ロードアクション）と`storeOp`（ストアアクション）を指定します。前節「タイルメモリのロード/ストア」で見たアクションの正体はこれです。
- サブパス：RenderPass内の描画の単位。各サブパスは出力アタッチメント（カラー/深度）と入力アタッチメントを宣言します。
- サブパス依存性：サブパス間の実行順序とメモリの依存関係の宣言。

ここで押さえておきたい性質が2つあります。

1. ロード/ストアはRenderPassの境界でのみ発生し、サブパス間では発生しませ

ん。つまり複数の処理を1つのRenderPassのサブパスとして構成できれば、その間の中間結果はタイルメモリに留まります。
2. 入力アタッチメントを使うと、前のサブパスの結果を同一ピクセル位置に限って読めます。これが次節で扱うフレームバッファフェッチのAPI上の姿です。

先ほどのジオメトリ描画をRenderPass/Subpassで表現すると次のようになります。

- ジオメトリ描画（RenderPass）
  - 深度プリパス（Subpass） — 出力: 深度
  - G-Buffer描画（Subpass） — 入力: 深度 / 出力: G-Buffer
  - ディファードライティング（Subpass）— 入力: G-Buffer / 出力: 不透明バッファ
  - 半透明描画（Subpass） — 入力: 深度 / 出力: 半透明バッファ
  - 合成（Subpass）— 入力: 不透明・半透明バッファ / 出力: フレームバッファ

この構成なら、深度・G-Buffer・不透明バッファ・半透明バッファはすべてサブパス間の受け渡しであり、RenderPass境界をまたぐのは最終出力のフレームバッファだけです。ストアが5回発生し得た構成が、ストア1回になります。ちなみにMetalにも同等の仕組みがあります。MetalのRender Passはロード/ストアアクションを持ち、タイルメモリ上のカラーアタッチメントをフラグメント関数から直接読むこと（およびタイルシェーダー）ができます。Vulkanのような明示的なSubpassの宣言はありませんが、Render Graphが行う最適化はMetal上でも同様に機能します。

1.3.3 UnityのNative Render Pass

では、Render GraphはこのVulkanのRenderPass/Subpassをどう扱うのでしょうか。まず押さえておきたいのは、`AddRasterRenderPass`で追加したパスがそのままサブパスになるわけではないという点です。`RecordRenderGraph`で追加されるパスは、あくまでRender Graphの論理的なパスです。Render Graphはフレーム内の全パスの記録が終わった後、コンパイル工程で次を行います。

1. リソース依存性からDAGを構築し、不要なパスを刈り取る（カリング）
2. 隣接するパスのうち、条件を満たすものを1つのNative Render Pass（VulkanのRenderPassに相当）にマージし、マージされた各パスをサブパスにする
3. 各アタッチメントのロード/ストアアクションを決定する（前節の「Render Graphがアクションを決定するルール」参照）
4. 可能なリソースをメモリーレス化する（「メモリーレス」の節）

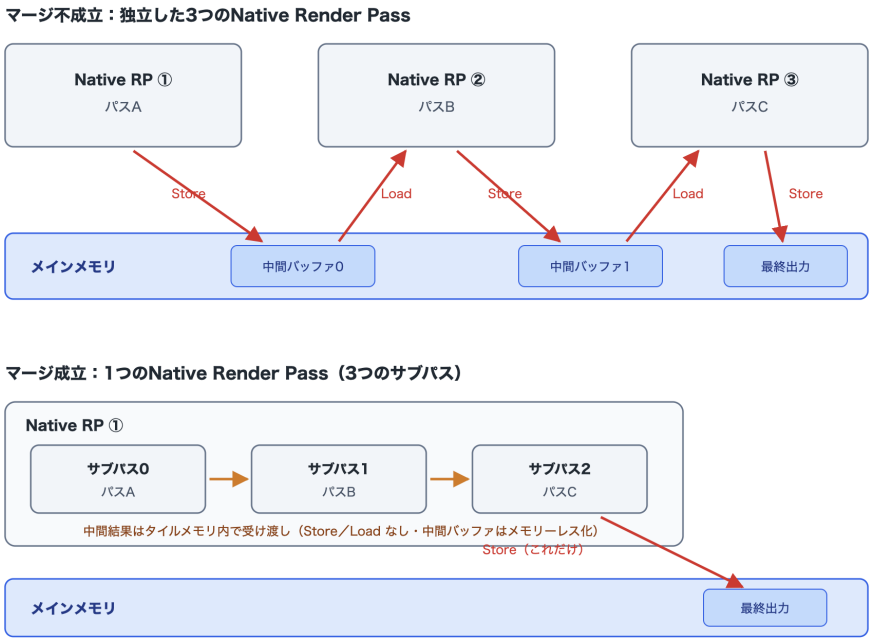
つまり開発者側の仕事は、サブパスを手で組むことではなく、マージ可能な形でパスを設計して、リソースの使い方を正しく申告することです。マージの成否はFrame Debuggerの(RP n:m)表記とRender Graph Viewerで確認していきます。

1.3.4 パスマージの条件

Render Graphがパスをマージする条件を整理します。

1. 両方がRaster Render Passである。AddComputePassやAddUnsafePassはマージされません。Rasterパスの列の間にComputeパスが挟まると、そこでNative Render Passが切れます。
2. パスが連続して記録されている。マージはグラフ上で隣接するパス同士で行われます。
3. 出力アタッチメントの解像度が一致している。中間バッファの解像度を変える（ダウンサンプリングする）と、その境界ではマージできません。
4. MSAAサンプル数が一致している。
5. 前のパスの出力を、後のパスがテクスチャとして利用していない。UseTextureでの利用はストア→サンプリングを要求するためマージを妨げます。SetInputAttachment（入力アタッチメント）としての利用はマージを妨げません。入力アタッチメントとテクスチャは別物です。
6. アタッチメントの合計数が上限（8枚）を超えない。マージ後のNative Render Passが使うアタッチメントの総数には上限があります。
7. グローバルテクスチャ化など、Render Graphの追跡を外れる利用がない。

条件を1つでも満たさないと、そのパスは単独のNative Render Passになります。マージされなかった理由はRender Graph ViewerのPass Break Reasoningに表示されるので、マージされないときはまずそこを確認してください。



▲図1.11　マージ前後のNative Render Pass構造の比較

1.4　フレームバッファフェッチ

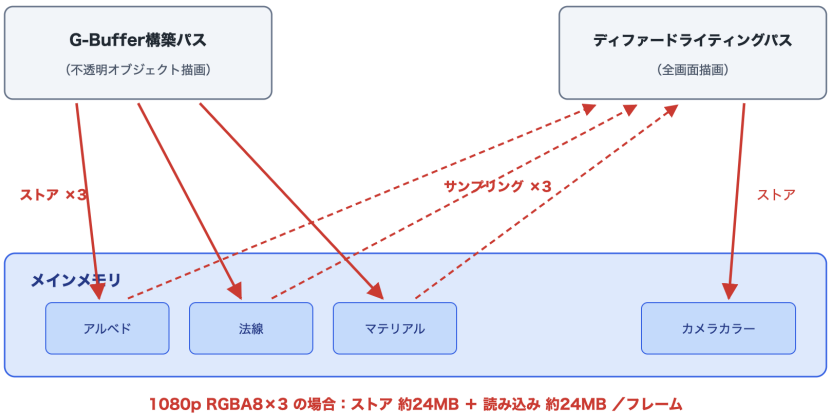
条件を満たすことでパスを1つのNative Render Passにマージできることが分かりました。この節では、マージされたサブパス間で前のサブパスの結果をタイルメモリから直接読む技術、フレームバッファフェッチ（Frame Buffer Fetch、以下FBF）について解説します。

1.4.1　フレームバッファフェッチとは

G-Bufferを作成してライティングを行うディファードレンダリングを例に考えます。シンプルに実装すると処理は次のようになります。

1. G-Bufferの作成（アルベド、法線、マテリアル情報をタイルメモリに書き込む）
2. アルベド、法線、マテリアル情報をメインメモリにストアする
3. ストアされたG-Bufferをテクスチャとしてサンプリングしてディファードライ

ティングを行う



▲図1.12　シンプルなディファードレンダリングのデータフロー

ここで一つ疑問が生まれます。ライティングのパスが読みたいのは「いま処理しているピクセルの」G-Bufferの値です。その値は、直前のパスが同じタイルのタイルメモリに書き込んだばかりです。メインメモリを往復せずに、タイルメモリから直接読むことはできないのでしょうか。それを可能にするのがFBFです。FBFを使うと処理は次のようになります。

1. G-Bufferの作成（タイルメモリに書き込む）
2. タイルメモリからG-Bufferを直接読んでディファードライティングを行う



▲図1.13　フレームバッファフェッチを使ったディファードレンダリングのデータフロー

G-Buffer3枚分のストアと、それをサンプリングするための読み込みがなくなります。1080pのRGBA8×3枚なら、ストアだけで約24MB/フレーム、60FPSで約1.4GB/sの帯域削減です。APIレベルでは、FBFは前節「VulkanのRenderPass/Subpass」で触れたVulkanの入力アタッチメント（Metalではカラーアタッチメントの直接読み取り）として実現されます。したがってFBFが使えるのは同一Render Pass内のサブパス間だけです。パスがマージされていることがFBFの前提条件になります。

1.4.2　制約の理解

FBFはテクスチャサンプリングの置き換えではありません。次の制約があります。

1. 処理中のフラグメントと同じピクセル位置しか読めません。タイルメモリには「いま処理しているタイル」の内容しかないため、オフセットサンプリング（周辺ピクセルの読み取り）は原理的に不可能です。
2. フィルタリングやミップマップは使えません。単一の値のロードだけです。
3. 入力側と出力側の解像度が一致している必要があります（マージ条件でもあります）。

制約1が効く実例を見てみましょう。「モノクロ/セピア合成の結果に、UVを歪ませるノイズ加工を重ねる」という処理を考えます。ノイズ加工のシェーダーは次のようにUVをずらしてサンプリングします。

▼リスト1.4　ノイズ加工シェーダー

```
// ノイズを使ってUVをゆがませる
float2 distortedUV = input.texcoord + noise * 0.02;
// ゆがませたUVでテクスチャをサンプリング。
// ロードする位置がフラグメントの位置と違うためフレームバッファフェッチは使えない。
half4 color = SAMPLE_TEXTURE2D_X(_BlitTexture, sampler_LinearClamp, distortedUV);
```

読みたい位置が処理中のフラグメントの位置と異なるため、このパスはFBFを使えません。合成結果をUseTextureで申告する必要があり、合成パスとノイズパスの間でストア→サンプリングの往復が発生します。このため、エフェクトを設計する段階で「このパスは同一ピクセルしか読まないか？」を確認しておくと、FBFが使えるかどうかを早めに判断できます。同一ピクセルで済む処理（トーンマッピング、カラーグレーディング、合成、ディファードライティング）はFBFの候補になり、周辺を読む処理（ブラー、歪み、Bloom）はテクスチャ利用になります。

1.4.3　シェーダーAPIの内部

Unityのシェーダーから FBFを使うには、SRP Coreが提供するマクロを利用します。

▼リスト1.5　フレームバッファフェッチの使い方

```
// 入力アタッチメントの番を宣言する
FRAMEBUFFER_INPUT_X_HALF(0);

half4 frag(Varyings input) : SV_Target
{
    // 入力アタッチメントの番から、処理中のフラグメント位置の値をロードする
    half4 color = LOAD_FRAMEBUFFER_X_INPUT(0, input.positionCS.xy);
    ...
}
```

マクロには精度と用途に応じたバリエーションがあります。

宣言マクロ	ロードマクロ	用途
FRAMEBUFFER_INPUT_HALF(n)	LOAD_FRAMEBUFFER_INPUT(n, pos)	half精度のカラー
FRAMEBUFFER_INPUT_FLOAT(n)	LOAD_FRAMEBUFFER_INPUT(n, pos)	float精度
FRAMEBUFFER_INPUT_X_HALF(n)	LOAD_FRAMEBUFFER_X_INPUT(n, pos)	XR（シングルパスインスタンシング）対応版

_X付きのバリエーションはTEXTURE2D_X系マクロと同じく、XRのレンダーターゲッ

ト配列に対応したものです。XR対応の可能性があるプロジェクトでは_X付きで統一しておくと安全です。このマクロがプラットフォームごとに何に展開されるかを知っておくと、挙動やfallbackを理解しやすくなります。

プラットフォーム	展開先	実体
Vulkan	subpassInput + subpassLoad	Render Passの入力アタッチメント
Metal	[[color(n)]]属性によるカラーアタッチメント読み取り	タイルメモリの直接読み取り
OpenGL ES	GL_EXT_shader_framebuffer_fetch等の拡張（対応GPUのみ）	拡張非対応時は下のfallback
非対応環境	通常のテクスチャ（_UnityFBInput0等）+ ロード命令	FBFにならず、通常のテクスチャ読み込みになる

注意したいのは最後の行です。マクロは非対応環境でも動作はするようにfallbackしますが、その場合は帯域削減の効果がありません。つまり、動作しているからといってFBFが効いているとは限りません。効果が出ているかどうかは、Render Graph Viewerのアクション表示（入力側バッファのストアがDontCareになっているか）と実機の帯域計測で確認してください。

1.4.4 SetInputAttachmentとUseTextureの違い

C#側では、FBFで読みたいリソースを`SetInputAttachment`で申告します。

▼リスト1.6　入力アタッチメントの申告

```
using (var builder = renderGraph.AddRasterRenderPass<PassData>("Lighting", out var
    passData))
{
    builder.SetRenderAttachment(cameraColor, 0, AccessFlags.Write);
    // シェーダーの FRAMEBUFFER_INPUT_X_HALF(0) に対応する
    builder.SetInputAttachment(gBufferAlbedo, 0);
    // FRAMEBUFFER_INPUT_X_HALF(1) に対応する
    builder.SetInputAttachment(gBufferNormal, 1);
    ...
}
```

`SetInputAttachment`の第2引数のインデックスが、シェーダー側の`FRAMEBUFFER_INPUT_X_HALF(n)`のnに対応します。ここで、「タイルメモリのロード/ストア」の節から繰り返し登場している`UseTexture`との違いを表で整理しておきます。

項目	UseTexture	SetInputAttachment
読める位置	任意のUV（オフセット、フィルタリング、ミップ可）	処理中のフラグメント位置のみ
前のパスとのマージ	妨げる	妨げない
前のパスの出力のストア	Storeが強制される	マージ成立時はDontCareにできる
帯域コスト	ストア＋サンプリング読み込み	ゼロ（タイルメモリ内で完結）
シェーダー側の読み方	SAMPLE_TEXTURE2D系	LOAD_FRAMEBUFFER系

1.4.5 深度のフェッチ

カラーだけでなく深度もタイルメモリ上にあります。深度をフェッチできれば、ソフトパーティクルやデカール、フォグのような「シーンの深度を参照する処理」からも`_CameraDepthTexture`のストア→サンプリング往復を消せる可能性があります。SRP Coreには深度用の入力アタッチメントを扱うためのマクロ群が用意されていますが、深度アタッチメントの入力化はプラットフォームとURPのバージョンによる制約が出やすい領域です。検証結果が出るまでの本章の推奨は、確実に動作する次の代替案です。代替案として、深度をカラーアタッチメントに書いてフェッチする方法があります。G-Buffer構築のサブパスで、線形化した深度値をカラーアタッチメントの1チャンネル（R16_Float等）に書き込み、後続サブパスではそれをFBFで読みます。アタッチメントが1枚増えますが、メモリーレス化されれば実メモリのコストはゼロであり、深度テクスチャのストア往復は確実になくせます。

1.4.6 プラットフォーム差と落とし穴

最後に、FBFを使う際に確認しておきたいプラットフォーム差をまとめます。

環境	FBFの利用	備考
Vulkan (Adreno)	○ 入力アタッチメント	広くサポート。ドライバ品質は端末による
Vulkan (Mali)	○ 入力アタッチメント	広くサポート
Vulkan (IMG / Pixel 10)	○ 入力アタッチメント	TBDR系。ドライバ固有の挙動差に注意（最終節の「GPUベンダー差」）
Metal (Apple)	○ カラーアタッチメント読み取り	安定して機能する

環境	FBFの利用	備考
OpenGL ES	△ 拡張依存	GL_EXT_shader_framebuffer_fetch対応GPUのみ。非対応はfallback
エディタ (D3D等)	× fallback動作	通常テクスチャとして動く。動作確認はできるが効果測定はできない

特に注意したい点は次の3つです。

1. エディタでは効果が見えません。fallbackで正しく描画されてしまうため、実機で計測して初めて差が分かることを前提にワークフローを組んでください。
2. MSAA併用時はマージ条件（サンプル数一致）に引っかかりやすくなります。MSAAが有効なカメラカラーと、MSAAなしで作った中間バッファはマージできません。中間バッファのTextureDescは`GetTextureDesc(cameraColor)`のように元のDescから複製し、サンプル数を引き継ぐようにしてください。
3. 1つのサブパスで読める入力アタッチメント数には上限があります。アタッチメント合計8枚の制約（前節の「パスマージの条件」）は入力側も含めた話です。G-Bufferを設計するときはチャンネルパッキングで枚数を節約してください。

1.5 メモリーレス

ここまでで、中間バッファのロードもストアも消せるようになりました。ここで一つ考えてみたいことがあります。ロードもストアもしないバッファに、メインメモリ上の実体は必要なのでしょうか。

1.5.1 メモリーレスとは

実は必要ありません。タイルメモリ上でだけ使われて、タイルメモリ上で破棄されるリソースには、メインメモリの確保そのものが要りません。この状態をメモリーレスと呼びます。Vulkan/Metalには、リソースを「タイルメモリ（オンチップメモリ）にのみ存在させ、メインメモリのバッキングストアを持たない」ものとして作成するモードがあります（Vulkanでは`LAZILY_ALLOCATED`メモリ、Metalでは`MTLStorageMode.memoryless`）。メモリーレスが指定されたリソースは次の性質を持ちます。

- メインメモリを消費しない（アプリのメモリフットプリントが減る）

- ロードアクションにLoadを、ストアアクションにStoreを指定できない（読み書きする実体がないため）

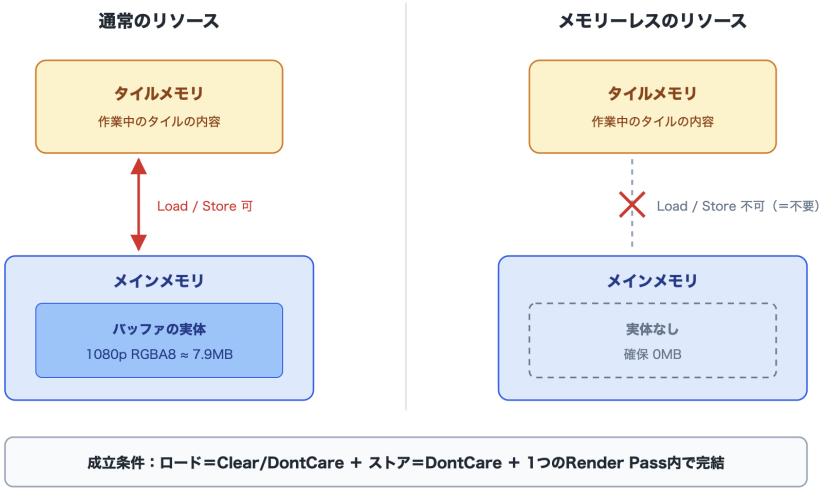
つまりメモリーレスは、ここまでの節でロード/ストアを消した結果として成立するものです。ロード/ストアが消えていればメモリーレス化でき、メモリーレス化できない場合は、どこかにまだロードかストアが残っています。

1.5.2 成立条件

あるリソースがメモリーレス化できる条件を、これまでの言葉で書くと次のようになります。

1. ロードアクションがClearまたはDontCareである（以前の内容をメインメモリから読まない。「AccessFlagsによるリソース利用の申告」の項で見たWriteAll/Discard申告、または初回利用のClear）
2. ストアアクションがDontCareである（後続のパスからテクスチャとして利用されず、次フレームにも持ち越さない。「Render Graphがアクションを決定するルール」の項を参照）
3. 利用が1つのNative Render Pass内で完結している（サブパス間の受け渡し、つまり入力アタッチメント/FBFは使ってもよい。「Render Pass / Subpass」の節のマージ、「フレームバッファフェッチ」の節のFBF）

見ての通り、この3条件はここまでの節で解説してきた内容そのものです。



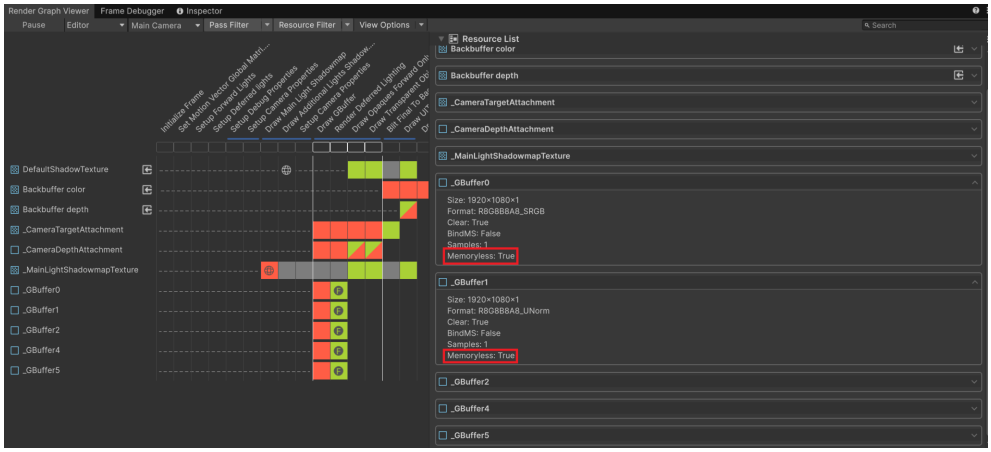
▲図1.14　メモリーレス化前後のメモリマップ比較

自動メモリーレス化と明示指定

UnityのRender Graphでは、メモリーレスは2つの経路で実現されます。

自動メモリーレス化

Render Graph管理のテクスチャ（`renderGraph.CreateTexture`で作成）が「成立条件」の項の3条件を満たす場合、Render Graphが自動的にメモリーレス化します。開発者は何も指定する必要がありません。メモリーレス化されているかどうかはRender Graph ViewerのResource Listで確認することができます。



▲図1.15　Render Graph ViewerでのMemoryless表示

明示指定

TextureDescのmemorylessフィールドで明示的に指定することもできます。

▼リスト1.7　メモリーレスの明示指定

```
var desc = renderGraph.GetTextureDesc(universalResourceData.cameraColor);
desc.memoryless = RenderTextureMemoryless.Color; // メモリーレスを指定
desc.name = "Intermediate Texture";
var handle = renderGraph.CreateTexture(desc);
```

明示指定には、設計意図をコードに残せるという利点があります。メモリーレスを指定したリソースをうっかり後続パスでUseTextureしようとすると、その時点で構成の矛盾が顕在化します。自動に任せていると、リファクタリングでどこかにUseTextureが紛れ込んだ場合に、メモリーレスが外れたこと（＝メモリ使用量と帯域が増えたこと）に気づけません。このリソースはタイルメモリ内で完結させる、という設計意図があるなら、明示指定でコードに残しておくのがおすすめです。

1.5.3　削減量の定量化

メモリーレス化の削減量は計算で求められます。1080pでの代表例を示します。

リソース構成	計算	削減量
RGBA8 中間バッファ1枚	1920×1080×4B	約7.9MB
RGBA16F 中間バッファ1枚	1920×1080×8B	約15.9MB
G-Buffer 3枚（RGBA8）＋深度	7.9MB×4	約31.7MB
4xMSAA RGBA8 カラー	1920×1080×4B×4	約31.7MB
4xMSAA 深度（D32）	1920×1080×4B×4	約31.7MB

表の通り、削減量が大きいのはMSAAとG-Bufferです。特に4xMSAAはカラーと深度を合わせて60MB超になるため、これが消えるかどうかはアプリ全体のメモリバジェットにかなり効いてきます。なお、メモリーレスによる削減は「メモリ使用量」であり、帯域の削減はここまでの節で行ってきたロード/ストア削減によるものです。両者は別の指標であることに注意してください（メモリーレスはロード/ストア削減の結果として成立するので、実務上はセットで達成されます）。

1.5.4 MSAAレンダーターゲットのメモリーレス化

メモリーレスの効果が最も大きいのがMSAAです。仕組みから見ていきましょう。4xMSAAでは、タイルメモリ上に1ピクセルあたり4サンプルのカラーと深度が保持されます。しかし最終的に画面に表示するのは、4サンプルを平均（リゾルブ）した1サンプル分だけです。ここでストアアクションの選択肢が効いてきます。

- Storeを選ぶと、4サンプル分のデータがそのままメインメモリに書き出されます（帯域4倍、メモリ4倍）。
- Resolveを選ぶと、タイルメモリ上でリゾルブが行われ、リゾルブ後の1サンプル分だけが書き出されます。

TBRのGPUではリゾルブはタイルメモリ上でほぼコストなしに行えます。そしてResolveを選んだ場合、4サンプル分のMSAAバッファ自体はストアされないため、メモリーレスの条件を満たします。つまり正しく構成されたMSAAは、

- MSAAカラーバッファ（4サンプル） → メモリーレス（メインメモリ確保なし、ストアなし）
- リゾルブ先バッファ（1サンプル） → 通常のストア
- MSAA深度バッファ（4サンプル） → メモリーレス＋ストアなし（深度を後続で使わない場合）

という構成になります。TBRのGPUでMSAAが（IMRと比べて）低コストだと言われ

るのはこのためです。URPでは、Universal Render PipelineアセットのMSAA設定を有効にすると、Render Graphがこの構成を自動的に組みます。確認すべきはそれが壊れていないかどうかです。Render Graph ViewerでMSAAカラーバッファのストアアクションがResolveになっているか、Memorylessがyesになっているかを確認してください。壊れる典型例は、MSAAバッファそのものを後続パスがテクスチャとして要求してしまうケース（カスタムパスでの誤った申告、Opaque Textureの構成ミスなど）です。

1.5.5 GPUベンダー差

本章で扱った技術はいずれも標準的なVulkan/Metalの機能の上に成り立っており、主要ベンダーで機能します。そのうえで、ベンダーごとの特性差を把握しておくと計測結果の解釈やトラブル対応がしやすくなります。

ベンダー	アーキテクチャ特性	注意点
Qualcomm Adreno	GMEMベースのビニング。描画内容によってダイレクトモード（IMR的動作）に切り替わることがある	ダイレクトモードに切り替わるとタイルメモリ前提の最適化効果が変わる。プロファイラでどちらのモードかを確認する
ARM Mali	16×16タイル基本のTBR	外部帯域カウンタが充実しており、本章の帯域計測の基準ベンダー
Imagination (IMG)	ハードウェア隠面除去（HSR）を持つTBDR。Pixel 10のIMG DXTが該当	HSRにより不透明オーバードローが元々少なく、最適化の効きどころが他ベンダーと異なる。新しいドライバのため挙動差の検証を推奨
Apple	PowerVR系譜のTBDR。Metalでタイルメモリを明示的に扱える	FBF/メモリーレスが安定して機能する。iOSで効果を確認してからAndroidの検証に進むのも有効

1.5.6 おわりに

本章で扱った4つのテーマを、最後にもう一度まとめます。Render Graphを使う場合、開発者がやることは、リソースの使い方を正しく申告することと、Frame DebuggerやRender Graph Viewerで結果を確認することの2つです。地味な作業ですが、その積

み重ねが端末の発熱やバッテリー消費の差になって表れます。本章が皆さんのプロジェクトの最適化に役立てば幸いです。

1.5.7 参考文献

- ARM, "Arm GPU Best Practices Developer Guide", https://developer.arm.com/documentation/101897/latest/
- Qualcomm, "Adreno GPU Best Practices", https://docs.qualcomm.com/doc/80-78185-2/topic/mobile_best_practices.html
- Apple, "Metal Best Practices Guide", https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/
- Unity, "Render graph system" (URP Documentation), https://docs.unity3d.com/Manual/urp/render-graph.html
- 『学園アイドルマスター』モバイルでも"ライティングと反射"を描き切る！【CEDEC2024】, https://gamemakers.jp/article/2025_01_29_90729/
- モバイルで爆速なDeferred Renderingを行う方法, CEDiL, https://cedil.cesa.or.jp/cedil_sessions/view/2457
- Unity6からRenderGraphを使いこなそう ― 基本機能編, https://blog.sge-coretech.com/entry/2024/06/04/171757
- Unity6からRenderGraphを使いこなそう！ 応用実装編1「RenderPassの構築」, https://blog.sge-coretech.com/entry/2024/09/11/105032
- シェーダー最適化入門 第2回「タイルベースレンダリングのGPUとは？」, https://blog.sge-coretech.com/entry/2025/01/27/145242

第2章

RenderGraphのUnsafePassを使うべき場面

チャン ユービン ZHANG YUBING
株式会社サイバーエージェント SGEコア技術本部

2.1　はじめに

Unity 6では、Render Graphによるレンダリングパイプラインの構築時に、処理内容に応じてRasterRenderPass、ComputePass、UnsafePassを使い分けます。本章では、UnsafePassを使うべき場面を紹介します。

Render Graphには依存関係の解析やSubPassマージといった最適化があります。そのため、通常の描画にはRasterRenderPassを、Compute Shaderを使う処理にはComputePassを優先するのが基本です。

UnsafePassを選ぶべき場面は、主に次の4つです。

- RasterRenderPassやComputePassでは実現できない処理がある場合
- CommandBuffer自身が必要な場合
- Compatibility ModeからRender Graphへ素早く移行したい場合
- 1つの描画処理が多くマージできないPassで構成されている場合

最初の3つは、UnsafePassを使用しなければ実現できない、または実装コストが大きい場合です。最後の1つは、必須ではないものの、Passを1つのUnsafePassにまとめることでCPU負荷の低減とRender Graph Viewerの見通しの改善が見込める場合です。

これから、これらの場面を順に解説していきます。

2.2 UnsafePassを選ぶべき場面

UnsafePassはC#のunsafe機能を意味するものではありません。Render Graphによる一部の自動管理や検証を抑え、CommandBufferに近いAPIを利用できるようにしたPassです。

2.2.1 RasterRenderPassやComputePassでは実現できない処理がある場合

RasterRenderPassやComputePassで使用するRasterCommandBuffer／ComputeCommandBufferは、CommandBufferのうち各Passの役割に必要なAPIだけを公開したラッパーであり、以下のようなAPIは利用できません。

- SetRenderTarget：Load/Store Actionを指定しながらレンダーターゲット、MRT、ミップ、配列スライスを細かく指定する
- CopyTexture：ミップ、配列要素、矩形領域を指定してテクスチャをコピーする
- GenerateMips：レンダーターゲットのミップマップを生成する
- SetRandomWriteTarget/ClearRandomWriteTargets：ピクセルシェーダーのランダム書き込みターゲットを設定または解除する
- RequestAsyncReadback：GPUリソースの非同期読み戻し要求を記録する

これらのAPIは、UnsafePassのUnsafeCommandBufferで利用できます。さらに、CommandBufferHelpers.GetNativeCommandBufferでネイティブのCommandBufferを取得すれば、より高度なAPIも利用できるようになります。

2.2.2 CommandBuffer自身が必要な場合

エンジンのコールバック、IMGUI、SDKの描画処理など、外部コードがCommandBufferを受け取って任意のコマンドを記録する処理では、記録される内容が実行時まで分かりません。記録時点で処理内容を宣言するRasterRenderPassやComputePassには、このようなブラックボックス処理を組み込めません。UnsafePassでネイティブのCommandBufferを取得し、外部コードへ渡すことで、Render Graphに組み込めます。

2.2.3 Compatibility ModeからRender Graphへ素早く移行したい場合

Compatibility ModeでExecuteメソッドに記述していたCommandBufferの処理は、コマンド列をほぼそのままUnsafePassのSetRenderFunc内へ移せます。そのため、少ない工数でRender Graphへの移行と動作確認を進めたい場合に有効です。

ただし、これはあくまで素早く移行するための暫定的な手段です。時間を確保できる場合は、処理内容に応じてRasterRenderPassやComputePassへ分類して移植し、UnsafePassは宣言的なAPIで表現できない部分だけに限定することを推奨します。

2.2.4 多くマージできないPassで構成されている場合

連続してマージできないPassを1つのUnsafePassにまとめることでCPU負荷の低減とRender Graph Viewerの見通しを改善する効果があります。

典型的なケース

- Ping-pong方式のBlur
- BloomのMip Pyramid生成
- 複数回のダウンサンプルとアップサンプル
- 前の処理の出力を次の処理でサンプリングするフィルタ処理

これらの処理では、Pass毎に描画ターゲットを切り替えたり、直前Passの出力を周辺サンプリングしたりすることで、SubPassマージが成立しません。

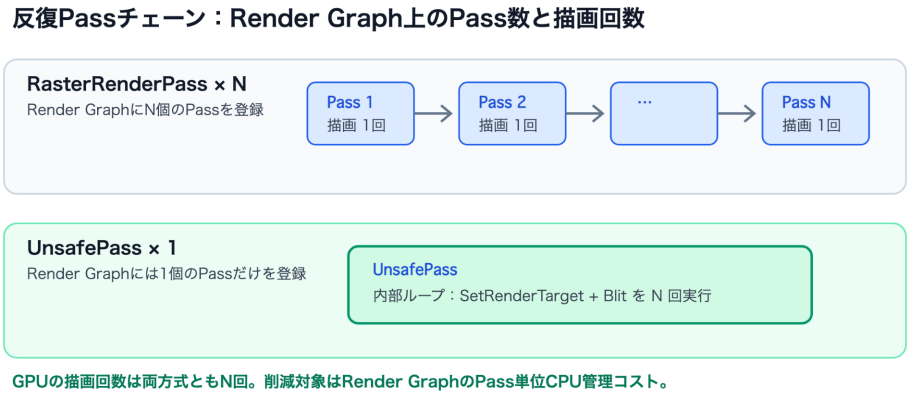
UnsafePassにまとめるとCPU負荷が下がる

RasterRenderPassを1つ登録するたびに、次の処理が発生します。

- Passの登録とPassDataの準備
- UseTextureやSetRenderAttachmentによるリソース依存関係の登録
- Passの依存関係解析とマージ候補の確認
- Profilingスコープの開始と終了
- レンダーターゲットの設定と実行コールバックの呼び出し

RasterRenderPassが増えるほど、CPU側のコストも増加します。マージできないPassチェーンをRasterRenderPassとして登録しても、マージの恩恵を得られないま

ま、登録や管理のコストが積み重なります。一方、それらを1つのUnsafePassにまとめると、Render Graph上では1つのPassとして登録されます。描画コマンドやレンダーターゲット切り替えのコストは変わりませんが、Render GraphがPass単位で行う登録/管理のコストを削減できます（図2.1）。



▲図2.1　マージできないPassチェーンにおけるRender Graph上のPass数と描画回数

CPU負荷軽減効果の測定例

本プロジェクトで実施したEditor上の測定では、256×256のRender Textureに対する4-tapのPing-pong Blurについて、Blur処理ごとにRasterRenderPassを登録する方式と、1つのUnsafePass内で一連のBlur処理を実行する方式を比較しました。測定条件は、Apple M4 Max（macOS 26.5）、Unity 6000.3.19f1、URP 17.3です。各組み合わせで500フレームを採取し、切り替え後の60フレームを除外した中央値を使用しました。測定値は、Render Graphの記録、ハッシュ計算、コンパイル、実行に関するCPU markerの合計です。

表2.1: Ping-pong BlurにおけるRender Graph関連CPU時間の比較

Blur回数	RasterRenderPass×N（μs）	UnsafePass×1（μs）	差分（μs）
5	286.4	280.0	6.4
10	334.8	313.6	21.2
20	427.8	389.1	38.7
50	675.9	592.9	83.0

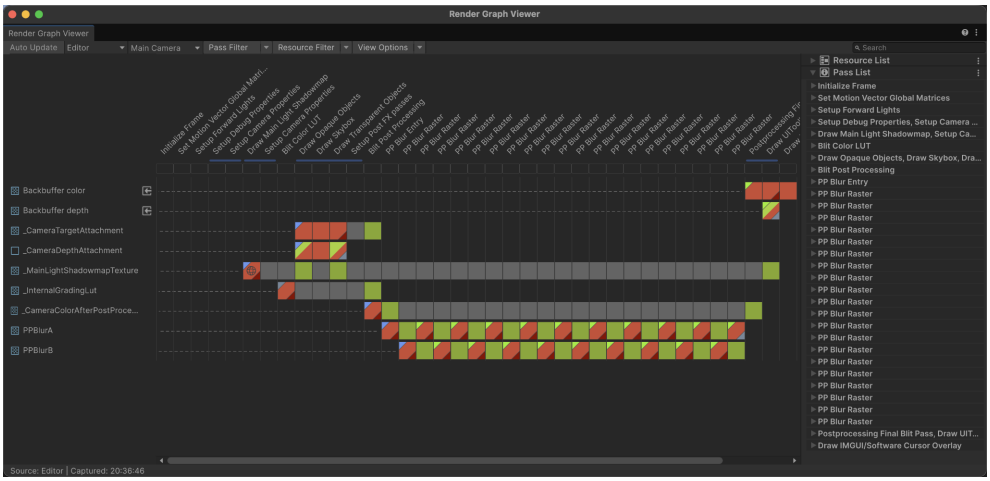
表2.1: Ping-pong BlurにおけるRender Graph関連CPU時間の比較

Blur回数	RasterRenderPass×N（μs）	UnsafePass×1（μs）	差分（μs）
100	1080.7	898.5	182.2

測定値はEditor環境における傾向確認用の値です。Blur回数が増えるほど、RasterRenderPass方式とUnsafePass方式の差が拡大しています。線形近似では、両方式の傾きの差、すなわちRasterRenderPassの追加固定コストは1Passあたり約1.8μsでした。この値はEditorでの測定値であり、実機での絶対値を示すものではありませんが、Blur回数に応じてCPU負荷の差が増えることを確認できます。この効果は、Render Graphの記録、実行、Profilingスコープ、レンダーターゲット設定など、Pass単位の登録/管理コストが削減されることによるものです。描画コマンドに差がないので、UnsafePass化によって描画負荷が減るわけではありません。

Render Graph Viewerの見通しも改善する

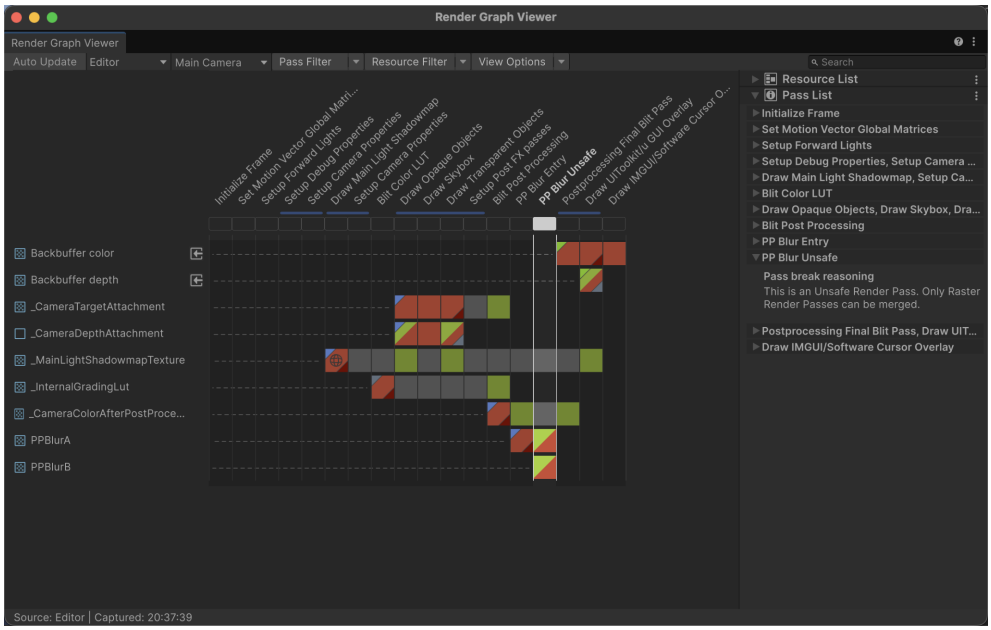
RasterRenderPassを多く登録すると、Render Graph Viewerには登録した数だけPassが表示されます。グラフが横方向に長くなるため、機能ごとの区切りが見づらくなり、リソースの依存関係も把握しにくくなります（図2.2）。



▲図2.2　Blur処理ごとにRasterRenderPassを登録した場合のRender Graph Viewer表示

機能内部のPassチェーンを1つのUnsafePassにまとめると、Viewerでは機能全体を

表す1つのPassとして確認できます（図2.3）。これにより、機能ごとの区切りが明確になり、リソースの依存関係も把握しやすくなります。



▲図2.3　一連のBlur処理を1つのUnsafePassにまとめた場合のRender Graph Viewer表示

CPU負荷の削減に加えて、プロファイリング時のノイズを減らせる点も大きな利点です。

ただし、Render Graph Viewerの見通しを改善するためだけに、異なる機能の処理やSubPassマージが成立する処理を無理にUnsafePassにまとめることは推奨されません。

2.3　UnsafePass使用時の注意事項

2.3.1　レンダリングターゲットの設定

RasterRenderPassでは記録時にSetRenderAttachmentでレンダーターゲットを宣言しますが、UnsafePassでは実行時にSetRenderTargetを呼び出して設定します。

2.3.2 依存関係を宣言する

UnsafePassを使用してもRender Graphから完全に切り離されるわけではありません。使用するテクスチャは、必ずUseTextureで読み書きの依存関係を宣言する必要があります。宣言が不足すると、リソースのライフタイム管理や同期が正しく行われません。

RasterRenderPassでは、レンダーターゲットの使用はSetRenderAttachmentで宣言されますが、UnsafePassではSetRenderTargetの呼び出しだけでは依存関係が登録されません。レンダーターゲットも必ずUseTextureで読み書きの依存関係を宣言する必要があります。

2.3.3 Load/Store Actionの設定

RenderBufferLoadActionはレンダーターゲットを有効にするときの既存内容の扱い、RenderBufferStoreActionは描画完了後の結果の扱いを指定します。Tile-Based GPUでは、不要なLoadやStoreがタイルメモリとメインメモリの転送を増やすため、適切な設定が重要です。

RasterRenderPassでは、宣言したAccessFlagsをもとにRender GraphがLoad/Store Actionを自動的に決定します。一方、UnsafePassでSetRenderTarget(target)のようにActionを省略すると、既定動作であるLoad／Storeが適用され、不要なメモリ転送によって性能が低下する可能性があります。

そのため、UnsafePassではSetRenderTargetの呼び出し時に、必ずLoad/Store Actionを明示します。

- Load側：全画面を上書きするならDontCare、既存内容を読む場合だけLoad
- Store側：結果を後続の処理で読むならStore、どこからも使わない場合だけDont Care

2.4 コード例：Ping-pong BlurのマージできないPassチェーン

本節に実際のコード例を紹介します。なお、掲載するコードは要点を示すための抜粋であり、そのまま動作する完全なコードではありません。

2枚のテクスチャを交互に読み書きしながらBlurを連続して適用する例です。同じ処理をRasterRenderPassで実装した場合を示し、その後に1つのUnsafePassへまとめた場合を示します。descはBlur用のRender Texture記述子、blurMaterialはBlur用のマテリアルです。

2.4.1 RasterRenderPass方式：Blur処理ごとにPassを登録する

RasterRenderPass方式では、リスト2.1 のように、ループ内でBlur処理ごとにAddRasterRenderPassを呼び出します。各Passの入力と出力は明確に宣言できますが、Blur回数が増えるほどRender Graph上のPass数も増加します。

▼リスト2.1　RasterRenderPassでPing-pong Blurを実装する

```
class BlurRasterPassData
{
    internal TextureHandle source;
    internal TextureHandle target;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var current = resourceData.activeColorTexture;
var writePing = true;

for (var i = 0; i < blurIterations; i++)
{
    var target = writePing ? ping : pong;

    using (var builder = renderGraph.AddRasterRenderPass<BlurRasterPassData>(
        "Ping-pong Blur Raster", out var passData))
    {
        passData.source = current;
        passData.target = target;
        passData.material = blurMaterial;
```

```
        builder.UseTexture(passData.source, AccessFlags.Read);
        builder.SetRenderAttachment(passData.target, 0, AccessFlags.Write);
        builder.SetRenderFunc((BlurRasterPassData data, RasterGraphContext context) =>
        {
            Blitter.BlitTexture(
                context.cmd, data.source, new Vector4(1, 1, 0, 0),
                data.material, 0);
        });
    }

    current = target;
    writePing = !writePing;
}
```

2.4.2 UnsafePass方式：1つのPass内で一連の処理を実行する

UnsafePass方式では、リスト2.2 のように、Render Graphへの登録を1回にして、SetRenderFunc内でレンダーターゲットを切り替えます。Blur処理ごとの描画コマンドは残りますが、Pass単位の管理を1回にまとめられます。

▼リスト2.2　Ping-pong Blurを1つのUnsafePassにまとめる

```
class PingPongBlurPassData
{
    internal TextureHandle source;
    internal TextureHandle ping;
    internal TextureHandle pong;
    internal int iterations;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var output = blurIterations == 0
    ? resourceData.activeColorTexture
    : blurIterations % 2 == 0 ? ping : pong;

using (var builder = renderGraph.AddUnsafePass<PingPongBlurPassData>(
    "Ping-pong Blur", out var passData))
{
    passData.source = resourceData.activeColorTexture;
    passData.ping = ping;
    passData.pong = pong;
    passData.iterations = blurIterations;
```

```
    passData.material = blurMaterial;

    builder.UseTexture(passData.source, AccessFlags.Read);
    builder.UseTexture(passData.ping, AccessFlags.ReadWrite);
    builder.UseTexture(passData.pong, AccessFlags.ReadWrite);
    builder.SetRenderFunc((PingPongBlurPassData data, UnsafeGraphContext context) =>
    {
        var cmd = CommandBufferHelpers.GetNativeCommandBuffer(context.cmd);
        var current = data.source;
        var next = data.pong;
        var alternate = data.ping;
        for (var i = 0; i < data.iterations; i++)
        {
            cmd.SetRenderTarget(
                next,
                RenderBufferLoadAction.DontCare,
                RenderBufferStoreAction.Store);
            Blitter.BlitTexture(
                cmd, current, new Vector4(1, 1, 0, 0),
                data.material, 0);

            current = next;
            (next, alternate) = (alternate, next);
        }
    });
}
```

2.5 まとめ

本章では、Render GraphでUnsafePassを使うべき場面と、使用時の注意事項を紹介しました。

UnsafePassを選ぶべき場面は次の4つです。

- RasterCommandBuffer/ComputeCommandBufferに公開されていないAPIが必要な場合
- 外部コードへ渡すCommandBufferそのものが必要な場合
- Compatibility ModeからRender Graphへ素早く移行したい場合
- マージできないPassチェーンを1つのPassにまとめたい場合

マージできないPassチェーンを1つのUnsafePassにまとめると、Pass単位の登録/管理コストが削減され、Render Graph Viewerの見通しも改善します。本プロジェクト

のEditor上の測定（表2.1）では、RasterRenderPass1つあたり約1.8µ sの固定コストを削減できました。ただし、描画コマンド自体は変わらないため、GPU負荷の削減を目的とした手法ではない点に注意が必要です。

一方で、UnsafePassはRender Graphによる自動管理や検証の一部を手放すPassでもあります。使用する際は、UseTextureによる依存関係の宣言と、SetRenderTargetでのLoad/Store Actionの明示を忘れないようにしてください。

基本はRasterRenderPassとComputePassを優先し、UnsafePassは宣言的なAPIで表現できない処理や、マージできないPassチェーンの整理に限定して使うことで、Render Graphの最適化とCommandBufferの自由度を両立できます。

2.6 参考資料

- Unity Manual - Use the CommandBuffer interface in a render graph (Unsafe Pass)
  `https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-unsafe-pass.html`
- Unity Manual - Optimize a render graph
  `https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-optimize.html`
- Unity Scripting API - CommandBuffer
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.CommandBuffer.html`
- Unity Scripting API - RenderBufferLoadAction
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferLoadAction.html`
- Unity Scripting API - RenderBufferStoreAction
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferStoreAction.html`

第3章

URPのFinalBlitPassを省略する方法

チャン ユービン ZHANG YUBING
株式会社サイバーエージェント SGEコア技術本部

3.1　FinalBlitPassとは

URPのレンダリングパイプラインにおいて、実際に画面へ表示されるのはBackBufferです。しかしBackBufferは書き込み専用で、読み込むことができません。そのためポストプロセスのように途中結果を入力として読む処理がある場合、カメラはいったん中間テクスチャへ描画され、最後にBackBufferへBlitされる仕組みになっています。この中間テクスチャからBackBufferへのBlitを担当するのが、URPのFinalBlitPassです。ただのコピーではなく、画面へ出す直前に必要な最終処理もまとめて担当しています。

- カメラのViewportを考慮した転送
- Linear→sRGB変換
- HDR Output時のエンコードとオーバーレイUIの合成
- UV原点が異なる場合のY反転
- スケーリング時のフィルタ切り替え

3.1.1　FinalBlitの機能を持つ3種類のFinalBlitPass

「中間テクスチャをBackBufferへ出力する」役割を持つパスは、実はURPに3種類あり、条件によってどれか1つだけが選ばれます。

UberPost

ポストプロセスが有効であれば追加されるパスですが、以下の条件を満たす場合にFinalBlitの機能も兼ねます。

- カメラスタックの最後のカメラであること
- 後述のFinalPostが不要であること
- AfterRenderingPostProcessing以降に実行されるカスタムパスが無いこと

FinalPost

ポストプロセスが有効で、かつUberPostでは処理しきれない「最終工程」がある場合に追加されるパスです。

1. FXAA
2. Linear以外のアップスケール … Point / FSR / STPなど。
3. TAA + CAS … RCASシャープニングを独立パスで実行する必要がある

FinalBlitPass

上2つのどちらもBackBufferを書いていない場合の、最後の砦です。実際に出るのは主に次のケースです。

- ポストプロセスが無効で、かつ中間テクスチャが存在する
- ポストプロセスは有効だが、その後ろにカスタムパスがある
- ポストプロセスは有効だが、Camera.AddCommandBufferAsyncなどのキャプチャ処理がある

3種類の関係を整理

表3.1: 最終出力を担当するパスと条件

最終出力を担当するパス	条件
UberPost	ポストプロセス有効、FinalPost不要、後続のカスタムパスもキャプチャも無し
FinalPost	ポストプロセス有効、かつFXAA / Linear以外のアップスケール / TAA + CAS のいずれか

表3.1: 最終出力を担当するパスと条件

最終出力を担当するパス	条件
FinalBlitPass	上2つがBackBufferを書いておらず、中間テクスチャがある場合
いずれも無し	中間テクスチャがない、またはカメラスタック上の最後のカメラではない

3.2 FinalBlitPassを省略する方法

URPのポストプロセスを使う時や通常のカスタムポストプロセスを作る時もFinalBlitPassを気にする必要がありません。ただし「最終工程」に相当するカスタムパスを入れた場合、URPのFinalBlitPassは余計なBlitを1回挟むだけの無駄なパスになってしまいます。そこでUnity 6.3から、カスタムパス側から「今からアクティブターゲットがBackBufferになる」と宣言し、URPのFinalBlitPassを省略できるようになりました。

▼リスト3.1　BackBufferへの切り替え

```
var resourceData = frameData.Get<UniversalResourceData>();
resourceData.SwitchActiveTexturesToBackbuffer();
```

実装は単純です（Runtime/FrameData/UniversalResourceData.cs）。

▼リスト3.2　SwitchActiveTexturesToBackbufferの実装

```
public void SwitchActiveTexturesToBackbuffer()
{
    activeColorID = UniversalResourceData.ActiveID.BackBuffer;
    activeDepthID = UniversalResourceData.ActiveID.BackBuffer;
}
```

frameData上のIDを2つ書き換えるだけで、BlitもコピーもRenderPassの追加も一切行いません。このAPIは「転送処理」ではなく、URPに対する宣言です。activeColorTexture / activeDepthTextureはこのIDを見て返すTextureHandleを切り替えるだけのswitchになっています。

▼リスト3.3　activeColorTextureの切り替え

```
public TextureHandle activeColorTexture
{
    get
```

```
    {
        switch (activeColorID)
        {
            case ActiveID.Camera:     return cameraColor;
            case ActiveID.BackBuffer: return backBufferColor;
            ...
        }
    }
}
```

SwitchActiveTexturesToBackbuffer()実行後、URPのFinalBlitPassは生成されなくなります。

3.2.1 使用例：カスタムFXAA

カスタムFXAAポストプロセスを例に、前後の比較をします。かつての典型的なフローは3回のBlitが必要でした。

▼リスト3.4　従来のFXAAフロー

```
CameraColor → [FXAA] → FXAABuffer → [Blit] → CameraColor → [URP FinalBlit] →
    BackBuffer
```

Unity 6.0以降はCameraColorを直接差し替えられるので2回に減りますが、FXAA Bufferという中間テクスチャは必要で、FinalBlitPassも引き続き作られます。

▼リスト3.5　Unity 6.0のFXAAフロー

```
CameraColor → [FXAA] → FXAABuffer → [URP FinalBlit] → BackBuffer
```

Unity 6.3でBackBuffer切り替えを使うと、1回で済みます。中間テクスチャも不要です。

▼リスト3.6　Unity 6.3のFXAAフロー

```
CameraColor → [FXAA] → BackBuffer
```

コードは次のようになります。なお以降のコードは、要点を示すために一部を抜粋したものです。そのままでは動作しません。

▼リスト3.7　BackBufferへ直接出力するカスタムFXAA

```
private class PassData
{
    public TextureHandle Source;
    public TextureHandle Target;
    public Material FXAAMaterial;
}

public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    var cameraData = frameData.Get<UniversalCameraData>();
    var resourceData = frameData.Get<UniversalResourceData>();

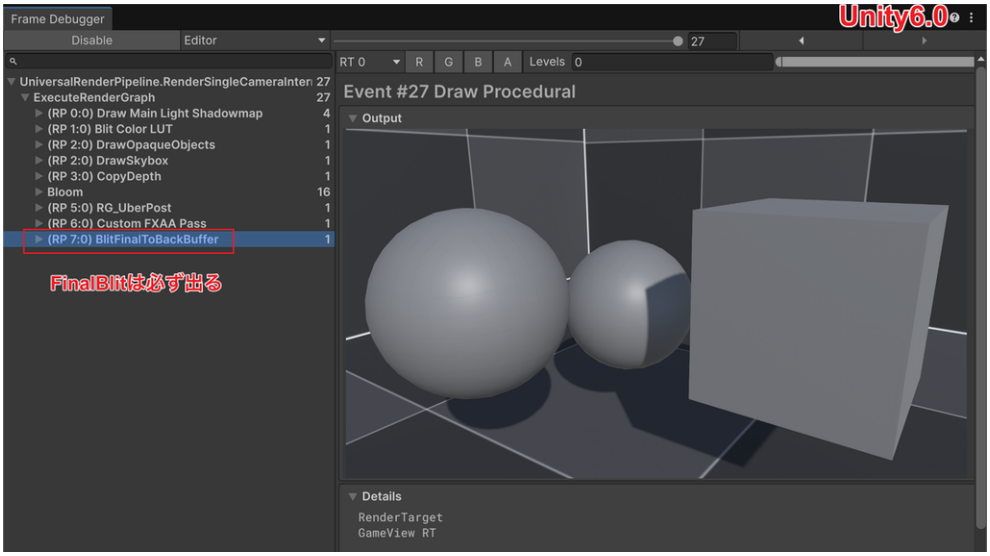
    // 切り替え前にCameraColorを取得しておく
    var cameraColor = resourceData.activeColorTexture;

    // 以降のactiveColorTextureはBackBufferColorを返す
    // これでURPのFinalBlitPassが生成されなくなる
    resourceData.SwitchActiveTexturesToBackbuffer();
    var backBufferColor = resourceData.activeColorTexture;

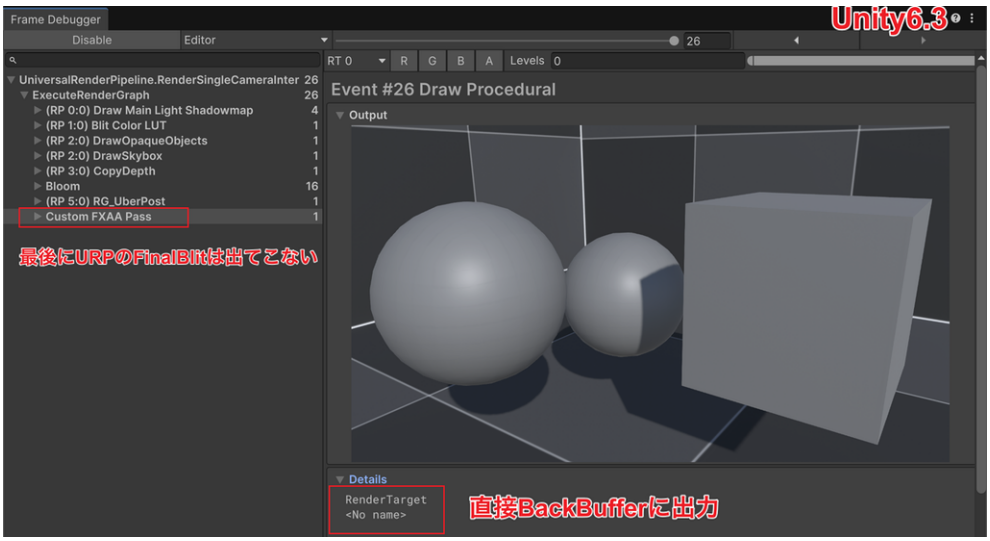
    // LinearToSRGBConversionが必要かどうかの判定
    var requireSrgbConversion = cameraData.targetTexture == null
                                && Display.main.requiresSrgbBlitToBackbuffer;
    CoreUtils.SetKeyword(_fxaaMaterial, ShaderKeywordStrings.LinearToSRGBConversion,
    requireSrgbConversion);

    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Custom FXAA Pass",
    out var passData))
    {
        passData.Source = cameraColor;
        passData.Target = backBufferColor;
        passData.FXAAMaterial = _fxaaMaterial;
        builder.SetRenderAttachment(backBufferColor, 0, AccessFlags.WriteAll);
        builder.UseTexture(passData.Source, AccessFlags.Read);
        builder.SetRenderFunc(static (PassData data, RasterGraphContext context) =>
        {
            // SourceとTargetのUV原点が異なる場合のみY反転が必要
            var yFlip = context.GetTextureUVOrigin(data.Source) != context.
    GetTextureUVOrigin(data.Target);
            var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
            Blitter.BlitTexture(context.cmd, data.Source, scaleBias, data.FXAAMaterial,
     0);
        });
    }
}
```

Unity 6.0ではFinalBlitPassが常に作成されていました。



Unity 6.3でBackBuffer切り替えを使うと、1回のBlitでFXAA処理が完了しBackBufferへ直接出力されます。



3.3 URPのFinalBlitを省略する時の注意事項

SwitchActiveTexturesToBackbuffer()は内部のテクスチャIDを書き換えるだけのAPIです。裏を返せば、辻褄を合わせる責任はすべて呼び出し側にあります。

3.3.1 カメラの最後にだけSwitchActiveTexturesToBackbuffer()を呼び出すこと

このAPIを呼んだ瞬間から、以降のすべてのパスのactiveColorTexture / activeDepthTextureはBackBufferを指します。しかしBackBufferはReadできないため、activeColorTexture / activeDepthTextureを読み込む後続処理はすべて破綻します。またcameraData.resolveFinalTarget == false（後続のカメラがある）場合も切り替えてはいけません。OverlayカメラはCameraColorに積み重ねて描画する設計のため、途中のカメラがBackBufferへ直接書くと後続カメラの描画結果と混ざるか上書きされて消えます。したがって呼び出しは、最後のカメラの、最終出力を行うカスタムパスの直前に限定してください。

3.3.2 最後に実行するカスタムパスにFinalBlitの機能を実装すること

FinalBlitPassを省略するということは、FinalBlitPassが担っていた最終処理も一緒に消えるということです。カスタムパス側で肩代わりする必要があります。

Y反転

SourceとTargetのUV原点が一致しない場合、反転しないと画面が上下逆になります。Unity 6.3で追加されたGetTextureUVOrigin()で両者を比較して判定します。このAPIはRasterGraphContextなど実行時のGraphContextのみが持つため、RecordRenderGraphの記録時ではなくSetRenderFunc内で呼ぶ必要があります。

▼リスト3.8　Y反転の判定

```
var yFlip = context.GetTextureUVOrigin(data.Source) != context.GetTextureUVOrigin(data.
    Target);
var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
```

なおSystemInfo.graphicsUVStartsAtTopはグラフィックスAPIのUV規約を示すだけで、「今回のBlitで反転が必要か」の判定には使えません。

sRGB変換

cameraData.requireSrgbConversionを見てLinearToSRGBConversionキーワードを設定する必要があり、Shader側でそのキーワードに応じた変換処理を実装する必要があります。

▼リスト3.9　sRGB変換キーワードの設定

```
var requireSrgbConversion = cameraData.targetTexture == null && Display.main.
    requiresSrgbBlitToBackbuffer;
cmd.SetKeyword(ShaderGlobalKeywords.LinearToSRGBConversion, data.requireSrgbConversion)
    ;
```

▼リスト3.10　sRGB変換

```
#ifdef _LINEAR_TO_SRGB_CONVERSION
    color = LinearToSRGB(color);
#endif
```

Viewport設定

Viewportは通常全画面に設定されているが、全画面以外に設定したい場合、CameraのViewportを設定する必要があります。

▼リスト3.11　Viewportの設定

```
var targetViewPort = new Rect(0, 0, targetWidth, targetHeight);
cmd.SetViewport(targetViewPort);
```

HDR OutputとオーバーレイUI合成

HDRディスプレイ出力時のエンコードや、cameraData.rendersOverlayUIがtrueのときのオーバーレイUI合成もFinalBlitPassの仕事でした。HDR出力対応が必要なタイトルでは、HDROutputUtilsを使って同等の処理を実装する必要があります。

要件が多いように見えますが、プロジェクトで実際に使う機能だけ実装すれば十分です。SDR固定・Viewport全画面・オーバーレイUI未使用のタイトルであれば、実質必要なのはY反転とsRGB変換だけになります。

3.4 まとめ

- URPでBackBufferへの最終出力を担当するパスはUberPost / FinalPost / FinalBlitPassの3種類あり、条件によってどれか1つだけが選ばれる
- カスタムパスをAfterRenderingPostProcessing以降に注入するとhasPassesAfterPostProcessingがtrueになり、FinalBlitPassがほぼ必ず追加される
- Unity 6.3のSwitchActiveTexturesToBackbuffer()を使うと、カスタムパスが直接BackBufferへ出力でき、このFinalBlitPassを省略できる
- ただしこのAPIはカメラのActiveColor/Depth TextureのIDを書き換える宣言にすぎない。最後のカメラの最終パスでのみ呼ぶことと、Y反転・sRGB変換・ViewportなどFinalBlitPassが担っていた処理を自前で持つことが前提になる

フルスクリーンのBlit1回は、高解像度・モバイル環境では無視できないコストです。カスタムポストプロセスを持つプロジェクトなら、検討する価値は十分にあります。

3.5 参考資料

- Unity Manual: Get data from the current frame in URP
  https://docs.unity3d.com/6000.3/Documentation/Manual/urp/accessing-frame-data.html
- Class UniversalResourceData | Universal Render Pipeline | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.3/api/UnityEngine.Rendering.Universal.UniversalResourceData.html
- Struct RasterGraphContext | Scriptable Render Pipeline Core | 17.3
  https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@17.3/api/UnityEngine.Rendering.RenderGraphModule.RasterGraphContext.html

第4章

モバイル向けシェーダー最適化：超越関数の計測と軽量化

永留 遼平 Nagatome Ryouhei

株式会社Colorful Palette

rngtm　X @rn49rn49

4.1　はじめに

シェーダー内で sin や exp などの超越関数を使用すると、GPUの処理負荷が高くなる傾向があります。本記事では、プロファイラの計測値を交えながら、これらの関数が実際のパフォーマンスにどの程度の影響を及ぼすのかを解説します。また、超越関数を近似計算に置き換える最適化手法を取り上げ、実際にどの程度の負荷削減効果が得られるのかを実測値とともに紹介します。

4.2　環境

この記事で使用したUnityバージョン、レンダーパイプライン、使用した端末は以下の通りです。

Unityバージョン	Unity6000.3.20f1
レンダーパイプライン	Universal Render Pipeline 17.3.0
iOS端末	iPhoneX iPhone14 iPhone15 Pro
Android端末	Galaxy S9

4.3 超越関数

超越関数（ちょうえつかんすう）とは、有限回の四則演算や累乗根では表せない関数のことです。代表的な例として、三角関数、指数関数、対数関数が挙げられます。

- sinやcosなどの三角関数
- pow, pow2などの指数関数
- log, log10などの対数関数

超越関数は負荷が高いため、多用することは避けることが望ましいとされています。

4.3.1 超越関数が重い理由

超越関数が重い理由は以下の3つです。

- クロック数が多い
- 超越関数の演算を行うユニット(SFU)の数が少ない
- レジスタプレッシャーによるオーバーヘッド

4.3.2 理由1：クロック数が多い

スマートフォンなどに搭載されているGPUは、算術演算を行うための演算ユニット(ALU)を多く搭載しています。ALUは、加算や乗算を1クロックサイクルで実行できるように設計されています。

しかし、超越関数は1回の命令で計算できる単純なものではありません。GPU内部で超越関数を計算する際、テイラー展開やミニマックス法などの多項式近似アルゴリズムを用いて、多数の加算と乗算の組み合わせに分解して計算しています。

単純な四則演算が「1命令」で終わるのに対し、超越関数はコンパイラによって「数十命令の四則演算」に展開されるため、処理時間が長くなります。

4.3.3 理由2：SFUの数が少ない

多くのGPUには超越関数などを高速に処理するための SFU (Special Function Unit) という専用ハードウェアが搭載されており、超越関数がSFUにて実行されることもあ

ります。

SFUの数はALUに比べて非常に少ないため、SFUを用いて超越関数を実行した場合でも処理に長い時間がかかります。

4.3.4 理由3：レジスタープレッシャーによるオーバーヘッド

sin() のような超越関数は、単純な足し算や掛け算と比べて計算が複雑であり、GPUの内部で計算を行うために、より多くの一時的なレジスタを必要とします。

GPUの各演算ユニット（SM: Streaming Multiprocessor）が持つレジスタの総量は物理的に決まっており、1スレッドあたりの消費レジスタ数が増える（レジスタプレッシャーが高まる）ため、1つのSMに詰め込めるスレッドの数が減ります。

一度に実行できるスレッド数が減るため、処理に長い時間がかかってしまいます。

4.4 様々な関数の負荷

ループ回数0〜 10それぞれについて、様々な超越関数を実行した時のGPU時間を計測し、どの超越関数が最も重いのかを調べてみることにします。GPU時間の計測にはFrameTimingManagerを使用しています。

4.5 負荷計測に使用したシェーダー

計測には、次のようなシェーダーを使用しました。シェーダーキーワードで実行対象の超越関数を選択することができ、変数 _LoopCount を使用して、ループ回数を変更できるようにしています。

```
Shader "Benchmark/Transcendental"
{
    Properties
    {
        // uGUI(CanvasRenderer)はテクスチャを "_MainTex" にバインドするので名前は固定
        [PerRendererData][MainTexture] _MainTex("Sprite Texture", 2D) = "white" {}
        [MainColor] _BaseColor("Tint", Color) = (1, 1, 1, 1)

        // Mask (UI/Mask) 用のステンシル設定。Maskコンポーネントが Materialを差し替えて書き込む
        _StencilComp("Stencil Comparison", Float) = 8
```

```
        _Stencil("Stencil ID", Float) = 0
        _StencilOp("Stencil Operation", Float) = 0
        _StencilWriteMask("Stencil Write Mask", Float) = 255
        _StencilReadMask("Stencil Read Mask", Float) = 255

        _ColorMask("Color Mask", Float) = 15

        [Toggle(UNITY_UI_ALPHACLIP)] _UseUIAlphaClip("Use Alpha Clip", Float) = 0
    }

    SubShader
    {
        Tags
        {
            "Queue" = "Transparent"
            "RenderType" = "Transparent"
            "RenderPipeline" = "UniversalPipeline"
            "IgnoreProjector" = "True"
            "PreviewType" = "Plane"
            "CanUseSpriteAtlas" = "True"
        }

        Stencil
        {
            Ref [_Stencil]
            Comp [_StencilComp]
            Pass [_StencilOp]
            ReadMask [_StencilReadMask]
            WriteMask [_StencilWriteMask]
        }

        Cull Off
        ZWrite Off
        ZTest [unity_GUIZTestMode]
        Blend SrcAlpha OneMinusSrcAlpha
        ColorMask [_ColorMask]

        Pass
        {
            HLSLPROGRAM

            #pragma vertex vert
            #pragma fragment frag
            // ループ回数がuniform(動的ループ)なので 2.0 ではなく 3.0 が必要
            #pragma target 3.0

            // RectMask2D / Mask のAlpha Clip対応
            #pragma multi_compile_local _ UNITY_UI_CLIP_RECT
```

```
        #pragma multi_compile_local _ UNITY_UI_ALPHACLIP

        // 計測する超越関数の切り替え。実行時に Shader.EnableKeyword で切り替えたいので
        // shader_feature(未使用variantが剥がれる)ではなく multi_compile を使う。
        // キーワードごとに別variantになるため、生成コードには選んだ関数だけが出る(分岐オーバーヘッド無し)。
        // グローバルキーワードなので TranscendentalBenchmark 側から一括で制御できる。
        // キーワードが1つも立っていない場合は先頭の _FUNC_NONE のvariantが使われる。
        #pragma multi_compile_fragment _FUNC_NONE _FUNC_SIN _FUNC_SIN_APPROX _FUNC_COS _FUNC_TAN _FUNC_TAN_TAYLOR _FUNC_TAN_PADE _FUNC_ASIN _FUNC_ACOS _FUNC_ATAN _FUNC_EXP _FUNC_EXP2 _FUNC_POW _FUNC_LOG _FUNC_LOG2 _FUNC_LOG10

        #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"

        struct Attributes
        {
            float4 positionOS : POSITION;
            float4 color : COLOR;
            float2 uv : TEXCOORD0;
            UNITY_VERTEX_INPUT_INSTANCE_ID
        };

        struct Varyings
        {
            float4 positionHCS : SV_POSITION;
            float2 uv : TEXCOORD0;
            float4 positionOS : TEXCOORD1; // _ClipRect と同じ空間(Canvasローカル)で保持
        };

        TEXTURE2D(_MainTex);
        SAMPLER(sampler_MainTex);

        CBUFFER_START(UnityPerMaterial)
            half4 _BaseColor;
        CBUFFER_END

        // TranscendentalBenchmark から設定するグローバル値。
        // マテリアル単位の値ではないので UnityPerMaterial には入れない(入れるとSRP Batcherが壊れる)
        int _TranscendentalLoopCount;
        float _PowExponent;

        // UnityUI.cginc の UnityGet2DClipping 相当(HLSLで自前定義)
        float Get2DClipping(float2 position, float4 clipRect)
        {
            float2 inside = step(clipRect.xy, position.xy) * step(position.xy,
```

```
    clipRect.zw);
                return inside.x * inside.y;
            }

            // 超越関数へ渡す値を (0.05, 0.95) に収める。
            // asin/acos の定義域 |x| <= 1、log系の定義域 x > 0、exp系のオーバーフローを
            // まとめて回避する。全variant共通の前処理なので、コストは差分では相殺される。
            float PrepareInput(float seed)
            {
                return seed;
                // return frac(seed) * 0.9 + 0.05;
            }

            float sinApprox(float x)
            {
                // sinの放物線近似。乗算とabsだけで済むので、本物のsin命令とのコスト差を見る。
                // 定義域は |x| <= PI(PrepareInputの出力はこの範囲に収まる)。
                // 係数は定数畳み込みされるので、実行時の除算コストは無い。
                return (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
            }

            float tanTaylor(float x)
            {
                // tanのテイラー展開の3次まで(乗算のみ)。本物のtan命令とのコスト差を見る。
                // 定義域は |x| < PI/2 で、π/4を超えると急速に精度が落ちる。
                float x2 = x * x;
                return x * (1.0 + x2 * 0.3333333); // 1.0/3.0
            }

            float tanPade(float x)
            {
                // tanのパデ近似(乗算+除算1回)。定義域は |x| < PI/2。
                // 分母の零点 x ≈ ±1.581 が tan の極 ±PI/2 の近くにあるため、極付近まで追従す
    る。
                float x2 = x * x;
                return x * (15.0 - x2) / (15.0 - 6.0 * x2);
            }

            // 計測対象の超越関数。キーワードごとに1つだけが残るので、
            // 生成コードには選んだ関数の展開結果だけが現れる。
            float Transcendental(float x)
            {
                #if defined(_FUNC_SIN)
                    return sin(x);
                #elif defined(_FUNC_SIN_APPROX)
                    return sinApprox(x);
                #elif defined(_FUNC_COS)
                    return cos(x);
```

```
                #elif defined(_FUNC_TAN)
                    return tan(x);
                #elif defined(_FUNC_TAN_TAYLOR)
                    return tanTaylor(x);
                #elif defined(_FUNC_TAN_PADE)
                    return tanPade(x);
                #elif defined(_FUNC_ASIN)
                    return asin(x);
                #elif defined(_FUNC_ACOS)
                    return acos(x);
                #elif defined(_FUNC_ATAN)
                    return atan(x);
                #elif defined(_FUNC_EXP)
                    return exp(x);
                #elif defined(_FUNC_EXP2)
                    return exp2(x);
                #elif defined(_FUNC_POW)
                    // 指数を定数にすると pow(x, 2) -> x * x のように畳まれてしまうので、
                    // uniform を渡して汎用のpow経路を通す
                    return pow(x, _PowExponent);
                #elif defined(_FUNC_LOG)
                    return log(x);
                #elif defined(_FUNC_LOG2)
                    return log2(x);
                #elif defined(_FUNC_LOG10)
                    return log10(x);
                #else // _FUNC_NONE : 超越関数なしのベースライン
                    return x;
                #endif
            }

            Varyings vert(Attributes IN)
            {
                Varyings OUT;
                OUT.positionOS = IN.positionOS;
                OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
                OUT.uv = IN.uv * 300.0;
                return OUT;
            }

            half4 frag(Varyings IN) : SV_Target
            {
                // ここが計測対象(見た目に意味は無い)。
                // UNITY_LOOP でループ展開を抑止し、_TranscendentalLoopCount 回ぶんのコストが
    そのまま乗るようにする。
                // 超越関数の中身以外はどのvariantでも完全に同じ形なので、
                // _FUNC_NONE との差分がその関数のコストになる。
                float accum = 0.0;
```

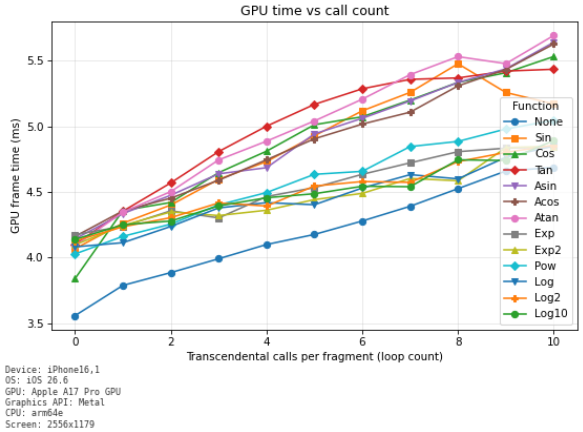
```
                UNITY_LOOP
                for (int i = 0; i < _TranscendentalLoopCount; i++)
                {
                    // ループ不変式にすると外へ巻き上げられてしまうので、iを引数に混ぜる
                    float x = IN.uv.x + i * 0.017;

                    accum += Transcendental(x);
                }
                return half4(saturate(accum.xxx) * 0.02, 1);
            }
            ENDHLSL
        }
    }
}
```

4.6 計測結果

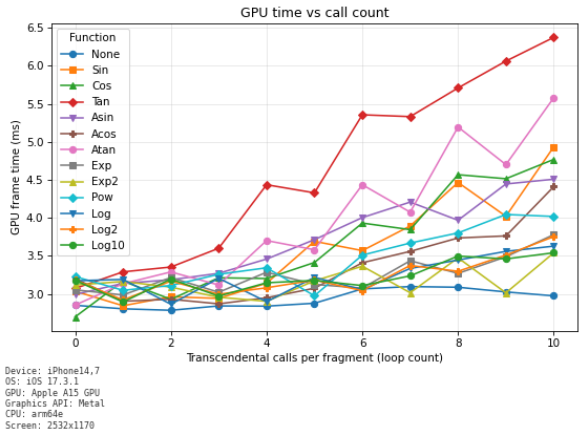
4.6.1 iPhone15 Pro

実行回数が7未満の場合はtanが最も重く、実行回数が7以上の場合はAtanが最も重くなりました。

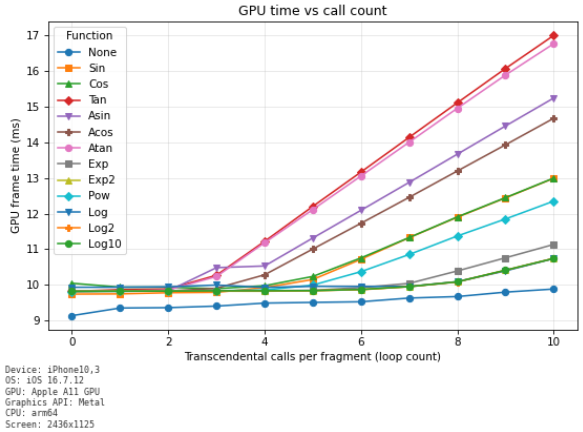


4.6.2 iPhone14

tanが最も負荷が高く、atanは2番目に負荷が高くなりました。

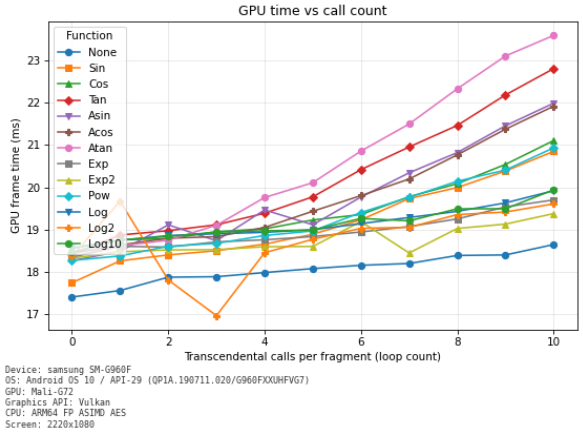


4.6.3 iPhoneX



4.6.4 Galaxy S9

Atanが最も負荷が高いようです。



4.6.5 まとめ

端末	特記事項
iPhone 15 Pro	実行回数が7未満の場合はtanが最も重い 実行回数が7以上の場合はatanが最も重い
iPhone 14	tanが最も重い
iPhone X	tanが最も重い
Galaxy S9	atanが最も重い

4.7　sin関数の負荷

超越関数の中でも代表的なsin関数についての負荷を計測します。

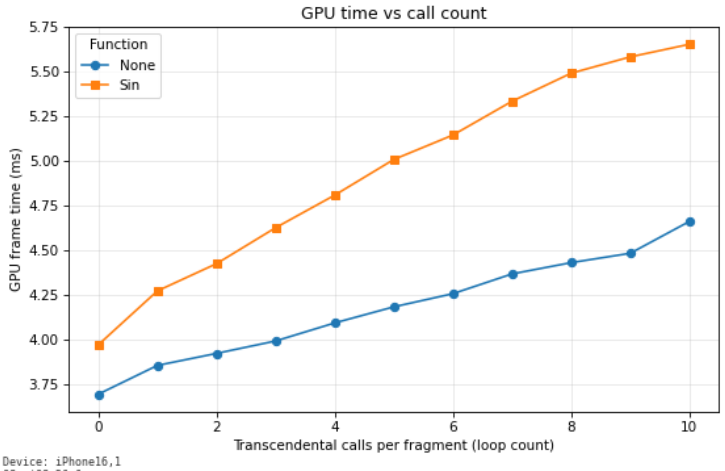
ループ回数0～ 10それぞれについて、フレーム内の処理に要したGPU時間を計測します。各計測点について、10秒間の計測を行い、GPU時間をグラフにまとめてみました。

1つ目のグラフ：ループ回数に対応したGPU時間2つ目のグラフ：ループ回数に対応したGPU時間とNoneの差分。

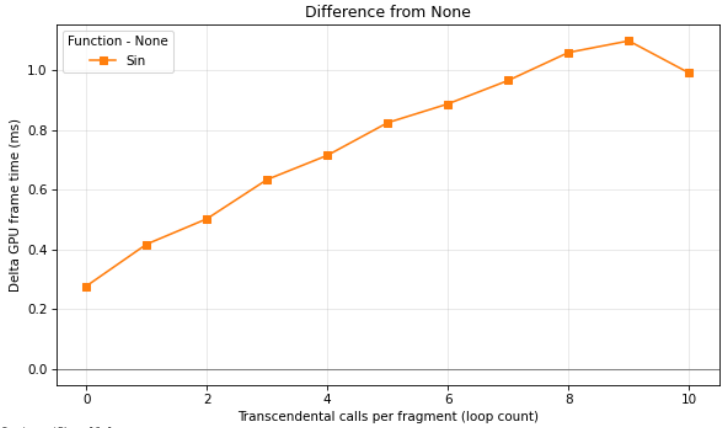
計測にはFrameTimingManagerを使用しました。

4.7.1 iPhone15 Pro

sin命令がシェーダー内に存在することで、0.25msほど増加していました。sin呼び出しが1回増えるごとに、0.1msほど処理負荷が増加しています。



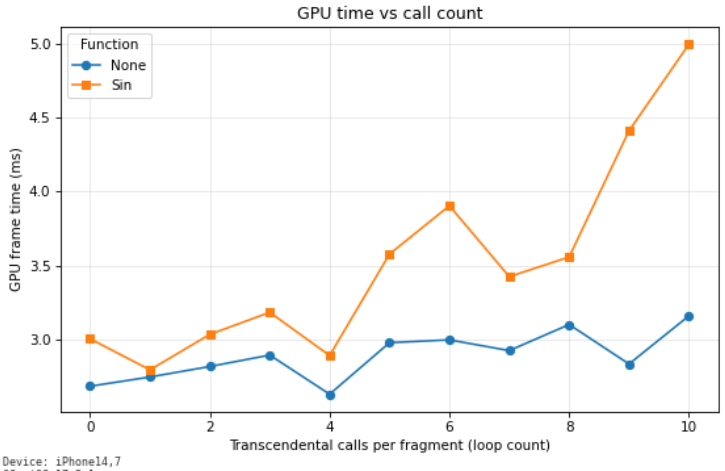

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```



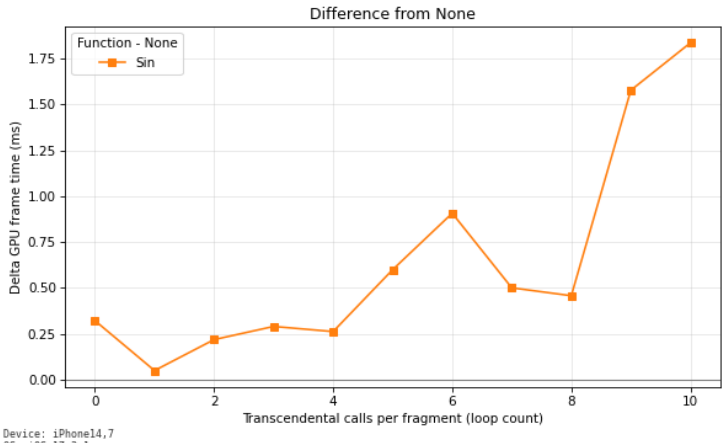

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

4.7.2 iPhone14

sin命令がシェーダー内に存在することで、GPU時間が0.25msほど増加していました。sin呼び出しが1回増えるごとに、0.1〜 0.2msほど処理負荷が増加しています。



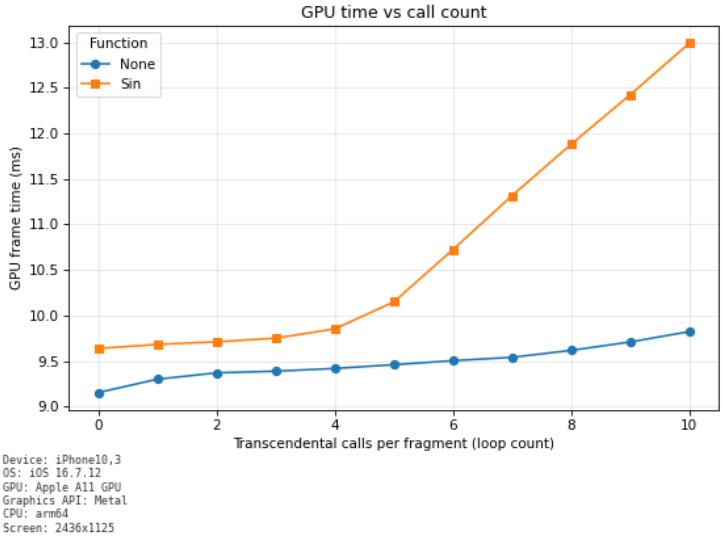

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

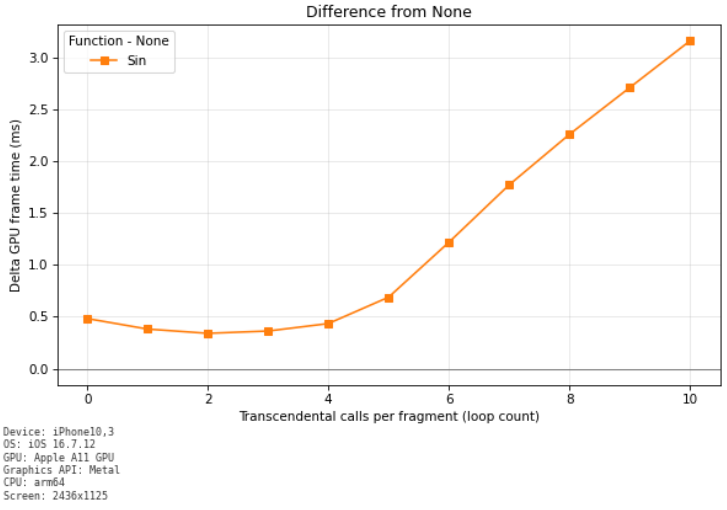



```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

4.7.3 iPhoneX

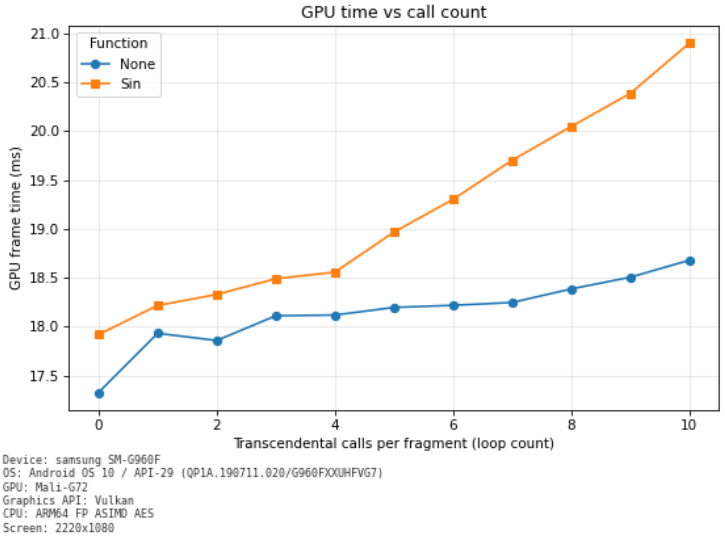
sin命令がシェーダー内に存在することで、0.5msほど増加していました。sin呼び出しが4回まででであれば、GPU時間の増加はほぼみられませんが、5回を超えた場合sin呼び出しが1回増えるごとに、0.6msほど処理負荷が増加しています。

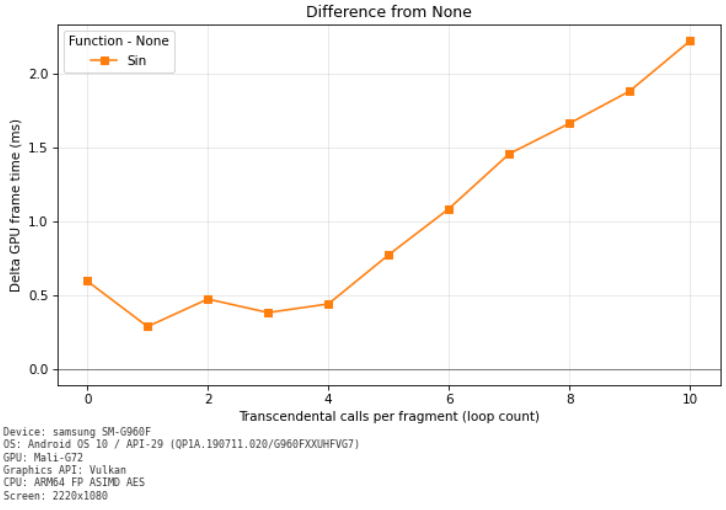




4.7.4 Galaxy S9

sin命令がシェーダー内に存在することで、0.5msほど増加していました。sin呼び出しが4回までであれば、GPU時間の増加はほぼみられませんが、5回を超えてから、sin呼び出しが1回増えるごとに、0.2〜0.3msほど処理負荷が増加しています。





4.7.5　sin関数の負荷のまとめ

端末	sin関数によるGPU負荷の増加	特記事項
iPhone 15 Pro	+0.25ms (ベースライン)	呼び出し1回につき約0.1ms増加
iPhone 14	+0.25ms (ベースライン)	呼び出し1回につき約0.1〜0.2ms増加
iPhone X	+0.5ms (ベースライン)	5回を超えると呼び出し1回につき約0.6ms増加
Galaxy S9	+0.5ms (ベースライン)	5回を超えると呼び出し1回につき約0.2〜0.3ms増加

※表の「ベースライン」はシェーダー内にsin関数が存在する場合の初期負荷増加を指します。

シェーダー内にsin関数が増加する場合、sin関数を実行しなくてもGPU時間の増加が確認できました。これはレジスタープレッシャーによる負荷の増加であると考えられます。

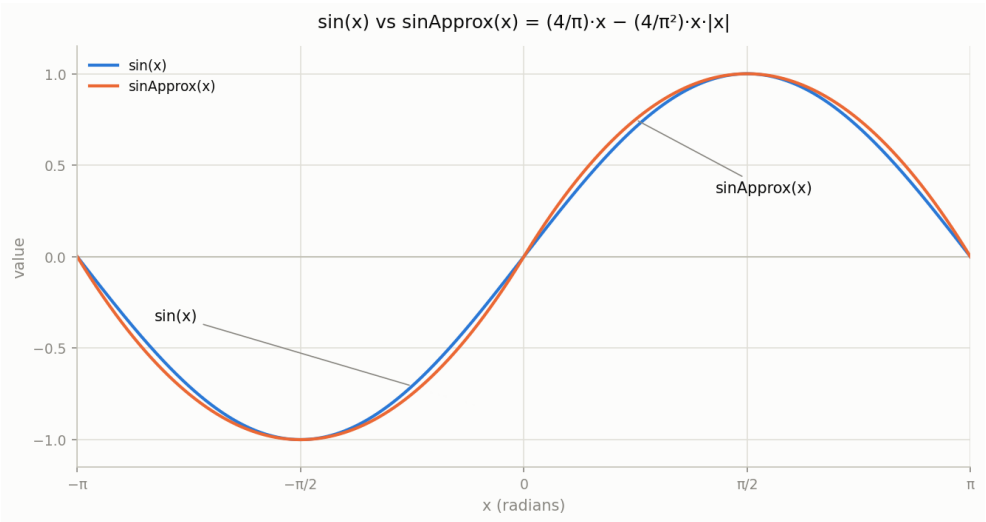
4.8　sin関数の放物線近似

次のような多項式を用いてsinを近似する手法が知られています。

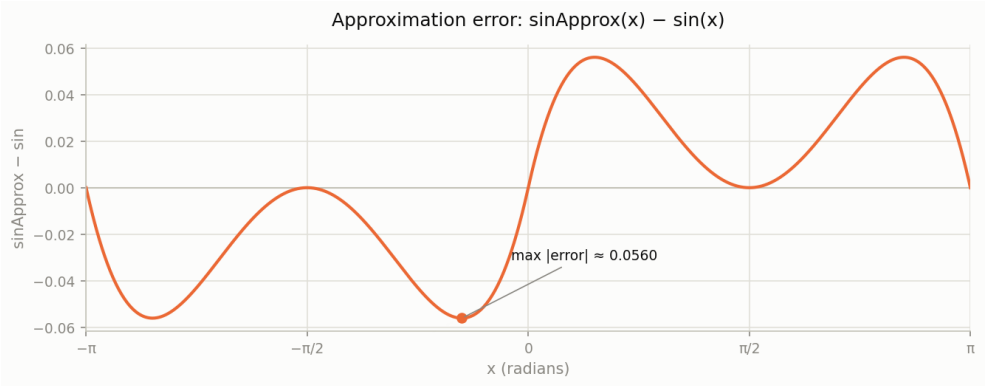
```
float sinApprox = (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
```

乗算と絶対値 abs() だけで計算できるため、非常に軽量です。

sinの放物線近似は、sinと非常に近い値を出力します (-PI<=x<=PI)



誤差の最大値は 0.056 程度です。

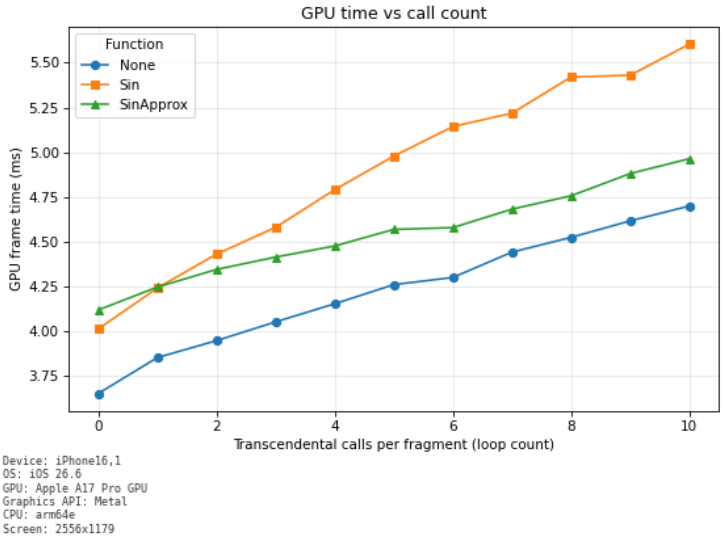


4.9　sin近似の負荷の計測

sinの放物線近似sinApproxの負荷を計測し、グラフにまとめてみました。

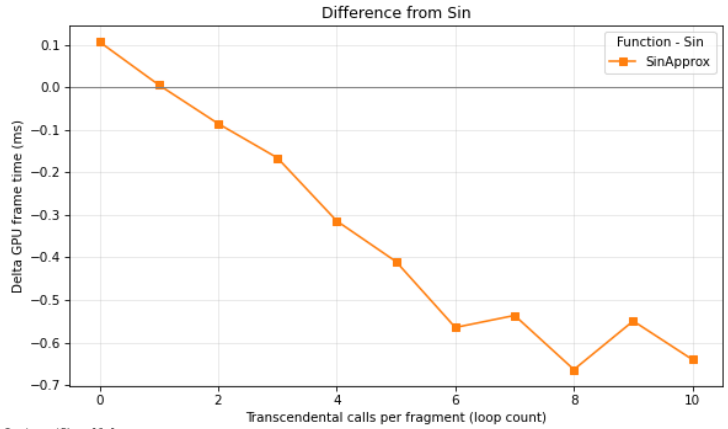
4.9.1　iPhone15 Pro

sin関数よりも負荷が低いことが確認できます。

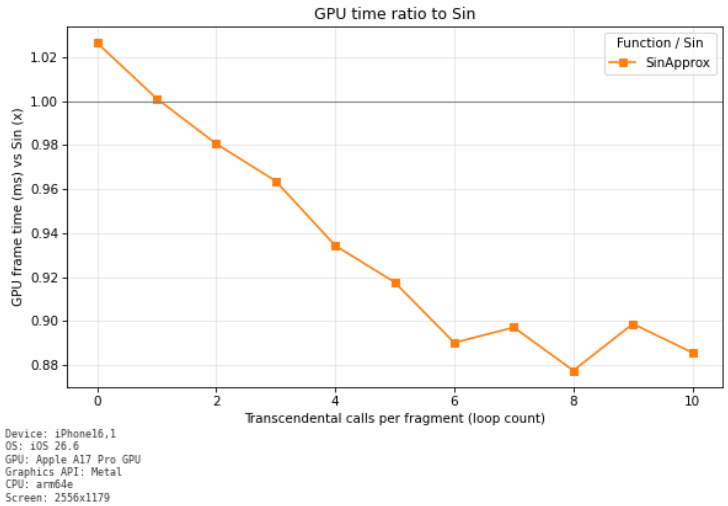


sin関数と近似計算の比較

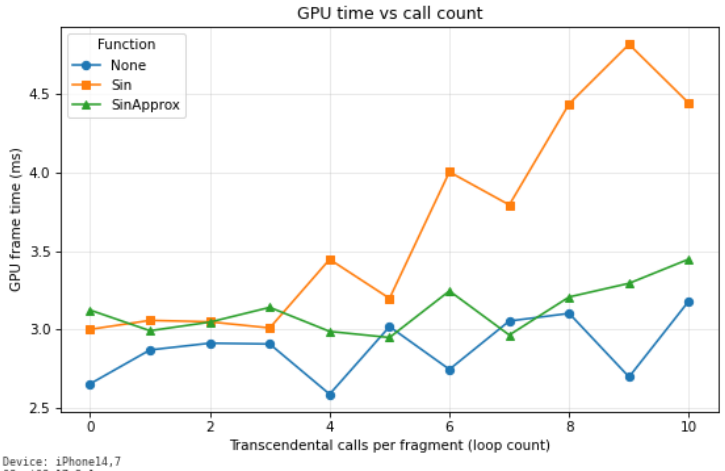
関数を10回実行した時、0.6msほど負荷が低くなっています。



関数を10回実行した時のsinApproxの負荷はsinの90%ほどとなりました。



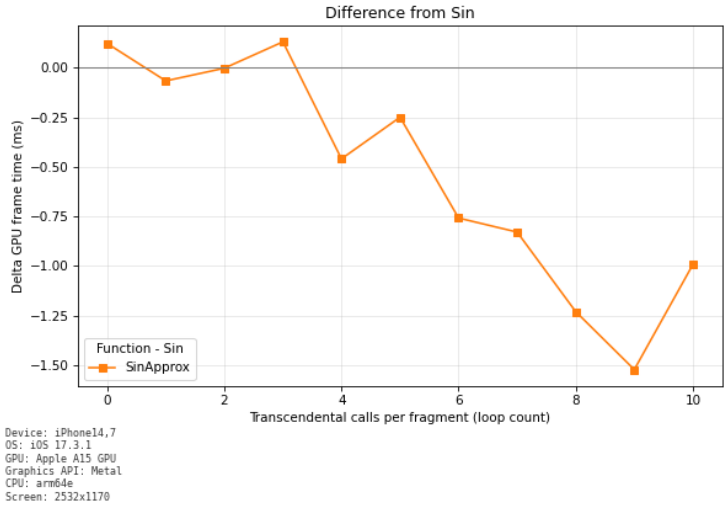
4.9.2 iPhone14



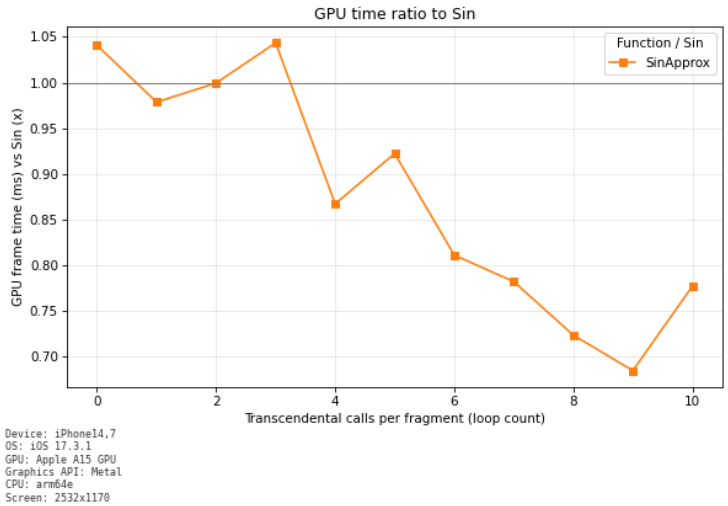

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

sin関数と近似計算の比較

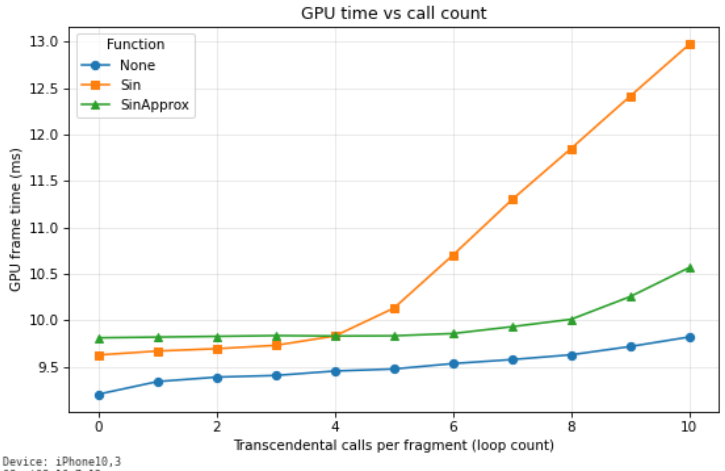
関数を10回実行した場合は、1msほどGPU時間が短くなっています。



sinApproxを10回実行した時の負荷は、sinの80%程度でした。



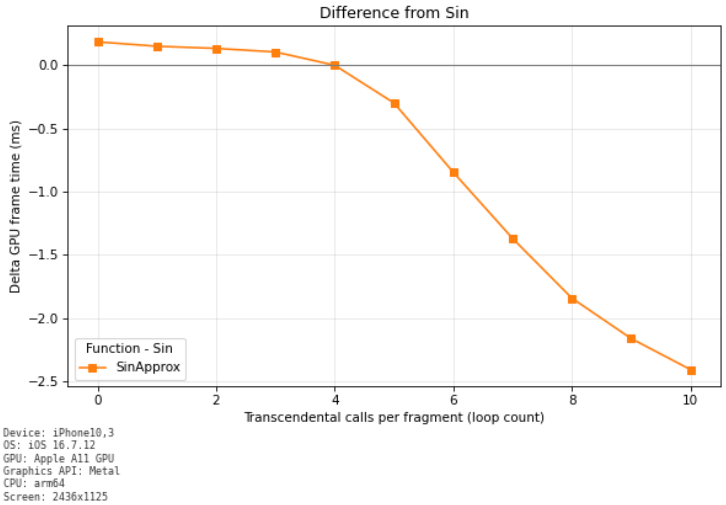
4.9.3 iPhoneX



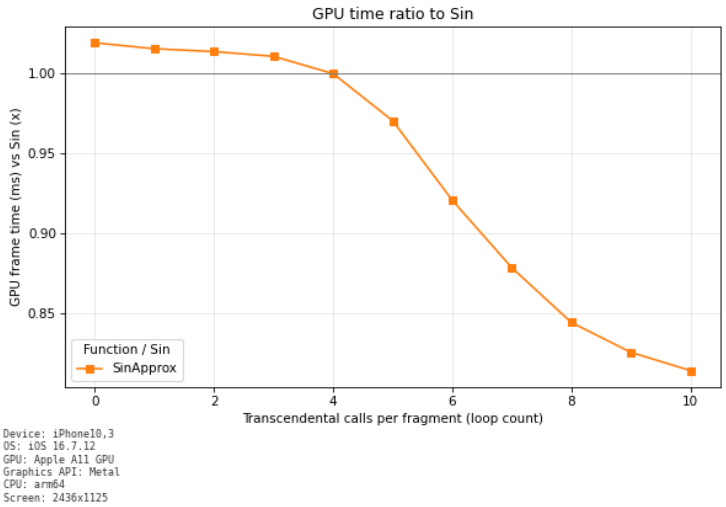

```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

sin関数と近似計算の比較

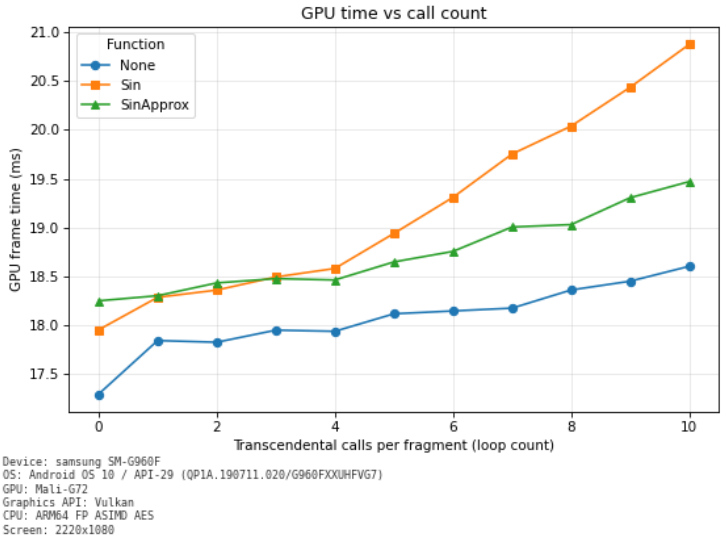
sinApproxを10回実行した時の負荷は、sinを実行した時と比較して2.5msほど短くなりました。



sinApproxを10回実行した時のGPU時間はsinの80%程度でした。

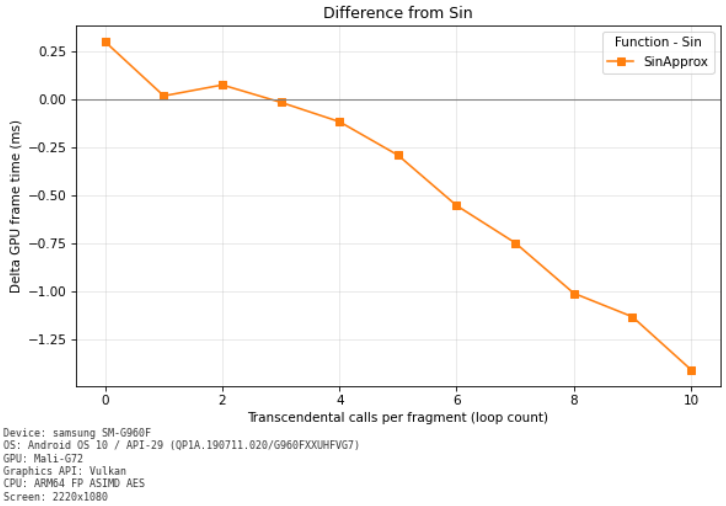


4.9.4 Galaxy S9

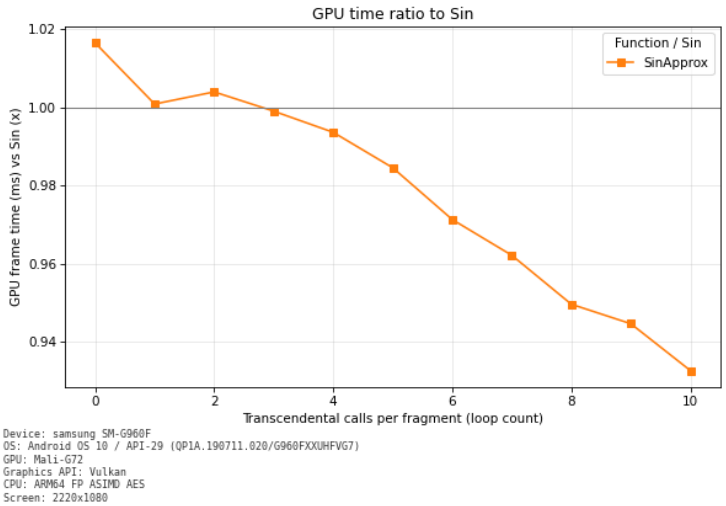


sin関数と近似計算の比較

関数を10回実行した場合は、1.3msほどGPU時間が短くなっています。



sinApproxを10回実行した時の負荷は、sinの94%程度でした。



4.9.5　sin近似の負荷のまとめ

sinApproxによるGPU削減効果を表にまとめると、このような結果になりました。

端末	sin関数の負荷	sinApproxによる削減効果	特記事項
iPhone 15 Pro	基準	約0.6ms 削減	sinApproxの負荷はsinの約90%
iPhone 14	基準	約1.0ms 削減	sinApproxの負荷はsinの約80%
iPhone X	基準	約2.5ms 削減	sinApproxの負荷はsinの約80%
Galaxy S9	基準	約1.3ms 削減	sinApproxの負荷はsinの約94%

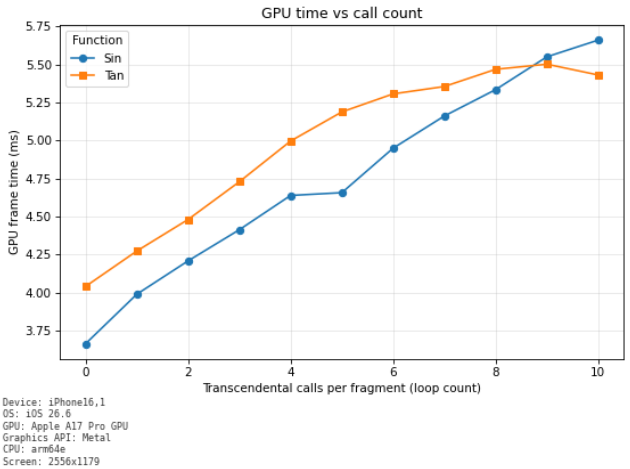
※削減効果は、ループ回数10回におけるGPU時間の差分です。

4.10　tan関数の負荷

tan関数の負荷も計測してみました。tan関数の内部では除算が呼ばれているため、sin関数やcos関数よりも負荷が高くなります。

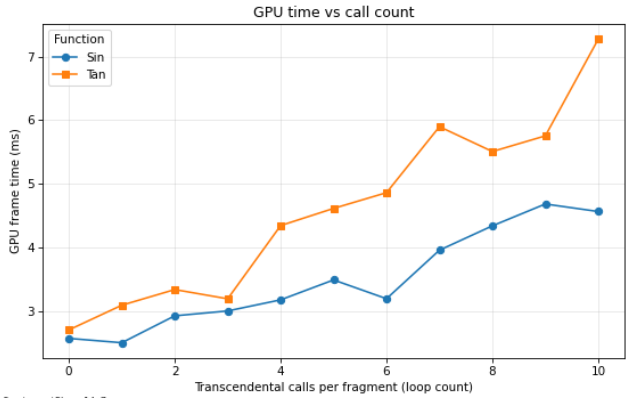
4.10.1　iPhone15Pro

sin関数よりも負荷が高いですが、実行回数が9回を超えると、sin関数の方が負荷が高くなりました。



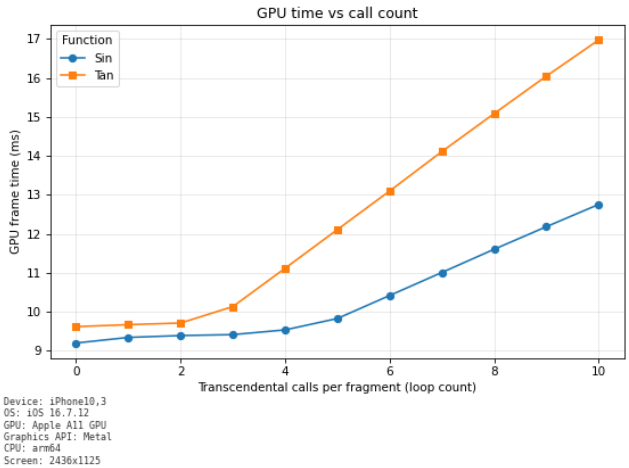
4.10.2 iPhone14

sin関数よりもtan関数の方が負荷が高くなりました。



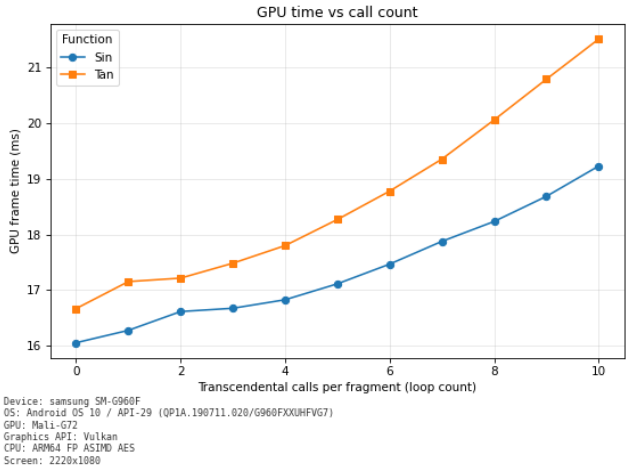
4.10.3 iPhoneX

sin関数よりもtan関数の方が負荷が高くなりました。



4.10.4 Galaxy S9

sin関数よりもtan関数の方が負荷が高くなりました。



4.11　tan関数の近似

tan関数の近似手法として、テイラー展開をそのまま活用する方法や、パデ近似を利用する方法があります。テイラー展開の方が負荷が低いですが、精度も低いです。パデ近似の方が精度が良いですが、負荷も高いです。

4.11.1　手法1. テイラー展開による近似

tanのテイラー展開を利用した近似計算です。

|x|がPI/2に近づくほど、誤差が大きくなります。

```
float tanTaylor(float x) {
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333);
}
```

4.11.2　手法2. パデ近似

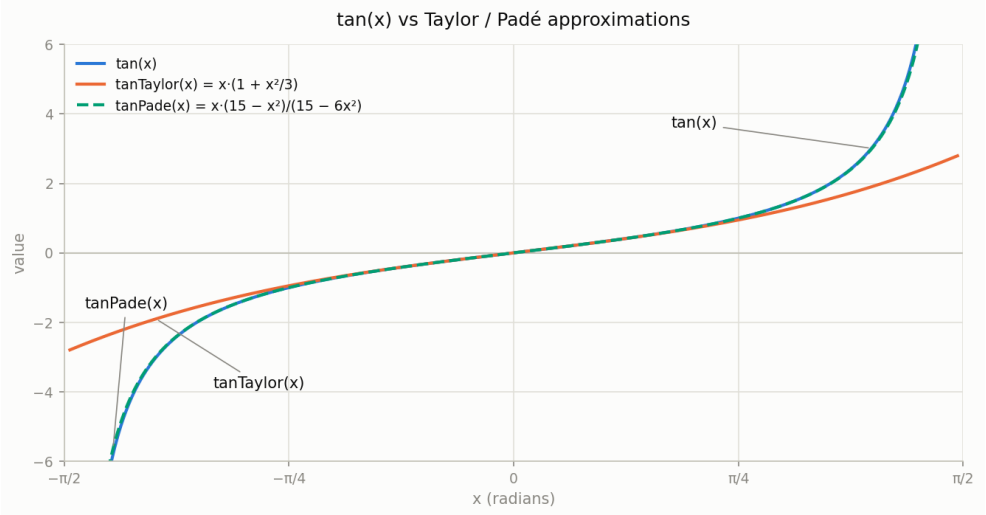
tanのpade近似を利用した手法です。

|x| < PI/4　の範囲で、tan(x)に非常に近い値を出力します。

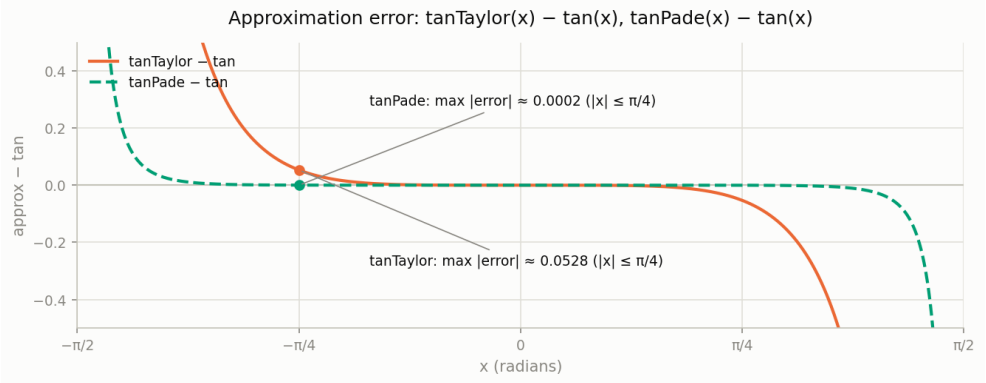
```
float tanPade(float x) {
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}
```

4.12　tan関数近似のグラフ

紹介したtanTaylorとtanPadeをグラフにプロットしてみました。



tan関数との誤差

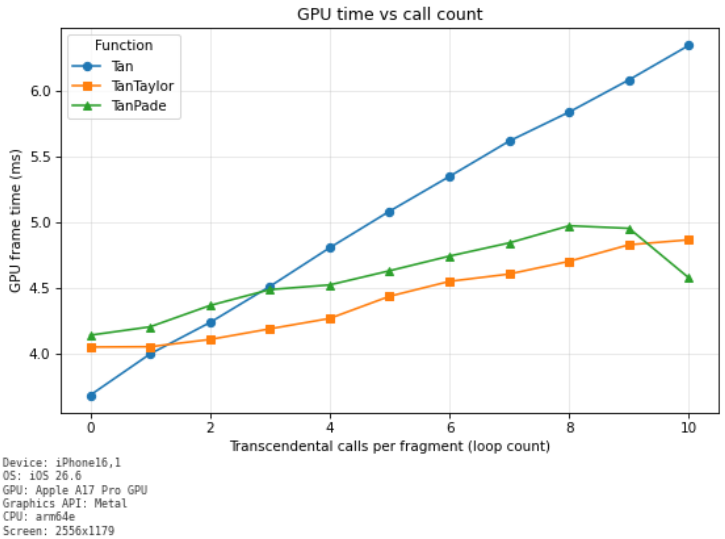


4.13　tan関数近似の負荷計測

tan関数の近似計算の負荷を計測してみました。

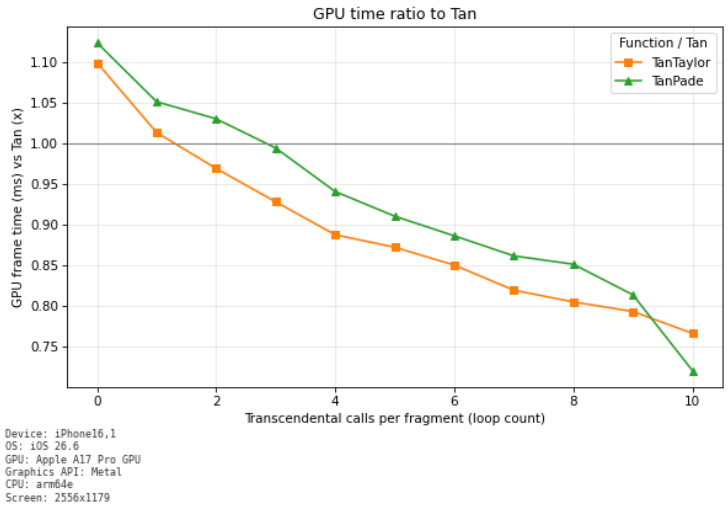
4.13.1　iPhone15 Pro

実行回数が少ない場合、レジスタープレッシャーによるオーバーヘッドが目立ちますが、実行回数が増えてくると、近似計算の方が負荷が低く抑えられています。

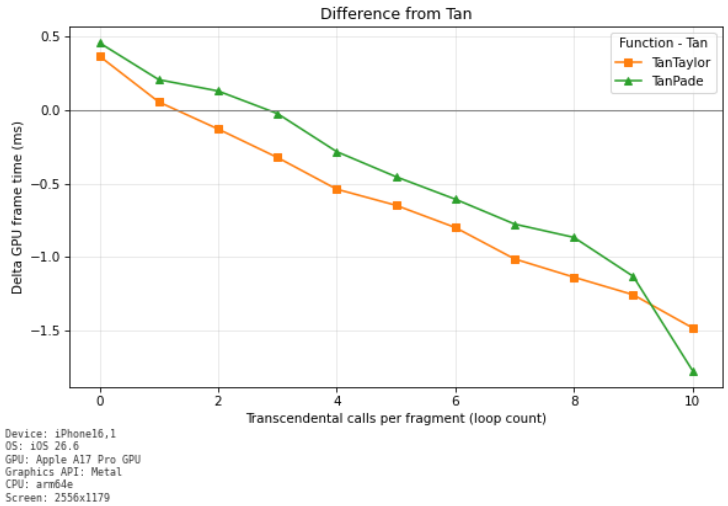


tan関数と近似計算の比較

関数を10回実行した場合、近似計算のGPU時間はtan関数の80%ほどのGPU時間となっていました。

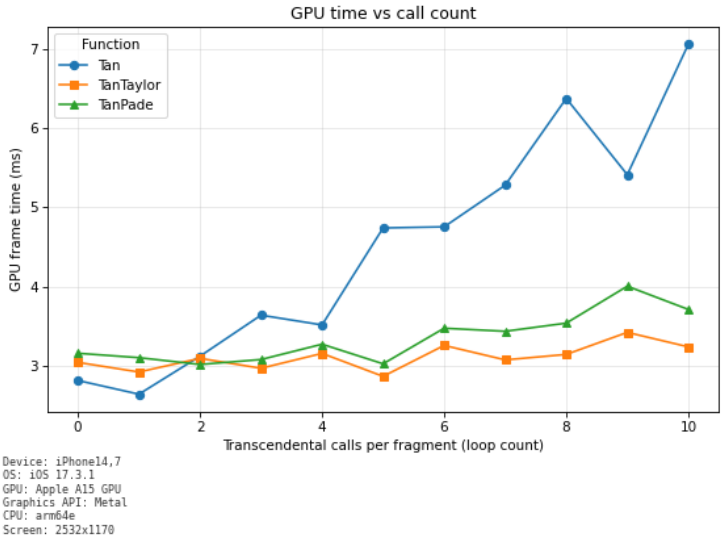


10回実行した時、近似計算の方が1msほど処理時間が少なくなりました。



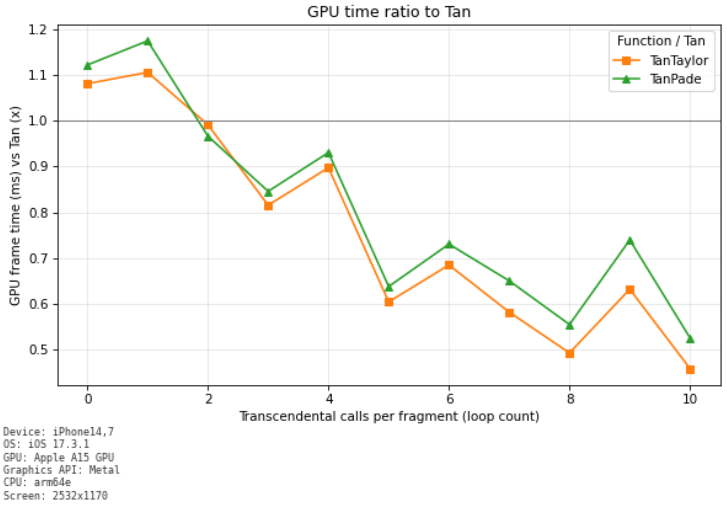
4.13.2 iPhone14

実行回数が少ない場合、レジスタープレッシャーによるオーバーヘッドがわずかにみられます。実行回数が増えてくると、近似計算の方が負荷が低く抑えられています。

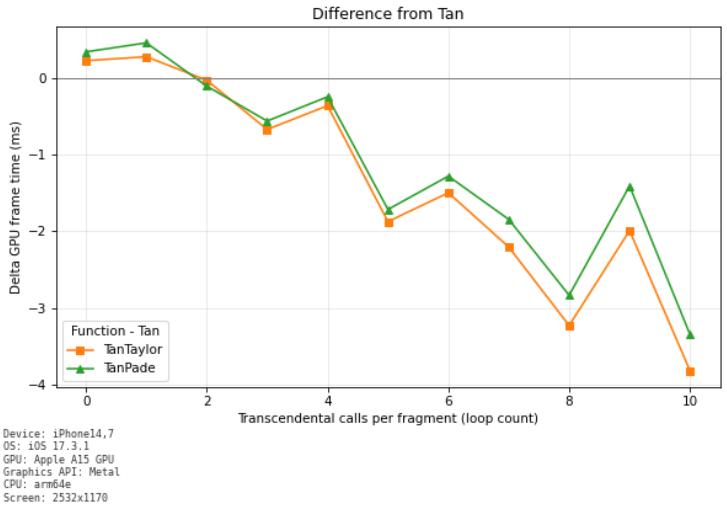


tan関数と近似計算の比較 (iPhone14)

関数を10回実行した場合、近似計算のGPU時間はtan関数の50%ほどのGPU時間となっていました。

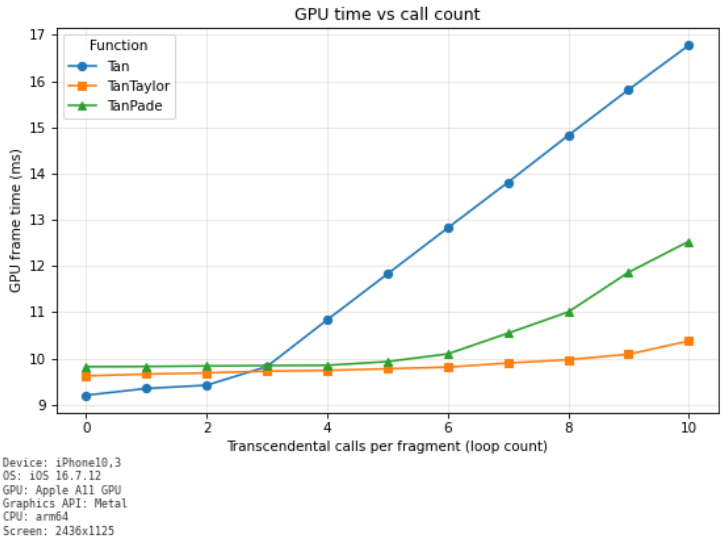


10回実行した時、近似計算の方が3msほど処理時間が少なくなりました



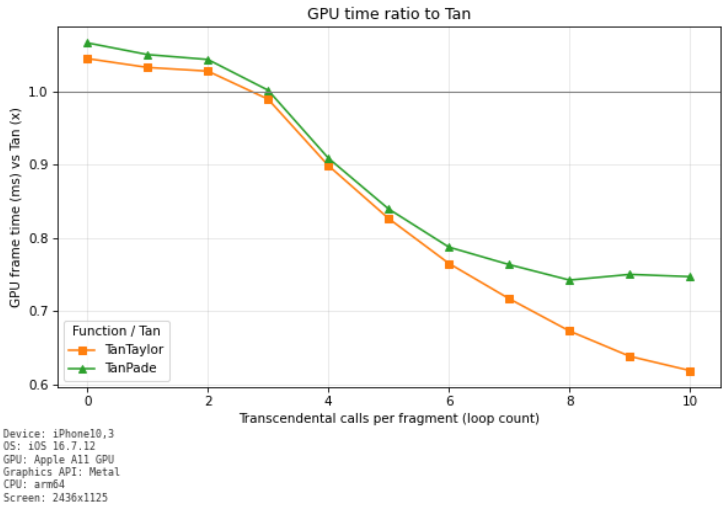
4.13.3 iPhoneX

実行回数が少ない場合、レジスタープレッシャーによるオーバーヘッドがわずかにみられます。実行回数が増えてくると、近似計算の方が負荷が低く抑えられています。テイラー展開の場合、処理負荷の増加はほとんどありませんが、パデ近似は処理負荷がわずかに上昇しています。

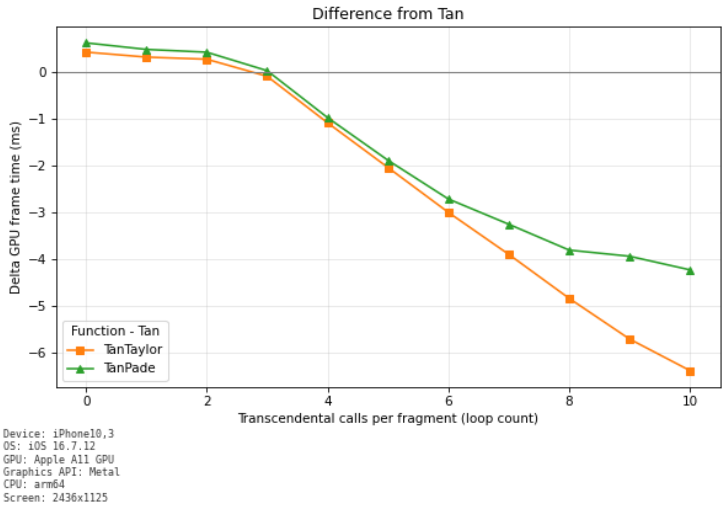


tan関数と近似計算の比較

関数を10回実行した場合、テイラー展開のGPU時間はtan関数の60%ほどのGPU時間となっていました。パデ近似は75%ほどのGPU時間となっていました。

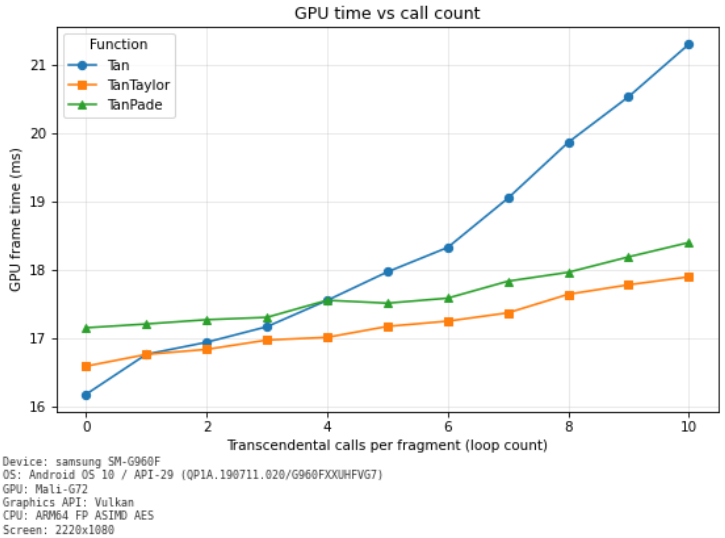


関数を10回実行した時、テイラー展開は6msほど短く、パデ近似は4msほど短い結果となりました。



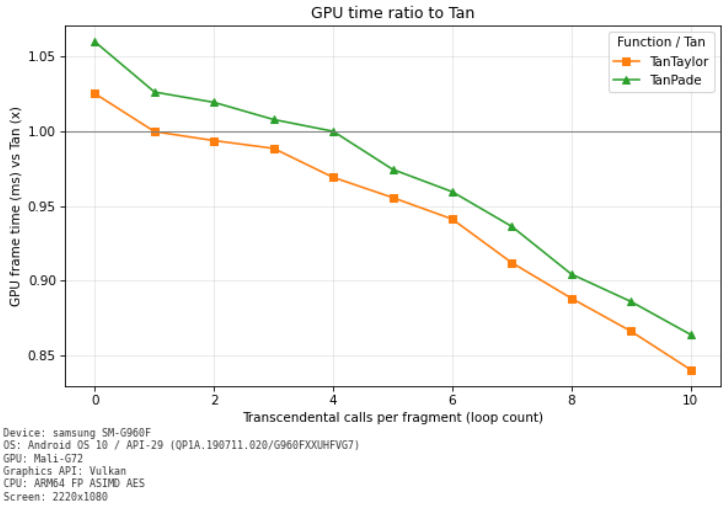
4.13.4 Galaxy S9

実行回数が少ない場合、レジスタープレッシャーによるオーバーヘッドがわずかにみられます。実行回数が増えてくると、近似計算の方が負荷が低く抑えられています。

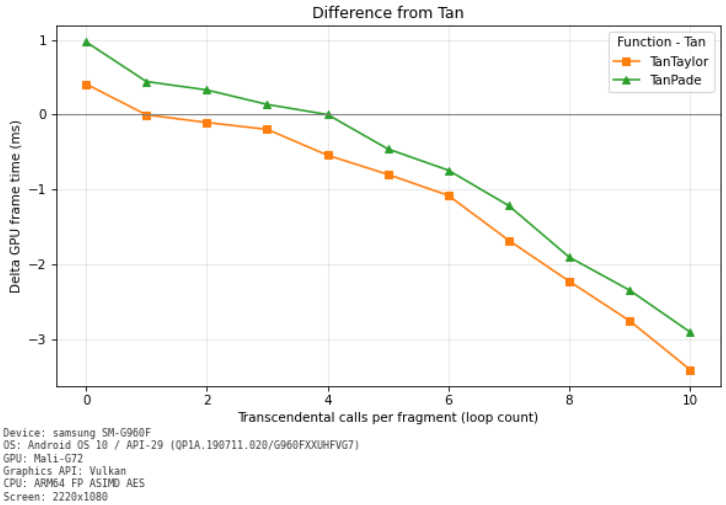


tan関数と近似計算の比較

関数を10回実行した時、近似計算はtan関数の85%ほどのGPU時間となっていました。



関数を10回実行した時、テイラー展開は3msほど短い結果となりました。



4.13.5　tan関数近似の負荷のまとめ

端末	GPU負荷の傾向	特記事項
iPhone 15 Pro	近似計算の方が**約20%低い**	近似計算により約1msの処理時間削減
iPhone 14	近似計算の方が**約50%低い**	近似計算により約3msの処理時間削減
iPhone X	近似計算の方が**約25〜40%低い**	テイラー展開は約6ms、パデ近似は約4ms削減
Galaxy S9	近似計算の方が**約15%低い**	近似計算により約3msの処理時間削減

※「GPU負荷の傾向」は、標準の tan 関数と比較した際の、近似計算（テイラー展開またはパデ近似）実行時のGPU時間のおおよその削減率です。

第5章

Renderer Shader User Value を活用したグラフィックス最適化

瀬戸 徳 Noboru Seto

株式会社サイバーエージェント SGEコア技術本部

ruccho

5.1 はじめに

Unity 6.3 より Renderer Shader User Value (RSUV) という機能が追加されました。RSUV は Renderer コンポーネントごとに設定できる32ビットの整数値で、シェーダーからも参照することができます。Unity 6000.3.0f1 以降では MeshRenderer, Skinned MeshRenderer で RSUV を利用でき、Unity 6000.5.0f1, 6000.4.4f1, 6000.3.13f1 以降は SpriteRenderer[*1], SpriteShapeRenderer, TilemapRenderer にも対応しています。使い方はシンプルで、 Renderer.SetShaderUserValue() でレンダラーに対して RSUV を設定し、シェーダーでは unity_RendererUserValue を使って参照するだけです。

```
uint value = /* ... */;
meshRenderer.SetShaderUserValue(value);
value = meshRenderer.GetShaderUserValue();
```

```
uint value = unity_RendererUserValue;
```

この例と同様のことはマテリアルパラメータを使っても実現できます。しかし、RSUV を使用すると、**特に大量描画系のシチュエーションでマテリアルよりもメモリ確保量や**

[*1] SpriteRenderer の RSUV 取得・設定は 2D Module が必要。

フレームレートを改善できる可能性があります。この章では RSUV が登場した背景、RSUV の効果的な利用方法を説明し、パフォーマンス計測結果も紹介します。

5.1.1 Unityの大量描画機能とRSUVが登場した背景

まずは RSUV が必要となった背景を振り返ってみます。Unity において大量のオブジェクトを同時に表示する際のパフォーマンスは常に頭痛の種でした。大量描画におけるパフォーマンスの問題は主に「マテリアル切り替えにおける CPU 負荷」「DrawCall の増大」「マテリアル複製によるメモリ確保」の3点が挙げられます。

マテリアル切り替えにおける CPU 負荷

異なるマテリアルを持った複数のオブジェクトを描画する際、フレーム描画のたびにオブジェクトごとにマテリアルのデータを収集して GPU に送信する必要があり、この際の CPU 処理コストが大きい問題がありました。SRP Batcher はこの問題を解決し、マテリアルのデータをGPUメモリ上に永続化させることで、シェーダーが同一であれば比較的少ないコストで異なるマテリアルを持った複数のオブジェクトを描画できるようになりました。

DrawCall の増大

GPU インスタンシングは、同じシェーダー・同じメッシュ・同じマテリアルを持った複数のオブジェクトを一つの DrawCall で描画する手法です。これは SRP Batcher より適用条件が厳しい代わりに、大量のオブジェクトを描画する際には SRP Batcher 以上の効果を発揮します。Unity 6 で GPU Resident Drawer が登場したことで、通常の MeshRenderer を使った描画でも条件を満たす場合には自動的に GPU インスタンシングが行われるようになりました。

マテリアル複製によるメモリ確保

最後の問題は、SRP Batcher や GPU インスタンシングを利用しながら、マテリアルにも変化を持たせたいシチュエーションです。Unity には昔からマテリアルインスタンスを複製せずに一部のパラメータのみをオーバーライドする MaterialPropertyBlock という機能が存在しましたが、これは SRP Batcher や GPU Resident Drawer と互換性がありませんでした。RSUV はこの問題を解決し、SRP Batcher や GPU Resident Drawer を利用しながら、オブジェクトごとに異なる32ビットのパラメータを**マテリアルを使わずに**シェーダーに渡すことができます。

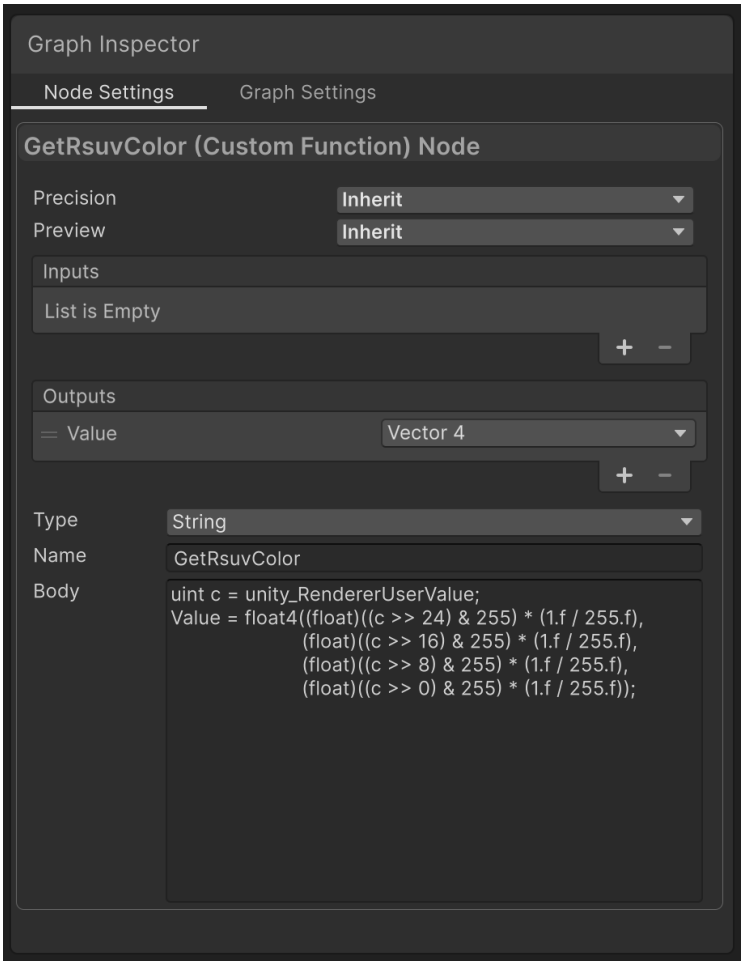
RSUV は上記の3つの問題を同時に解決するために必要だった最後のピースでした。SRP Batcher でマテリアル切り替え時のコストを抑えながら、GPU Resident Drawer で効率的な GPU インスタンシングを行いつつ、インスタンスごとにバリエーションのある描画を行えます。GPU Resident Drawer による自動インスタンシングが効かない状況でも、マテリアルインスタンスの複製を避けてメモリ消費を抑えられるという恩恵が受けられます。

5.2 シンプルな使い方

実際に RSUV の使い方を見てみましょう。RSUV は C# / HLSL において32ビットの整数（uint）として表現されるので、例えば32ビットのカラー（RGBAの4チャンネルが各 8ビット）を渡すには次のようにビット演算によるパッキングを行います。

```
Color32 color = /* ... */;
var colorEncoded = (uint)(color.r << 24 | color.g << 16 | color.b << 8 | color.a);
meshRenderer.SetShaderUserValue(colorEncoded);
```

続いてシェーダー側の実装ですが、ここでは ShaderGraph の使用を想定します。ShaderGraph には RSUV を取得する専用のノードが用意されていないため、Custom Function ノードを使用して次のように設定し（図5.1）、コードを入力します。



▲図5.1　ノードの設定

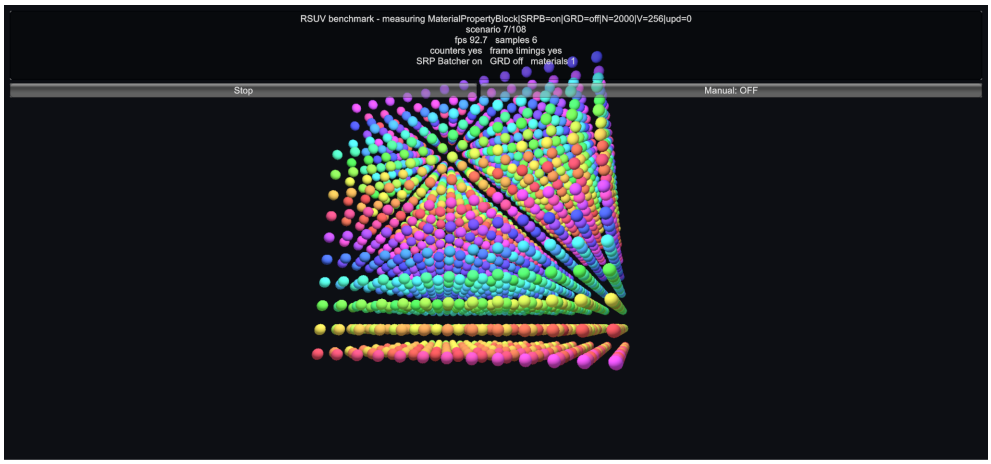
```
uint rsuv = unity_RendererUserValue;
Value = float4((float)((rsuv >> 24) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 16) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 8) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 0) & 255) * (1.f / 255.f));
```

ノードを接続して RSUV が受け取れていることを確認します。

5.3　パフォーマンス計測

RSUV が大量描画系のシチュエーションで効果を発揮することを確認するため、以下のような条件でメモリ確保量とフレームレートを計測しました。

- Unity 6000.3.15f1
- URP 17.3
- iPhone 12
- シーン：10000個の Sphere
- SRP Batcher 有効
- フレームごとのパラメータ変化あり・なしで比較
- GPU Resident Drawer 有効・無効で比較
- Sphereごとに異なるシェーダーパラメータを3つの方法で設定
  - Renderer ごとのマテリアル：10000個の Material を作成しアサイン
  - RSUV：1個の Material で Renderer ごとに RSUV を設定
  - MaterialPropertyBlock：1個の Material と10000個の MaterialPropertyBlock（SRP Batcher / GPU Resident Drawer の対象外となる）



▲図5.2　ベンチマーク画面

まずは各パラメータ設定手法におけるメモリ確保量の比較です（表5.1）。

表5.1: メモリの増分

条件	メモリの増分
Rendererごとのマテリアル	15.79 MB
MaterialPropertyBlock	3.32 MB
RSUV	1.66 KB

マテリアルが複製されないことにより、当然ながらメモリの確保量も大きく減少しています。次にフレームレートを見てみます。まずはフレームごとにパラメータが変化しない場合です（表5.2）。

表5.2: フレームごとにパラメータが変化しない場合のフレームレート (FPS)

条件	GPU Resident Drawer なし	GPU Resident Drawer あり
Renderer ごとのマテリアル	33.8	38.8
RSUV	49.9	59.8
MaterialPropertyBlock	28.9	28.9

このように、RSUV の方が Renderer ごとのマテリアルよりも性能が高く、GPU Resident Drawer を併用することでその効果はより高まることがわかります。RSUV 単体ではマテリアルの切り替えコストが削減され、さらに GPU Resident Drawer を使った GPU インスタンシングが可能になることでレンダースレッド上での描画コマンドの発行負荷が軽減されます。次に、フレームごとにパラメータが変化する場合を見てみます（表5.3）。

表5.3: フレームごとにパラメータが変化する場合のフレームレート (FPS)

条件	GPU Resident Drawer なし	GPU Resident Drawer あり
Renderer ごとのマテリアル	11.1	8.1
RSUV	48.0	60.0
MaterialPropertyBlock	26.1	25.2

パラメータ更新がない場合と比べると、Renderer ごとのマテリアルでは性能が大きく低下しているのに対し、RSUV ではほとんど劣化していません。1つのマテリアルパラメータを変更するとマテリアル全体の GPU への再アップロードが必要になるのに対し、RSUV はフレームごと・Renderer ごとに常に GPU に送信されるバッファの一

部として送られる[*2]ため、RSUV の更新によって追加で発生するコストは低く抑えられます。興味深いことに、マテリアルパラメータの大量更新と GPU Resident Drawer は相性が悪く、GPU Resident Drawer が無効の場合と比較してむしろ性能が低下するようです。以上のように「パラメータの更新」と「描画コマンド発行」の二面において RSUV が有利であることが見て取れます。特に RSUV によって GPU インスタンシングが可能になる状況（シェーダーやメッシュが同一のオブジェクトが大量に存在する場合など）で RSUV の効果は大きく高まります。

5.4 32ビットよりも大きいデータを渡す

このようにパフォーマンス向上に役立つ RSUV ですが、32ビットに収まる値しか渡せないのが大きな弱点です。これを克服するテクニックを紹介します。RSUVで32ビットに収まらないデータを渡すには、別の GraphicsBuffer に目的のデータを格納しつつ、RSUV にはその GraphicsBuffer に対するインデックスを渡す、という方法が有効です。この方法では GraphicsBuffer のアップロードに追加のコストがかかるので、GraphicsBuffer の更新頻度やサイズには一定の配慮が必要かもしれません。

```
[StructLayout(LayoutKind.Sequential)]
struct VariantGpuData
{
    public Vector4 baseColor;
    public Vector4 surface;
}
private static readonly int VariantBufferId = Shader.PropertyToID("_RsuvVariantBuffer")
    ;

// GraphicsBuffer のアップロードと割り当て
var buffer =  new GraphicsBuffer(GraphicsBuffer.Target.Structured, count, UnsafeUtility
    .SizeOf<VariantGpuData>());
var cpuBuffer = new VariantGpuData[count];
FillCpuBufferValues(cpuBuffer);
buffer.SetData(cpuBuffer, 0, 0, count);
Shader.SetGlobalBuffer(VariantBufferId, buffer);

// RSUV のセット
renderer.SetShaderUserValue(index);
```

[*2] インスタンシングされない場合に限る。インスタンシングされる場合はほかのインスタンスパラメータとともに GraphicsBuffer を通して渡され、全体のパラメータが変更され GraphicsBuffer がダーティになったタイミングでのみアップロードされる。

```
// ...

// 不要になったら解放
buffer.Dispose();
```

```
struct RsuvVariantRecord
{
    float4 baseColor;
    float4 surface;
};

StructuredBuffer<RsuvVariantRecord> _RsuvVariantBuffer;

// アクセス
RsuvVariantRecord record = _RsuvVariantBuffer[unity_RendererUserValue];
```

5.4.1 まとめ

RSUV が登場した背景、パフォーマンス計測、基本的・発展的な使い方を紹介しました。RSUV はマテリアルと比べると32ビットまでという制限がつくものの、状況によってはパフォーマンスに大きな改善をもたらします。RSUV が効果を発揮する状況を見極めて有効に活用しましょう。

第6章

GraphicsStateCollectionによるシェーダーウォームアップ

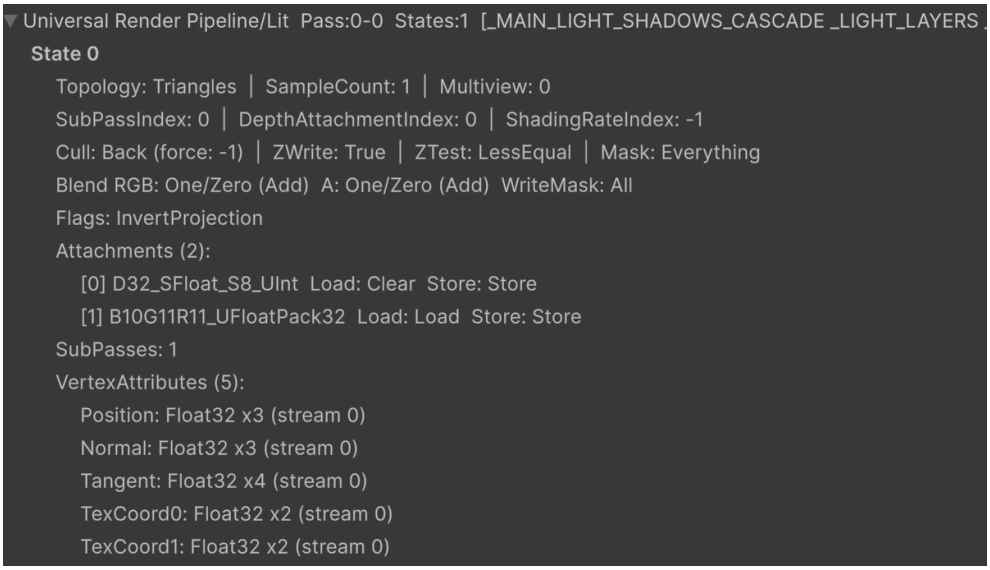
天野 択郎 Amano Takuro

株式会社サイバーエージェント SGEコア技術本部

シェーダーが初めて描画に使用されるとき、ランタイムでコンパイル処理が実行され、シェーダーコンパイルスタッターと呼ばれる処理落ちを引き起こすことがあります。本章では、モバイル環境（Metal・Vulkan）を対象に、GraphicsStateCollectionによるシェーダーウォームアップについて基本的な使い方から導入後にスタッターが残る実例とその対処までを紹介します。なお、本章の内容はUnity 6.3 LTS（6000.3）を対象としています。

6.1　GraphicsStateCollectionの概要

GraphicsStateCollection（GSC）は、Unity 6で使用できるようになったAPIです（出典ID: gsc_docs）。ゲームプレイ中に使用されるシェーダーバリアントとグラフィックス状態（描画時のレンダリング設定）の組み合わせを記録し、まとめて事前コンパイル、つまりウォームアップする仕組みです。Metal・Vulkan・D3D12などのモダングラフィックスAPI向けの、ShaderVariantCollectionの代替として位置づけられています。なお、Unity 6000.3時点では`UnityEngine.Experimental.Rendering`名前空間に属するExperimental APIであり、将来のバージョンで仕様が変更される可能性があります。



▲図6.1　GSCに記録される情報の例

図は、GSCに記録された情報をスクリプトで取得し、EditorWindowに表示した例です。記録内容の取得方法は、本章の最後（リスト6.11）で紹介します。

シェーダーはそのままではGPUで実行できず、GPUのマシン命令へのコンパイルが必要です。ただし、マシン命令はGPUの機種やドライバによって異なるため、すべての端末向けに事前コンパイルした状態で配布することはできません。そのため、シェーダーは以下の流れを経てGPUで実行されます。

1. 開発者は、HLSLなどのシェーダー言語でシェーダー（アセット）を記述する
2. ビルド時に、Unityがシェーダーをグラフィックス APIごとの中間的な表現へコンパイルし、アプリやAssetBundleに格納する。VulkanではSPIR-V、MetalではMetal Shading Language（MSL）がこの中間的な表現にあたる（出典ID: gsc_spirv_msl）
3. アプリの実行中、シェーダーが初めて描画に使用されるときに、格納された中間的な表現からシェーダープログラムを生成する
4. 続けて、シェーダープログラムとレンダリング設定を組み合わせて、GPUドライバがGPUのマシン命令へコンパイルする

手順3・4の実行時コンパイルを初回描画より前にまとめて済ませておくことが、シェーダーウォームアップの基本的な考え方です。

なお、1つのシェーダーからは、パス（描画工程）ごと、さらにライトの影響やフォグの有無といった条件（シェーダーキーワード）の組み合わせごとに、複数の実行プログラムが生成されます。この生成される単位をシェーダーバリアントと呼びます。

6.1.1 Pipeline State Object

モダングラフィックスAPIにおいて描画時にコンパイルされる単位を、Pipeline State Object（PSO）と呼びます。PSOは、シェーダーバリアントとレンダリング設定（頂点レイアウト、カラーフォーマットやMSAAなどのレンダーパスアタッチメント、ブレンドステート、デプステスト、ステンシルステート）を一体として、GPUドライバがGPU向けのマシン命令にコンパイルしたオブジェクトです。生成されたPSOは端末にキャッシュされ、同じレンダリング設定であれば再利用されます。つまり、同じシェーダーバリアントであっても、レンダリング設定が異なれば別のPSOとしてコンパイルされます。

6.1.2 シェーダーコンパイルスタッターの発生原因

スタッターの原因は、ランタイムで実行される2段階のコンパイル処理です。GPUが初めてシェーダーバリアントを描画するとき、Main Threadでのシェーダープログラムの生成（手順3）と、Render ThreadでのGPUドライバによるPSO生成（手順4、マシン命令へのコンパイル）が発生します。

Unity Profilerでは、前者はMain Threadの`Shader.CreateGPUProgram`、後者はRender Threadの`GpuProgramMetal.GetCachedPipeline`（iOS）または`CreateGraphicsPipelineImpl`（Android）として観測できます。特にMetal（iOS）・Vulkan（Android）ではPSO生成コストが大きく、初回描画時のスタッターの主因になります。GSCを使ったウォームアップは、このPSO生成をゲーム起動直後などの任意のタイミングで済ませておく手段です。

従来のShaderVariantCollectionは「どのシェーダーバリアントをコンパイルするか」は管理できますが、レンダリング設定を持ちません。そのためShaderVariantCollectionのウォームアップは手順3のシェーダープログラム生成までにとどまり、実際の描画時にはGPUドライバが改めてPSOを生成するため、スタッターが残ります。



▲図6.2　Main ThreadとRender Threadでのコンパイル発生例

図は、Vulkan（Android）でコンパイルが発生したフレームを、ProfilerのTimelineビューで表示した例です。上はMain Threadの`Shader.CreateGPUProgram`を選択した様子で、対象のシェーダー名・パス名・キーワードが確認できます。下はRender Thread の`CreateGraphicsPipelineImpl`を選択した様子で、この例では1回のPSO生成に数十ミリ秒かかっており、スタッターの主因になることがわかります。

6.2　GSCを使ったウォームアップのフロー

GSCを使ったウォームアップは、「レンダリング設定の収集（トレース）」と「ウォームアップ（事前コンパイル）」の2つのフェーズに分かれます。

6.2.1　レンダリング設定の収集

トレースの基本形を リスト6.1 に示します。トレース中に描画されたオブジェクトのレンダリング設定が、GraphicsStateCollection.GraphicsStateとしてGSCに記録されます。リストでは1つのメソッドにまとめていますが、実際にはBeginTrace()とEndTrace()は別々のタイミングで呼び、複数フレームにわたって描画を収集します。

▼リスト6.1　レンダリング設定のトレース

```
private GraphicsStateCollection _gsc;

void Trace() {
    // 収集開始
    _gsc = new GraphicsStateCollection();
    _gsc.BeginTrace();

    // ここで複数フレームにわたり、ウォームアップ対象のオブジェクトを描画する
    // 描画されたレンダリング設定がGraphicsStateとして記録される

    // 収集終了・保存
    _gsc.EndTrace();
    _gsc.SaveToFile(path);
}
```

生成されるGraphicsStateCollection.GraphicsStateはグラフィックスAPIごとに異なります。トレースは、ビルドしたアプリを実際に動かすデバイス上で行いましょう。

6.2.2　ウォームアップ（事前コンパイル）

収集したGSCをロードし、ウォームアップを実行します（リスト6.2）。この例では、プロジェクトに取り込んだGSCをAddressablesのアセットとしてロードしています。

▼リスト6.2　WarmUpProgressivelyによる分散ウォームアップ

```
async Task WarmUpAsync() {
    // シェーダーを含むアセットのロード後に実行する
    _gsc = await Addressables.LoadAssetAsync<GraphicsStateCollection>(address).Task;

    // 段階的にWarmUp(Jobsベース。毎フレームn件ずつコンパイル)
    StartCoroutine(WarmUpCoroutine());
}

IEnumerator WarmUpCoroutine() {
```

```
    JobHandle handle = default;
    int requestedCount = 0;
    // エラー時にisWarmedUpが真にならないケースがあるため、依頼件数で終了判定する
    while (requestedCount < _gsc.totalGraphicsStateCount) {
        handle = _gsc.WarmUpProgressively(4, handle);
        handle.Complete();
        requestedCount += 4;
        yield return null;
    }
}
```

ウォームアップを実装する際のポイントは以下のとおりです。

- `WarmUp(dependency)`でもウォームアップを実行できますが、一括で全てのコンパイルを行うためスパイクが発生します。`WarmUpProgressively(count, dependency)`で1回あたりのコンパイル件数を指定し、複数フレームに分散させましょう
- シェーダーアセットのロード後に実行します。シェーダーのインスタンスが存在しない場合、そのバリアントのウォームアップはスキップされます
- 完了判定は`isWarmedUp`や`completedWarmupCount == totalGraphicsStateCount`で確認できます
- ただし、ウォームアップエラーが発生すると`isWarmedUp`が真にならない場合があります（Unity 6000.3で確認）。このため、リストでは依頼件数の合計と`totalGraphicsStateCount`の比較で終了判定し、無限ループを回避しています

6.3 収集シナリオの実装例

アプリ起動時に`BeginTrace()`を呼び、手動でプレイしてレンダリング設定を収集する方法もあります。しかしアプリの規模によっては、バージョンアップのたびに手動で網羅するのは困難です。そこで、収集専用シーンでアセットを順次描画するシナリオを機械的に実行して収集する方法を紹介します。実装例にはAddressablesとUniTask（出典ID: gsc_unitask）を使用します。これらを使用していない場合も、アセットの非同期ロードと1フレームの描画待ちを自身のプロジェクトの仕組みに置き換えることで、同じ流れを実装できます。全体のフローは以下のとおりです。

1. 必要なアセットをダウンロードする（Built-inアセットのみで完結する場合は不要）
2. 収集に必要なアセットのAddressをAddressablesのコンテンツカタログ（`IResourceLocator`）から取得する

3. Quality Settingsを切り替える（Quality Settingsでシェーダーが変化しない場合は切り替え不要）
4. 収集を開始する（`BeginTrace()`）
5. シーンを加算ロードし、収集対象アセットを順次ロードして描画する（モデル描画・`ParticleSystem`再生・Timeline再生）
6. 収集を完了する（`EndTrace()`）
7. 不要なシェーダーをGSCから削除する
8. GSCを保存する（`SaveToFile(path)`）
9. シーンを破棄し、Quality Settingsを切り替えながら3〜8を繰り返す

コンテンツカタログからレンダリング設定に影響するアセット（シーン）、キャラクターやプロップなどのモデルアセット、エフェクトアセット、Timelineアセットを取得し、レンダリング設定を切り替えながら順次描画することで網羅的に収集します。Quality Settingsごとに収集するのは、Quality LevelによってMSAAやHDRなどPSOに影響するレンダリング設定が変わるためです。

処理の骨格を リスト6.3 に示します。Quality Settingsごとにトレースを繰り返し、Quality Level名を付けて別ファイルに保存しています。実行時は、現在のQuality Levelに対応するファイルを選択してロードし、ウォームアップします。

▼リスト6.3　収集シナリオの骨格

```
private GraphicsStateCollection _gsc = new GraphicsStateCollection();

private async UniTaskVoid StartAsync() {
    // コンテンツカタログの取得と必要アセットのダウンロード
    // Prefixによる各種Address一覧の取得を事前に行う
    await AddressableSetupAsync();

    var qualitySettingNames = QualitySettings.names;
    for (var i = 0; i < qualitySettingNames.Length; i++) {
        QualitySettings.SetQualityLevel(i);
        await TraceAsync();
    }
}

private async UniTask TraceAsync() {
    // GSC収集開始
    _gsc.ClearVariants();
    _gsc.BeginTrace();
```

```
    // 収集シナリオを実行
    await ExecuteScenarioAsync();

    // GSC収集終了
    _gsc.EndTrace();

    // 収集対象外のシェーダーをGSCから削除する
    RemoveUnnecessaryVariants();

    var qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
    _gsc.SaveToFile(_gscSavePathFormat + $"_{qualityLevelName}");
}
```

シナリオ本体では、シーンごとにモデル・エフェクト・Timelineを順次描画します。リスト6.4 はキャラクターモデルを数体ずつバッチで生成・描画する部分の抜粋です。1フレーム描画を待つことで、そのフレームのレンダリング設定がGSCに記録されます。

▼リスト6.4　モデルアセットの順次描画（抜粋）

```
// キャラクターモデルを収集
for (var i = 0; i < _charaAddressArray.Length; i += _characterNum) {
    // キャラクターモデルを数体ずつ生成
    var batchCount = Mathf.Min(_characterNum, _charaAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var charaHandle = Addressables.LoadAssetAsync<GameObject>(_charaAddressArray[i
    + j]);
        var charaPrefab = await charaHandle.ToUniTask();
        // カメラに収まる位置に均等配置
        var pos = new Vector3((j - batchCount * 0.5f) * 2f, 0f, 5f);
        _charaObjectArray[j] = Object.Instantiate(charaPrefab, pos, Quaternion.identity
    );
        Addressables.Release(charaHandle);
    }

    // 1フレーム描画待ち
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_charaObjectArray[j]);
        _charaObjectArray[j] = null;
    }
}
```

ParticleSystemも同様に数個ずつバッチで生成しますが、Simulate()で再生途中の状態（この例では0.5秒後）を同期的に再現してから描画する点が異なります（リスト6.5）。実時間の再生を待つ必要がないため、収集時間を短縮できます。

▼リスト6.5　ParticleSystemの収集（抜粋）

```
// ParticleSystemを収集
for (var i = 0; i < _particlesAddressArray.Length; i += _effectNum) {
    var batchCount = Mathf.Min(_effectNum, _particlesAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var particleHandle = Addressables.LoadAssetAsync<GameObject>(
    _particlesAddressArray[i + j]);
        var particlePrefab = await particleHandle.ToUniTask();
        var pos = new Vector3((j - batchCount * 0.5f) * 3f, 0f, 5f);
        _particleObjectArray[j] = Object.Instantiate(particlePrefab, pos, Quaternion.
    identity);
        Addressables.Release(particleHandle);

        // _effectSimulateTime秒後の状態を同期処理で再現(Playは不要)
        var ps = _particleObjectArray[j].GetComponentInChildren<ParticleSystem>();
        if (ps != null) {
            ps.Simulate(_effectSimulateTime, withChildren: true, restart: true);
        }
    }

    // 1フレーム描画待ち
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_particleObjectArray[j]);
        _particleObjectArray[j] = null;
    }
}
```

Timelineを使った演出は、PlayableDirectorで実際に再生して収集します（リスト6.6）。再生時間の分だけ待つため、対象のTimelineアセットは収集に必要なものへ絞るのがよいでしょう。

▼リスト6.6　Timelineの収集（抜粋）

```
// Timelineを収集
foreach (var timelineAddress in _timelineAddressArray) {
    var timelineHandle = Addressables.LoadAssetAsync<GameObject>(timelineAddress);
    var timelinePrefab = await timelineHandle.ToUniTask();
    var timelineObj = Object.Instantiate(timelinePrefab, Vector3.zero, Quaternion.
    identity);
    Addressables.Release(timelineHandle);
    var director = timelineObj.GetComponent<PlayableDirector>();
    if (director != null) {
        director.Play();
        await UniTask.Delay((int)(director.duration * 1000));
```

```
    }
    DestroyImmediate(timelineObj);
}
```

収集が終わったら、ウォームアップ対象にしたいシェーダーだけを残します（リスト6.7）。残すシェーダーを許可リスト（_allowShaderNames）で指定し、対象外のシェーダーと参照切れ（null）のバリアントをGSCから削除しています。

▼リスト6.7　収集対象外のバリアント削除

```
private void RemoveUnnecessaryVariants() {
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    _gsc.GetVariants(variants);

    foreach (var variant in variants) {
        if (variant.shader == null) {
            _gsc.RemoveVariant(null, variant.passId, variant.keywords);
            continue;
        }

        if (_allowShaderNames.Contains(variant.shader.name)) {
            continue;
        }

        // 不要なシェーダーをGSCから削除
        _gsc.RemoveVariant(variant.shader, variant.passId, variant.keywords);
    }
}
```

シナリオやアセットの管理方法はプロジェクトによって異なるため、シーン構成や収集対象は自身のプロジェクトに合わせて調整してください。

6.4　スタッターが残る実例と対処

GSCによるウォームアップを導入したにもかかわらずスタッターが残ることがあります。ここではその実例と対処を紹介します。

6.4.1　Shader Strippingによるバリアントの欠落

ウォームアップ時のログにvariant not foundエラーが発生し、かつランタイムでも該当バリアントのコンパイル（ProfilerのShader.CreateGPUProgramなど）が観測されないケースです。

原因は、GSCに含まれるシェーダーバリアントがアプリまたはAssetBundleに含まれていないことです。**Project Settings > Graphics > Shader Stripping**の設定によっては、ビルド時に使用されないバリアントが除外されます。シェーダーやアセットの変更によって必要なバリアントが変わった際に発生しやすい問題です。

Shader Strippingの設定を確認し、バリアントに影響のある変更を行った際はGSCを再度収集しましょう。

なお、GSCにバリアントは記録されているもののGraphicsStateが0件の場合にも、同じvariant not foundエラーが発生します。この場合はランタイムでそのバリアントのコンパイルが発生するため、ランタイムでのコンパイル発生の有無で切り分けられます。

6.4.2 メタデータ不一致によるmismatch警告

ウォームアップ実行時に、コンソールに以下の警告が出るケースです。

```
WarmUp may be inaccurate due to GraphicsDeviceType mismatch.
WarmUp may be inaccurate due to RuntimePlatform mismatch.
WarmUp may be inaccurate due to Quality Level mismatch.
```

原因は、GSCファイルに保存されたgraphicsDeviceType / runtimePlatform / qualityLevelNameのメタデータが実行環境と一致していないことです。主な発生ケースは以下の2つです。

- 異なるプラットフォームで収集したGSCを流用している（例: iOS実機で収集したGSCをAndroidビルドで使用）
- EditorスクリプトでGSCを編集・保存した際に、メタデータがEditorのPlatform設定で上書きされた

対処は発生ケースによって異なります。異なるプラットフォームで収集したGSCは、メタデータを書き換えても中身のGraphicsStateが実行環境と一致しないため、該当プラットフォームで収集し直します。EditorのPlatform設定で上書きされた場合は、メタデータを実機の値へ修正します。なお、qualityLevelNameの不一致は、PSOに影響するレンダリング設定（MSAAやHDRなど）やシェーダーがQuality Levelによって変化しない場合に限り無視できます。その場合は、リスト6.8 のようにウォームアップ直前に書き換えることで警告を回避できます。

▼リスト6.8　qualityLevelNameの書き換えによる警告回避

```
// GSCロード
var gsc = LoadGsc();
// qualityLevelNameを実行環境に合わせて変更
gsc.qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
// ウォームアップ実行
```

6.4.3　iOS（Metal）でのAssetBundle重複パッキングによる二重インスタンス

iOS（Metal）でのみ発生するケースです。ウォームアップ時のログにエラーはなく、ProfilerのMain ThreadでもShader.CreateGPUProgramは発生していないのに、Render ThreadではGpuProgramMetal.GetCachedPipelineが発生します。

原因は、AssetBundleの重複パッキングによるシェーダーの二重インスタンス化です。同一シェーダーが複数のAssetBundleに重複してパッキングされると、Unityはメモリ上に別々のシェーダーインスタンスを生成します。ウォームアップはGSCが参照しているシェーダーインスタンスに対してのみPSOを作成するため、同じシェーダーであっても別インスタンスになっているシェーダーはウォームアップされません。なお、Vulkanではこの問題は発生しません。PSOキャッシュの管理がMetalとVulkanで異なるためと考えられます。

発生する構成の例は以下のとおりです。同一のシェーダーXが、依存関係によって2つのバンドルの両方にパッキングされています。

```
- アセット
  - シェーダーX
  - マテリアルA(シェーダーXを使用)
  - マテリアルB(シェーダーXを使用)
  - GraphicsStateCollection

- AssetBundle構成
  - バンドルA
    - GraphicsStateCollection
    - マテリアルA(シェーダーXを使用)
    - シェーダーX ← 依存関係によるパッキング
  - バンドルB
    - マテリアルB(シェーダーXを使用)
    - シェーダーX ← 依存関係によるパッキング
(シェーダーXにはアドレスが振られていない状態)
```

この構成でバンドルAとバンドルBをロードすると、以下の流れでマテリアルBの描画時にスタッターが発生します。

```
- ランタイムの流れ
  1. バンドルAとバンドルBをロード
     - バンドルAのシェーダーX(instanceId=100)
     - バンドルBのシェーダーX(instanceId=200)
  2. GraphicsStateCollectionのウォームアップを実行
     - GSCが参照しているシェーダーX(instanceId=100)でウォームアップが行われる
  3. マテリアルAを描画
     - シェーダーX(instanceId=100)のPSOキャッシュは作成済みのためスタッターなし
  4. マテリアルBを描画
     - シェーダーX(instanceId=200)のPSOキャッシュはなく、
       Render ThreadでPSO作成が実行されスタッターが発生
```

対処は、シェーダーを単一のバンドル（シェーダー専用グループ）にまとめ、コンテンツバンドルへの暗黙のパッキングを排除することです。単一バンドルへの集約が難しい場合は、バンドルごとにGSCを用意します。重複しているシェーダーは リスト6.9のようなコードで確認できます。Addressablesを使っている場合は、**Addressables Report**ウィンドウでもシェーダーの重複を確認できます。

▼リスト6.9　シェーダーインスタンスの重複確認

```
using System.Linq;

// AssetBundleロード後に実行してインスタンスの重複を確認
var all = Resources.FindObjectsOfTypeAll<Shader>();
var duplicates = all.GroupBy(s => s.name).Where(g => g.Count() > 1);
foreach (var g in duplicates)
    foreach (var s in g)
        Debug.Log($"Duplicate: {s.name} | InstanceID: {s.GetInstanceID()}");
```

6.4.4　iOS（Metal）でのSubPass使用シェーダーのウォームアップ失敗

ここまではプロジェクト設定やバンドル設計のミスによって引き起こされる問題でした。次はGSCの仕様上発生する問題を紹介します。

1つのレンダーパスを複数の描画ステップに分割し、前のステップの描画結果をメインメモリへ書き戻さずに次のステップから参照できるようにする仕組みを、SubPassと呼びます。モバイルGPUでは、この一連のステップをチップ内の高速なメモリ領域（タイルメモリ）上で完結でき、メモリ帯域を節約できます。SubPassはMetal・Vulkan

ではNative Render Passとして実行されます。iOS（Metal）でのみ、ウォームアップを実施しログにもエラーが出ていないのに、ゲームプレイ中にPSOコンパイルが発生するケースがあります。原因は、iOS（Metal）のウォームアップ処理が、このSubPass（Native Render Pass）内で描画されるシェーダーに対して想定どおり機能しないことです。前のSubPassの出力を読み込むinput attachmentsの有無にかかわらず発生します。なお、ウォームアップの内部処理は非公開APIのため、ここでは発生条件の紹介にとどめます。

影響を受ける条件は以下のとおりです。

- ターゲットがiOS / Metal（Apple A/Mチップの実機）
- URPの描画処理基盤（RenderGraph）の設定である`RenderGraph.nativeRenderPassesEnabled`が有効（Unity 6000.5以降ではこの設定は廃止され、常時有効）
- SubPass（Native Render Pass）内で描画するシェーダーを使用している

書き込みのみのSubPass（URP DeferredのGBufferパスなど）でも、input attachmentsを持つSubPass（同Deferred Lightingパスなど）でも同様に発生します。影響を受けるSubPassの記述例として、input attachmentsを設定するケースを リスト6.10 に示します。

▼リスト6.10　SubPassにinput attachmentsを設定するケース

```
// ScriptableRenderContext.BeginRenderPassで開始した
// レンダーパス内で、SubPassを設定する例
var inputAttachments = new NativeArray<int>(new[] { 0, 1 }, Allocator.Temp);
context.BeginSubPass(
    colors,
    inputAttachments,   // 前SubPassの出力バインド
                        // Metalではframebuffer fetchに変換されるが、
                        // GSCウォームアップ時のPSO生成には反映されない
    false
);
```

Unity 6000.3では根本的な対処は難しく、SubPassの利用を見直すか、プリレンダリングでプレイ中のスタッターを回避します。プリレンダリングは、対象のシェーダーを使うオブジェクトをロード画面中などに1回実際に描画し、PSOキャッシュを作っておく手法です。本章の収集シナリオと同様の順次描画を、ウォームアップの代わりにゲーム内で行うイメージです。

6.4.5　スタッターが残る場合の診断ポイント

ここまで見てきたように、GSCを使ったシェーダーウォームアップはプロジェクトのアセット管理設計（Addressablesの構成・シェーダーキーワード定義・バンドル設計）と密接に関係するため、導入後のデバッグが必要になるケースが多くあります。ウォームアップ時のログの確認や、Profilerを使った繰り返しの計測を行い、ウォームアップ漏れがないか確認しましょう。特に以下の点を押さえておくと診断が速くなります。

- ウォームアップ時のログにウォームアップエラーや`variant not found`が出ていないか
- ProfilerでMain Threadの`Shader.CreateGPUProgram`、Render Threadの`GpuProgramMetal.GetCachedPipeline`（iOS）/ `CreateGraphicsPipelineImpl`（Android）が発生していないか
- Shader Strippingの設定は適切か
- AssetBundleのパッキングは想定どおりになっているか

GSCに記録されているバリアントの確認には、次節で紹介するバリアント一覧の出力（リスト6.11）が利用できます。

6.5　GSCファイルの操作例

最後に、GSCファイルの操作例を2つ紹介します。いずれもEditorスクリプトから実行する想定で、収集したGSCの中身の確認や、複数回の収集結果の統合に利用できます。

6.5.1　バリアント一覧とState数の確認

リスト6.11 は、GSCファイルをロードしてメタ情報と記録されているバリアントの一覧を出力する例です。

▼リスト6.11　バリアント一覧とState数の確認

```
void PrintVariantInfo() {
    // GSCのロード
    var gsc = new GraphicsStateCollection();
    if (!gsc.LoadFromFile(path))
        return;
```

```
    // バリアントの取得
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    gsc.GetVariants(variants);

    // GSCのメタ情報を出力
    Debug.Log($"[GSC] Variants={variants.Count} TotalStates={gsc.
    totalGraphicsStateCount} "
            + $"Platform={gsc.runtimePlatform} Device={gsc.graphicsDeviceType} Quality
    ={gsc.qualityLevelName}");

    // バリアントの詳細を出力
    foreach (var v in variants) {
        int stateCount = gsc.GetGraphicsStateCountForVariant(v.shader, v.passId, v.
    keywords);
        string kws = v.keywords.Length > 0
            ? string.Join(" ", v.keywords.Select(k => k.name))
            : "<no keywords>";
        Debug.Log($"  {v.shader?.name} | states={stateCount} | {kws}");
    }
}
```

6.5.2　GSCのマージ

　リスト6.12 は、2つのGSCをマージする例です。複数回に分けて収集した結果を1つに統合できます。

▼リスト6.12　2つのGSCのマージ

```
void MergeGsc() {
    // 2つのGSCをロード
    var src = new GraphicsStateCollection();
    var dst = new GraphicsStateCollection();
    if (!src.LoadFromFile(srcPath) || !dst.LoadFromFile(dstPath))
        return;

    // バリアントを取得
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    var states = new List<GraphicsStateCollection.GraphicsState>();
    src.GetVariants(variants);

    // srcのGSCからdstのGSCに追加
    foreach (var v in variants) {
        // バリアントに含まれるGraphicsStateを取得
        src.GetGraphicsStatesForVariant(v.shader, v.passId, v.keywords, states);
        foreach (var s in states) {
            // Stateを追加
```

```
            dst.AddGraphicsStateForVariant(v.shader, v.passId, v.keywords, s);
        }
    }

    dst.SaveToFile(dstPath);
}
```

AssetBundleの構成によっては、収集したGraphicsStateを別のGSCへ移動・統合するといった対応が必要になる場合があります。二重インスタンスの対処で紹介した、バンドルごとにGSCを用意するケースが典型です。その際は、本節の操作例を組み合わせて活用してください。

Unity®

PERFORMANCE TUNING TIPS 2026

CHAPTER 02

第2部

スクリプティング・アニメーション最適化

CyberAgent Smartphone Games & Entertainment

2026年のスクリプティング・アニメーション動向

スクリプティングの動向

Unity 6世代ではスクリプティングに関する状況はそれほど変化していません。ただしUnity 7では**CoreCLR**と**ECS for All**という、C# コードのパフォーマンスチューニングに関する前提を大きく変える機能の追加が控えており、Unity 6世代でも一部それに向けたAPIの変更が行われています。この章ではUnity 6世代のAPIの紹介とともに、Unity 7で予定されている変更なども一部紹介します。なお、Unity 7に関する情報は執筆時現在のものであり、実際のリリースとは異なっている可能性があることをご了承ください。

非同期処理

AwaitableはUnity 2023で登場したUnity標準のクラスで、従来は**UniTask**などの外部ライブラリを使って行っていたような効率的なasync / await処理をUnity標準クラスのみで記述できるようになりました。ただしAwaitableはUniTaskと比較すると機能セットが少ないので、既にUniTaskを利用している場合はそのままUniTaskを継続利用してもよいでしょう。外部パッケージに依存できない状況下ではTaskやValueTask, コルーチン の代わりにAwaitableを使用することで性能の向上が期待できます。また、第2部5章では非同期処理で問題になりやすいCancellationTokenの管理について解説しています。

.NETランタイム

Unity 6時点の .NETランタイムはこれまで通りMonoとIL2CPPが存在し、.NET Standard 2.1のAPIセットとC# 9が利用可能です。Unity 7ではMonoが廃止され、エディタ

と一部のデスクトッププレイヤーで**CoreCLR**の導入が予定されています。CoreCLRは.NET本家（.NET Core / .NET）で利用されているランタイム実装であり、パフォーマンスにおいてMonoより先行しています。モバイルやコンソールプラットフォームでは当面CoreCLRは導入されず、引き続きIL2CPPが第一級の選択肢となりますが、エディタはCoreCLRに移行するほか、実行時に利用できる標準APIの追加や実装の最適化など一部で恩恵があります。

CoreCLRのJIT

CoreCLRではC# コードをネイティブコードに変換するJITに多数の最適化が加えられています。ほとんどの場面ではMonoからCoreCLRに移行するだけでC# コードの実行速度は向上するでしょう。Unityから公開されたベンチマーク結果によると、GC利用のないワークロードではCoreCLRはMonoと比較して実行速度が2 ～ 3倍に上昇し、IL2CPPやBurstに匹敵する性能が得られることが示されています[*1]。これまで、特にMonoとIL2CPPのどちらかを選択できるデスクトッププラットフォームではパフォーマンス観点でIL2CPPが最適な選択肢でしたが、CoreCLR登場以降はこの前提も変化するかもしれません。

.NET 10のBCL

これまで .NET BCL[*2]はMonoに付属する実装が利用されており、.NET Standard 2.1相当のAPIセットに制限されていましたが、CoreCLRの導入に伴い .NET 10相当のBCLに更新される見込みです。利用できるAPIが大きく増えるほか、内部実装がより洗練され、性能の改善が期待できます。新しいBCLはIL2CPPでも利用可能になる予定で、CoreCLRが導入されないモバイルやコンソールプラットフォームにおいても性能面と機能面において恩恵があります。

CoreCLRのGC / メモリ管理

CoreCLRはGarbage Collector (GC)の実装もMono / IL2CPPと異なります。これまでUnityのMonoとIL2CPPでは**Boehm GC** (Boehm-Demers-Weiser GC, bdwgc)と呼ばれるGC実装が利用されていましたが、CoreCLRのGCはBoehm GCとは大きく異なる特徴を持っています[*3]。

[*1] Path to CoreCLR | GDC 2026 - Unity - YouTube `https://youtu.be/_t6xVfrmEWU?si=edILni6YnObyWkoj&t=218`

[*2] Base Class Library。System名前空間などの標準クラスライブラリ実装のこと

[*3] Fundamentals of garbage collection - .NET | Microsoft Learn - `https://learn.microsoft.com/en-us/dotnet/standard/garbage-collection/fundamentals`

まず、CoreCLR GCは**世代別GC (generational GC)**です。世代別GCは確保されたオブジェクトを「世代」と呼ばれるグループごとに管理します。新たに確保されたオブジェクトはまずgen0として扱われます。gen0に対するGCは比較的頻繁に行われますが、gen0 GCを生き延びたオブジェクトは次の世代gen1に移動し、gen0よりも低い頻度でGCが実行されるようになります。これは「新たに確保されたオブジェクトのほとんどが短命で、ある程度生き残ったオブジェクトはその後も長く生き残る場合が多い」という一般的な観測に基づいており、若い世代に対して頻繁に実行されるGCにかかる時間を短くする目的があります。CoreCLR GCでのメモリ最適化では、確保されたオブジェクトがどの世代に属しているか、どの世代に対するGCがいつ実行されているかなどが重要なポイントになるでしょう。

また、CoreCLR GCは**コンパクション**を行います。Boehm GCは一度確保されたオブジェクトを移動させないため、オブジェクトの確保と解放が繰り返されると次第にヒープが断片化し、実際に必要なサイズよりもヒープが大きくなりやすいという問題がありました。CoreCLR GCのコンパクションでは断片化したヒープ上でオブジェクトを移動させ、オブジェクトの間の隙間を埋めます。これによってヒープの拡大が抑えられます。

なお、IL2CPPでは引き続きBoehm GCが利用されるため、これまでと同様のメモリ最適化戦略が有効です[*4]。

GCのほかに、CoreCLRではJITによるメモリ確保の最適化も積極的に行われています。例えば**Object Stack Allocation**はメソッド内で利用が完結するような非常に短命なオブジェクトをヒープではなくスタック上に確保するJITの機能です。特にループの内側などホットパスで大量に確保される短命なオブジェクトなどは、MonoやIL2CPPほど問題を引き起こさなくなるかもしれません。

C# 14サポート

Unity 6時点で公式にサポートされているのはC# 9ですが、Unity 7ではC# 14が利用可能になる予定です。近年のC# リリースではパフォーマンス改善のための新機能も多数追加されており、パフォーマンスチューニング観点で知っておくべき言語機能も増えることになります。

[*4] Understanding Memory Management in Unity - Unity Discussion - `https://discussions.unity.com/t/understanding-memory-management-in-unity/1700107`

Unity 6世代に先行導入されている機能

CoreCLRの正式導入はUnity 7で行われる予定ですが、一部のプラットフォームではUnity 6系で実験的なCoreCLRの導入が始まっています。執筆時現在最新のUnity 6.7 alpha (6000.7.0a6)ではWindows, macOS, Linuxのスタンドアロンプレイヤーで CoreCLRが利用可能です。ただしAPIセットはまだ .NET Standard 2.1に制限されています。同じくUnity 6.7 alphaでiOSプレイヤー(IL2CPP)では .NET 10相当のBCL実装が利用可能になっていますが、こちらも .NET Standard 2.1に制限されています。現段階ではUnity 7でのCoreCLR移行に向けたパフォーマンスと互換性の検証にのみ利用できると考えるべきでしょう。

DOTS

Unity 6ではDOTS (Data-Oriented Technology Stack)機能群が引き続き有効です。LLVMを活用してC# コードを高度に最適化されたネイティブコードに変換する**Burst**, コードを安全にマルチスレッド化する**C# Job System**, それらを活用してMonoBehaviourよりも高速にゲームの状態を更新する仕組みである**Entity Component System (ECS)**が利用できます。

Unity 7では**ECS for All**と題した機能群によってECSとGameObjectの連携が強化される見込みです。これまでECSでコンポーネントを格納するためのコンテナであるEntityはGameObjectとは互換性がなく、ECSの状態はGameObjectから完全に独立して管理されていました。Unity 7ではGameObjectがEntityとしても機能するようになり、GameObjectに対してECSのコンポーネントをアタッチできるようになります。これまでよりもECSとGameObjectを混在・連携させやすくなることで、ボトルネックになりやすいMonoBehaviourのUpdate処理群をECSに移動しやすくなり、メインスレッド負荷の軽減とフレームレートの向上に役立つでしょう。

アニメーションの動向

Unity 6世代ではアニメーションのパフォーマンスにいくつか改善が加えられています。Unity 6.3でAnimatorとAnimationPlayableGraphに**ResetControllerState**メソッドが追加され、Animatorを容易にプーリングできるようになりました。詳しくは第2部2章で解説しています。Unity 6.5では**AnimationEventInfo**構造体が追加され、Animation Eventを受け取る際のAnimationEventクラスによるヒープメモリ確保を回避できるようになりました。

第1章

Unity 3D キャラクターアニメーションのチューニング

斉藤 俊介 Shunsuke Saito

株式会社Colorful Palette

1.1　はじめに

本章では「Unityでの3Dキャラクターアニメーション機能の最適化手法」について、アセット設定や実装の複数の観点で紹介します。

1.1.1　計測環境

本章の実装・計測は以下の環境で行いました。

表1.1: 計測環境

項目	値
Unity	6000.5.7f1
プラットフォーム	macOS Standalone (StandaloneOSX)
Scripting Backend	IL2CPP / IL2CPP Compiler Configuration = Release / Code Generation = OptimizeSpeed
Graphics API	Metal / GPU Skinning = 有効
Quality Level	PC / Skin Weights = Four Bones

1.1.2　使用アセットとその構造

計測に使うキャラクターはunity-chanをベースに、**Generic**リグとして再インポートしたものです。 後述するOptimize Game Objectsの適用前後で以下のようなスペックとなっています。

表1.2: 計測に使うプレハブの Transform 数と頂点数

プレハブ	Transform数	頂点数
Base	166	13,391
Optimized（Optimize Game Objects）	24	13,391

1.2　Unity Animationの基本制御について

まず、Unityでアニメーションを制御する3つの手法を整理します。

1.2.1　Animatorコンポーネントを使用した制御

`Animator`に`RuntimeAnimatorController`を割り当てる、もっとも一般的な方法です。ステートマシン・トランジション・ブレンドツリー・パラメータといった仕組みが揃っており、Mecanimのワークフローがそのまま使えます。

▼リスト1.1　Animator コンポーネントでの制御

```
var animator = GetComponent<Animator>();
animator.runtimeAnimatorController = controller;   // ステートマシンを割り当てる
animator.SetFloat("Blend", 0.5f);                  // パラメータで制御する
```

重要なのは、Animatorは内部的に**Playable Graphを組み立てて実行している**ことです。ステートマシンの評価もクリップのサンプリングも、内部ではグラフのノードとして処理されています。次の2手法は、この内部グラフを自分の手で組む方法にあたります。

1.2.2　Playable GraphをPlayable APIで構築し、自動更新で制御

`PlayableGraph`を自分で組み立てる方法です。ステートマシンを介さずにブレンド

構造を完全に制御でき、後述するAnimation Job(`AnimationScriptPlayable`)をグラフの任意の位置に差し込めます。

▼リスト1.2　Playable API でのグラフ構築（自動更新）

```
var graph = PlayableGraph.Create("MyGraph");
var output = AnimationPlayableOutput.Create(graph, "Animation", animator);

var clipA = AnimationClipPlayable.Create(graph, waitClip);
var clipB = AnimationClipPlayable.Create(graph, walkClip);
var mixer = AnimationMixerPlayable.Create(graph, 2);
graph.Connect(clipA, 0, mixer, 0);
graph.Connect(clipB, 0, mixer, 1);
mixer.SetInputWeight(0, 0.5f);
mixer.SetInputWeight(1, 0.5f);

output.SetSourcePlayable(mixer);
graph.Play();
```

更新モードは既定で`DirectorUpdateMode.GameTime`です。このモードではUnityのライフサイクル（UpdateとLateUpdateの間にあるAnimationフェーズ）で、Animatorコンポーネントと同じタイミングで**自動的に評価**されます。呼び出し側で毎フレーム何かをする必要はありません。

1.2.3　Playable GraphをPlayable APIで構築し、手動更新で制御

同じグラフに`DirectorUpdateMode.Manual`を設定すると、Unityは一切自動評価をしなくなり、`Evaluate()`を呼んだときだけグラフが評価されます。

▼リスト1.3　手動更新でのグラフ評価

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    graph.Evaluate(Time.deltaTime);   // 呼んだときだけ評価される
}
```

評価のタイミングと頻度を完全に制御できるため、「特定のキャラクター（グラフ）だけ更新を止める」「数フレームに1回だけ更新する」といった制御がそのまま実装できます。

3手法の関係をまとめると次の通りです。

表1.3: アニメーション制御 3 手法の比較

手法	グラフの構築	評価のタイミング
Animatorコンポーネント	Unity（内部）	UnityのAnimationフェーズ
Playable API + 自動更新	自分	UnityのAnimationフェーズ
Playable API + 手動更新	自分	`Evaluate()`を呼んだとき

1.3　アセットの設定

コード側の最適化を紹介する前に、アセット側の設定で適用できる最適化から見ていきます。この節のテーマは**ボーンのTransform階層をなくす**ことです。同じ結果を得る手段が2つあります。

1.3.1　Optimize Game Objects（インポート時）

FBXインポート設定（Rigタブ）の`Optimize Game Objects`を有効にすると、実行時にボーンのGameObject / Transformが丸ごと生成されなくなり、Animatorが内部スケルトンだけを持つようになります。今回のキャラクターでは**Transformが166個から24個**（ルート + Renderer 23個）に減りました。

※ボーンがGameObjectとして存在しないため、階層をたどってボーンを触るコードは動かなくなります。武器のアタッチポイントなど特定のボーンだけが必要な場合は、`Extra Transforms to Expose`で個別に残します。

1.3.2　Optimize Hierarchy（ランタイム）

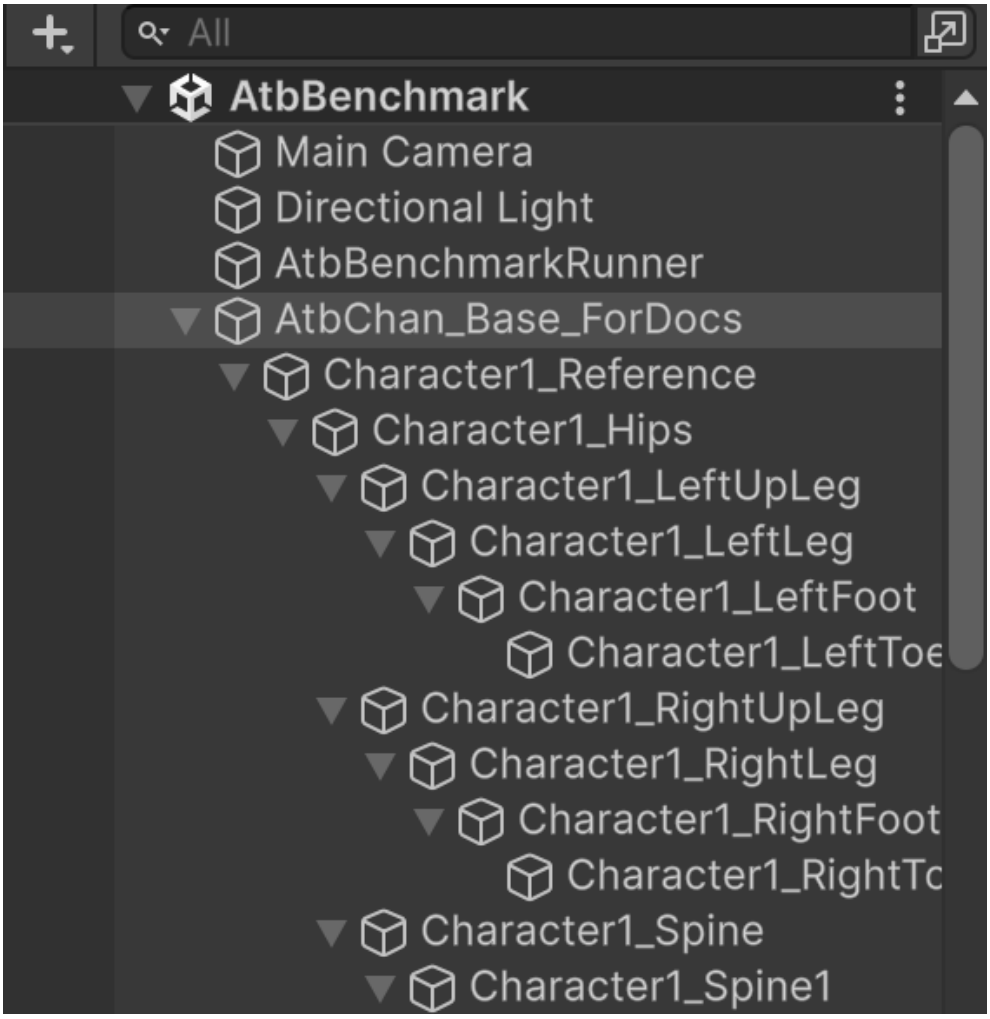
同じことを実行時に行うAPIが`AnimatorUtility.OptimizeTransformHierarchy`です。

▼リスト1.4　ランタイムでの階層フラット化

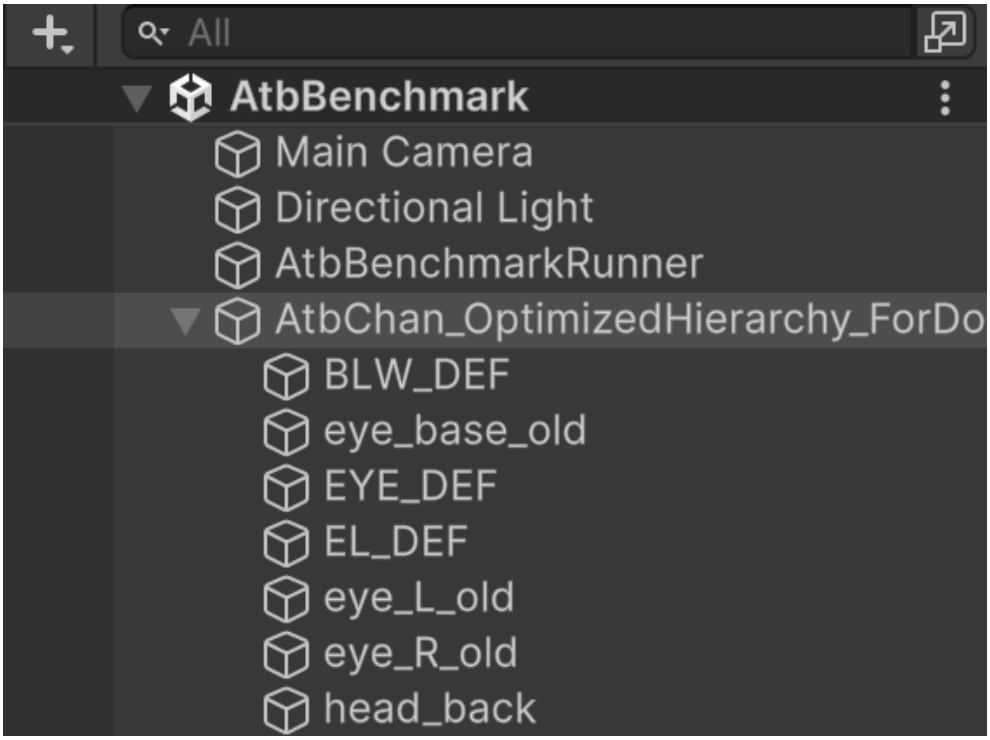
```
// go は Animator(と有効な Avatar)を持つルート。第 2 引数は露出させたいボーン名の配列
AnimatorUtility.OptimizeTransformHierarchy(go, null);

// 逆変換もある(エディタ拡張やデバッグで階層を一時的に戻したいとき)
AnimatorUtility.DeoptimizeTransformHierarchy(go);
```

インポート設定を触れないアセットでも、生成時に実行することで同じ構造になります。実際に確認したところ、Baseプレハブ（Transform 166個）に適用すると**Transform 24個**・`hasTransformHierarchy == false`となり、インポート時にOptimize Game Objectsを適用したプレハブと完全に同じ構造になりました。



▲図1.1　最適化前のHierarchy。`Character1_Hips`以下にボーンのTransform階層が166個続く。



▲図1.2 `AnimatorUtility.OptimizeTransformHierarchy`適用後。ボーン階層が消え、ルート直下にSkinnedMeshRenderer 23個が並ぶだけになる（Transform 24個）。

1.3.3 実測: 階層フラット化の効果

描画負荷を2段階に変えて測りました。1つめはRendererを全部有効にした構成（300体 × 23 Renderer = 6,900 Renderer）、2つめは各キャラクターで頂点数が最大のSkinnedMeshRenderer 1個だけを有効にして描画負荷を下げた構成です。フレーム時間は中央値です。

表1.4: 階層フラット化の効果（300 体・全 Renderer と 600 体・1 Renderer）

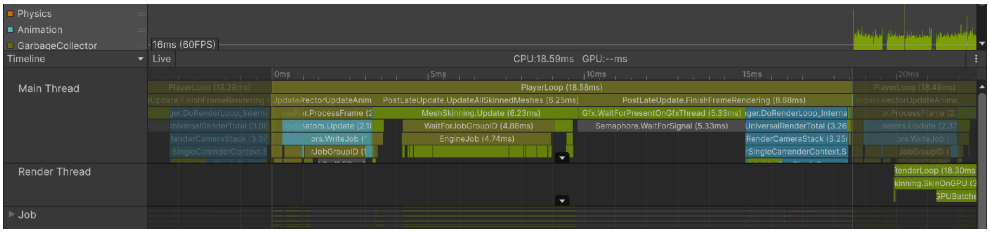
構成	バリアント	フレーム時間	`MeshSkinning.Update` (main)	`MeshSkinning.CalcMatrices` (all)	`Semaphore.WaitForSignal`
300体・全Renderer	Base	18.59 ms	6.23 ms	36.633 ms	5.33 ms
300体・全Renderer	Optimize Game Objects	18.497 ms (±0)	**1.537 ms** (− 75.7 %)	**0 ms**	**10.255 ms**
300体・全Renderer	Optimize Hierarchy	18.307 ms (±0)	**1.504 ms** (− 76.2 %)	**0 ms**	**9.977 ms**
600体・1 Renderer	Base	6.797 ms	0.735 ms	4.385 ms	5.741 ms
600体・1 Renderer	Optimize Game Objects	**5.991 ms (− 11.9 %)**	0.207 ms	**0 ms**	4.901 ms
600体・1 Renderer	Optimize Hierarchy	**6.009 ms (− 11.6 %)**	0.208 ms	**0 ms**	5.010 ms

CPU側の処理は、どちらの構成でも同じ割合で減っています。メインスレッドのスキニング処理が6.321 → 約1.5 ms（全Renderer）、0.735 → 約0.21 ms（1 Renderer）で、いずれも7割超の削減を達成しています。スキニング行列の計算(`MeshSkinning.CalcMatrices`)に至っては全スレッド合計36.6 ms / 4.4 msが**完全に消滅**しています。

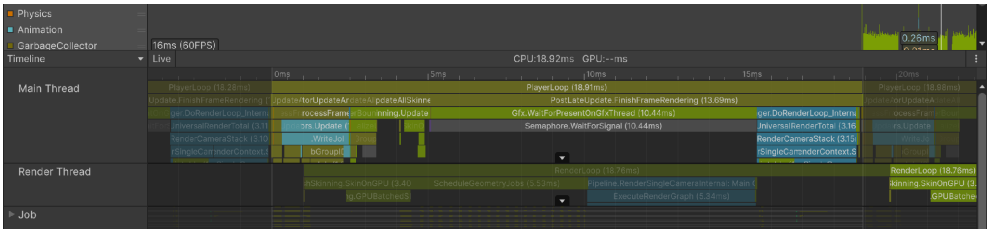
ただし「全Renderer構成」では、レンダースレッドの処理がフレーム時間に対するクリティカルパスとなっているため、フレーム時間の削減は見られません。

描画負荷を下げた1 Renderer構成では、同じ削減がそのまま**12 %前後のフレーム時間短縮**として現れます。300体でも同じ傾向で、3.312 → 2.899 / 2.901 ms（− 12.5 % / − 12.4 %）でした。**インポート時のOptimize Game ObjectsとランタイムのOptimize Hierarchyの差は0.1 %未満**で、構造が同じになる以上、効果も同じだということがわかります。

ProfilerのTimelineでも同じことが起きています。どちらも300体・全Renderer、フレーム時間の中央値付近のフレームです。



▲図1.3 Base（Transform 166個）。フレーム時間18.59 msのうち、メインスレッドで`MeshSkinning.Update`が6.23 msを占め、`Gfx.WaitForPresentOnGfxThread` / `Semaphore.WaitForSignal`が5.33 ms。



▲図1.4 `AnimatorUtility.OptimizeTransformHierarchy`適用後。`MeshSkinning.Update`のブロックはラベルが読めない細さまで縮む一方、`Semaphore.WaitForSignal`が5.33 → 10.44 msに伸び、`PlayerLoop`は18.58 → 18.91 msとほぼ変わっていない。減らしたCPU時間が待ち時間に置き換わっている。

1.3.4 実測: Renderer全無効（アニメーション評価だけ）

描画とスキニングを完全に外し、アニメーション評価そのものへの寄与を見ます。300体です。

表1.5: Renderer を全無効にしたときのフレーム時間

バリアント	フレーム時間
Base	2.702 ms
Optimize Game Objects	2.687 ms (− 0.6 %)
Optimize Hierarchy	2.496 ms (− 7.6 %)

クリップのサンプリングやブレンドといった評価処理はTransform階層の有無に関係なく走るため、こちらへの寄与はほぼないと言えます。このことから判明するのは、こ

の最適化の目的は主に**スキニング行列の計算とTransformへの書き戻し**に対するもので、アニメーション評価そのものではない、ということです。

1.3.5 この章の結論

- 階層フラット化は**スキニング行列計算(`MeshSkinning.CalcMatrices`)**の削減に繋がる
- インポート時(Optimize Game Objects)とランタイム(`AnimatorUtility.OptimizeTransformHierarchy`)で効果は同等
- どちらを選ぶかは運用の問題: 自前アセットならインポート設定で済ませる、インポート設定を触れない事情があるならランタイムAPIを使う
- **モデル自体の頂点数・骨数を制作段階で適正化しておくこと**が最優先

1.4 Job Threadの活用

1.4.1 Unityのアニメーション並列化

Animatorコンポーネント、および自動更新のPlayable Graphは、Unityが**すべてのグラフをまとめてスケジュールし、Animation Job Threadに分散**します。

問題は手動更新です。`PlayableGraph.Evaluate()`は**そのグラフ内部の評価処理を並列化する**もので、グラフをまたいで並列化はしません。したがって「1キャラクター = 1 Playable Graph」という直感的な構成でN体を制御すると、`Evaluate()`をN回呼ぶことになり、**N個のグラフが直列に処理されます。**

これはUnityのPlayable Graphの避けられない仕様なのですが、対策はシンプルで、**N体を1つのグラフにまとめる**ことです。

▼リスト1.5　全キャラクターを 1 本のグラフに集約する

```
// 1 本のグラフに、キャラクターごとの出力とサブツリーをぶら下げる
_sharedGraph = PlayableGraph.Create("Shared");
_sharedGraph.SetTimeUpdateMode(DirectorUpdateMode.Manual);

foreach (var character in characters)
{
    var output = AnimationPlayableOutput.Create(_sharedGraph, "Animation", character.
    Animator);
```

```
    output.SetSourcePlayable(BuildClipSource(_sharedGraph));   // 各キャラのミキサー
}

_sharedGraph.Play();

// 更新は 1 回だけ
void Update() => _sharedGraph.Evaluate(Time.deltaTime);
```

1.4.2 実測1: 制御手法 × キャラクター数

Rendererは全無効にして、アニメーション評価のコストだけを取り出しています。フレーム時間の中央値です。

表1.6: 制御手法とキャラクター数ごとのフレーム時間

制御方法	100体	300体	600体
Animatorコンポーネント	0.899 ms	2.413 ms	**5.004 ms**
Playable API + 自動更新	0.899 ms	2.498 ms	**5.093 ms**
Playable API + 手動更新（1体1グラフ）	1.395 ms	3.925 ms	**8.396 ms**
Playable API + 手動更新（全体で1グラフ）	0.800 ms	2.115 ms	**4.500 ms**

AnimatorとPlayable自動更新はほぼ同じコストです。「Playable APIのほうが軽い」という言説は、少なくとも「シンプルなアニメーション制御」の条件では成立しません。Animatorのステートマシン評価（`AnimatorControllerPlayable.PrepareFrame`）は600体でも全スレッド合計2.624 ms、うちメインスレッドは0.161 msしかなく、この処理がボトルネックになる可能性は極めて低いと言えます。（ただし、2 msを大きい/小さいどちらと捉えるかはゲーム次第だと考えます。）

手動更新はグラフの構造で1.87倍変わります。同じ600体でも、グラフのまとめ方しだいで自動更新より速くも遅くもなります。

1.4.3 実測2: スレッドの使われ方

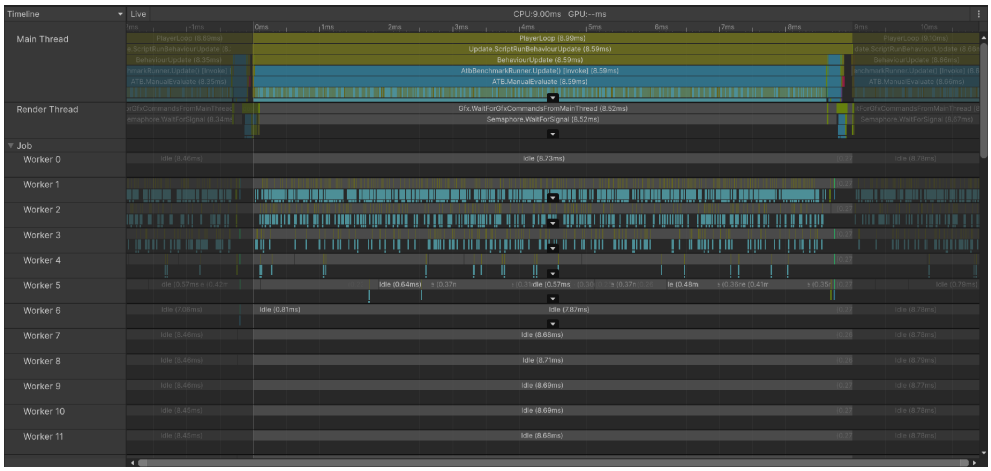
600体、Renderer無効。Unity Profilerのキャプチャからスレッド別に集計した結果です。

表1.7: グラフ粒度によるスレッドの使われ方の違い（600 体）

1体1グラフ（600グラフ）	600体1グラフ	
フレーム時間	**9.490 ms**	**4.928 ms**
メインスレッドのEvaluate	9.028 ms	4.556 ms
アニメーション処理を担ったワーカースレッド数	**5**	**15**
Animator.ProcessGraph全スレッド合計	0.920 ms	**19.034 ms**
ワーカー1本あたりのAnimator.ProcessGraph	0.32 / 0.15 / 0.03 / 0.02 / 0.002 ms	各約**1.19 ms**（15本ほぼ均等）

1体1グラフのとき、アニメーション処理はほぼ全部メインスレッド上にあります。600回のEvaluate()が順番に走っているだけで、ワーカースレッドはほとんど働いていません。

600体1グラフのとき、15本すべてのワーカースレッドがほぼ均等にAnimator.ProcessGraphを実行しています。



▲図1.5 1体1グラフ（600グラフ）のTimeline。メインスレッドのATB.ManualEvaluateが約8.6 msを占め、Jobワーカーの大半が「Idle」のまま。



▲図1.6 600体1グラフのTimeline。メインスレッドのATB.ManualEvaluateは約4.3 msに半減し、全ワーカーが約1.2 msずつのEngineJobを同時に実行している。

数値の読み方に1点補足があります。

この表の数値はUnity Profilerを接続した状態のキャプチャから取っているため、Profilerなしで走らせた計測（4.500 ms / 8.396 ms）よりわずかに重く出ています。章末のまとめ表はProfiler非接続の数値を使っています。

1.4.4 この章の結論

表1.8: 600 体でのフレーム時間まとめ

構成	600体のフレーム時間
1体1グラフ + 手動更新	8.396 ms
Animator（自動）	5.004 ms
600体1グラフ + 手動更新	**4.500 ms**

- Animatorと自動更新のPlayable Graphは、Unityが全グラフをまとめてAnimation Job Threadに分散してくれる
- 手動更新はグラフ単位でしか並列化されない
- 手動更新を選ぶ場合は、グラフの粒度を必ず設計項目に考慮すること
- Playable APIへの置き換え自体は最適化効果はほぼない。Playable APIを用いる

真の価値は「Animation Jobを差し込める」「グラフ構造と更新タイミングを制御できる」といった**アニメーション制御の柔軟性**にある

1.5 Animation C# Jobsの利用とSIMD対応について

「プロシージャルにアニメーションを制御したい」というニーズはゲーム開発中に必ず発生します。例えば「キャラクターの足を地面の形状に接地するように動かしたい」などの場合です。

この場合とりうる手段としてTwo-Bone IKという手法があります。

実際にTwo-Bone IKのようなプロシージャルアニメーション制御を実装するとき、選択肢は3つあります。

1. `LateUpdate`でTransformを直接書き換える
2. `AnimationScriptPlayable`に`IAnimationJob`を差し込む
3. 2に加えてBurstでコンパイルする

今回はこれらの手法のパフォーマンスに着目します。

3つとも**完全に同一の演算**を行うように実装しました。解法は1つのstaticメソッドに切り出し、LateUpdate版・Job版・Burst版がそれを共有しているため、違うのは実行される場所だけです。

1.5.1 Animation C# Jobsとは

Animation C# Jobsは、`AnimationScriptPlayable`と`IAnimationJob`によって**アニメーション評価パイプラインの中に自前の処理を差し込む**仕組みです。ジョブはクリップの評価やブレンドと同じ流れの中で呼ばれ、ボーンには`AnimationStream`と`TransformStreamHandle`を通してアクセスします。

`LateUpdate`との違いは、コードが動く場所です。`LateUpdate`はメインスレッド上で、アニメーション評価が終わったあとのTransformを直接書き換えます。処理はキャラクター数分メインスレッドに直列に積み上がります。Animation Jobは評価パイプラインの内部（Animation Job Thread）で呼ばれるため、Unityは他のアニメーション評価と同じようにワーカースレッドへ分散できます。同じ演算でも、メインスレッドの

負担になるかどうかが変わります。

AnimationStream経由というアクセス制約はその代償です。ジョブから見えるのは事前に束ねたハンドルのぶんだけで、任意のTransformやGameObjectは触れません。

1.5.2 実装

ジョブ本体は、ProcessAnimationで解法を呼ぶだけです。

▼リスト1.6 Two-Bone IK の Animation Job（Burst 対応）

```
[BurstCompile]
public struct TwoBoneIkJobBurst : IAnimationJob
{
    public TransformStreamHandle Root;
    public TransformStreamHandle Mid;
    public TransformStreamHandle Tip;
    public NativeArray<float3> Target;    // 長さ 1。毎フレーム、メインスレッドから書き込む
    public float Weight;
    public int Iterations;

    public void ProcessRootMotion(AnimationStream stream) { }

    [BurstCompile]
    public void ProcessAnimation(AnimationStream stream)
    {
        TwoBoneIkCore.SolveStreamIterations(
            ref stream, Root, Mid, Tip, Target[0], Weight, Iterations);
    }
}
```

ボーンの読み書きは、TransformStreamHandleとAnimationStreamの組で行います。

▼リスト1.7 AnimationStream からボーンを読み書きする

```
static void SolveStream(
    ref AnimationStream stream,
    TransformStreamHandle root, TransformStreamHandle mid, TransformStreamHandle tip,
    float3 target, float weight)
{
    var rootPosition = (float3)root.GetPosition(stream);
    var midPosition = (float3)mid.GetPosition(stream);
    var tipPosition = (float3)tip.GetPosition(stream);
    var rootRotation = (quaternion)root.GetRotation(stream);
    var midRotation = (quaternion)mid.GetRotation(stream);
```

```
    Solve(rootPosition, midPosition, tipPosition, target, rootRotation, midRotation,
    weight,
        out var newRootRotation, out var newMidRotation);

    mid.SetRotation(stream, newMidRotation);
    root.SetRotation(stream, newRootRotation);
}
```

ハンドルはAnimator.BindStreamTransformで作り、ジョブはクリップの後段としてグラフに挿入します。

▼リスト1.8　Animation Job をグラフに挿し込む

```
var job = new TwoBoneIkJobBurst
{
    Root = animator.BindStreamTransform(rootTransform),
    Mid = animator.BindStreamTransform(midTransform),
    Tip = animator.BindStreamTransform(tipTransform),
    Target = ikTarget,   // NativeArray<float3>
    Weight = 1f,
    Iterations = iterations,
};

var ikPlayable = AnimationScriptPlayable.Create(graph, job, 1);
graph.Connect(clipSource, 0, ikPlayable, 0);   // clipSource = クリップ側のミキサー
ikPlayable.SetInputWeight(0, 1f);
output.SetSourcePlayable(ikPlayable);
```

IKターゲットのような毎フレーム変わる入力はNativeArrayに置き、メインスレッドから書き込みます。

BurstはBurstCompile属性をクラスに付与することで適用されます。

実際にBurst Compilerによる最適化が効いているかはBurst Inspectorなどのツールを用いて確認することができます。

1.5.3　実測

300体、Renderer無効。1体あたりのIK解法回数を1 / 16 / 64と変えています。

表1.9: IK 解法回数ごとのフレーム時間（300 体・Renderer 無効）

実装	1回	16回	64回
IKなし（基準）	2.501 ms	2.601 ms	2.596 ms
LateUpdate	2.699 ms	4.400 ms	**9.094 ms**
Animation Job	2.411 ms	2.496 ms	**2.795 ms**
Animation Job + Burst	2.494 ms	2.496 ms	**2.792 ms**

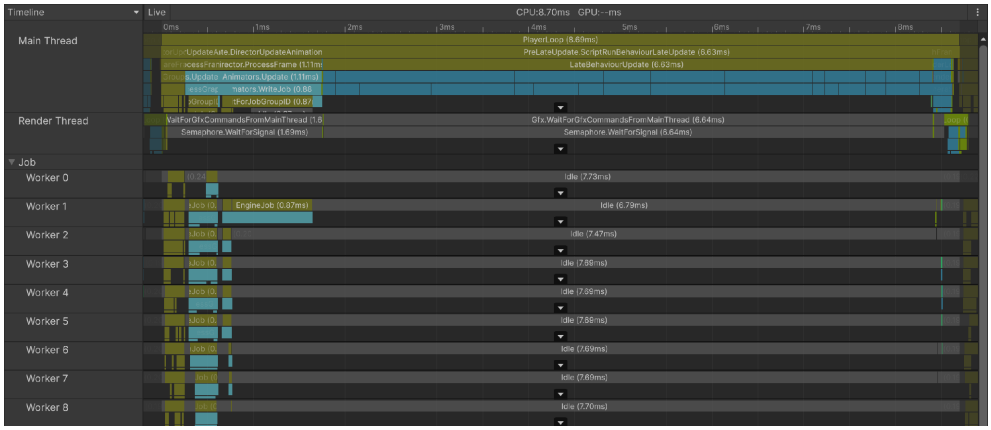
CPU時間の内訳（64回のとき）：

表1.10: IK 実装ごとの CPU 時間の内訳（解法 64 回）

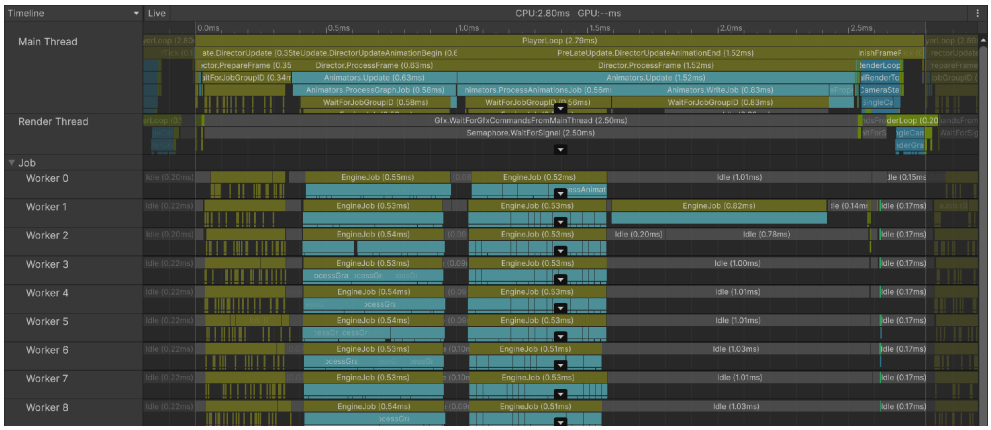
実装	IKのCPU時間（全スレッド）	うちメインスレッド	フレーム時間への上乗せ
LateUpdate	6.910 ms	**6.910 ms（100 %）**	+6.498 ms
Animation Job	7.790 ms	0.487 ms（6.3 %）	+0.199 ms
Animation Job + Burst	6.932 ms	0.436 ms（6.3 %）	+0.196 ms

1.5.4 Profiler Timelineで見る実行スレッドの違い

同じ演算がどのスレッドで実行されているかは、Unity EditorのProfilerに現れます。以下は解法64回・300体のキャプチャをProfilerウィンドウで開き、フレーム時間の中央値付近のフレームを選択したものです。Profiler接続時のキャプチャなので、上の表の数値とは完全には一致しません。



▲図1.7 LateUpdate実装。フレーム時間8.698 msのうち`LateBehaviourUpdate`が6.63 msを占める。ワーカースレッド15本はフレームの大半（約6.8〜7.7 ms）が`Idle`のまま。



▲図1.8 Animation Job実装。同じ演算が`AnimationScriptPlayable.ProcessAnimation`としてワーカースレッド15本に分散し、1本あたり約0.5 msずつ実行されている。フレーム時間は2.796 ms。

LateUpdate実装では、300体分の`ATB.TwoBoneIk.LateUpdate`（合計6.577 ms）がすべてメインスレッドの`LateBehaviourUpdate`の中にあります。このときワーカースレッドが担っているのはベースのアニメーション評価（`Animator.ProcessGraph`が1本あたり0.12〜0.29 ms）だけで、IKの計算は1つも載っていません。

Animation Job実装では`AnimationScriptPlayable.ProcessAnimation`の全スレッド合計が7.891 msで、うちメインスレッドは0.498 msです。残りの7.393 msはワーカースレッド15本にほぼ均等（1本あたり0.47〜0.51 ms）に分散しています。総CPU時間はLateUpdate実装より多いものの、それが16本のスレッドに分かれるため、フレーム時間は8.698 → 2.796 msに収まります。

1.5.5 読み取れること

このケースの最適化について、最も効果的なことはBurstではなく、実行スレッドを移すことです。 LateUpdate実装はIKのCPU時間を丸ごとメインスレッドで払うため、64回ではフレーム時間が2.596 → 9.094 msに膨らみます。同じ演算をAnimation Jobに移すとメインスレッドの負担はほぼ消え、フレーム時間への上乗せで**33倍の差**がつきます。

一方で、**総CPU時間ではLateUpdate版のほうが少なくなっています。**Animation Jobには`AnimationScriptPlayable`ノードごとの固定コストがあり、300体分では約2.16 msです。それでも並列化の利得がはるかに上回ります。

実運用に近い「1体1回」の条件では、3実装の差はほぼ測定ノイズの範囲です。300体程度のTwo-Bone IKなら、どの実装を選んでも体感差はありません。

1.5.6 Burstは属性を付けるだけでは効かない

`[BurstCompile]`を付ければSIMDになって数倍速くなる、という理解は誤りです。今回の実測では、Burstによる総CPU時間の削減は11.0 %です。ノード固定コストを差し引いた「解法1回あたりの限界コスト」で見ると、

- Burstなし: (7.790 − 2.157) / 63 = **0.0894 ms**
- Burstあり: (6.932 − 2.167) / 63 = **0.0756 ms**（− 15.4 %）

Burstが行うのはLLVMによるネイティブコード生成で、SIMD命令が出るかどうかは書かれたコードの形で決まります。**適切にSIMD化されるコードを実装しなければ、大きな恩恵は得られません。**

Animation C# Jobsには、SIMD化を難しくする性質が2つあります。

AnimationStreamの読み書きが処理の中に頻繁に現れます。解法1回ごとにGetPosition・GetRotationでボーンを読み、SetRotationで書き戻します。反復のたびにストリームの状態が変わるため反復間に依存関係が残り、複数回分を束ねて1つのベクトル命令にはできません。さらにIAnimationJobが受け取るAnimationStreamは1体分なので、「4体分を同時に計算する」ためのデータがそもそもジョブの中にありません。

超越関数の利用が多発します。余弦定理から曲げ角を得るacos、quaternion.AxisAngleが内部で呼ぶ三角関数、math.lengthの平方根。これらはレーンをまたぐ計算やスカラーのライブラリ呼び出しになり、ベクトル化を止めます。

BurstのLLVMも、解法を繰り返し呼ぶループを**ベクトル化できないと明示的に報告します**（最適化リマークloop not vectorized: instruction return type cannot be vectorized）。

1.5.7 Burstの恩恵をフル活用するための作戦

Burstの恩恵をフル活用したいなら、C# Job Systemを使って**AnimationStreamを介さずにTransformを読み書きする**構成が候補になります。Transformの読み書きはTransformAccessArrayとIJobParallelForTransformでジョブ側にまとめられますし、ボーンの収集と書き戻しだけをAnimation Jobに任せ、重い計算はN体分をまとめたバッチジョブに切り出す形も取れます。どちらもキャラクター単位ではなく**N体分のデータがひとつのジョブの中に揃う**ため、SIMD化の前提が成立します。

そのうえで、バッチジョブの中身は次のように整える必要があります。

- 1成分1配列のSoA（Structure of Arrays）にして、キャラクターをメモリ上で連続させる
- 分岐をmath.selectやmath.maxに置き換え、ループ本体を1本の直線にする
- 超越関数をループから追い出す（恒等式で書き換える、テーブル化する、近似する）
- float3ではなくfloat4で「4体分を1レーンずつ」書き、自動ベクトル化の判断に依存しない

代償は設計の複雑さです。IAnimationJobは、グラフに1ノード挿すだけで評価パイプラインの正しいタイミングでボーンの最新状態を触れる、という手軽さが最大の利点で、この作戦はそれを捨てることになります。本記事ではこの構成の実測を行って

いないため、どれだけ速くなるかはここでは示しません。Burstに数倍の効果を期待するならこの方向の設計変更が前提になる、という提案として扱ってください。

1.5.8 この章の結論

- Animation Jobの価値は**Animation Job Threadが活用できる**こと
- 演算量が少ないうちは`AnimationScriptPlayable`のノード固定コストのほうが大きい。素直にLateUpdateで書いたほうが総CPU時間は少ない
- `[BurstCompile]`は付けるだけではSIMDにならない。SIMDが適用されることを見越したデータ・処理の設計を行う必要がある

1.6 アニメーションLODの実装

1.6.1 仕組みと効果的な場面

ゲーム仕様にもよりますが、基本的には遠くのキャラクターまで毎フレーム更新する必要はありません。Nフレームに1回だけ更新すれば、そのキャラクターの平均負荷は1/Nになります。これが「アニメーションLOD」です。メッシュのLODが描画コストを距離で段階制御するのと同じ発想を、**アニメーションの更新頻度**に適用します。

効果的なのは次のような場面です。

- 画面内に数百体規模のキャラクターがいて、大半は遠景やモブである
- 遠いキャラクターの30 fps / 15 fps相当の動きが体感上許容できる（カメラが遠いほど間引きは気づかれにくい）
- アニメーション評価がフレーム時間の中で無視できない割合を占めている

逆に、近距離でカメラに大写しになるキャラクターに適用するとカクつきがすぐ分かります。距離やスクリーン占有率に応じて間引き段階を切り替えるのが想定される実運用でしょう。

1.6.2 シンプルな実装: `Animator.Update()`を自分で呼ぶ

`animator.enabled = false`にしてUnityの自動更新を止め、必要なフレームだけ`Animator.Update()`を明示的に呼びます。飛ばしたフレーム分の時間は溜めて渡します。

この実装にはこれまで説明してきたAnimation Job Threadでの並列性を失うという副作用があります。

▼リスト1.9　Animator.Update() を自分で呼ぶ間引き実装

```
animator.enabled = false;   // 初期化時。Unity の自動更新を止める

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;   // Slot = index %
    interval

        character.Animator.Update(character.PendingDeltaTime);    // 溜めた時間で明示更新
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

1.6.3　Playable APIを直接用いた実装例

手動更新（DirectorUpdateMode.Manual）のPlayable Graphなら、Evaluate()の呼び出しそのものを間引くだけです。

▼リスト1.10　手動更新グラフの Evaluate() を間引く実装

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;

        character.Graph.Evaluate(character.PendingDeltaTime);
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

1つ注意があります。前章で推奨した「全キャラクターを1グラフに集約する」構成

では、グラフ全体を1回のEvaluate()で評価するため、**キャラクター単位の間引きが表現できません。**上のコードのように1体1グラフにすれば間引けますが、そのぶん集約による並列化を失います。

どちらの実装でも重要なのがSlot（位相）です。全キャラクターを同じフレームに更新すると、そのフレームだけ負荷が集中します。index % intervalで位相をずらせば、毎フレーム1/N体ずつが更新される形になります。

1.6.4 実測

600体、Renderer無効。まずAnimatorでの実装です。

平均値は12秒間の総負荷を、**中央値とp95の開き**はフレーム時間の暴れ具合を表します。

表1.11: 間引きと位相ずらしのフレーム時間統計（Animator・600 体）

方式	平均	中央値	p95	p99	最大	標準偏差
毎フレーム更新（Animator自動）	5.037 ms	5.000 ms	5.303 ms	5.707 ms	16.901 ms	0.433
2フレームに1回・位相なし	4.781 ms	0.648 ms	9.607 ms	9.816 ms	12.194 ms	4.544
2フレームに1回・位相ずらし	4.831 ms	4.803 ms	5.099 ms	5.212 ms	8.804 ms	0.171
4フレームに1回・位相なし	2.613 ms	**0.382 ms**	**9.470 ms**	9.658 ms	11.171 ms	3.863
4フレームに1回・位相ずらし	2.578 ms	**2.593 ms**	**2.707 ms**	2.908 ms	11.722 ms	0.185

Playable API（1体1グラフ・手動更新）でも同じ傾向です。

表1.12: 間引きと位相ずらしのフレーム時間統計（Playable API・1 体 1 グラフ）

方式	平均	中央値	p95	標準偏差
毎フレーム更新	8.496 ms	8.493 ms	9.001 ms	0.337
4フレームに1回・位相なし	2.438 ms	0.369 ms	8.855 ms	3.570
4フレームに1回・位相ずらし	2.486 ms	**2.495 ms**	**2.613 ms**	0.173

1.6.5　位相ずらしは「負荷を減らす」のではなく「平らにする」

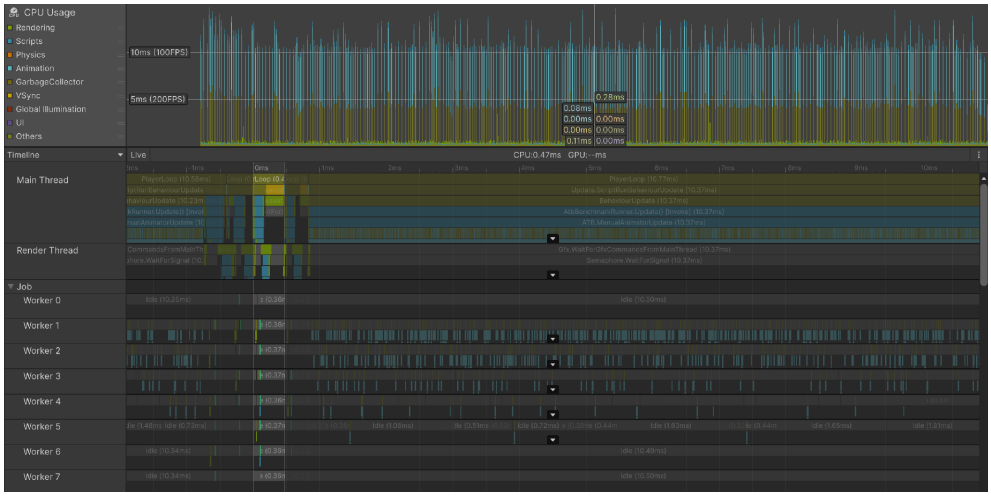
4分の1に間引いたときの平均値は位相の有無でほとんど変わらず、**総負荷はほぼ同じ**です。位相ずらしは仕事を減らしているわけではありません。

一方で、フレーム時間の分布には次の違いが出ています。

- 位相なし: 4フレームのうち3フレームは何もせず、1フレームで600体分をまとめて処理する。中央値は下がるがp95と標準偏差が跳ね上がる
- 位相ずらし: 毎フレーム150体ずつ処理する。中央値とp95がほぼ一致し、ばらつきが小さい

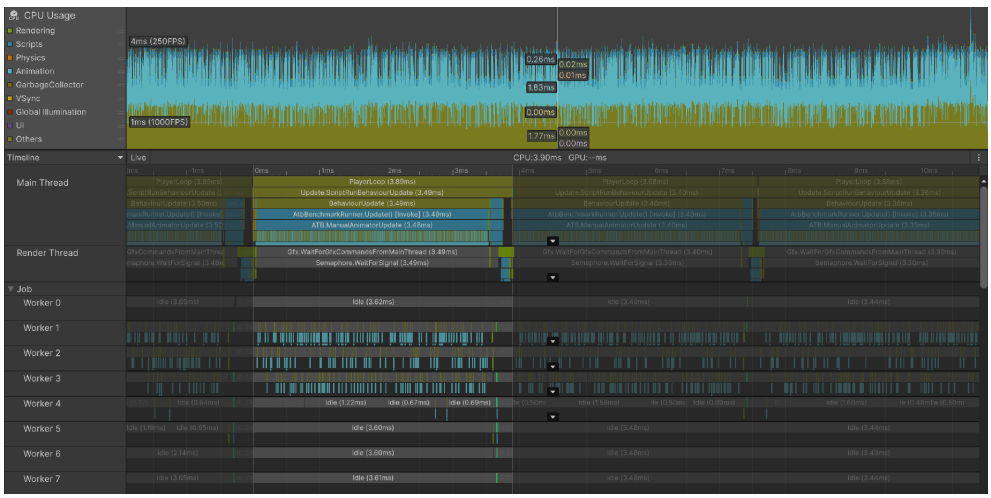
中央値0.382 msという数字だけを見て「1/13になった」と喜ぶと、実際には9.5 msのスパイクが4フレームに1回刺さるゲームができあがります。**位相ずらしは体感品質のための施策**であり、標準偏差0.185は毎フレーム更新の0.433よりも安定しています。

この分布の違いはProfilerのCPUチャートにそのまま現れます（いずれもProfiler接続時のキャプチャなので、上表の非接続の数値より重めに出ています。またチャートの縦軸スケールは自動調整のため2枚で異なります）。



▲図1.9

4フレームに1回・位相なし。CPUチャートが「ほぼ0 msの谷3フレーム + 約10 msのスパイク1フレーム」の櫛状になる。下のTimelineでは選択中の空フレーム（0.47 ms）の両隣に、600体分をまとめて処理するATB.ManualAnimatorUpdate（10.37 ms）のフレームが写っている。



▲図1.10

4フレームに1回・位相ずらし。毎フレーム150体ずつ処理するため波形は平坦（約3.5 ms）で、スパイクが消えている。

1.6.6 素朴な実装は並列化を失う

2フレームに1回の間引きは、ほとんど何も削っていません。更新回数を半分にしたのに、平均フレーム時間の削減はわずか**4.1 %**です。

この理由は、Profilerマーカーの内訳から分かります。

表1.13: 素朴な間引き実装で失われる並列性

方式	1フレームに更新する体数	メインスレッドのアニメーション更新時間	`Animator.ProcessGraph`全スレッド合計	うちメインスレッド
毎フレーム更新（Unityのバッチ更新）	600	`Animators.Update` 4.046 ms	20.104 ms	1.257 ms
2フレームに1回・位相ずらし	300	`ATB.ManualAnimatorUpdate` 4.532 ms	0.549 ms	0.317 ms
4フレームに1回・位相ずらし	150	`ATB.ManualAnimatorUpdate` 2.316 ms	0.278 ms	0.180 ms

毎フレーム更新では、仕事のうちメインスレッドに残るのは6 %だけで、残りはワーカースレッドに散っています。一方`Animator.Update()`を明示的に呼ぶと**その場で同期的に評価される**ため、ワーカースレッドがほとんど使われません。Unityのバッチ更新が持っていた並列化を、間引きと引き換えに手放しています。

1体あたりのメインスレッドコストに直すと、

- 毎フレーム更新: 4.046 ms / 600体 = **6.7 µs/体**（並列）
- 2フレームに1回: 4.532 ms / 300体 = **15.1 µs/体**（直列）
- 4フレームに1回: 2.316 ms / 150体 = **15.4 µs/体**（直列）

直列化した瞬間に1体あたり約2.3倍になり、その後は間引き倍率によらず一定です。つまり**2倍の間引きで得られる2倍の削減が、直列化による2.3倍の悪化でほぼ相殺されます。**実測の4.1 %はその残差にすぎません。

素朴な実装のままでは、4倍間引き（− **48.8 %**）でようやく実質的な効果が出ます。

ただし、これは避けられる損失です。

1.6.7 並列性を保ったまま間引く

`Animator.Update()`が同期的なのは避けられません。しかし**「Unityのバッチ更新に参加させる体を毎フレーム選ぶ」**という形にすれば、間引きながら並列性を保てます。2通りの実装があり、どちらもAnimator（とそのステートマシン）を使ったままです。

方法1: `Animator.enabled`を切り替える

Unityのアニメーション更新は、その時点で有効なAnimatorを集めて1つのバッチにし、Animation Job Threadに分散します。したがって`Animator.Update()`を呼ぶ代わりに、**そのフレームに更新したい体だけ`enabled = true`にしておけば済みます。**評価はUnityが行うので並列化はそのまま残ります。

▼リスト1.11　Animator.enabled を切り替える間引き実装

```
// 初期化時。飛ばしたフレーム分は speed で取り戻す
foreach (var character in _characters)
{
    var animator = character.Animator;
    animator.keepAnimatorStateOnDisable = true;    // 無効化でステートを巻き戻さない(必須)
    animator.speed = interval;                     // interval フレームに 1 回しか更新され
    ないぶんを補正
}

void Update()
{
    foreach (var character in _characters)
    {
        var scheduled = _frameIndex % interval == character.Slot;
        if (character.Animator.enabled != scheduled)
        {
            character.Animator.enabled = scheduled;   // Unity のバッチ更新に参加する体を
    選ぶだけ
        }
    }
    _frameIndex++;
}
```

`keepAnimatorStateOnDisable = true`は忘れてはいけません。既定値の`false`ではAnimatorを無効化するたびにコントローラのステートが巻き戻り、アニメーションが進まなくなります。

制約はAnimator.speedが**レート**であることです。Animator.Update(pendingDeltaTime)のように「溜めたΔtを正確に渡す」ことはできず、更新フレームのTime.deltaTime × intervalだけ進みます。フレーム時間が揺れる環境では進み方も揺れますが、平均の再生速度は保たれます。

方法2: 位相ごとにグラフを分ける

前章の結論は「N体を1つの手動グラフにまとめれば並列化される」でした。これを間引きに組み合わせます。**位相の数だけグラフを作り、1フレームに1本だけEvaluate()する**のです。1本のグラフにはN/interval体が入っているので、その1回のEvaluate()の中で並列化が効きます。

ステートマシンはAnimatorControllerPlayableとしてグラフに持たせれば、Animatorコンポーネントに割り当てたときと同じ評価が走ります。

▼リスト1.12　位相ごとにグラフを分ける間引き実装

```
// 初期化時：位相バケットごとに手動グラフを 1 本
for (var bucket = 0; bucket < interval; bucket++)
{
    var graph = PlayableGraph.Create($"Bucket_{bucket}");
    graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
    _bucketGraphs[bucket] = graph;
}

foreach (var character in _characters)
{
    var graph = _bucketGraphs[character.Slot];
    character.Animator.runtimeAnimatorController = null;   // 評価はグラフ側に任せる
    var output = AnimationPlayableOutput.Create(graph, "Animation", character.Animator)
    ;
    output.SetSourcePlayable(AnimatorControllerPlayable.Create(graph, controller));
}

foreach (var graph in _bucketGraphs) graph.Play();

void Update()
{
    for (var bucket = 0; bucket < interval; bucket++) _pendingDeltaTime[bucket] += Time
    .deltaTime;

    var scheduled = _frameIndex % interval;
    _bucketGraphs[scheduled].Evaluate(_pendingDeltaTime[scheduled]);   // 1 フレームに 1
     バケットだけ
```

```
    _pendingDeltaTime[scheduled] = 0f;
    _frameIndex++;
}
```

こちらはEvaluate()に**溜めたΔtをそのまま渡せる**ので、進み方が正確です。代わりにグラフを間引き段階数だけ持つことになり、LOD段階を動的に変えるにはキャラクターをグラフ間で移す（出力を作り直す）必要があります。

実測: 3つの実装の比較

600体、Renderer無効。平均フレーム時間と、毎フレーム更新（5.037 ms）に対する削減率です。

表1.14: 3 つの間引き実装のフレーム時間比較（600 体）

実装	1/2間引き	1/4間引き
Animator.Update()を明示呼び出し（位相ずらし）	4.831 ms（- 4.1 %）	2.578 ms（- 48.8 %）
Animator.enabledの切り替え	**3.350 ms（- 33.5 %）**	**1.835 ms（- 63.6 %）**
位相ごとの手動グラフ（AnimatorControllerPlayable）	**3.082 ms（- 38.8 %）**	**1.541 ms（- 69.4 %）**

素朴な実装がいちばん割に合いません。とくに1/2間引きでの差が大きく、- 4.1 %が- 33.5 % / - 38.8 %になりました。「2倍の間引きは並列化の損失に食われる」という前節の結論は、**実装を変えれば成立しません。**

並列性が保たれていることはProfilerマーカーで確認できます。

表1.15: 3 つの間引き実装のスレッド別内訳

方式	Animator.ProcessGraph全スレッド合計	うちメインスレッド	メインスレッド側の更新時間
毎フレーム更新（600体/フレーム）	20.104 ms	1.257 ms	Animators.Update 4.046 ms
Animator.Update()・1/4（150体/フレーム）	0.278 ms	0.180 ms	ATB.ManualAnimatorUpdate 2.316 ms
Animator.enabled・1/4（150体/フレーム）	**4.424 ms**	0.288 ms	Animators.Update 1.104 ms + 切り替え 0.125 ms

表1.15: 3 つの間引き実装のスレッド別内訳

方式	Animator.ProcessGraph全スレッド合計	うちメインスレッド	メインスレッド側の更新時間
位相ごとの手動グラフ・1/4（150体/フレーム）	**4.253 ms**	0.279 ms	ATB.BucketEvaluate 1.278 ms

新しい2実装では、Animator.ProcessGraphの仕事量が毎フレーム更新のちょうど**1/4**になり（1フレームに150体しか評価しないので当然です）、その仕事は依然としてワーカースレッドに散っています。一方Animator.Update()版はこのマーカーにほとんど計上されず、仕事の本体はメインスレッドのATB.ManualAnimatorUpdateの中にあります。

1体あたりのメインスレッドコストで並べると差がはっきりします。

- 毎フレーム更新: 4.046 ms / 600体 = **6.7 µs/体**
- Animator.enabled切り替え・1/4: 1.229 ms / 150体 = **8.2 µs/体**
- 位相ごとの手動グラフ・1/4: 1.278 ms / 150体 = **8.5 µs/体**
- Animator.Update()・1/4: 2.316 ms / 150体 = **15.4 µs/体**

新しい2実装は1体あたりのコストがほぼ毎フレーム更新のままです（差は切り替えやEvaluateの呼び出しオーバーヘッド）。そのため間引き倍率がそのまま削減率に近づきます。

再生速度が保たれていることも確認しました。最後にスポーンしたキャラクター（位相スロットが最大の体）の指先ボーンが1秒あたりに動く距離を測ると、毎フレーム更新0.509 m/sに対しAnimator.enabled切り替え0.496 m/s、位相ごとの手動グラフ0.517 m/sです。間引きで動きが直線的に近似されるぶんの誤差の範囲で、どちらも速度が変わっていません。

分布については1点注意があります。1/2間引きの手動グラフ版はp99が9.102 ms、標準偏差1.086と、enabled切り替え版（p99 4.100 ms、標準偏差0.562）より裾が重く出ました。1/4では逆に手動グラフ版のほうが安定します（標準偏差0.306対0.340）。グラフの本数と1本あたりの体数の兼ね合いで挙動が変わるので、採用するなら平均だけでなくp99も見てください。

どちらを選ぶかは要件次第です。Animatorのステートマシンとインスペクタのワークフローをそのまま使い続けたいなら`Animator.enabled`の切り替えが圧倒的に手軽です（数行で済み、既存コードへの影響もほぼありません）。進み方の正確さや、Animation Jobを差し込むといったグラフ側の制御が要るなら位相ごとの手動グラフです。**どちらにしても`Animator.Update()`を自分で呼ぶ実装を選ぶ理由はありません。**

1.6.8 この章の結論

- アニメーションLODは「更新頻度の間引き + 位相ずらし」のセットで設計する
- 位相ずらしは総負荷を減らさない（平均2.613対2.578 ms）。減らすのは**フレーム時間のばらつき**（標準偏差3.863 → 0.185）であり、これは体感品質のための施策
- **`Animator.Update()`を自分で呼ぶ実装は選ばない。**その場で同期評価されるためジョブ並列化を失い、1体あたりのメインスレッドコストが6.7 → 15.4 µ sと約2.3倍になる
- 代わりに**Unityのバッチ更新に参加する体を毎フレーム選ぶ。**`Animator.enabled`の切り替え（`keepAnimatorStateOnDisable = true`と`Animator.speed`での補正が必要）か、位相ごとに手動グラフを分けて`AnimatorControllerPlayable`を載せる形。1体あたり8.2〜8.5 µ sに収まり、並列化も残る
- 実装を変えると**2倍の間引きでも効く**: − 4.1 %が− 33.5 % / − 38.8 %になった。4倍なら− 63.6 % / − 69.4 %
- キャラクター単位の間引きは1本の集約グラフでは表現できない。ただし「位相の数だけグラフを分ける」なら並列化と更新頻度制御を両立できる
- 中央値だけで判断しないこと。総負荷は平均で、体感はp95・p99と標準偏差で見る

1.7 まとめ

各章の実測結果です。

表1.16: 各章の実測結果まとめ

章	施策	条件	結果
アセットの設定	Optimize Game Objects	600体・Renderer 1個	6.797 → 5.991 ms（− **11.9 %**）
アセットの設定	Optimize Hierarchy（ランタイム）	600体・Renderer 1個	6.797 → 6.009 ms（− **11.6 %**）
アセットの設定	上記2手法	300体・全Renderer	18.205 → 18.3〜18.5 ms（改善なし・描画律速）
Job Thread	手動更新グラフを1本に集約	600体	8.396 → 4.500 ms（− **46.4 %**）
Animation C# Jobs	Two-Bone IKをAnimation Job化	300体・64回解法	9.094 → 2.795 ms（− **69.3 %**）
Animation C# Jobs	さらにBurstでコンパイル	300体・64回解法	2.795 → 2.792 ms（フレーム時間は差なし）
アニメーションLOD	1/4間引き・`Animator.Update()`を明示呼び出し	600体	平均5.037 → 2.578 ms（− **48.8 %**）
アニメーションLOD	1/4間引き・位相ごとの手動グラフ	600体	平均5.037 → 1.541 ms（− **69.4 %**）
アニメーションLOD	1/2間引き・`Animator.Update()`を明示呼び出し	600体	平均5.037 → 4.831 ms（− 4.1 %）
アニメーションLOD	1/2間引き・位相ごとの手動グラフ	600体	平均5.037 → 3.082 ms（− **38.8 %**）

1.7.1 最終的な最適化アプローチ

この記事の実測を踏まえた、大量キャラクターのアニメーション最適化の進め方です。

1. アセット設定を最適化する

まずはモデル自体の頂点数や骨数の仕様を適切な値に定めましょう。基準となるサポート端末でベンチマークを計測し、適切なモデルレギュレーションを定めることがまずは理想です。そのうえでUnity側でOptimize Game Objectsの適用を検討しましょう。3Dタイトルにおいて、上流に位置するモデル制作を量産が進んだプロジェクト終盤で大きく修正していくことは難しいため、プロジェクト序盤に行えることが最適です。

2. Animation Job Threadでの処理の並列性を高める

AnimatorとPlayable GraphはUnityが自動で並列化しますが、手動更新はグラフ単位でしか並列化されません。キャラクター制御の設計を行う時点で「Animation Job Threadを有効活用できるか」を念頭に入れましょう。

3. Animation JobのBurst適用を含めた最適化を行う

IKなどの後処理はまずメインスレッドからAnimation Jobへ移します。

さらにパフォーマンスが必要とされる場合はSIMD化を適用できるデータ・処理設計を適用しましょう。

4. アニメーションLODなどのアプリケーション的な工夫を行う

アセット・エンジン設定・コード最適化を尽くしたら、最後は「そもそもアニメーションを処理しない」という工夫です。

プロジェクトの要件と相談になりますが、ゲーム体験的に許容できるラインを探りながら処理を抑える仕組みを検討しましょう。

1.8 脚注

1.8.1 Unity公式ドキュメント

- Playable API
  https://docs.unity3d.com/Manual/Playables.html – Playable Graphの構築とDirectorUpdateMode
- AnimatorUtility.OptimizeTransformHierarchy
  https://docs.unity3d.com/ScriptReference/AnimatorUtility.OptimizeTransformHierarchy.html – ランタイムでの階層フラット化
- Model Import Settings: Rig（Optimize Game Objects）
  https://docs.unity3d.com/Manual/FBXImporter-Rig.html
- IAnimationJob
  https://docs.unity3d.com/ScriptReference/Animations.IAnimationJob.html – AnimationScriptPlayableに差し込むジョブ
- AnimationScriptPlayable
  https://docs.unity3d.com/ScriptReference/Animations.AnimationScriptPlayable.html
- C# Job System
  https://docs.unity3d.com/Manual/JobSystem.html – AnimationStreamを介さずに並列処理を書くための仕組み

1.8.2 使用アセット

- ユニティちゃん3Dモデルデータ
  https://unity-chan.com/ – © Unity Technologies Japan/UCL

第2章

ResetControllerStateで Animatorプーリングを安全に行う

塩塚 勇気 Yuki Shiotsuka

QualiArts

2.1　はじめに

本記事では、Unity 6.3で追加されたAnimator.ResetControllerStateを取り上げます。

ResetControllerStateを「プール返却時の初期化を明確かつ簡単に実装するためのAPI」として整理し、Unity 6000.3.21f1で確認した挙動をもとに、安全なプール返却処理を説明します。

2.2　Animatorプーリングで起きる問題

ObjectPoolは、生成済みのインスタンスを使い回すことで、頻繁な生成と破棄の負荷を減らすための定番手法です。UnityのObjectPoolも、取得、返却、破棄のタイミングに処理を差し込める仕組みを持ちます[*1]。

プールのためにGameObjectを非アクティブにすると、再びアクティブにしたときにAnimatorがRebindを行い、プールで得た性能を打ち消す可能性があります。そのため、Unity公式のAnimatorクラスの説明では、GameObjectではなくコンポーネントを無効にすることが推奨されています[*2]。本記事では、この推奨に沿ってGameObjectは

[*1] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html

[*2] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html

アクティブのまま保管し、返却時にAnimatorとRendererを無効にして、取得時に再び有効にするプールを前提とします。

一方で、プールでインスタンスを再利用する以上、GameObjectやコンポーネントは前回利用時の状態を持ったまま戻ってくる可能性があります。Animatorの場合も、無効にして止まるのは評価だけで、内部の状態はそのまま残ります。何もしなければ、前回利用時のStateや各種Parameterを引き継いだまま次の利用が始まるため、前回の途中のStateから再生されたり、残ったParameterによって意図しない遷移が起きることがあります。

ResetControllerStateは、AnimatorControllerの各LayerのStateとParameterをまとめて既定の状態へ戻せるAPIです。リリースノートでも、アニメーションするキャラクターのプーリングを容易にするための追加として紹介されています[*3]。

2.3 ResetControllerStateでリセットされる項目

ResetControllerStateの引数resetParametersにtrueを指定すると、Stateに加えてParameterも既定値へ戻ります。falseを指定した場合は、Stateだけが戻ります[*4]。Animator付きキャラクターをプールへ返す場合は、通常はResetControllerState(true)を使い、StateとParameterをまとめて戻します。未使用、通常遷移、AnyState遷移、BlendTreeの用途にかかわらず、Float、Bool、Intを既定値へ戻し、Triggerをリセットします。また、すべてのLayerの現在Stateを既定のStateへ戻します。

ただし、反映されるタイミングは項目によって異なります。Parameterの値は呼び出し直後から既定値になりましたが、GetCurrentAnimatorStateInfoは、次にAnimatorが評価されるまでリセット前のStateを返しました。

AnimatorControllerPlayableにも、ControllerのStateとParameterをリセットするResetControllerStateメソッドが用意されています[*5]。

[*3] https://unity.com/releases/editor/whats-new/6000.3.0f1

[*4] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html

[*5] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

2.4 ResetControllerStateではリセットされない項目

一方、次の項目はResetControllerStateだけでは既定値へ戻りません。

- SetLayerWeightで変更したLayer Weight
- Animator.speed
- StateMachineBehaviourに実装した独自の実行時変数

Layer WeightやAnimator.speedは、ControllerのStateやParameterとは別のプロパティです。プール返却時に変更後の値を残したくない場合は、明示的に戻します。StateMachineBehaviourの独自変数も、個別の初期化が必要です。

また、Rigidbody、ParticleSystem、AudioSourceなど他のコンポーネントの状態や、独自ロジックの状態は、AnimatorControllerの外で管理されるため、ResetControllerStateの対象ではありません。

2.5 実装例

リスト2.1は、プール返却時の初期化と無効化をまとめた例です。ObjectPool<T>のactionOnReleaseから呼び出すことを想定しています。プロジェクトの仕様によっては不要な処理も含まれます。

▼リスト2.1　プール返却時のAnimator初期化例

```
// プール返却時に必要な初期化と無効化をまとめる
public void ReturnToPool()
{
    // AnimatorControllerの各LayerのStateとParameterを戻す
    animator.ResetControllerState(true);
    // defaultLayerWeightは、再利用時に戻す値として別途設定しておく
    animator.SetLayerWeight(layerIndex, defaultLayerWeight);
    // 再生速度を戻す
    animator.speed = 1f;
    // 必要な場合のみ。リセット後の最初の評価を、返却処理の中で済ませる
    animator.Update(0f);
    // Animator外の独自状態を戻す
    ResetGameState();
    // GameObjectはアクティブのまま、評価と描画を止めて保管する
    animator.enabled = false;
    characterRenderer.enabled = false;
}
```

2.6 まとめ

ObjectPoolは、生成と破棄の回数を減らすだけでは完成しません。再利用するGameObjectが次回利用時にどの状態から始まるべきかを決め、その状態へ戻す処理まで含めて設計する必要があります。

GameObjectを非アクティブにせず、Animatorを無効にして保管する場合、Animatorの内部状態はそのまま残ります。ResetControllerState(true)を使うと、Animator Controllerの各LayerのStateと、Float、Bool、Int、TriggerのParameterをまとめて既定値へ戻せます。一方、Layer Weight、Animator.speed、StateMachineBehaviourの独自変数、他のコンポーネントや独自ロジックの状態はリセットされません。

初期化対象をAnimatorController内と外に分けておくと、プール返却処理の責務が明確になります。ResetControllerStateを軸に、プロジェクト固有の状態だけを補う形で返却処理を組み立てるとよいでしょう。

2.7 参考資料

- Unity Scripting API - ObjectPool<T0>
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html`
- Unity Scripting API - Animator
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html`
- Unity 6000.3.0f1 Release Notes
  `https://unity.com/releases/editor/whats-new/6000.3.0f1`
- Unity Scripting API - Animator.ResetControllerState
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html`
- Unity Scripting API - AnimatorControllerPlayable.ResetControllerState
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html`

第3章

VATによるボーンアニメーション負荷軽減

畳 悠樹 Tatami Yuuki

株式会社サイバーエージェント SGEコア技術本部

X @tatamin0207

3.1　はじめに

Unityにおける一般的なボーンアニメーションでは、ボーンの更新といった再生用の計算を毎フレームCPUが行います。そのため、アニメーションするオブジェクトが増えるほどCPU負荷が積み上がり、フレームレート低下の原因になります。本章では、この実行時の計算をまるごと事前計算に置き換えることで負荷を軽減する手法、VATを紹介します。

3.2　VATとは

VATはVertex Animation Textureの略で、アニメーションによる頂点の変形情報をあらかじめテクスチャにベイクしておき、頂点シェーダーで読み取って再生する手法です。テクスチャのピクセル1つ1つが「ある頂点の、あるフレームでの位置」に対応しており、再生時間に合わせて読み取る場所をずらしていくことでメッシュが動いて見える仕組みです（図3.1）。なお、位置は精度を確保するため0から1に正規化した値で格納されており、頂点シェーダーで実際の座標へ復元します。



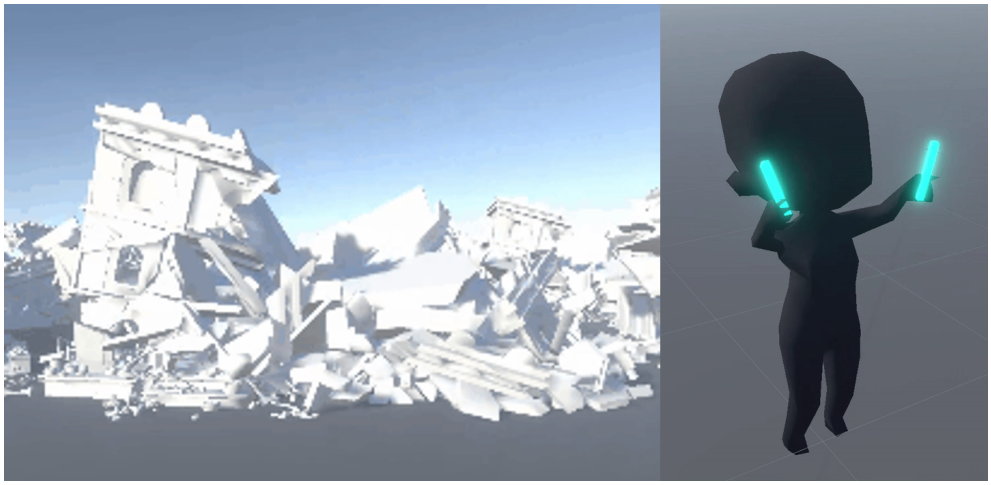
▲図3.1　VATテクスチャの仕組み。横軸が頂点、縦軸がフレームに対応する

VATを使用することで、ボーンの更新といった実行時の再生計算をまるごと事前計算に置き換えられます。これはGPU Skinningでもできない効率化です。

一方で、以下の短所があります。

- VAT用の専用アセットを用意する必要があります
- 頂点数とフレーム数に応じてテクスチャサイズが大きくなります
- 正確な値が必要なためテクスチャ圧縮と相性が悪く、事実上非圧縮で扱います
- テクスチャに焼き込んだ時点で動きが固定されるため、工夫次第である程度の制御はできるものの、細かい制御には不向きです

これらの特性から、VATは大量のオブジェクトに決まったアニメーションを再生させる場合にとくに有効です。たとえば建物の破壊表現や群衆、背景の装飾物のような、個別の制御を必要としない演出に適しています（図3.2）。



▲図3.2　VATによるアニメーション（左：剛体、右：ソフトボディ）

3.3　性能検証

VATの性能を、他のアニメーション方式と実機で比較した結果を紹介します。

3.3.1　計測環境と比較条件

計測環境を 表3.1 に示します。

表3.1: 計測環境

項目	内容
実機	Pixel 8a（GPU: Mali-G715）
グラフィックスAPI	Vulkan
Unityバージョン	6000.3.2f1
計測API	FrameTimingManager、ProfilerRecorder
モデル	建物の倒壊アニメーション（85,209頂点、破片1,315個）
アニメーション	300フレーム
計測方法	オブジェクト数を0〜16体まで変化、各条件300フレーム×3回の中央値

比較した3つの方式を 表3.2 に示します。

表3.2: 比較条件

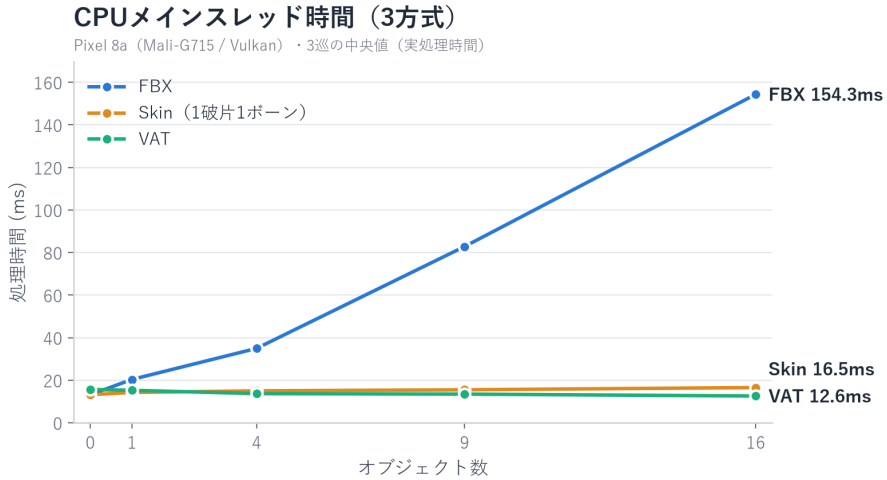
方式	構成	有効にした最適化
FBX方式	各破片を個別のMesh Rendererとして配置し、Animatorで再生	SRP Batcher
Skin方式	1破片1ボーンで1つのSkinned MeshRendererに統合し、Animatorで再生	SRP Batcher
VAT方式	全破片の動きをテクスチャに焼き込み、1つのメッシュとして描画（Rigid-Body Dynamicsモード）	GPU Instancing

FBX方式は、各破片を個別のMesh Rendererとして持ち、Animatorで再生する構成です（破片はTransformで動く剛体で、スキニングは不使用）。Skin方式は、破片1つごとにボーン1本を割り当てて、全体を1つのSkinned Meshにまとめた構成です。1,315個の破片がそのまま1,315本のボーンになり、Rendererは1つです。アニメーションの再生はCPUのAnimatorが担い、頂点の変形はGPU Skinningで行われます。VAT方式は、全破片の動きをテクスチャに焼き込み、1つのメッシュとして描画します。見た目は3方式とも同一です。

描画の効率化設定は、それぞれの方式に有利なものを有効にした状態で比較しています。SkinnedMeshRendererはGPU Instancingに対応していないため、Skin方式に使える最善はSRP Batcherになります。なお、Skin方式のボーン化にも専用の変換パイプラインが必要で、この点はVATと同条件です。

3.3.2 CPU処理時間

CPU処理時間（メインスレッド）の計測結果を 図3.3 に示します。16体表示時、FBX方式が154.3msまで伸びるのに対し、Skin方式は16.5ms、VAT方式は12.6msとほぼ横ばいでした。



▲図3.3　オブジェクト数ごとのCPUメインスレッド処理時間

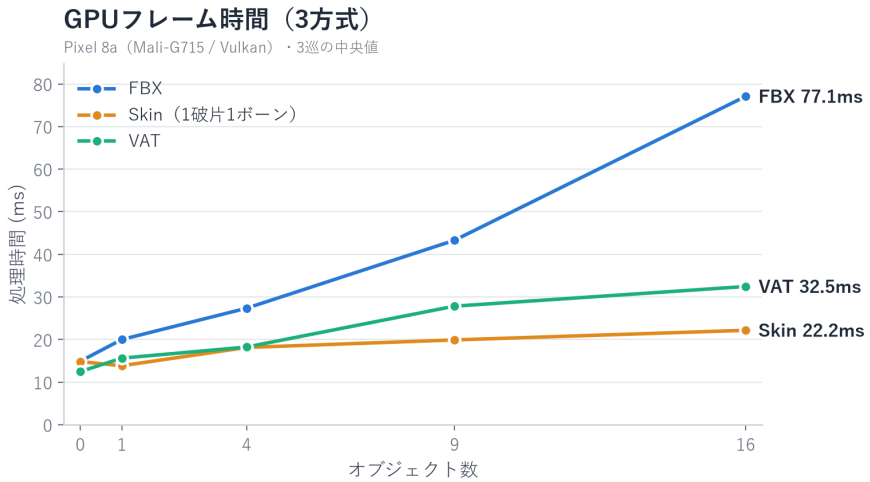
内訳を 表3.3 に示します。FBX方式と他2方式の差の最大の要因はドローコールの送信処理です。FBX方式ではモデル1つごとに1,315個の破片が描画対象になるのに対し、1つのメッシュにまとまっている2方式ではこの処理がほぼ消えます。破片ごとのバウンディングボックス計算やカリングも同様です。一方、Skin方式とVAT方式の差はAnimatorの更新です。Skin方式は再生をCPUが担うため16体で4.5msが残りますが、VAT方式は再生そのものがGPUへ移るため、まるごとなくなります。この差は体数に応じて広がります。

表3.3: 16体表示時のCPU処理時間の内訳（ms）

処理	FBX方式	Skin方式	VAT方式
メインスレッド合計	154.30	16.53	12.56
うちドローコール送信（SRPBatcher.Flush）	76.17	0.03	0.02
うちAnimator更新	5.30	4.54	0.02

3.3.3　GPU処理時間

GPU処理時間の計測結果を 図3.4 に示します。16体時はFBX方式の77.1msに対し、Skin方式が22.2msで最速、VAT方式は32.5msでした。



▲図3.4　オブジェクト数ごとのGPU処理時間

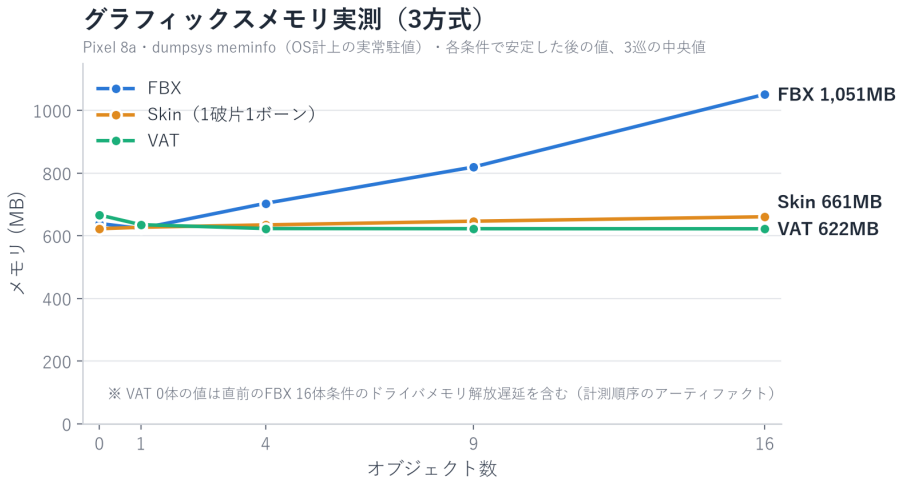
FBX方式と他2方式の差の主因は、描画命令の分割数です。FBX方式は破片が各描画パスで個別に描画命令を発行するため1フレームあたり約21万回に達していたのに対し、VAT方式はインスタンス描画にまとまり99回でした。Skin方式も1体につきRendererが1つのため同水準に収まります。描画命令には1回ごとに固定のコストがかかるため、分割数の差がそのまま処理時間の差として現れます。

Skin方式とVAT方式の差は、変形計算の回数によるものです。GPU Skinningは変形をフレームに1回だけ計算し、その結果をシャドウやデプスなどすべての描画パスが再利用します。対してVATは、パスのたびに頂点シェーダーでテクスチャから変形をやり直します。今回の環境はシャドウのカスケードが4枚という、パス数が多いシーン構成のため、その分Skin方式との差が開きました（シャドウマップ描画はFBX方式44.3ms、Skin方式7.4ms、VAT方式16.0ms）。頂点計算を行うパス数が少ない構成では、この差は縮まります。

3.3.4　GPUメモリ確保量

GPUメモリの計測結果を 図3.5 に示します。値はOSが計上するグラフィックスドライバの実測値（GL mtrack）です。0体から16体への増分は、FBX方式が+411.8MB（1体あたり約26MB）、Skin方式が+38.2MB（1体あたり約2.4MB）、VAT方式はほぼ±0で

した。



▲図3.5　オブジェクト数ごとのGPUメモリ確保量

この差は、1体増えたときに何が増えるかという構造の違いによるものです。FBX方式は、行列などの描画用データが破片の数だけGPUメモリに確保されます。Skin方式で増えるのは、スキニングの変形結果を書き出す頂点バッファが1体につき1本だけです。VAT方式は変形をその場で計算するだけで新しいバッファを持たないため、増えません。なお、VATのテクスチャは非圧縮で約6.6MBありますが、増分の差に対して十分小さい値です。

3.3.5　検証結果のまとめ

16体表示時の実測値を 表3.4 にまとめます。

表3.4: 16体表示時の実測値まとめ

指標	FBX方式	Skin方式	VAT方式
CPUメインスレッド（ms）	154.3	16.5	12.6
GPUフレーム（ms）	77.1	22.2	32.5
描画命令数（回/フレーム）	約210,000	VAT方式と同水準	99

表3.4: 16体表示時の実測値まとめ

指標	FBX方式	Skin方式	VAT方式
GPUメモリ増分（0→16体、MB）	+411.8	+38.2	ほぼ±0

FBX方式に対しては、Skin方式・VAT方式のどちらも全指標で大きく優位でした。要因は「1つのメッシュとして扱えること」に集約されます。

Skin方式とVAT方式の比較は一長一短で、GPU処理時間はSkin方式、CPUメインスレッドとメモリはVAT方式が有利です。VAT方式のGPU処理時間は、頂点変形の処理が多くかかるパス構成かどうかに大きく依存します。そのため、頂点計算を行うパス数が少ない構成や、CPU負荷の削減を目的とする場合は、VATが第一候補になります。

なお、今回の比較は剛体破壊という、ボーンで表現しやすい題材でした。布や人体のように、ボーン数が多くなるか近似処理になってしまうモデルや、オブジェクト数が多い場合はCPUの負荷が顕著になるため、よりVATが優位になると考えられます。

3.4 VATアセットの作成

VATアセットの作成には、Houdiniに付属するLabs Vertex Animation Textures ROP（以降、VAT ROP）を使用する方法が手軽です[*1]。HoudiniはSideFX社のDCCツールで、プロシージャル生成に強みがあります。VAT ROPにはUnity用のプリセットと対応シェーダーが用意されており、Houdini内のシミュレーション結果だけでなく、他のDCCツールで作成したFBXアニメーションも入力にできます。

VAT ROPには 表3.5 の4つのモードがあります。

表3.5: VAT ROPのモード

モード	用途
Soft-Body Deformation	基本的な頂点変形。トポロジーが変化しないことが条件
Rigid-Body Dynamics	剛体破壊に最適化。破片単位で動きを記録するためテクスチャサイズを小さく抑えられる

[*1] SideFX Labsパッケージに含まれるツールです。本章の検証にはバージョン3.0を使用しています。

表3.5: VAT ROPのモード

モード	用途
Dynamic Remeshing	液体や破ける布など、トポロジーが変化するものに対応
Particle Sprites	パーティクルシミュレーション用

手順は、Input GeometryにVAT化したいアニメーションのノードを、Modeに使用するモードを、Target EngineにUnityを指定し、Render Allボタンを押すだけです。表3.6のアセットが一括出力されます。

表3.6: VATの再生に必要なアセット

アセット	役割
ポジション用テクスチャ	各頂点の各フレームにおける位置（0〜1に正規化）
回転用テクスチャ	変形後の面の向き。法線・接線を再構築して陰影を保つ
頂点カラー用テクスチャ	Houdini側アトリビュートのベイク先。補間品質の向上などに使える自由枠
FBX	テクスチャの参照位置を示す識別番号がUVに埋め込まれたメッシュ
マテリアル	フレーム数・FPS・バウンディングボックスなどの補助データが設定済み
シェーダー	各モードに対応した再生計算の本体（別途インストール）

シェーダーは出力ファイルに含まれません。計算処理はアニメーションによらず固定のためです。UnityのPackage ManagerからSideFX Vertex Animation Texturesパッケージを別途インストールします（格納先はVAT ROPのReal-Time Shadersタブから確認できます）。パッケージには4モード対応のShader Graphのほか、ドキュメントとインポート設定用のPresetが同梱されています。

3.5 導入時の注意点

VATは精度が足りないとアニメーションが壊れてしまうため、テクスチャに適したフォーマットを指定する必要があります。同様に、FBX側にも推奨されるインポート設定があります。パッケージ同梱の「VAT Import Settings Guide」に従い、テクスチャ圧縮の無効化をはじめとする専用のインポート設定を適用しましょう。

また、公式のShader Graphはあくまでテンプレートという位置づけで、実用上はカスタマイズが必要です。テンプレートを直接利用するのではなく、プロジェクト用にコピーしてから使うことになります。ただし、コピー先のShader Graphはパッケージ側のSub Shader Graphを参照したままになるため、コピーした側へ参照を挿げ替えておく点に注意しましょう。

なお、出力ファイルのインポート設定や、マテリアルへのテクスチャ・シェーダーの割り当てといった参照設定は手作業で行う必要があり、設定漏れによるトラブルが起きやすい部分です。量産するプロジェクトでは、Scripted Importerなどでインポート設定と参照解決を自動化する余地があります。このあたりの改良についてはCEDiLにアップロードされている資料を確認してください[*2]。

3.6 まとめ

VATは、アニメーションをテクスチャに焼き込み、GPUで再生する手法です。ドローコールを大きく削減できるのが特徴で、破片ごとにRendererを持つ構成に対しては、Skin方式と並んで大幅に効率的でした。両者は一長一短で、GPU処理時間はSkin方式、CPU処理時間とメモリはVAT方式が有利です。頂点計算を行うパス数が少ない構成や、CPU負荷の削減が目的の場合は、VATが第一候補になります。専用アセットの準備や動きが固定されるといった制約はありますが、大量のオブジェクトに決まったアニメーションを再生させる場面では強力な選択肢になります。

[*2] https://cedil.cesa.or.jp/cedil_sessions/view/3263

第4章

ParticleSystemで長い時間Simulateをすると巨大なスパイクが発生する

加藤 有也 KATO YUYA

株式会社Colorful Palette

4.1　はじめに

UnityのParticleSystemにはパーティクルのシミュレーション時間を任意の時間で制御することのできるParticleSystem.Simulateが用意されています。しかし、ParticleSystem.Simulateに大きな秒数を指定するとシミュレーションを開始した際に大きなスパイクが発生してしまうケースがあります。本記事では、その具体的な事例と注意点について紹介します。

4.2　ParticleSystem.Simulateとは？

ParticleSystem.Simulateについての公式ドキュメント[*1] ParticleSystem.Simulateを使用すると第1引数で指定した時間までパーティクルの演出をシミュレーションし、指定した時間からパーティクルを再度再生することが可能です。

▼リスト4.1　0秒時点から再生し直す

```
ParticleSystem.Simulate(0f, false, true);
```

[*1] https://docs.unity3d.com/ja/2022.3/ScriptReference/ParticleSystem.Simulate.html

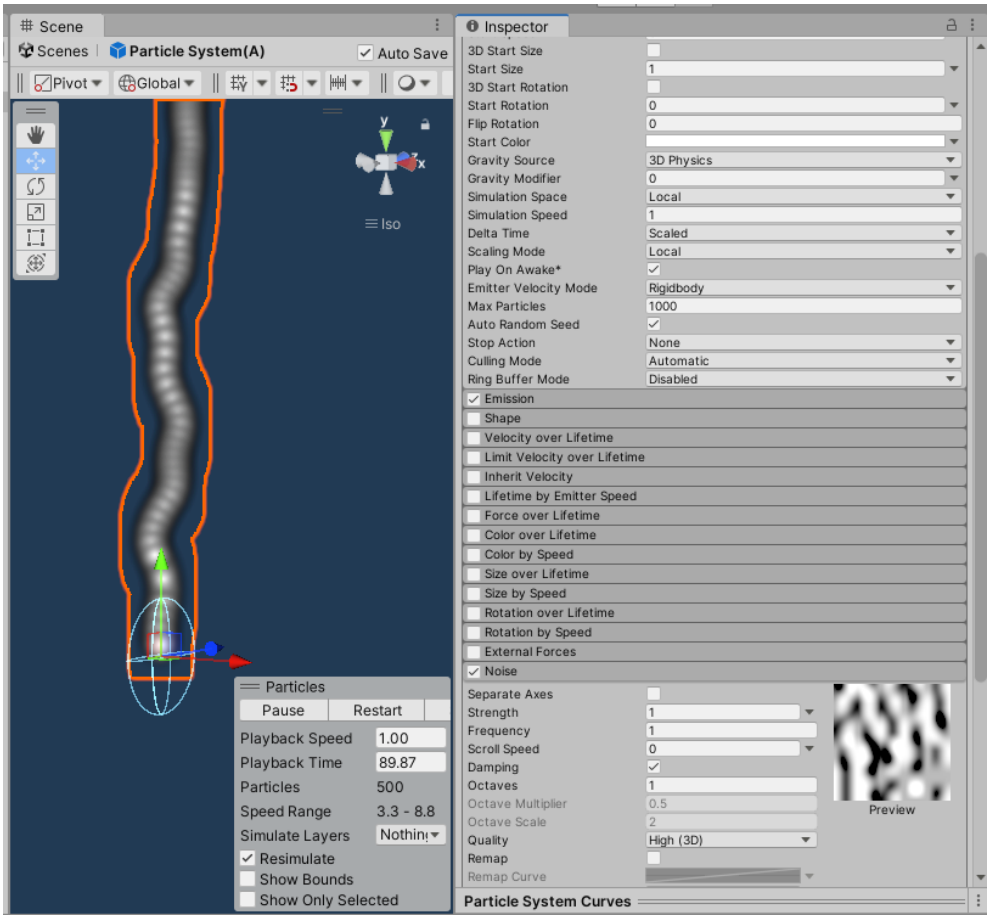
リスト4.1のように0秒を指定すると途中まで再生していたパーティクルを最初から再生し直す仕組みを作れます。

▼リスト4.2　5秒時点から再生し直す

```
ParticleSystem.Simulate(5f, false, true);
```

リスト4.2のように5秒を指定すると再生中のパーティクルを5秒時点から再生し直す仕組みを作れます。

4.3 問題がある使用方法



▲図4.1　問題のあるパーティクル

Unityのデフォルトで作成できるパーティクルに追加でNoiseモジュールをオンにします。他に調整しているパラメータはパーティクルを見やすくするためにEmissionモジュールで一直線になるように発生を調整しています。

▼リスト4.3　20秒時点から再生し直す

```
ParticleSystem.Simulate(20f, false, false);
```

そしてこのようなNoiseモジュールを使用したパーティクルをリスト4.3のように第1引数の秒数指定に大きすぎる秒数を入れてシミュレーションしてしまうと処理中に大きなスパイクが発生します。

▼リスト4.4　シミュレーションの負荷計測

```
void OnClickA()
{
    // 再生中のパーティクルがあれば破棄
    if (_activeParticle != null)
    {
        Destroy(_activeParticle);
        _activeParticle = null;
    }

    // 新規生成して再生
    _activeParticle = Instantiate(particlePrefab, Vector3.zero, Quaternion.identity);
    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        ps.Play();
    }
}

void OnClickB()
{
    if (_activeParticle == null) return;

    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        Profiler.BeginSample("再シミュレーション");
        // 再シミュレーション
        ps.Simulate(20f, false, false);
        Profiler.EndSample();

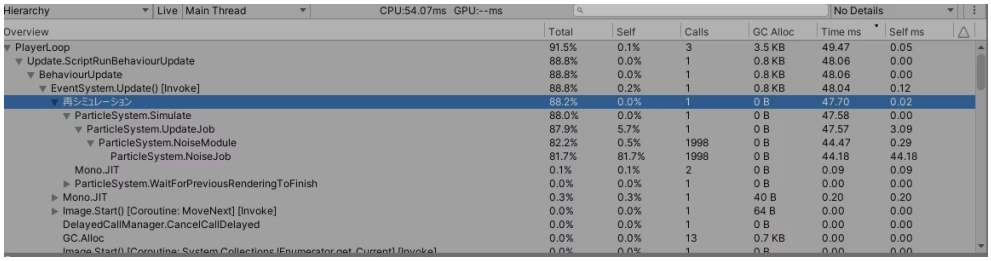
        ps.Play();
    }
}
```

どれほどの負荷がかかるのか、Profilerで確認します。ButtonAで作成したパーティクルを生成、ButtonBでパーティクルをシミュレートするコードです。UnityのPro-

filerで確認した時にParticleSystem.Simulateに問題があるのか調べやすくするためにProfiler.BeginSample、Profiler.EndSampleで囲い、調査しやすくします。



▲図4.2　スパイクが発生している様子



▲図4.3　処理時間

このようにParticleSystem.Simulate時、ParticleSystemのNoiseモジュールでNoise Jobが約2000回コールされ、これにより約50msのスパイクが発生しています。このコール数はParticleSystemのMax Particlesを下げることで少しは減らすことが可能です。

4.4　おわりに

ParticleSystem.Simulateを使用して任意の秒数からパーティクルを発生させる仕組みを作る必要がある場合、

- 長い時間を指定して途中からパーティクルを発生させる場面があるのか？
- Noiseモジュールを使用しているのか？

を確認し、特にNoiseモジュールを有効にしているパーティクルを使用する場合はParticleSystem.Simulateの利用を控えた方が良いです。

第5章

UniTaskとキャンセル処理のライフタイム管理

稲田 健人 Inada Taketo

株式会社サイバーエージェント SGEコア技術本部

UniTaskを用いた非同期処理では、終了条件やキャンセルを適切に管理しないと、完了しない処理や不要な参照が残り続けることがあります。

前作『Unity パフォーマンスチューニング バイブル』では、MonoBehaviourが破棄された後もUniTaskが待機し続ける事例を紹介しました。本節では、CancellationTokenSourceの破棄漏れによって、不要なオブジェクトが参照され続ける事例を紹介します。

なお、CancellationTokenSourceやCreateLinkedTokenSourceはUniTask固有の機能ではなく、.NETのキャンセル処理を構成するAPIです。

本節で扱うコードのうち、using宣言（using var）はC# 8の構文で、Unity 2020.2以降で利用できます。また、後述するCreateLinkedTokenSourceの内部実装は、Unity TechnologiesがUnity 2022.3向けMono実装（unity-2022.3-mbeブランチ）を参照したもので、Unityのバージョンによって変わる可能性があります。

5.1 CancellationTokenSourceのDispose

CancellationTokenは、処理へキャンセル要求を伝えるための値型です。CancellationToken自体はIDisposableを実装していないため、Disposeを呼び出す必要はありません。

ただし、CancellationToken.Registerでコールバックを登録した場合、その戻り値で

あるCancellationTokenRegistrationはIDisposableです[*1]。キャンセル前に登録されたコールバックは発行元のCancellationTokenSourceに保持されるため、登録が不要になった時点でDisposeして解除しないと、コールバックが参照するオブジェクトが解放されないことがあります。

一方、CancellationTokenSourceはIDisposableを実装しています。使用を終えたCancellationTokenSourceは、所有者がDisposeを呼び出します。

▼リスト5.1　CancellationTokenSourceをDisposeする例

```
private async UniTask LoadCancelableAsync()
{
    using var cts = new CancellationTokenSource();

    // キャンセル要求を行う処理は省略
    await LoadAsync(cts.Token);
}
```

リスト5.1では、using宣言によってCancellationTokenSourceを管理しています。非同期処理が正常終了した場合だけでなく、キャンセルや例外によって終了した場合にも、メソッドを抜ける際にcts.Dispose()が自動的に呼び出されます。

CancelとDisposeの役割は異なります。Cancelは処理へキャンセルを要求し、DisposeはCancellationTokenSourceが保持するタイマーやコールバック登録などを解除・解放します。Disposeを呼び出しただけでは、処理にキャンセル要求は通知されません。

なお、Disposeを呼び忘れても、CancellationTokenSourceと関連するオブジェクトがすべて外部から参照されなくなれば、GCの回収対象になります[*2]。ただし、長寿命のCancellationTokenSourceへのコールバック登録などによって参照が残る場合がある

[*1] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokenregistration.dispose（参照日：2026年9月8日）

[*2] https://docs.unity3d.com/ja/2022.3/Manual/performance-managed-memory.html（参照日：2026年9月8日）

ため、使用を終えた時点でDisposeすることを原則としましょう。

5.2 CancellationTokenSource.CreateLinkedTokenSource

CancellationTokenSource.CreateLinkedTokenSourceは、複数の元トークンを監視する新しいCancellationTokenSourceを生成します。元トークンのいずれかがキャンセルされると、生成されたCancellationTokenSourceのTokenがキャンセル状態になります。

元トークン同士がキャンセルされるわけではなく、キャンセルは元トークンからリンク先へ伝播します。

以下では、tokenAとtokenBの発行元となるCancellationTokenSourceを、それぞれA、Bとします。

Unity TechnologiesがGitHubで公開しているUnity 2022.3向けMono実装[*3]では、CreateLinkedTokenSourceは元トークンにキャンセルコールバックを登録します。参照関係に関わる部分を簡略化すると、リスト5.2のようになります。

▼リスト5.2 CreateLinkedTokenSourceの内部実装の要点

```
linkedCts.m_linkingRegistrations[0] =
    tokenA.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);

linkedCts.m_linkingRegistrations[1] =
    tokenB.InternalRegisterWithoutEC(
```

[*3] https://github.com/Unity-Technologies/mono/blob/unity-2022.3-mbe/mcs/class/referencesource/mscorlib/system/threading/CancellationTokenSource.cs（参照日：2026年9月8日）

```
        s_LinkedTokenCancelDelegate,
        linkedCts);
```

元トークンに登録されたコールバックからlinkedCtsが参照され、linkedCtsもAとBへの登録情報を保持します。

そのため、Bがアプリケーション全体で使用される長寿命のCancellationTokenSourceであり、長期間キャンセルされない場合、linkedCtsをDisposeしないと次の参照が残ることがあります。

長寿命のB → コールバック登録 → linkedCts → Aへの登録情報 → 短命なA

この参照関係だけで必ずメモリリークするわけではありません。すべてのオブジェクトが外部から参照されなくなれば、まとめてGCの回収対象になります。しかし、Bが生存し、リンク用のコールバックが解除されていない間は、linkedCtsはBから参照され続け、そこから短命なAも到達可能なまま残ります。

また、linkedCts.Tokenに登録されたコールバックがほかのオブジェクトを参照している場合、それらも保持されることがあります。

linkedCtsをDisposeすると[*4]、内部でAとBへのCancellationTokenRegistrationが破棄され、元トークンに登録されたコールバックが解除されます。

CreateLinkedTokenSourceの戻り値は、リンクしたトークンを使用する処理のスコープ内で、using宣言などにより管理するとよいでしょう。

▼リスト5.3　リンクしたCancellationTokenSourceをDisposeする例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
```

[*4] https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokensource.dispose（参照日：2026年9月8日）

```
{
    using var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

リスト5.3では、LoadAssetAsyncが正常終了、キャンセル、または例外によって終了した後にlinkedCts.Dispose()が呼び出され、元トークンへの登録が解除されます。

リンクされたCancellationTokenSourceがすでにキャンセル状態であっても、ほかの元トークンへの登録が残っている場合があります。CreateLinkedTokenSourceの戻り値は、不要になった時点で必ずDisposeしましょう。

5.3 IDisposableオブジェクトのDispose漏れをチームで防ぐ

ここまで、CancellationTokenSourceを例にIDisposableの破棄漏れを見てきました。対策は、オブジェクトの所有者が適切なタイミングでDisposeを呼び出すことを前提としています。しかし、プロジェクトの規模が大きくなると、すべての生成箇所をコードレビューだけで確認し続けることは困難です。特にCreateLinkedTokenSourceは短い記述で生成できるため、戻り値がIDisposableであることを見落としやすくなります。

チームで再発を防ぐには、個人の注意に加えて、破棄が必要なオブジェクトを扱いやすいAPIに寄せること、Analyzerで問題を早期に検知すること、必要に応じてCIで新しい違反を検出することを組み合わせるとよいでしょう。Analyzerだけに責務を集中させず、設計、静的解析、運用のそれぞれから同じ方針を支えると効果的です。

ここで紹介するusingによる破棄とAnalyzerによる検出は、CancellationTokenSource

に限らず、IDisposableを実装するほかのリソースにも応用できます。

本節では、こうした仕組み化のうち、C#コードを自動的に検査できるRoslyn Analyzerを使った方法を紹介します。

5.4 Roslyn Analyzerとは

Roslyn Analyzerは、C#コードをコンパイル時やIDE上で静的に解析し、問題の可能性がある箇所へ警告や提案を表示する仕組みです。構文、型情報、処理経路などを基にバグの兆候やコーディング規約違反を検出でき、Code Fixを用意すれば修正候補も提示できます。

Analyzerが何を問題として検出するかは診断ルールとして定義され、各ルールにはCA2000のような識別用の診断IDが付けられます。CA2000は.NETが提供する既存ルールで、後述するUTK001とUTK002はプロジェクト独自に追加するルール案です。

Unityプロジェクトでは、AnalyzerのDLLをAssets以下へ配置し、RoslynAnalyzerラベルを付与して導入します[*5]。asmdefを利用して適用対象のアセンブリを限定することもできます。

5.5 既存のAnalyzerを最初の防波堤にする

まず検討したいのは、.NETのコード品質規則に含まれるCA2000です[*6]。CA2000は、ローカルで生成したIDisposableのオブジェクトが、破棄されないままスコープを外れる経路を検出します。CancellationTokenSourceもIDisposableであるため、new CancellationTokenSource()の破棄漏れを検出する最初の防波堤として利用できます。

ただし、利用しているAnalyzerのバージョンや解析設定によって、CreateLinkedTokenSourceのようにオブジェクトを生成して返すメソッドの戻り値やメソッド間の所有権移譲をどこまで追跡できるかは異なります。導入時には、new CancellationTokenSource()とCreateLinkedTokenSourceの両方について、検出される例と検出されない例を小さ

[*5] https://docs.unity3d.com/ja/2022.3/Manual/roslyn-analyzers.html（参照日：2026年9月8日）

[*6] https://learn.microsoft.com/ja-jp/dotnet/fundamentals/code-analysis/quality-rules/ca2000（参照日：2026年9月8日）

な検証プロジェクトで確認するとよいでしょう。既存規則で十分に検出できないパターンは、必要に応じて後述する独自規則で補えます。

5.6 非同期処理向けの独自ルール設計例

既存AnalyzerでCreateLinkedTokenSourceの戻り値を十分に追跡できない場合は、プロジェクト固有のRoslyn Analyzerで補う方法があります。以下は、独自Analyzerを実装する場合のルール設計例です。説明のために、仮の診断IDとしてUTK001とUTK002を使用します。

UTK001では、new CancellationTokenSource()またはCancellationTokenSource.CreateLinkedTokenSource()で生成されたローカル変数が、using文、using宣言、またはfinally節のDisposeによって保護されているかを検査することを想定します。すべての正常終了経路だけでなく、returnや例外でメソッドを抜ける経路も解析対象とします。

▼リスト5.4 UTK001が検出する例

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

リスト5.4では、linkedCtsがDisposeされないままメソッドを抜けます。この規則で

は、CreateLinkedTokenSourceの呼び出し位置へ警告を出し、using宣言またはtry-finallyで破棄を保証するよう案内することを想定します。

UTK002では、CancellationToken.Registerの戻り値であるCancellationTokenRegistrationが、そのまま破棄されていないかを検査することを想定します。Registerの戻り値を受け取らずに捨てるコードは、登録解除の責務が不明確です。コールバックを非同期処理の間だけ有効にするなら、using宣言で管理するとよいでしょう。フィールドへ保持する場合は、所有者の終了処理でDisposeします。

▼リスト5.5　CancellationTokenRegistrationを管理する例

```
private async UniTask WaitAsync(CancellationToken token)

{

    using var registration = token.Register(OnCanceled);



    await WaitForSignalAsync(token);

}
```

リスト5.5では、WaitForSignalAsyncが終了した時点でregistrationが破棄されます。一方、アプリケーション全体で共有するTokenへ永続的に登録するなど、登録とTokenの寿命を一致させる設計もあります。そのような例まで一律に警告すると誤検知が増えるため、UTK002はまず情報レベルで試し、プロジェクトで頻出する安全なパターンを除外できるようにするとよいでしょう。

フィールドに保持されたCancellationTokenSourceは、ローカル変数よりも所有権の判定が難しくなります。第二段階の規則として、フィールドへ新しいCancellationTokenSourceを代入する前に古い値をDisposeしているか、MonoBehaviourのOnDestroyなど所有者の終了処理でDisposeしているかを検査できます。ただし、別オブジェクトへの所有権移譲を伴う場合があるため、ローカル変数の規則とは分けて導入するとよいでしょう。

5.7 Code Fixで修正方法を揃える

Roslyn Analyzerには、検出した問題をIDE上で自動修正するCode Fixも用意できます。たとえば、UTK001で所有権がメソッド内に完結する単純なケースに、varをusing varへ変換するCode Fixを提供すると、安全に機械変換できるパターンの修正方法をチーム内で統一しやすくなります。フィールドへの代入や戻り値による所有権移譲など、破棄タイミングを一意に決められないケースには自動修正を提示せず、設計の確認を促す診断に留めるとよいでしょう。

コードレビューではAnalyzerの結果だけでなく、生成、Cancel、Disposeの担当がコードから読み取れるかも確認するとよいでしょう。Analyzerに警告がないことだけでは、安全性を証明できない点にも注意が必要です。

5.8 Analyzerだけでは検知できない問題

静的解析には限界があります。CancellationTokenSourceをコレクションへ格納した場合や、別のオブジェクトへ渡して所有権を移した場合などは、最終的な破棄箇所を正確に判断できないことがあります。誤検知を避けるために解析を弱くすると、今度は検出漏れが増えます。

この領域は、Development Buildやテスト環境での実行時検証によって補う方法があります。プロジェクト共通の生成用の仕組みや、寿命を管理する型を経由してCancellationTokenSourceを生成し、生成数と破棄数を記録すれば、シーン遷移やテスト終了時に未破棄のスコープが残っていないかを確認できます。生成時のスタックトレースも記録する場合は、負荷を考慮して開発環境だけで有効にするとよいでしょう。

必要に応じて、Profilerでメモリの増加を確認する調査、実行時の未破棄検知、Roslyn Analyzerによる編集時の警告を組み合わせるとよいでしょう。問題が発生してから担当者が調査する流れだけでなく、コードを書いた時点で修正方法まで提示できる流れにすると、チームとしての再発防止につながります。

5.9 参考資料

Unity Manual『Roslynアナライザーとソースジェネレーター』

Microsoft Learn『CA2000: スコープを失う前にオブジェクトを破棄』

Unity Technologies『Mono CancellationTokenSource.cs（unity-2022.3-mbeブランチ）』

Microsoft Learn『CancellationTokenSource.Dispose Method』

Microsoft Learn『CancellationTokenRegistration.Dispose Method』

Unity Manual『マネージメモリ』

第6章

UIシステムの使い分け － IMGUI / uGUI / UI Toolkit

財津 大睦 Zaitsu Hiromu

株式会社サムザップ

X @RomGameclient

前作『Unity パフォーマンスチューニング バイブル』[*1]では、Canvasの分割やRaycast Targetの調整といった、uGUIの負荷への対処を扱いました。本章では一歩さかのぼり、UIシステムの選択そのものを扱います。どれを選ぶかが、UIの負荷構造と取れる対処の種類を決めるからです。UnityにはIMGUI、uGUI、UI Toolkitの3つのUIシステムがあります。この3つを順に取り上げ、それぞれがランタイムとエディター拡張でいまどう位置づけられているかを整理します。本章の記述はUnity 6.5時点の情報に基づきます。

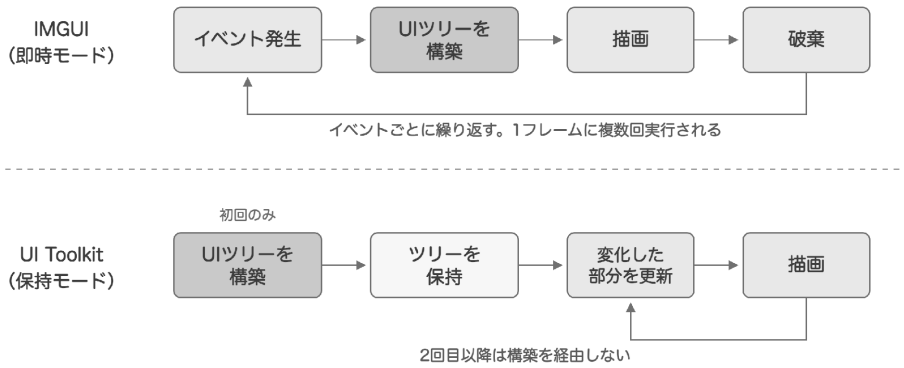
6.1 IMGUI

ランタイムのIMGUIは、ゲーム内UIの選択肢から外れています。公式マニュアルは、プレイヤーが操作する通常のゲーム内UIに使うことを意図していないと明記しており、想定用途としてゲーム内のデバッグ表示とツール、カスタムインスペクター、エディター拡張を挙げています。デバッグオーバーレイやランタイムのログビューアという用途では、現在も使用されています。一方、エディター拡張においてIMGUIは公式に現役です。公式のUIシステム比較ページ[*2]はIMGUIを代替手段として挙げ、利点としてエディターの拡張機能に制限なくアクセスできることと、画面にすばやくUIを描画できる

[*1] https://cyberagentgameentertainment.github.io/UnityPerformanceTuningBible/ja/preface.html

[*2] https://docs.unity3d.com/6000.5/Documentation/Manual/UI-system-compare.html （2026年8月6日参照）

軽量なAPIであることの2点を示しています。公式のIMGUI移行ガイド[*3]には、UI Toolkitに等価物がないAPIが明記されています。実務で影響が大きいのは InspectorTitlebar()、Toolbar()、IntPopup()、LinkButton() あたりです。いずれもエディターウィンドウの見た目をUnity本体に寄せるためのAPIで、必要な場面では現状IMGUIを使うことになります。ただし、エディター拡張として使ううえで知っておきたい負荷特性があります。IMGUIは OnGUI() の中にUIを毎回記述する即時モードのシステムで、UIツリーの情報を保持せず、イベントのたびに構築と破棄を繰り返します。保持モードであるUI Toolkitとの違いは 図6.1 のとおりです。OnGUI() は1フレームに複数回実行され、そのたびにヒープ確保が発生しやすくなります。常時再描画されるエディターウィンドウでは、これがGCへの負荷になります。



▲図6.1　即時モードと保持モードの描画サイクル

6.2　uGUI

CanvasとGameObjectをベースにしたシステムです。要素ひとつひとつがGameObjectであり、Transform階層を持ちます。ランタイム専用で、エディター拡張には使えません。ランタイムでは事実上の標準です。利点は3つあります。・Sceneビューで直接配置して、その場でプレビューできる・AnimatorとTimelineでUIを動かせる・

[*3] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-IMGUI-migration.html

Asset StoreにuGUIを前提としたパッケージが大量にある負荷の面では、要素に変更があったCanvasではメッシュの再構築が走り、要素数に比例してTransform階層の更新やRaycast Targetの走査のコストが乗ります。前作で扱ったCanvasの分割やRaycast Targetの調整は、いずれもこの構造に由来する負荷への代表的な対処です。

6.3 UI Toolkit

VisualElement のツリーを一度構築し、以後は変化した部分だけを更新する保持モードのシステムです。ロジックをC#、UI構造をUXML、スタイルをUSSに分けて記述します。ランタイムとエディター拡張の両方で使え、エディター拡張では公式の第一推奨です。ただし、公式は「複雑なエディターツールを作るなら」という書き方をしています。数個のフィールドを並べるだけのツールであれば、UXMLとUSSを持ち込まずC#だけで組んでも困りません。負荷の面では、uGUIとの違いがそのまま対処の違いになります。要素をひとつ変えるとCanvas全体のメッシュが作り直されるuGUIと違い、UI Toolkitは変化した部分だけを更新するため、Canvasの分割にあたる対処は不要です。ただし要素数が増えればレイアウト計算のコストは同じように増えるので、要素を減らす工夫は引き続き必要になります。Unity 6系では、イベントディスパッチの高速化やジオメトリ生成のジョブ化、レイアウト計算のキャッシュ改善といった性能改善も続いています。

6.3.1 uGUIと比べて不足している点

Unity 6.5時点でできないこととしては、次のようなものが挙げられます。いずれもUnityが開発中としている項目ですが[*4]、いま採用を判断するなら、できない前提で考える必要があります。・VisualElement のキーフレームアニメーションをAnimationウィンドウで扱う機能・SceneビューでのUI編集・UXMLで定義した VisualElement をMonoBehaviourスクリプトから参照する仕組み・カスタムシェーダー向けの頂点データ対応エコシステムも薄いままです。Asset Storeの資産はGameObject階層を前提にしたものが多く、そのままでは動きません。TextMeshProもUI Toolkitからは使えず、UI Toolkitは専用のテキストシステム（6.5で既定になったAdvanced Text Generator）を使います。DOTweenも VisualElement 用のショートカットを追加していますが、対象はUnity 6.3以降です[*5]。uGUIから移行する場合、アンカーとピボットに直接の等価物は

[*4] https://discussions.unity.com/t/ui-toolkit-development-status-and-next-milestones-november-2025/1698009

[*5] https://dotween.demigiant.com/download.php

ありません。UI Toolkitのレイアウトは、親要素の中に子要素を順に流し込んで並べる方式です。要素ごとに位置とサイズを指定していたuGUIとは、組み立て方そのものが変わります。

6.4 公式の推奨の変遷

公式のUIシステム比較ページはマニュアルのバージョンごとに公開されており、読み比べるとランタイムUIの推奨理由が入れ替わっています[*6]。その内訳を 表6.1 に示します。

表6.1: ランタイムUIの推奨理由の変化

要件・用途	Unity 2022.3〜6.1	Unity 6.5
大量のUIを扱う	UI Toolkit	UI Toolkit
アーティストとデザイナー向けの制作ワークフローが必要	UI Toolkit	UI Toolkit
テクスチャレスなUI描画	UI Toolkit	UI Toolkit
3D空間に配置・ライティングされるUI	uGUI	UI Toolkit
カスタムシェーダーとマテリアルによる表現	uGUI	UI Toolkit
MonoBehaviourからの参照が容易	uGUI	uGUI

2022.3から6.1まで一貫してuGUI側にあった2つは[*7]、ワールドスペースUIが6.2、カスタムシェーダーが6.3でURP向けに実装されたことでUI Toolkit側へ移りました。残る「MonoBehaviourからの参照が容易」も、UXMLで定義した要素を参照する仕組みが開発中の項目に入っています。ただし、比較ページの構成そのものはUnity 6.5でも変わっていません。ランタイムUIはuGUIを既定として紹介し、UI Toolkitをその代替として並べる形のままです。入れ替わったのは推奨の理由だけで、どちらを既定に置くかは動いていません。同じページに載っている用途別の一覧も、見出しが2022.3から6.2までは「Recommendation（推奨）」、6.3以降は「Often used for（よく使われる）」に変わりました。Unity自身、これを推奨とは呼ばなくなっています。

[*6] https://docs.unity3d.com/6000.0/Documentation/Manual/UI-system-compare.html （2026年8月6日参照）

[*7] https://docs.unity3d.com/2022.3/Documentation/Manual/UI-system-compare.html （2026年8月6日参照）

6.5 UIシステムを併用するときの制約

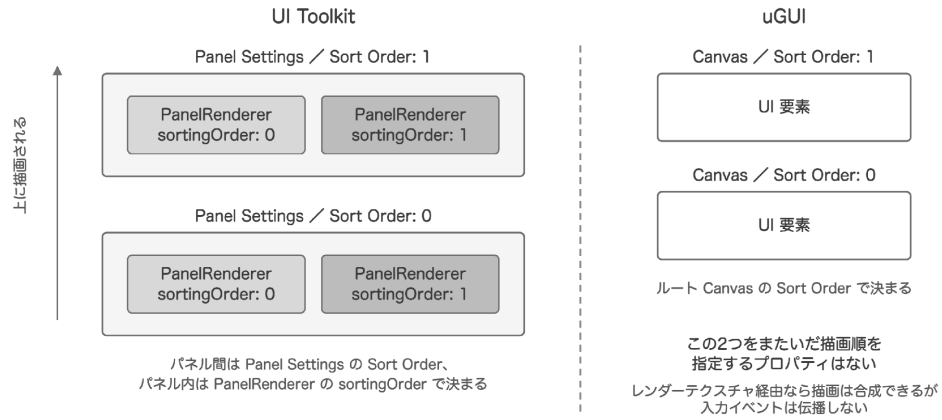
1つのプロジェクトで複数のUIシステムを使うこと自体はできますが、組み合わせるといくつか問題が出ます。ランタイムとエディター拡張に分けて見ていきます。

6.5.1 ランタイム

uGUIとUI Toolkitは同じプロジェクトで使えますが、一方を他方の中に埋め込むことはできません。ナビゲーションとスタイルの不整合も、公式が制約として挙げています。描画順は、それぞれのシステムの中でしか制御できません（図6.2）。UI ToolkitでUIを画面に出すには、GameObjectに Panel Renderer コンポーネントを付けます。この役割はUnity 6.4以前では UIDocument が担っており、6.5で置き換わりました[*8]。LTSの6.3を使っている場合は UIDocument になります。この Panel Renderer が並び替えの単位で、パネル同士はPanel SettingsのSort Orderで、同じパネル内の Panel Renderer 同士は PanelRenderer.sortingOrder で順序が決まります。uGUIはルートCanvasのSort Orderで並べます。この2つをまたいだ順序を指定するプロパティはありません。uGUIからの移行ガイド[*9]はレンダーテクスチャで描画を合成する手も挙げていますが、入力イベントは届かないと明記しています。操作するUIには使えないため、uGUIとUI Toolkitを重ねる設計は避けてください。

[*8] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-create-ui-document-component.html

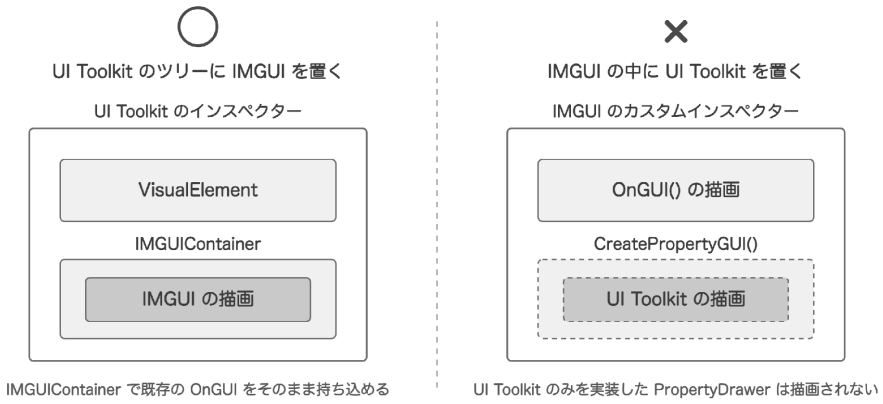
[*9] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-Transitioning-From-UGUI.html



▲図6.2　UI ToolkitとuGUIの描画順の決まり方

6.5.2　エディター拡張

エディター拡張では、2つのシステムを入れ子にできる向きが片方だけに限られます（図6.3）。UI Toolkitのツリーの中にIMGUIの描画を置くことは IMGUIContainer でできますが、その逆、IMGUIの描画の中にUI Toolkitのツリーを置くことはできません。これが問題になるのは、インスペクターの表示を自作するときです。インスペクターのカスタマイズは、コンポーネント全体を受け持つカスタムインスペクターと、フィールド単位で見た目を差し替える PropertyDrawer の2段になっており、内側の PropertyDrawer は自分がどちらのシステムのインスペクターから呼ばれるかを選べません。UI Toolkitだけで実装した PropertyDrawer が、IMGUIで書かれたカスタムインスペクターや、IMGUIの PropertyDrawer の中から呼ばれると、そのフィールドは値の欄が「No GUI Implemented」（GUIが実装されていない）という表示に置き換わります。Unity 2022.2以降はインスペクターの既定がUI Toolkitなので多くの場合は起きませんが、IMGUIのカスタムインスペクターやPropertyDrawerが残っているプロジェクトでは、PropertyDrawer側にIMGUI版の描画も用意しておく必要があります。



▲図6.3　PropertyDrawerの入れ子の可否

6.6　おわりに

調べてみて印象的だったのは、公式の推奨理由がUnity 6.5でほぼUI Toolkit側へ移っていたことです。ランタイムはuGUIが標準のままで、動いているものを移す理由はありませんが、新規開発でUI Toolkitを避ける理由は確実に減っています。迷ったら、AnimatorとTimelineの演出とAsset Storeの資産にどれだけ依存しているかで判断するのがよさそうです。エディター拡張は、複雑なツールはUI Toolkit、Unity本体の見た目に寄せる場面はIMGUIという整理で当面は困らないと感じました。UIシステムの選択は、前作で扱った対処が必要になるかどうかを決める、いちばん上流のチューニングです。

Unity®

PERFORMANCE TUNING TIPS 2026

CHAPTER 03

第3部

アセット・設定・ビルド最適化

CyberAgent Smartphone Games & Entertainment

2026年のアセット・設定・ビルド動向

パフォーマンスチューニングは、自分たちのコードやアセットを直すことだけではありません。同じコードとアセットでも、エンジンのバージョンや設定ひとつで、ビルド時間、アプリ容量、メモリ使用量、実行速度は変わります。エンジン側ですでに解決された問題を自前の仕組みで抱え続けていないか、新しく追加された設定を知らないまま既定値で使っていないか。それを確かめるためにも、エンジン自体の変化を追っておくことはチューニングの前提になります。

Unityにとっても、パフォーマンスはエンジンとして優先度の高い領域です。Unity 6世代の各リリースのWhat's Newには、新機能と並んで、ビルド時間の短縮、プレイヤーの容量削減、メモリ管理の改善が毎回挙がっています。第3部で扱うアセット・設定・ビルドは、その変化がとくに大きい領域です。

執筆時点の2026年9月では、長期サポート版の最新はUnity 6.3 LTS、正式版の最新はUnity 6.6です。Unity 6.7はLTSとなる予定のバージョンとしてベータ版が公開されており[*10]、Unity 7は2026年12月に初期のベータ版、2027年第1四半期に正式版のリリースが予定されています[*11]。以降では、Unity 6.0から6.7までの変化をリリースの順にたどり、同じバージョンの中では効果の大きいものから取り上げます。第3部の各章で詳しく扱う変化は要点だけにとどめ、該当する章を示します。なお、本文で触れるUnityのバージョンや既定値は執筆時点のものであり、以降のリリースで変わる可能性があることをご了承ください。

[*10] Unity Discussions, Unity 6.7 Beta is now available `https://discussions.unity.com/t/unity-6-7-beta-is-now-available/1736830`

[*11] Unity, Unity 7 roadmap revealed at Unite Seoul `https://unity.com/news/unity-7-roadmap-revealed-at-unite-seoul`

Unity 6.0

iOSのメモリ上限をAPIで設定する

Increased Memory LimitとExtended Virtual Addressingは、アプリが使えるメモリの上限そのものに関わるiOSのエンタイトルメントです。以前のUnityでも、書き出したXcodeプロジェクトのEntitlementsファイルを手で編集するか、ビルド後処理で書き換えれば設定できました。Unity 6000.0.11f1以降では `ProjectCapabilityManager` に専用のAPIが追加され、Entitlementsへの追加とXcodeプロジェクトへの反映をまとめて行えます。それぞれの役割と実機での上限の確認方法は、第3部6章「iOSのメモリ上限と仮想アドレス空間」で扱います。

Build Profilesで構成ごとに設定を持つ

Unity 6.0で導入されたBuild Profilesは、1つのプラットフォームに対して複数のビルド構成を持てる仕組みです。導入時点から、プロファイルごとにシーンの一覧、Player Settingsの上書き、スクリプトのDefine Symbolsの追加を持てます。Unity 6.1からはGraphics設定とQuality設定をプロファイルごとに上書きできるようになり[*12]、6.3では上書きする項目を選んで追加する形になりました。6.5では `BuildProfile.CreateBuildProfile` でスクリプトからプロファイルを作成することもできます。

パフォーマンスの面での利点は、プラットフォームや配信先ごとの設定の違いを、ビルド前に設定を書き換えるスクリプトではなく、プロファイルとして固定しておける点です。たとえばDefine Symbolsを分ければ、デバッグ用の計測やログのコードをリリース用のプロファイルからだけ除外でき、Quality設定を上書きすれば、ローエンド向けの構成ではビルドに含める品質レベルそのものを絞り込めます。第3部3章「端末に応じた描画設定の切り替え」は、実行時に端末の性能に合わせて描画負荷を振り分ける設計を、Quality Levelを切り替える形式とRender Pipeline Assetの値を直接書き換える形式で比べます。Build Profilesは、それをビルドの時点で切り替える手段として組み合わせられます。

[*12] Unity Manual 6000.1, What's new in Unity 6.1 `https://docs.unity3d.com/6000.1/Documentation/Manual/WhatsNewUnity61.html`

Managed Stripping LevelのLowが非推奨に

Unity 6.0のマニュアルでは、Managed Stripping LevelのLowが将来の廃止対象とされ、MinimalかMediumを使うよう案内されています[*13]。Lowを使っていたプロジェクトでは、Mediumへ上げてバイナリを小さくするか、Minimalへ下げるかを選ぶことになります。Mediumへ上げるとリフレクションで参照されるコードが削られやすくなり、その対処は第3部4章「Unity Managed Code Stripping 実践ガイド」で扱います。

Unity 6.1

端末ごとにGraphics Jobsのモードを選ぶ

Unity 6.1で追加されたVulkan Device Filter Assetでは、Androidの端末ごとにGraphics Jobsのモードを指定できます。特定のGPUやドライバーでだけ遅くなる、あるいは不安定になる設定を、ほかの端末の設定を変えずに避けられます。

Unity 6.2

インポート時に自動生成されるMesh LOD

Unity 6.2で追加されたMesh LODは、モデルのインポート設定でGenerate Mesh LODsを有効にすると、インポート時にLODを自動生成する機能です[*14]。LODを用意していなかったモデルでも、設定ひとつで遠景のポリゴン数を減らし、GPU負荷を下げられます。すでにLOD Groupを使っている場合、ポリゴンを減らす効果は変わらず、差が出るのはメモリです。Mesh LODは頂点を追加せず、すべてのLODが元のメッシュの頂点バッファを共有して、インデックスバッファだけが増えます。LODごとにメッシュ全体とGameObjectを持つLOD Groupより、メモリの増加を小さく抑えられます。

ただし、自動生成されるLODの品質は元のメッシュの形状に左右され、手作業で作ったLODと同じ品質は期待できません。公式ドキュメントでも、生物や岩のような有機的な形状には向く一方、曲率の高い形状や離れたパーツの多いメッシュでは、テクス

[*13] Unity Manual 6000.0, Configure managed code stripping `https://docs.unity3d.com/6000.0/Documentation/Manual/managed-code-stripping-configure.html`

[*14] Unity Manual 6000.6, Introduction to Mesh LOD `https://docs.unity3d.com/6000.6/Documentation/Manual/lod/mesh-lod-introduction.html`

チャのずれや、樹木の葉が消えるといった崩れが起きる例が示されています。生成されたLODはInspectorで段階ごとに確認し、崩れる段階からはLimit LODsで生成を止めるか、そのメッシュだけLOD Groupに戻すといった判断が必要です。

また、減らせるのはポリゴンだけで、マテリアルは変わりません。Particle System、VFX Graph、Static Batchingでは常にLOD0が使われるため効果がなく、Skinned Mesh Rendererでは、どのLODが選ばれていても変形の計算はLOD0に対して行われます。LODのクロスフェードにはGPU Resident Drawerが必要で、LOD Groupとの併用は推奨されていません。

Unity 6.3

シェーダーバリアントを設定で減らす

シェーダーバリアントはキーワードの組み合わせごとにコンパイルされるため、キーワードが増えるほど掛け算で増えていきます。バリアントを減らせば、ビルド時間、ビルド容量、シェーダーの読み込み時間、実行時のメモリがまとめて減ります。

Unity 6.3で追加されたShader Build Settingsでは、シェーダーのソースを書き換えずに、キーワードの種類を `shader_feature` や `dynamic_branch` に上書きしたり、特定のキーワードのコードをビルドから除外したりできます[*15]。この設定はBuild Profileごとに上書きできるため、プラットフォームごとに不要なバリアントだけを削れます。ビルトインのキーワードを除外すると必要なバリアントまで消えることがあるため、欠落を検出できるStrict Shader Variant Matchingとの併用も欠かせません。ビルドに残ったバリアントの実行時メモリは、2021 LTSから使えるShader Variant Loadingの設定で、読み込むチャンクのサイズと数を制限して抑えられます[*16]。

[*15] Unity Manual 6000.3, What's new in Unity 6.3 `https://docs.unity3d.com/6000.3/Documentation/Manual/WhatsNewUnity63.html`

[*16] Unity Blog, Improvements to shader build times and memory usage in 2021 LTS `https://unity.com/blog/engine-platform/2021-lts-improvements-to-shader-build-times-and-memory-usage`

Unity 6.4

エディタに同梱されたProject Auditor

Unity 6.4では、Project Auditorがエディタに同梱されました。Addressablesのバンドル間の重複のような肥大化を機械的に洗い出せます。6.4以降は診断ルールが別パッケージに分かれているため、導入手順がバージョンによって異なります。調査の手順は、第3部2章「運用フェーズのアセット肥大化とProject Auditorによる調査」で扱います。

Physicsモジュールを無効にする

Unity 6.4では、組み込みのPhysicsモジュールをPackage Managerから無効にできるようになりました[*17]。3D物理を使わないタイトルでは、その分ビルド容量を減らせます。ただしLocal Volumeを使う場合は有効のままにする必要があります。

Unity 6.5

Read/Writeの暗黙の有効化がなくなった

テクスチャやメッシュのRead/Writeを有効にすると、GPUに転送したデータとは別にCPU側にもコピーが残り、その分メモリを消費します。これまでのUnityは、Particle SystemのShapeモジュールやTerrainのディテール、Mesh ColliderがCPUからの読み取りを必要とする場合に、ビルド時にこの設定を暗黙的に有効にしていました。Unity 6.5ではテクスチャ、Unity 6.6ではメッシュについてこの暗黙の有効化がなくなり、必要なアセットで明示的に有効にしていなければ、Inspectorに警告が表示され、ビルドが失敗するようになりました。

公式ドキュメントはこの変更の理由としてビルド時間の短縮を挙げていますが、メモリの面でも意味があります。どのアセットがCPU側にコピーを持つのかがアセットの設定として見えるようになるため、本当にCPUからの読み取りが必要かを1つずつ判断でき、不要なコピーを減らす手がかりになります。

[*17] Unity Manual 6000.4, What's new in Unity 6.4 https://docs.unity3d.com/6000.4/Documentation/Manual/WhatsNewUnity64.html

AndroidのLTO

Unity 6.5では、Androidにリンク時最適化 (LTO) の設定が追加されました。このトグルが効くのはエンジン側のライブラリだけで、Strip Engine Codeの無効化を前提とするため、実行速度の改善と容量の増加を比べて採用を判断することになります。実測の結果は、第3部5章「UnityのLTO設定とAndroid・iOSでの性能比較」で扱います。

mimallocの追加

Unity 6.5では、ネイティブメモリのアロケーターとしてmimallocが追加されました。mimallocは複数のスレッドが同時にメモリを確保・解放できるアロケーターで、6.5では選択制でしたが、Unity 6.6では既定で有効になっています[*18]。ジョブやワーカースレッドでネイティブメモリの確保が重なる場面では、スレッド間の待ちの削減が期待できます。一方で、アロケーターが変わればメモリ使用量や断片化の傾向も変わり、公式には効果の数値も示されていません。6.6へ更新するときは、ネイティブメモリの推移を更新前と比べておくのが安全です。問題があればMemory Settingsから無効にできます。

iOSの熱状態に応じた目標フレームレート

Unity 6.5では、iOSの熱状態がSeriousやCriticalになったときの目標フレームレートをPlayer Settingsで指定できるようになりました。発熱時にフレームレートを下げる処理を自前で書かずに、サーマルスロットリングによる急な性能低下を和らげられます。

AssetBundleのTypeTreeを分離する

AssetBundleには、シリアライズされたデータの型情報であるTypeTreeが含まれています。TypeTreeがあるおかげで、バンドルをビルドしたときとプレイヤーのUnityバージョンが異なっても読み込めますが、同じ型の情報がバンドルごとに重複して含まれ、容量を押し上げます。Unity 6.5とAddressables 2.9.1以降では、Extract TypeTree Dataを有効にすると、TypeTreeを別ファイルに分けてAssetBundleの容量を減らせま

[*18] Unity Manual 6000.6, mimalloc `https://docs.unity3d.com/6000.6/Documentation/Manual/performance-mimalloc.html`

す[*19]。配信するバンドルが多いタイトルほど、ダウンロード量と端末のストレージの両方に効きます。ただし、この設定はすでに配信しているすべてのAssetBundleを変更するため、公式には新規プロジェクトでの利用が推奨されています。

Unity 6.6

AssetBundleの後継となるContent Directories

Unity 6.6では、AssetBundleの後継となる新しいコンテンツビルドの仕組みとしてContent Directoriesが追加されました[*20]。公式ドキュメントが挙げる利点は、暗黙的な重複排除、読み込み時間の短縮、メモリのオーバーヘッドの削減です。AssetBundleでは、複数のバンドルから参照されるアセットが各バンドルに複製され、容量とメモリを二重に消費することがありました。第3部2章「運用フェーズのアセット肥大化とProject Auditorによる調査」で扱うバンドル間の重複は、Content Directoriesでは自動的に排除されます。アセットの読み込みと解放は `Loadable<T>` APIで行い、async/awaitにも対応しています。Addressables 4.0ではContent Directory用のスキーマが追加され、既存のグループを変換できます[*21]。

ただし、Content Directoriesが現時点で対応するのはローカルコンテンツのビルドだけで、リモート配信には対応していません。ベータ版のUnity 6.7でも同じで、リモートのコンテンツへの対応はUnity 7世代で予定されています[*22]。Addressables 4.0でも、リモートの読み込みパスを持つグループを変換すると、スキーマは無効な状態で追加されます。ダウンロードでコンテンツを追加していく多くのモバイルタイトルでは、配信するコンテンツは引き続きAssetBundleで扱い、アプリに同梱するコンテンツから移行を検討することになるでしょう。参照の循環に含まれるアセットの合計が64MBを超えるとビルドが失敗するという制約もあります。

どちらの仕組みを使う場合でも、アセットをいつ読み込み、いつ解放するかという設

[*19] Unity Manual 6000.5, What's new in Unity 6.5 `https://docs.unity3d.com/6000.5/Documentation/Manual/WhatsNewUnity65.html`

[*20] Unity Manual 6000.6, Introduction to content directories `https://docs.unity3d.com/6000.6/Documentation/Manual/content-directories-introduction.html`

[*21] Addressables 4.0 Manual, Content build systems `https://docs.unity3d.com/Packages/com.unity.addressables@4.0/manual/content-build-systems.html`

[*22] Unity Blog, Content directories: Beyond the AssetBundle `https://unity.com/blog/content-directories-beyond-the-assetbundle`

計はそのまま当てはまります。第3部1章「AssetBundleのライフタイム管理」は、読み込みと解放を基盤層に集約し、読み込んだ時点でアセットの寿命を決めることで、解放漏れや不要なアセットの常駐を構造で防ぎます。

シェーダー定数をBuild Profileごとに指定する

Unity 6.6では、シェーダー定数の値をプロジェクト全体またはBuild Profileごとに指定できるようになりました[*23]。プラットフォームごとに値だけが違う処理を、キーワードでバリアントを増やさずに書き分けられます。

Build HistoryとBuild Analysis

Unity 6.6では、Build HistoryとBuild Analysisウィンドウが追加され、プレイヤーのビルドレポートが自動で蓄積されるようになりました。ビルドごとの容量の推移を追えるため、肥大化の原因となった変更を特定しやすくなります。ただしAssetBundleのビルドへの対応は限定的です。

Unity 6.7（発表段階）

次の機能は2026年7月のUnite Seoulで発表されたもので、執筆時点ではドキュメントで詳細を確認できない発表段階の情報です[*24]。

Neural Texture Compression

Neural Texture Compression (NTC) は、学習済みのモデルを使い、テクスチャをほぼ劣化なく圧縮する技術です。発表では、見た目の劣化なしにテクスチャの実行時メモリを50%以上、ディスク容量を約70%削減した例が示されました。NTCは、Unity 6.7から順次提供が始まるUnity Neuralの一部として紹介されています。

Fast Build Profile

Fast Build Profileは、Unity 6.7で追加される予定の新しいビルドプロファイルで、シェーダーのビルド時間を最大90%短縮できるとされています。あわせて、PSOのト

[*23] Unity Manual 6000.6, What's new in Unity 6.6 `https://docs.unity3d.com/6000.6/Documentation/Manual/WhatsNewUnity66.html`

[*24] Unity Blog, Unite Seoul 2026 keynote recap `https://unity.com/blog/unite-seoul-keynote-2026-recap`

レースとウォームアップによって実行時のスタッターを減らす仕組みも紹介されています。

第3部に向けて

Unity 6.0から6.7までをたどると、エンジン側の変化は、これまで自前のスクリプトや運用で補ってきたことを設定として持てるようにする方向に進んでいます。一方で、これらの変化は既定値のまま待っていれば効果が出るものではありません。mimallocのように更新するだけで挙動が変わるものは、更新前後の計測で確かめる必要があります。Content Directoriesのように、配信の要件によってはまだ移行できないものもあります。アセットをいつ読み込んで解放するか、どこまでコードを削るか、端末ごとに何を切り替えるかといった設計の判断は、エンジンのバージョンが変わっても残ります。

第3部の各章は、その判断に必要な設計と計測を、実際のプロジェクトでの事例とともに扱います。新しいバージョンへの更新を検討するときの判断材料として、役立てていただければ幸いです。

第1章

AssetBundleのライフタイム管理

稲田 健人 Inada Taketo

株式会社サイバーエージェント SGEコア技術本部

ゲームでは、テクスチャ、メッシュ、オーディオなど、多くのアセットを実行時にロードします。これらを各実装者が個別に管理すると、解放漏れや不要なアセットの常駐が発生しやすくなります。

そのため、AssetBundleやAddressablesなどから動的にロードするアセットは、原則としてロードと解放の窓口をシステム基盤へ集約しましょう。ロード時にアセットのライフタイムを決定し、その終了時に基盤側でまとめて解放できる仕組みにします。

本節でいう「解放」は、ロード基盤が管理する参照や解放用ハンドルを解放することを指します。実際にアセットがメモリからアンロードされるかどうかは、基盤の実装や参照数、依存関係の状態によります。

なお、以降で紹介する分類や実装方法は参考例です。各プロジェクトのゲーム構造に合った方法を選択してください。

1.1　アセットのライフタイムを分けて管理する

まず、ゲーム内でアセットが必要となる期間を整理します。たとえば、次のように分類できます。

表1.1: アセットのライフタイムの分類例

ライフタイム	保持期間の例
Common	ログアウト、アカウント切り替え、タイトルへの復帰など、共通データをリセットするまで
Scene	対象のシーンを終了するまで
View	対象の画面やダイアログを閉じるまで
Custom	バトルやイベントなど、独自に定めた期間が終了するまで

重要なのは、Common、Scene、Viewといった名前ではなく、それぞれのライフタイムがいつ始まり、いつ終了するのかを明確にすることです。

分類が粗すぎると不要なアセットが長期間残り、細かすぎるとロードと解放の回数や管理コストが増えます。また、寿命の異なるアセットが同じAssetBundleに含まれている場合、AssetBundle単位のアンロードでは一部のアセットだけを選んでアンロードできません。ライフタイムの分類は、AssetBundleの粒度（前作『Unity パフォーマンスチューニング バイブル』第5章「Tuning Practice - AssetBundle」）とあわせて設計しましょう。

1.2 ロード時にライフタイムを決定する

ライフタイムを分類したら、アセットをロードするときに登録先を決定します。

実装例の1つとして、ロード関数の引数にライフタイムを指定する方法があります。

▼リスト1.1 ライフタイムを指定してロードする例

```
var background =
    await AssetLoader.LoadAsync<GameObject>(
        address,
        AssetLifetime.Scene);
```

リスト1.1では、ロードしたアセットをSceneライフタイムへ登録しています。シーンの終了時には、基盤側が登録されたアセットをまとめて解放します。

別の実装例として、ライフタイムごとにロード関数を分ける方法もあります。

▼リスト1.2 ライフタイムごとにロード関数を分ける例

```
var commonIcon =
    await AssetLoader.LoadCommonAsync<Sprite>(
        commonAddress);

var stagePrefab =
    await AssetLoader.LoadSceneAsync<GameObject>(
        stageAddress);

var dialogPrefab =
    await AssetLoader.LoadViewAsync<GameObject>(
        dialogAddress,
        viewId);
```

LoadCommonAsync、LoadSceneAsync、LoadViewAsyncのように関数を分けると、呼び出し箇所からライフタイムを判断でき、登録先の指定ミスも防ぎやすくなります。一方、引数で指定する方法は、独自のライフタイムを追加しやすいという特徴があります。

どちらの方法でも重要なのは、ロード時にライフタイムが必ず決まり、その終了時に基盤側で解放できることです。

非同期ロードの完了前にライフタイムが終了する場合に備え、ロード開始時から処理を追跡し、途中終了時にはキャンセルするか、完了後の結果を解放できるようにします。また、同じアセットを複数のライフタイムから使用する場合は、参照数を管理し、最後の利用が終了した時点で解放します。

1.3 GameObjectにライフタイムを紐づける

別の管理方法として、GameObjectをライフタイムの所有者にする方法があります。

画面やダイアログのルートGameObjectに専用のComponentを追加し、ロードしたアセットの解放用ハンドルを登録します。GameObjectが破棄されたときに、OnDestroyから登録済みのハンドルを解放します。

▼リスト1.3　GameObjectにライフタイムを紐づける例

```
public sealed class AssetLifetimeScope : MonoBehaviour
{
    private readonly List<IDisposable> _handles =
        new List<IDisposable>();
```

```
    private bool _released;

    public void Add(IDisposable handle)
    {
        if (_released)
        {
            handle.Dispose();
            return;
        }

        _handles.Add(handle);
    }

    private void OnDestroy()
    {
        _released = true;

        for (var i = _handles.Count - 1; i >= 0; i--)
        {
            _handles[i].Dispose();
        }

        _handles.Clear();
    }
}
```

リスト1.3では、GameObjectの破棄後に非同期ロードが完了してAddが呼ばれた場合も、追加されたハンドルを即座に解放します。

ただし、GameObjectを破棄するだけで、動的にロードしたアセットが自動的に解放されるわけではありません。専用のComponentから、ロード基盤に対応した解放処理を呼び出す必要があります。

管理用のGameObjectはアセットごとに作成せず、1つの画面やダイアログなど、1つのライフタイム単位につき1つにまとめます。この方法はGameObjectとアセットの寿命が一致する場合に有効ですが、複数のシーンをまたいで使用するアセットなどには適さないことがあります。必要に応じて、ライフタイムごとの管理テーブルと使い分けましょう。

1.4 解放漏れを確認する

Memory Profilerで画面やシーンの遷移前後のスナップショットを取得し、終了したライフタイムのアセットが残っていないかを比較します。同じ遷移を複数回繰り返し、アセットやロードハンドルが増え続けていないかを確認することも有効です。

計測は、必要な解放処理が完了してから同じ基準状態どうしで比較し、最終的には実機のプレイヤービルドで確認しましょう。ただし、ほかの機能から参照されているアセットや、キャッシュとして意図的に保持しているアセットもあるため、残っていることだけで解放漏れとは判断せず、実際の参照元まで確認します。

アセット管理は、開発後期に変更すると多くのロード処理へ影響します。開発の早い段階で必要なライフタイムを整理し、実装者が個々の解放タイミングを意識しなくても、プロジェクトに合ったライフタイムが適用される仕組みを設計しておきましょう。

第2章

運用フェーズのアセット肥大化とProject Auditorによる調査

財津 大睦 Zaitsu Hiromu

株式会社サムザップ

X @RomGameclient

運用が続くプロジェクトでは、アプリの容量とメモリ使用量が少しずつ増加します。この章で扱うのは、対策を知っているチームでも肥大化が再発する理由と、その止め方、そして増え続けているアセットを見つける手段です。個々のアセットの軽量化そのものは扱いません。

2.1　運用フェーズで肥大化が進む理由

肥大化は、日々の運用のなかで少しずつ蓄積します。シーズンイベントのために追加された高解像度のバナー、期間限定キャラクターのボイスとカットイン、コラボ企画で受け入れた外部制作の素材、告知が終わったのに残っているキャンペーン用のアセット。1件ずつは許容範囲でも、リリースを重ねるうちに無視できない量になります。しかも蓄積が進むほど見直しのコストが上がるため、整理の優先度は下がっていきます。原因は技術よりも、運用のルールが緩むところにあります。運用フェーズでは、次の4つが繰り返し起こります。・規約が古くなる：端末や要件が変わって規約と実態がズレても、誰も規約を書き直さない。今回だけと言って通した例外が積み重なる。・チェックが甘くなる：締め切りが近づくと納品素材の確認が省略され、担当が交代したときも引き継ぎが中途半端になる。・一時的な対応が残り続ける：急ぎで無圧縮テクスチャや個別出力のスプライトを入れたまま、後で戻す作業が行われない。・判断基準が記録されていない：設定の根拠がドキュメントに残っていないため、担当者が変わると同じ基準を再現できない。いずれもツールの導入だけでは解決しません。

2.2　規約を運用し続ける

達成したい指標を決め、動作保証端末を選び、最も負荷の掛かるシーンで検証する。この規約の作り方そのものは、前作『Unity パフォーマンスチューニング バイブル』の第1章が扱っています。運用フェーズで問題になるのは、決めた後です。規約は決めたそばから実態とズレていきます。そこで、動作保証端末の変更、Unityやレンダリングパイプラインの更新、計測方法の変更といったタイミングを、見直すきっかけとして先に決めておきます。とくに計測方法が変わると同じアセットでも数値が動くため、測り方も規約と一緒に残しておきます。規約の数値は、書き出しやインポート、自動ビルドのいずれかで検査が走る形にします。止めるか通すかは項目ごとに決め、通す場合も実測値をビルドのログやレポートとしてチーム全員が見える場所に残してください。本人の画面にだけ警告を出す形は読み飛ばされます。規約を超えているアセットの修正は、値を直せば終わりではありません。圧縮設定を変えるなら見た目が許容範囲に収まるか、バンドルを組み替えるなら参照が外れていないかを、本番と同じ条件で確認する必要があります。前作が警告しているとおり、量産後の作り直しは発注から品質チェック、デバッグまでを含む工程です。まとめて直すのではなく、リリースのたびに計測して、増えた分をその都度修正するのが理想です。

2.3　肥大化のパターン分類

素材の種類ごとに対策をまとめると、どの素材が重いかは分かっても、なぜ重くなったのかまでは分かりません。原因が残れば同じ肥大化が再発します。そこで素材の種類を横断し、原因ごとに6つに分類します。

表2.1: 肥大化の6パターン

パターン	何が起きているか
圧縮設定の誤り	インポート設定が未圧縮や既定値のまま放置され、必要な数倍の容量を占めています
解像度の過剰	最大解像度を指定しないまま取り込み、面積に比例して増えています
余分な付随データ	使わない補助データを有効にしたまま同梱し、メモリ上にも保持しています
形式の選択ミス	用途やプラットフォームに合わないフォーマットを選び、容量を余分に使うか、実行時に展開されてメモリを消費しています

表2.1: 肥大化の6パターン

パターン	何が起きているか
まとめ方の不一致	Sprite Atlasやバンドルのまとめ方が実際の利用単位と合っておらず、1枚のSpriteの表示でアトラス全体を読み込むなど、使わないものまで一緒にロードしています
バンドル間の重複	共有アセットを登録しないまま参照し、参照元のバンドルの数だけ同梱しています

前半4つはアセットのImport時に判定できますが、後半2つは他のアセットとの関係で決まるため、その時点では判定できません。この違いが、次節で対策を二段に分ける理由になります。

2.4 Import時の自動適用とビルド前の検査

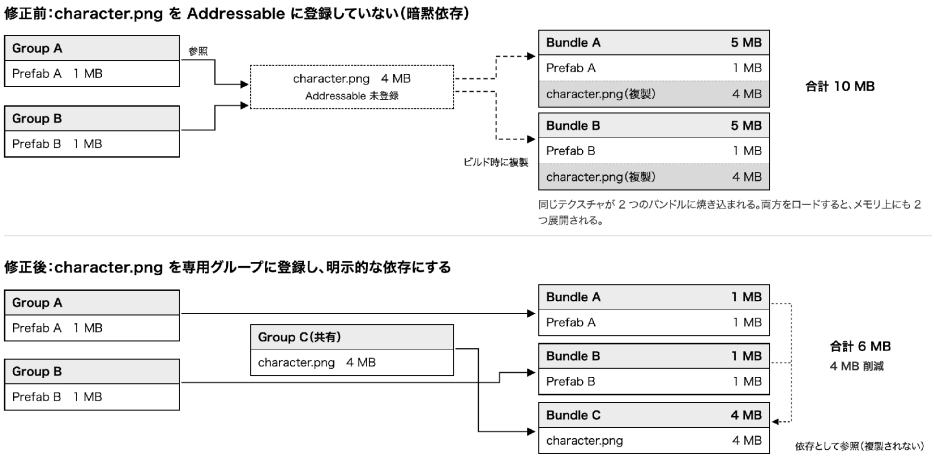
再発を止めるには、アセットのImport時に正しい設定を適用したうえで、それをすり抜けたものをビルド前に見つけます。自動適用の手段は2つあります。条件によって値を変えるなら、Importの前後に処理を差し込めるAssetPostprocessorを使います。型ごとの初期値を固定するだけならPresetで足ります。ただし、Import時の適用だけではすり抜けが起きます。.metaファイルごと持ち込まれたアセットは設定を保持したまま入ってくるうえ、前節の後半2つはこの段階では判定できません。そこでビルド前や自動ビルドの段階で、規約に反するアセットが混ざっていないかを検査します。テクスチャの最大解像度やメッシュのポリゴン数などの制限規約をテストとして書いておけば、エディターと自動ビルドの両方で同じ基準を使えます。AssetRegulationManagerのように、この用途のOSSも公開されています[*1]。こうした検査を仕組みにせず、規約を文書に書いて周知するだけの運用は、冒頭に挙げた規約が古くなる状態と、判断基準が残らない状態をそのまま招きます。ルールを決めるのは人の仕事ですが、守らせる部分は機械的に任せてください。

2.5 バンドル間の重複が起きる仕組み

6パターンのうち、最も見えにくいのがバンドル間の重複です。原因が個々のアセットの設定ではなく、アセットどうしの参照関係にあるためです。Addressablesはアセッ

[*1] https://github.com/CyberAgentGameEntertainment/AssetRegulationManager/blob/main/README_JA.md

トをグループ単位で管理し、ビルド時にグループごとのバンドルへまとめます。このとき、Addressableに登録していないアセットは暗黙の依存として扱われ、参照している側のバンドルそれぞれに複製されます。



▲図2.1　登録していない共有アセットの複製（上）と、専用グループに登録した場合（下）

図の例では、4MBのテクスチャを2つのPrefabが参照しているだけで、バンドルの合計が10MBになります。このテクスチャを専用グループに登録して明示的な依存にすれば、複製が解消されて合計は6MBまで下がります。参照元が増えればその数だけ複製され、同時にロードすればメモリにも同じだけ展開されるため、放置するほど影響が大きくなります。なお、実際のバンドルは圧縮されるため、数値は仕組みを示す目安です。こうした重複は目視では追いきれないため、ツールで検出します。Addressablesの場合はAddressables Analyze[*2]が使えます（2.0系で一時削除され、2.3.1で復活しています）。検出した重複を自動で解消する機能もありますが、グループ構成が書き換わるため、適用後はビルド結果まで確認してください。

2.6　肥大化したアセットを調査する

規約と検査で新しい肥大化は止められますが、すでに積み上がっているアセットは別途探し出す必要があります。アセットのプロファイリングには、用途の異なるツー

[*2] https://docs.unity3d.com/Packages/com.unity.addressables@latest

ルが複数あります。

表2.2: 容量とメモリの調査に使うツール

ツール	分かること
Build Report / Editor Log	ビルドサイズのカテゴリ別内訳
Addressables Analyze	Addressablesの重複バンドル依存の検出と修正
Project Auditor	プロジェクト全体のインポート設定と静的な構成の問題
Memory Profiler	実行時のメモリ使用量とオブジェクトの参照関係

このうちMemory Profilerは、前作の第3章「プロファイリングツール」が扱っています。本章では、前作で紹介していないProject Auditorを取り上げます。

2.7 Project Auditorで洗い出す

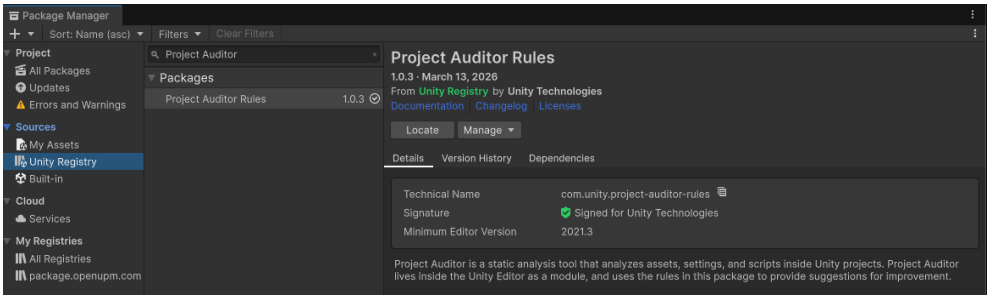
Project Auditorは、プロジェクト内のアセット、コード、設定を静的に解析し、問題のある箇所を一覧として出力するツールです。規約に落とすところまで手が回っていない段階でも、何が問題になりうるかを把握できます。長く実験段階のパッケージでしたが、2025年2月に1.0がリリースされ、正式版になりました。以降の画面と項目は、Unity 6.4とRulesパッケージ1.0.3で確認したものです。

2.7.1 導入方法はUnityのバージョンで異なる

Unity 6.4より前では、Package Managerからcom.unity.project-auditorを追加します[*3]。6.4以降はEditorに同梱されたため追加は不要ですが[*4]、判定に使うルールがcom.unity.project-auditor-rulesに分離されたので、そちらを導入しないと解析できません（図2.2）。

[*3] https://docs.unity3d.com/Packages/com.unity.project-auditor@1.0/manual/index.html

[*4] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/project-auditor-introduction.html

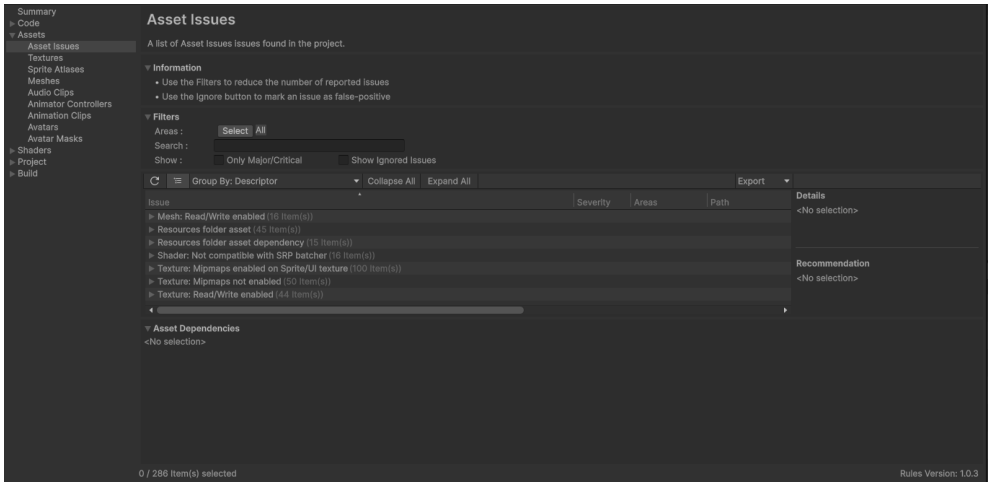


▲図2.2　Rulesパッケージ。解析本体はEditorのモジュールとして存在する

ルールを本体から切り離したことで、Editorのバージョンを上げなくてもルールだけを更新できます。

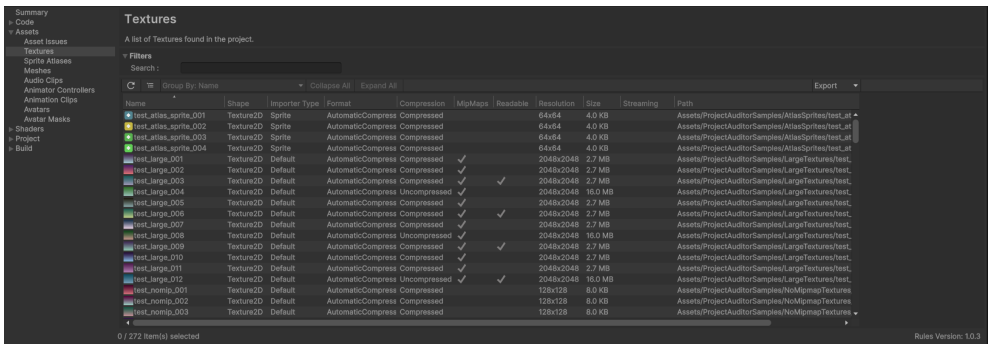
2.7.2　Assetsビューの使い方

Window > Analysis > Project Auditorを開いて解析を実行すると、結果はSummary、Code、Assets、Shaders、Project、Buildのビューに分かれます（ビューはUnityのバージョンによって増えることがあります）。容量とメモリの調査で使うのはAssetsビューで、Asset IssuesやTextures、Sprite Atlasesなど9つに分かれています。最初に開くのはAsset Issuesです（図2.3）。検出された問題が種別ごとにまとまり、それぞれ何件見つかったかが表示されます。



▲図2.3　Asset Issuesビュー。検証用に用意したプロジェクトでの検出例

種別を展開すると個々のアセットが表示され、選択すると右側に詳細と推奨される対処が出ます。全体像がつかめたら、Texturesなど個別のビューでその内訳を見ます（図2.4）。



▲図2.4　Texturesビュー。設定と結果のサイズが横に並ぶ

このビューの利点は、設定と結果が同じ行に並ぶことです。同じ2048×2048のテクスチャでも、Compressedの行は2.7MB、Uncompressedの行は16.0MBを占めています。設定1か所の違いで6倍近い差がつくことが、Inspectorを開かずに分かります。Size列を降順にして上位からCompression、Resolution、Formatを確認すれば、圧縮設定の誤り、解像度の過剰、形式の選択ミスが見つかります。

2.7.3 検出結果を減らす

運用中のプロジェクトで解析を実行すると、検出結果は数千件を超えることがあります。この状態で上から潰そうとしても進みません。前掲の図2.3にあるFiltersで表示を絞ります。Only Major/Criticalにチェックを入れれば重要度の高いものだけが残り、Areasで対象領域を選べば容量に関わる指摘だけを表示できます。検出された問題には重要度が付きます。ドキュメントではMajor、Moderate、Minor、Ignoredの4種類と説明されていますが、定義上はCriticalも存在します。Ignoredは重要度の段階ではなく、無視すると指定した状態を指します。対処はMajorから順に進め、Criticalが検出された場合はそちらを最優先します。静的解析では意図した設定も指摘されるため、1件ずつ確認してから直すのが安全です。誤検知と判断した指摘にはIgnoreを付けます。Ignoreは開いているレポートに記録される状態のため、設定ファイルには書き出されず、解析をやり直すと外れます。検出のしきい値のような判定パラメーターはProjectSettings/ProjectAuditorSettings.assetに保存されるため、こちらはコミットすれば全員で揃えられます。恒常的に指摘を抑制したい場合は、自動ビルド側のスクリプトでAnalysisParams.WithAdditionalDiagnosticRulesに無効化のルールを渡します。

2.7.4 自動ビルドに組み込む

Project Auditorはコマンドラインからも実行できます[*5]。自動ビルドに組み込んで検出件数をリリースごとに残しておけば、どのリリースで増えたかを追跡できます。Preferencesには、ビルド後に検出件数をログ出力する設定と、検出をエラーとして扱う設定があります。失敗判定を自前で書く前に、こちらで足りるかを確認してください。一方、保存した2つのレポートを比較するAPIは提供されていないため、前回との差分を取るには自前の処理が必要です。なお、ビルド後の自動解析が完了せず止まったままになる、解析中にメモリを使い切るといった報告もあります[*6]。タイムアウトを設定し、解析の失敗がビルド全体を止めないようにしてください。

[*5] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/run-from-command-line.html

[*6] https://github.com/Unity-Technologies/ProjectAuditor/issues

2.8 おわりに

肥大化対策は、個々のアセットを軽くする技術よりも、チームとして再発を止める仕組みづくりの比重が大きいテーマです。運用のなかで規約が緩んでいくこと自体は、どのプロジェクトでも起こります。だからこそ、基準を決めることと見直すことに人の時間を使い、守らせる部分は機械に任せる形が、結局いちばん長続きすると感じています。Project Auditorは正式版になったばかりのツールですが、規約づくりまで手が回っていないプロジェクトが現状を把握する入り口として十分に実用的です。まずは一度解析を実行して、自分たちのプロジェクトの傾向を眺めるところから始めてみてください。

第3章

端末に応じた描画設定の切り替え

佐久間 尚輝 NAOKI SAKUMA

株式会社サイバーエージェント SGEコア技術本部

スマートフォン向けゲームでは幅広い端末でプレイされることが想定されます。一律で同じ描画設定にすると、低スペック端末ではクラッシュ、処理落ち、発熱の原因となってしまうケースがあります。逆に高スペック端末では端末の性能を活かしてより良い品質にすることができる可能性があります。そこで、端末に応じた適切な描画設定に切り替える仕組みを紹介します。

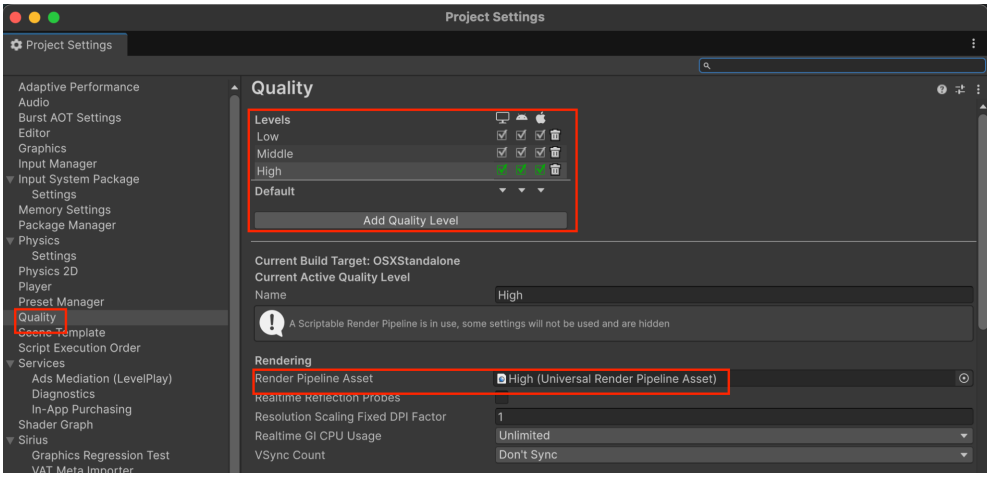
3.1　切り替えの手法

低・中・高のようなプリセット形式と、ユーザーが直接値を操作できる形式の2つがゲームではよくある手法かと思います。

3.1.1　プリセット形式

プリセット単位で切り替えたい場合の手法です。ユーザーにプリセットを選ばせる場合や、端末での自動判定でもこの手法を用います。

ProjectSettingsのQualityから、Add Quality Level を押下し、プリセット分追加します。その後、各プリセットに沿ったRenderPipelineAssetをアタッチします。必要に応じて、Qualityのその他の項目（Rendering, Textures等）を設定することも可能です。



コード上で、QualitySettings.SetQualityLevel を呼び出すことにより、任意のQuality Levelに切り替えることができます。

3.1.2　値を直接操作する形式

ユーザーにパラメータを操作させる場合は、こちらの手法になります。以下のようなコードで、パラメータを直接操作することができます。

```
var urpAsset = (UniversalRenderPipelineAsset)GraphicsSettings.currentRenderPipeline;
// 解像度の変更
urpAsset.renderScale = value;
// 影のカスケードの枚数の変更
urpAsset.shadowCascadeCount = value;
// その他､影の解像度であったり､追加ライトの個数等
```

3.2　端末に応じた切り替え

端末によって、プリセットを推奨、自動で設定したいケースもあるかと思います。その場合は、 SystemInfo型が便利です。

特に、deviceModel、graphicsDeviceName、systemMemorySizeで見ると識別しやすい値を取得することができます。以下は返ってくる値の例です。

表3.1: 端末毎のSystemInfoの返ってくる値の例

機種	deviceModel	graphicsDeviceName	systemMemorySize
iPhone 13	iPhone14,5	Apple A15 GPU	約3800
iPad Pro 11-inch (第3世代)	iPad13,4	Apple M1 GPU	約7800
Galaxy S23	samsung SM-S911B	Adreno (TM) 740	約7500
Pixel 8	Google Pixel 8	Mali-G715	約7400

iOSはdeviceModelとgraphicsDeviceNameの命名規則が一貫しているのと、基本的に同じ系列内であれば A もしくは M の後の数字が大きいほどスペックが高いです。ですので、組み合わせることで一定程度切り分けられます。

Androidは、各機種によって返ってくる値がまばらですので、iOSのように一概に比較することは難しいです。ですので、

- RAMで割り切る
- マスタに逃がして比較する
- 起動時にベンチマークツールを走らせる

等で切り分けるのが良いです。

3.3 自作RendererFeature

自作のRendererFeature等を作成した際にも、描画設定を切り替えられるAPIを準備しておくと便利です。例えば、

- 一時テクスチャのフォーマットを下げる、解像度を下げる
- Blitの数を減らす
- 簡易なシェーダーに置き換える

等の対応が挙げられます。

第4章

Unity Managed Code Stripping 実践ガイド

向井 祐一郎 Mukai Yuichiro

株式会社アプリボット

yucchiy　X @yucchiy_　blog.yucchiy.com

アプリサイズの削減手段として、Unityには未使用のコードをビルド時に取り除くManaged Code Strippingという機能について紹介します。設定1つで有効にできる手軽な機能ですが、削減を強めると「エディター上では動くのに、ビルドしたアプリだけで特定の機能が壊れる」という問題を引き起こすことがあります。本章では、Managed Code Strippingの仕組みと使い方、コードが必要以上に削減されてしまう典型的なパターンとその対処方法などを紹介します。

検証環境は以下の通りです。

- Unity 6000.3.19f1
- Addressables 4.0.1
- macOS Sequoia 15.7.3

先に結論を書くと、Managed Code Strippingは設定をHighにして終わりにできる最適化ではありません。

静的解析から見えない依存を明示し、ビルドしたアプリのサイズと動作を検証し続ける運用とあわせて、初めて安全に使える機能です。

4.1 Managed Code Stripping とは何か

4.1.1 機能の概要

Managed Code Strippingは、ビルド時に未使用のマネージドコード（C#コード）を検出し、ビルドしたアプリから取り除くUnity標準の機能です。

シーンやスクリプトから利用されていないクラスやメソッドが除去されることで、アプリのコードサイズを削減できます。

削減の対象はユーザーコードだけではなく、UnityEngineのモジュールや.NETのクラスライブラリ、プロジェクトに含まれるパッケージやプラグインのアセンブリも対象になるため、自分では手を入れづらいライブラリ内部の未使用コードまで削減することができます。

とくにIL2CPPビルドでは、マネージドコードが削減されることでその後のIL2CPPで生成されるC++コードも減るため、アプリサイズやビルド時間の短縮にもつながります。

なお、似た名前の設定に Strip Engine Code がありますが、こちらはUnityエンジンのネイティブコードを対象とした削減設定であり、マネージドコードを対象とするManaged Code Strippingとは別の機能です。

Managed Stripping Level

どの程度積極的にコードを削るかは、Managed Stripping Levelという設定で制御します。

Edit > Project Settings > Player > Other Settings > Optimization の「Managed Stripping Level」から設定でき、下記のレベルを選択できます。

表4.1: Managed Stripping Levelの選択肢

Level	概要
Disabled	ストリッピングを行わない。Monoビルドでのみ選択できる

表4.1: Managed Stripping Levelの選択肢

Level	概要
Minimal	UnityEngineと.NETのクラスライブラリのみを対象に削減し、ユーザーコードは削除しない。IL2CPPビルドの既定値
Low	ユーザーコードも解析対象に含めるが、保守的に削減する
Medium	全アセンブリを対象に、サイズと安全性のバランスを取って削減する
High	全アセンブリを最も積極的に削減する

MonoビルドではDisabledが既定値のため、何も設定しなければストリッピングは行われません。

一方、IL2CPPビルドではMinimalが既定値であり、Disabledは選択できないため、程度の差はあれ必ずストリッピングが行われます。

なお、LowはUnity 6.3のドキュメントで将来的な廃止が予告されており、利用は推奨されていません。

参考: Managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping.html` / Configure managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-configure.html`

4.1.2 基本的な動作原理

Managed Code Strippingは、**UnityLinker**というUnity内製のツールにより実現されています。

UnityLinkerはアプリのビルドパイプラインの中で、C#コードのコンパイルが完了した後に実行され、コンパイル済みのアセンブリ（DLL）から未使用コードを取り除きます。

IL2CPPビルドでは、このストリッピングを経たアセンブリが、その後のC++コード変換のための入力になります。

UnityLinkerは、次の三段階で未使用コードを判定します。

1. 解析の起点となるコード（**root**）を決める
2. rootから静的に参照されている型やメンバーを再帰的にたどり、使用中の印を付ける（**mark**）
3. 最後まで印が付かなかったコードを、到達不能とみなしてアセンブリから取り除く

rootとして扱われる代表的なものは下記の通りです。

- ビルドに含まれるシーンやアセットから参照されている型
- MonoBehaviourやScriptableObjectを継承した型
- [RuntimeInitializeOnLoadMethod]などの属性が付いたメソッド
- link.xmlや[Preserve]属性で明示的に保持を指定したコード（後述の対処方法で扱います）

ここで重要なのは、コードを残すかどうかが「実行時に実際に使われるか」ではなく、「ビルド時の静的解析でrootから到達できるか」で判断される点です。

静的解析とは、コードを実行せずに、コンパイル済みのコードに現れる参照だけを機械的にたどる解析のことです。

例えば new による型の生成やメソッドの直接呼び出しは、コンパイルされたコードに参照先の型やメンバーの情報がそのまま残るため、UnityLinkerがたどれます。

一方で、コード上に参照が現れない形で呼び出されているコードは、実行時に使われるかどうかにかかわらず到達不能とみなされます。

そのため、静的解析から見えない参照しか持たないコードは、実行時には必要であっても未使用と判定されて削除されます。

また、markは型単位ではなくメンバー単位で行われるため、残った型でも使われていないメソッドは個別に削除されます。

実際にどのようなコードが削除されてしまうのかは、次節で具体的なコードと計測結果とともに確認します。

何が削られたかをMono.Cecilで確認する

本章では、この後の節で「どの型やメソッドがビルドに残ったか」を繰り返し確認します。

その確認には、.NETのアセンブリを解析するライブラリである**Mono.Cecil**を用いています。

ビルド出力に含まれるアセンブリ（macOS向けMonoビルドなら「アプリ名.app/Contents/Resources/Data/Managed/Assembly-CSharp.dll」）を読み、型とメソッドの一覧をファイルに書き出します。

▼リスト4.1　型とメソッド一覧の出力

```
// アセンブリを読み、型とメソッド(.ctor 含む)の一覧を書き出す
using var assembly = AssemblyDefinition.ReadAssembly(dllPath);
var lines = assembly.MainModule.Types
    .Where(type => type.Name != "<Module>")
    .SelectMany(type => new[] { type.FullName }
        .Concat(type.Methods.Select(method => $"  {method.Name}")));
File.WriteAllLines(outputPath, lines);
```

メソッドの一覧にはコンストラクタ（.ctor）も含まれるため、後述する「型は残っているのにコンストラクタだけが消える」症状の確認にも使えます。

Mono.CecilはUnity Package Manager経由でインストールして利用できます。エディター用のAssembly Definitionで overrideReferences を有効にして precompiled References に Mono.Cecil.dll を追加すると、エディターコードから参照できます。

Levelを変えた2つのビルドでこの一覧を取って差分を見れば、どの型が消え、どのメソッドがメンバー単位で削られたかを特定できます。

ビルド後には、ストリップ後のアセンブリが中間生成物としてLibrary配下に残ることがあります。

今回の環境では、Library/Bee/artifacts/MacStandalonePlayerBuildProgram/ManagedStripped/ にビルド出力と同一内容のDLLが残っており、ビルド成果物を展開しなくてもこちらを解析対象にできます。

このパスにストリップ後のアセンブリが保存されることは公式ドキュメントには明記されてないため、見つからない場合はビルド出力側を解析します。

以降の節で「型とメソッドを確認した」と書いている箇所は、基本的にはこの方法による確認です。ストリッピング前後のDLLを比較すれば、削減の内訳まで確認できます。

参考: Unity linker marking rules `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-marking-rules.html`

4.1.3 なぜコードを削りたいのか

Managed Code Strippingを使う主目的は、アプリサイズの削減です。

副次的には、未使用コードがメモリ上に乗らないため、ランタイムでのメモリ使用量の削減も期待できます。

どの程度の効果があるかを、終了処理のスクリプトを1つ置いただけのシンプルなシーンをビルドし、Managed Stripping LevelのMinimalとHighを比較します。

下記に計測結果を示します（表4.2）。

表4.2: シンプルなシーンでのMinimalとHighの比較

指標	Minimal	High	削減
マネージドアセンブリの合計サイズ	7,702,016 bytes	7,188,480 bytes	約6.7%
アセンブリ（DLL）の数	52	51	1個
アプリ全体のサイズ	89,069,307 bytes	88,548,299 bytes	約0.6%

小さなサンプルプロジェクトでも、マネージドコードは6.7%減りました。

内訳を見ると、どこからも使われていないSystem.Runtime.Serialization.dllが丸ごと削除され、mscorlib.dllが約272KB減り、シーンから参照されないコードを多く含むAssembly-CSharp.dllも22.5%減っています。

マネージドコードの削減量は、プロジェクトが参照するライブラリの量と、その中で実際に使われていないコードの割合で決まります。

大規模なプロジェクトでは多くのライブラリを導入する傾向にありますが、実際にライブラリの中でも使われない機能が多くなることがよくあり、それによってコード削減の恩恵を受けやすいでしょう。

IL2CPPビルドでは、削減されたマネージドコードの分だけC++への変換とコンパイルの対象も減るため、ネイティブバイナリのサイズとビルド時間にも効果が波及します。

4.2 Managed Code Stripping を適用する上での注意点

前節で見たとおり、Managed Code Strippingの判断はビルド時の静的解析に依存します。

そのため、静的解析から見えない参照しか持たないコードは、実行時には必要であっても削除されてしまいます。

この問題が厄介なのは、症状がビルドしたアプリでしか現れないことです。

エディターでの実行や、ストリッピングが無効な既定のMonoビルドでは正常に動くため、開発中には気づきにくく、Stripping Levelを上げた途端に特定の機能だけが壊れる、という形で不具合が表面化します。

この節では、コードが必要以上に削られてしまう典型例の2つのパターンを取り上げます。

ひとつはリフレクションを介したコード呼び出し、もうひとつはUnityのエントリーポイントから参照されないコードです。

4.2.1 リフレクションによるコード呼び出しが消える

前節で説明した「静的解析でrootから到達できるか」の判断を、具体的なコードで確認します。

参照のされ方が異なる3つのクラスを用意します（リスト4.2）。

▼リスト4.2　参照のされ方が異なる3つのクラス

```
// new で直接生成される
public class DirectlyUsedService
{
    public string Describe() => "DirectlyUsedService is alive";
}

// typeof で型だけ参照される
public class TypeofOnlyService
{
    public string Describe() => "TypeofOnlyService is alive";
}

// リフレクションを介してのみ生成される
public class ReflectionOnlyService
{
    public string Describe() => "ReflectionOnlyService is alive";
}
```

シーンに置いたMonoBehaviourから、この3つを次のように使います（リスト4.3）。

▼リスト4.3　3つのクラスを使うMonoBehaviour

```
public class StrippingProbe : MonoBehaviour
{
    private static readonly string ReflectionTypeName = "ReflectionOnlyService";
    private static readonly string TypeofTypeName = "TypeofOnlyService";
    private static readonly string TypeofMethodName = "Describe";

    private void Start()
    {
        // 静的な参照：参照先の型とメソッドの情報がコンパイル済みコードに残る
        var direct = new DirectlyUsedService();
        Debug.Log($"direct: {direct.Describe()}");

        // 静的な参照が型だけ：typeof は型を参照するが、メンバーへの静的参照はない
        Debug.Log($"typeof: type is referenced: {typeof(TypeofOnlyService).Name}");

        // メンバーが残ったかは、静的解析から見えない経路で型を取り直して確認する
        var typeofType = Type.GetType(TypeofTypeName);
        var describeMethod = typeofType?.GetMethod(TypeofMethodName);
        if (describeMethod == null)
        {
            Debug.Log($"typeof: type {typeofType?.Name} exists but Describe is stripped
```

```
    ");
        }
        else
        {
            Debug.Log($"typeof: {describeMethod.Invoke(Activator.CreateInstance(
    typeofType), null)}");
        }

        // 静的解析から見えない参照：リフレクションを介して生成する
        var type = Type.GetType(ReflectionTypeName);
        if (type == null)
        {
            Debug.Log($"reflection: Type.GetType(\"{ReflectionTypeName}\") returned
    null");
        }
        else
        {
            var instance = Activator.CreateInstance(type);
            Debug.Log($"reflection: {type.GetMethod("Describe")?.Invoke(instance, null)
    }");
        }
    }
}
```

このサンプルを、Managed Stripping LevelをMinimalとHighに変えてビルドして実行すると、結果は次のようになります。

```
# Minimal
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: TypeofOnlyService is alive
reflection: ReflectionOnlyService is alive

# High
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: type TypeofOnlyService exists but Describe is stripped
reflection: Type.GetType("ReflectionOnlyService") returned null
```

Highでは、typeofのケースは型は取得できるのにGetMethodがnullを返し、リフレクションのケースはType.GetType自体がnullを返します。

まとめると下記のような挙動になります（表4.3）。

表4.3: MinimalとHighでの残り方

対象	Minimal	High
StrippingProbe	残る	残る
DirectlyUsedService（newで参照）	残る	型もメソッドも残る
TypeofOnlyService（typeofで型だけ参照）	残る	型は残るがDescribeは削除される
ReflectionOnlyService（リフレクションのみ）	残る	型ごと削除される

Minimalはユーザーコードを削除しないため、3つともそのまま残ります。

Highでは、参照のされ方の違いがそのまま結果に現れます。

DirectlyUsedServiceはnewで型を参照し、Describeも直接呼び出しているため、型もメソッドも残ります。

typeofで参照されるTypeofOnlyServiceは型としては残りますが、どこからも呼ばれていないDescribeはメンバー単位で削除されています。

そして、静的な参照がまったくないReflectionOnlyServiceは、型ごと削除されています。

なお、この実験で型名やメソッド名の文字列をすべてstatic readonlyフィールド経由で読んでいるのは、意図的な作りです。

UnityLinkerはリフレクションの典型的な使い方を一部理解しており、Type.GetType("ReflectionOnlyService") と定数文字列を直接渡した場合はHighでも型が保持されることを確認しています。

また、typeofで得たTypeに対して直接GetMethodを呼んだ場合は、メソッド名が定数でなくても、その型のpublicメソッドが一括で保持されました。

一方で、フィールド経由で渡した文字列まではUnityLinkerは追跡しません。

型名やメンバー名が設定ファイル、シリアライズされたデータ、サーバー応答など

ビルド時に確定しない経路から来る実際の状況を、フィールド読み出しで再現しています。

フィールドをconstにすると使用箇所へ定数文字列として埋め込まれるため、定数を直接渡した場合と同じ扱いになります。

DIコンテナーにおける型解決が壊れる

ここまでの例は説明のために単純化していますが、同じ構造の問題は実際のライブラリで起こります。代表的なのがDIコンテナーです。

多くのDIコンテナーは、インターフェイスと実装型の対応を登録しておき、解決時にリフレクションでコンストラクタを呼び出してインスタンスを生成します。

この動きを模した最小のコンテナーで確認します。実際のDIコンテナーと同じく、Registerで登録しResolveで解決するインターフェイスにします（リスト4.4）。

▼リスト4.4　最小のDIコンテナー

```
public sealed class MiniContainer
{
    private readonly Dictionary<Type, Type> _registrations = new Dictionary<Type, Type
    >();

    // 実装型を型引数で登録する。TImplementation は typeof 相当の静的な参照になる
    public void Register<TInterface, TImplementation>() where TImplementation :
    TInterface
    {
        _registrations[typeof(TInterface)] = typeof(TImplementation);
    }

    // 登録された実装型を、リフレクションでインスタンス化して返す
    public TInterface Resolve<TInterface>()
    {
        var implementationType = _registrations[typeof(TInterface)];
        return (TInterface)Activator.CreateInstance(implementationType);
    }
}
```

登録する実装型は、コンテナー経由でのみ使われる想定で、どこからも静的に参照しません（リスト4.5）。

▼リスト4.5　登録するインターフェイスと実装

```
public interface INotificationService
{
    string Notify();
}

// コンテナー経由でのみ使われ、どこからも静的に参照されない実装
public sealed class NotificationService : INotificationService
{
    public string Notify() => "NotificationService.Notify succeeded";
}
```

シーンに置いたMonoBehaviourから、登録と解決を行います（リスト4.6）。

▼リスト4.6　登録と解決を行うMonoBehaviour

```
public sealed class DiContainerProbe : MonoBehaviour
{
    private void Start()
    {
        try
        {
            var container = new MiniContainer();
            container.Register<INotificationService, NotificationService>();
            Debug.Log($"[DiContainerProbe] explicit: {container.Resolve<
    INotificationService>().Notify()}");
        }
        catch (Exception exception)
        {
            Debug.Log($"[DiContainerProbe] explicit failed: {exception.GetType().Name}:
     {exception.Message}");
        }
    }
}
```

これを先ほどと同じ条件でビルドすると、結果は次のようになります。

```
# Minimal
[DiContainerProbe] explicit: NotificationService.Notify succeeded

# High
[DiContainerProbe] explicit failed: MissingMethodException: Default constructor not
    found for type NotificationService
```

Highでは、Register<INotificationService, NotificationService>()の型引数が静的な参照になるため、NotificationServiceの型そのものは残ります。

また、インターフェイス経由で静的に呼ばれているNotifyも残ります。

しかし、リフレクションでしか呼ばれないデフォルトコンストラクタは静的な参照を持たないため、メンバー単位で削除されます。

その結果、Activator.CreateInstanceがコンストラクタを見つけられず、MissingMethodExceptionで解決に失敗します。

型は登録してあるのに、ビルドしたアプリでだけ解決に失敗するという不具合の原因です。

実際のDIコンテナーでも、構造は同じです。

コンストラクタの解決がリフレクション頼みである以上、明示的に登録した型でも暗黙のデフォルトコンストラクタは削除の候補になります。

アセンブリから実装型をスキャンして自動登録する方式であれば、静的な参照を持たない実装型が型ごと削除され、登録自体が失敗します。

4.2.2 Unity エントリーポイントから参照がないコードが消える

もうひとつのパターンは、シーンからもスクリプトからも参照されず、AssetBundleなどの外部コンテンツを介してのみ必要になるコードです。

前節で見たとおり、rootの供給源はビルドに含まれるシーンやアセットです。

AssetBundleとして別にビルドされるコンテンツはアプリ本体のビルドに含まれないため、そのコンテンツが参照する型はrootとして扱われません。

したがって、コンテンツ側だけが必要とする型は、アプリのコードに静的な参照がなければ、実行時に読み込まれるかどうかにかかわらず未使用と判定されます。

このパターンを、Timelineアセットで再現します。

TimelineアセットをAssetBundleにだけ含め、アプリ側はUnity.Timelineを静的に参

照せず、アセット名で読み込んでPlayableDirectorから再生します（リスト4.7）。

▼リスト4.7　AssetBundleからTimelineアセットを再生するMonoBehaviour

```
public class TimelineAssetBundleProbe : MonoBehaviour
{
    private const string BundleDirectoryName = "TimelineBundles";
    private const string BundleName = "timeline.bundle";
    private const string TimelineAssetName = "TimelineOnly";

    private void Start()
    {
        var bundlePath = Path.Combine(Application.streamingAssetsPath,
    BundleDirectoryName, BundleName);
        var bundle = AssetBundle.LoadFromFile(bundlePath);
        if (bundle == null)
        {
            Debug.Log("timeline bundle: failed to load asset bundle");
            return;
        }

        var asset = bundle.LoadAsset(TimelineAssetName);
        if (asset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} failed to load");
            bundle.Unload(true);
            return;
        }

        Debug.Log($"timeline bundle: loaded {asset.GetType().AssemblyQualifiedName}");
        var playableAsset = asset as PlayableAsset;
        if (playableAsset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} is not playable");
            bundle.Unload(true);
            return;
        }

        var director = gameObject.AddComponent<PlayableDirector>();
        director.playableAsset = playableAsset;
        director.Play();
        Debug.Log($"timeline bundle: play state={director.state}, graph valid={director
    .playableGraph.IsValid()}");
        bundle.Unload(false);
    }
}
```

AssetBundleの作成後に元のTimelineアセットをプロジェクトから削除してからアプ

リをビルドすることで、アプリとは別にコンテンツを更新する方式を模しています。

Managed Stripping LevelをMinimalとHighに変えてビルドします。どちらのレベルでも同じ結果になります。

```
# Minimal / High とも
The referenced script (UnityEngine.Timeline.TimelineAsset) on this Behaviour is missing
    !
timeline bundle: asset TimelineOnly failed to load
```

また、ビルドしたアプリのManagedフォルダーを確認すると、どちらのビルドでもUnity.Timeline.dll自体が存在しませんでした。

注意したいのは、MinimalとHighで結果が変わらないことです。

このパターンへの対処はStripping Levelの調整ではなく、必要な型やアセンブリの保持を明示することです。

その手段は次節で扱い、Addressablesを使う場合の仕組みは応用編で扱います。

参考: How code stripping affects content `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-content.html` / タイムラインアセットが消える事例（Qiita, 2021） `https://qiita.com/KyoheiOkawa/items/0cf9cc09679eababec75`

4.3 コードを削りすぎる問題への対処方法

4.3.1 基本的な対処方法

必要なコードが削られるのを防ぐ最も基本的な手段は、UnityLinkerに「このコードは使われる」と明示的に伝えることです。

Unityはそのための手段として、link.xmlファイルと[Preserve]属性を提供しています。

どちらで指定したコードも解析のrootとして扱われ、静的な参照がなくても削除されなくなります。

link.xml

link.xmlは、保持したいアセンブリや型、メンバーを列挙するXMLファイルです。

プロジェクトのAssets配下に置くと、UnityLinkerがビルド時に読み取ります。

前節のサンプルで型ごと削除されたReflectionOnlyServiceは、次のlink.xmlで保持できます（リスト4.8）。

▼リスト4.8　ReflectionOnlyServiceを保持するlink.xml

```
<linker>
  <assembly fullname="Assembly-CSharp">
    <type fullname="ReflectionOnlyService" preserve="all" />
  </assembly>
</linker>
```

このlink.xmlをAssets直下に置いてHighでビルドし直すと、実行結果が次のように変わりました。

```
# High(link.xml なし)
reflection: Type.GetType("ReflectionOnlyService") returned null

# High(link.xml あり)
reflection: ReflectionOnlyService is alive
```

ビルドに含まれるアセンブリを確認しても、ReflectionOnlyServiceがDescribeごと保持されていることが確認できます。

link.xmlは保持対象をコードの外側で指定できるため、自分では編集できないDLLやパッケージの型にも使えます。

保持範囲はアセンブリ単位、型単位、メソッドやフィールドなどのメンバー単位で指定できます。

なお、アセンブリ全体をpreserve="all"で保持する書き方は、手軽ですがそのアセンブリに対するストリッピングの効果を打ち消します。

問題の切り分けには使えますが、恒久的な対応では必要な型・メンバーまで絞りこめ

る方が良いです。

[Preserve] 属性

自分で編集できるコードであれば、UnityEngine.Scripting名前空間の[Preserve]属性を対象に直接付けられます。

注意したいのは、[Preserve]は付けた場所によって保持範囲が変わることです。

この違いを、型に付けたクラスと、メソッドにだけ付けたクラスの2つで確認します（リスト4.9、リスト4.10）。

▼リスト4.9　型にPreserveを付けた場合

```
using UnityEngine.Scripting;

// 型に [Preserve] を付ける。メソッドには付けない
[Preserve]
public sealed class TypePreservedService
{
    public string Describe() => "TypePreservedService.Describe is alive";
}
```

▼リスト4.10　メソッドにだけPreserveを付けた場合

```
using UnityEngine.Scripting;

// 型には付けず、メソッドにだけ [Preserve] を付ける
public sealed class MethodPreservedService
{
    [Preserve]
    public string Describe() => "MethodPreservedService.Describe is alive";
}
```

この2つを、静的な参照を持たない形でリフレクションから生成して呼び出すと、Highでの結果は次のようになりました（表4.4）。

表4.4: Preserveを付けた位置とHighでの保持範囲

対象	型	デフォルトコンストラクタ	Describe
TypePreservedService	残る	残る	削除される
MethodPreservedService	残る	削除される	残る

型に付けた[Preserve]は、型とデフォルトコンストラクタを保持しますが、属性を付けていないDescribeは保持しません。

メソッドに付けた[Preserve]は、その型とメソッドを保持しますが、今度はデフォルトコンストラクタが保持されず、Activator.CreateInstanceによる生成がMissingMethodExceptionで失敗します。

リフレクションで生成して呼び出すところまで動かすには、型とメソッドの両方に[Preserve]を付ける必要があります。

「[Preserve]を付けたのに動かない」という場合は、まず属性を付けた位置と保持範囲の対応を疑います。

参考: Preserving code using annotations `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-preserving.html`

4.3.2 応用編

Preserve 以外の属性の活用

[Preserve]が「この要素を残す」という点の指定なのに対し、UnityEngine.Scripting名前空間には「このインターフェイスの実装をすべて残す」のように、パターンで保持を宣言する属性があります。

「利用者が実装した型をリフレクションで探して呼び出す」といったフレームワークによくある構造では、実装型を1つずつ[Preserve]で指定する代わりに、インターフェイスや基底クラスに属性を付けることで対応できます。

- RequireImplementorsAttribute

https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireImplementorsAttribute.html：インターフェイスに付けると、そのインターフェイスを実装する型を保持する

- RequireDerivedAttribute
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireDerivedAttribute.html：クラスに付けると、その型を継承する型を保持する
- RequiredMemberAttribute
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredMemberAttribute.html：メンバーに付けると、その型が残る場合にそのメンバーも保持する
- RequiredInterfaceAttribute
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredInterfaceAttribute.html：型に付けると、指定したインターフェイスの実装情報を保持する

代表として[RequireImplementors]の挙動を確認します。

インターフェイスに属性を付け、実装はどこからも静的に参照しません（リスト4.11）。

▼リスト4.11　RequireImplementorsを付けたインターフェイスと実装

```
// インターフェイスに付けると、実装型が保持の対象になる
[RequireImplementors]
public interface IRequiredContract
{
}

// どこからも静的に参照されない実装
public sealed class RequiredContractImpl : IRequiredContract
{
}
```

Highでビルドすると、RequiredContractImplはデフォルトコンストラクタごと保持されました。

静的な参照を持たない実装型が、インターフェイス側の宣言だけで残ります。

[RequireDerived]を付けた基底クラスの派生型でも、同様に保持を確認できました。

IUnityLinkerProcessor でビルド時に link.xml を生成する

保持する対象のコードが決まっている場合は、link.xmlに対象のクラスやメンバーを指定するだけで十分です。

一方で、ビルドターゲットごとに保持したい型が違う、ビルドに含めるコンテンツによって必要な型が変わる、といった状況では、link.xmlを手で更新し続けるのは現実的ではありません。

IUnityLinkerProcessorを使うと、ビルド時に動的にlink.xmlを生成してUnityLinkerへ渡せます。

IUnityLinkerProcessorはエディター用のインターフェイスで、実装クラスをエディターアセンブリに置くとビルド時に自動で呼ばれます。

GenerateAdditionalLinkXmlFileが返したパスのlink.xmlが、UnityLinkerの入力に追加されます。

最小の実装は次のとおりです（リスト4.12）。

▼リスト4.12　link.xmlを生成する最小のIUnityLinkerProcessor実装

```
using System.IO;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEditor.UnityLinker;
using UnityEngine;

public sealed class PreserveListLinkerProcessor : IUnityLinkerProcessor
{
    private const string LinkXmlContent = @"<linker>
  <assembly fullname=""Assembly-CSharp"">
    <type fullname=""GeneratedPreserveService"" preserve=""all"" />
  </assembly>
</linker>
";

    public int callbackOrder => 0;

    // 返したパスの link.xml が UnityLinker の入力に追加される
    public string GenerateAdditionalLinkXmlFile(BuildReport report,
```

```
    UnityLinkerBuildPipelineData data)
    {
        var path = Path.Combine(Path.GetTempPath(), "managed-stripping-sample-generated
    -link.xml");
        File.WriteAllText(path, LinkXmlContent);
        Debug.Log("[PreserveListLinkerProcessor] generated 1 assembly / 1 type.");
        return path;
    }
}
```

この例では説明のために生成内容を固定していますが、実際にはこの部分を、設定アセットの読み取りやコンテンツの依存解析など、ビルド時の入力から組み立てる処理に置き換えます。

静的な参照を持たないGeneratedPreserveServiceを用意してHighでビルドすると、Assets配下にlink.xmlを置いていなくても、型・コンストラクタ・メソッドが保持されることが確認できました。

```
[GeneratedLinkXmlProbe] type=present
[GeneratedLinkXmlProbe] ctor=present, method=GeneratedPreserveService.Describe is alive
```

参考: IUnityLinkerProcessor `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Build.IUnityLinkerProcessor.html`

Addressables が生成する link.xml

Addressablesでは、前節で見た「コンテンツ側だけが必要とする型が消える」問題に、コンテンツビルドの成果物としてlink.xmlを自動生成することで対応しています。

実際に何が生成されるかを確認します。コンテンツは最小構成にし、Cubeのメッシュを表示するだけのprefab（GameObjectにMeshFilterとMeshRendererを付けたもの）を1つ作って、Addressablesのグループに登録します。

コンテンツビルドを実行すると、link.xmlが生成されます。

Addressables 4.0.1では、生成先はLibrary/com.unity.addressables/aa/<platform>/AddressablesLink/link.xmlです。

今回生成された内容は次のとおりでした（リスト4.13）。

▼リスト4.13　Addressablesが生成したlink.xml

```
<linker>
  <assembly fullname="Unity.Addressables, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.AddressableAssets.Addressables" preserve="all" />
  </assembly>
  <assembly fullname="Unity.ResourceManager, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    AssetBundleProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    BundledAssetProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    IAssetBundleResource" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.InstanceProvider"
    preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.SceneProvider"
    preserve="all" />
  </assembly>
  <assembly fullname="UnityEngine.CoreModule, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null">
    <type fullname="UnityEngine.GameObject" preserve="all" />
    <type fullname="UnityEngine.MeshFilter" preserve="all" />
    <type fullname="UnityEngine.MeshRenderer" preserve="all" />
    <type fullname="UnityEngine.Transform" preserve="all" />
  </assembly>
</linker>
```

内容は2種類に分かれています。

ひとつはAddressablesのランタイム（Unity.AddressablesとUnity.ResourceManager）のprovider型で、グループ構成に応じて必要になるものです。

もうひとつは、登録したコンテンツが参照する型です。今回はprefabを構成するGameObject、Transform、MeshFilter、MeshRendererが、そのまま保持対象に挙がっています。

Addressablesがコンテンツ側の依存を解析し、アプリのコードから参照がない型でも保持対象に含めていることが分かります。

参考: Addressables Build Artifacts `https://docs.unity3d.com/ja/Packages/com.unity.addressables@1.20/manual/BuildArtifacts.html`（link.xmlの記述は

1.20のマニュアルにあります）

リフレクションでのコード呼び出しを Source Generator による静的呼び出しに置き換える

ここまでの対処方法は、どれも必要なコードが削られないように保持を指定するものでした。

これに対して、静的解析から見えない依存を生んでいるリフレクション自体を、**Source Generator**によるコード生成で置き換える方法もあります。

Source Generatorは、C#コンパイラーがコンパイル時にソースコードを解析し、追加のソースコードを生成する仕組みです。

型の探索と生成をコンパイル時に済ませてしまえば、依存は最初から静的な参照として表現され、保持指定そのものが不要になります。

例として、コマンド名の文字列を受け取り、対応するハンドラーを実行するディスパッチ処理を考えます。

ハンドラーは属性でコマンド名を宣言します。これを実行時にリフレクションで作るなら、属性の付いた型をアセンブリから探して生成することになる構造です（リスト4.14）。

▼リスト4.14　属性でコマンド名を宣言するハンドラー

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class CommandHandlerAttribute : Attribute
{
    public CommandHandlerAttribute(string command)
    {
        Command = command;
    }

    public string Command { get; }
}

public interface ICommandHandler
{
    string Execute();
}
```

```
// ハンドラーはどこからも静的に参照されない
[CommandHandler("greet")]
public sealed class GreetCommandHandler : ICommandHandler
{
    public string Execute() => "GreetCommandHandler.Execute succeeded";
}

[CommandHandler("farewell")]
public sealed class FarewellCommandHandler : ICommandHandler
{
    public string Execute() => "FarewellCommandHandler.Execute succeeded";
}
```

2つのハンドラーはどこからも静的に参照されないため、実行時スキャンの作りのままHighでビルドすれば、前節で見たとおり型ごと削除されます。

このコマンド名に対するハンドラーの解決を、Source Generatorを用いて実現します。

[CommandHandler]が付いたクラスをコンパイル時に収集し、コマンド文字列から具象型を引くディスパッチャーを生成するジェネレーターを用意します。

ジェネレーターがやることは2段階で、前半（ForAttributeWithMetadataName）が属性付きクラスからのコマンド名と型名の収集、後半（GenerateDispatcher）が集めたペアのテンプレートへの流し込みです。

以下にジェネレーターのコード全文を示します（リスト4.15）。

▼リスト4.15　ディスパッチャーを生成するSource Generator

```
#nullable enable
using System.Collections.Immutable;
using System.Linq;
using System.Text;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.Text;
namespace StrippingSample.SourceGenerators
{
    [Generator]
    public sealed class CommandDispatcherGenerator : IIncrementalGenerator
```

```
{
    public void Initialize(IncrementalGeneratorInitializationContext context)
    {
        // [CommandHandler] が付いたクラスから、コマンド名と型名のペアを収集する
        var handlers = context.SyntaxProvider
            .ForAttributeWithMetadataName(
                "CommandHandlerAttribute",
                static (_, _) => true,
                static (syntaxContext, _) => (
                    Command: (string)syntaxContext.Attributes[0].
ConstructorArguments[0].Value!,
                    TypeName: ((INamedTypeSymbol)syntaxContext.TargetSymbol)
                        .ToDisplayString(SymbolDisplayFormat.FullyQualifiedFormat))
)
            .Collect();
        // 収集したペアからディスパッチャーを生成する
        context.RegisterSourceOutput(handlers, static (sourceContext,
commandHandlers) =>
        {
            if (!commandHandlers.IsDefaultOrEmpty)
            {
                sourceContext.AddSource(
                    "CommandDispatcher.g.cs",
                    SourceText.From(GenerateDispatcher(commandHandlers), Encoding.
UTF8));
            }
        });
    }
    private static string GenerateDispatcher(
        ImmutableArray<(string Command, string TypeName)> handlers)
    {
        // 出力を決定的にするため、コマンド名で並べ替えてから流し込む
        var cases = string.Join("\n", handlers
            .Distinct()
            .OrderBy(static handler => handler.Command, System.StringComparer.
Ordinal)
            .Select(static handler => $$"""
                        case {{SymbolDisplay.FormatLiteral(handler.Command, true)
}}:
                            return new {{handler.TypeName}}().Execute();
            """));
        return $$"""
            // <auto-generated/>
            public static class CommandDispatcher
            {
                public static string Execute(string command)
                {
                    switch (command)
```

```
                        {
                {{cases}}
                            default:
                                throw new global::System.ArgumentException($"Unknown
    command: {command}");
                        }
                    }
                }
                """;
        }
    }
}
```

コマンド名で並べ替えているのは、同じ入力から同じコードを生成し、ビルドのたびに出力が揺れないようにするためです。

このジェネレーターにより、コンパイル時に下記のようなコードが生成されます（リスト4.16）。

▼リスト4.16　生成されるCommandDispatcher

```
// <auto-generated/>
public static class CommandDispatcher
{
    public static string Execute(string command)
    {
        switch (command)
        {
            case "farewell":
                return new global::FarewellCommandHandler().Execute();
            case "greet":
                return new global::GreetCommandHandler().Execute();
            default:
                throw new global::System.ArgumentException($"Unknown command: {command}
    ");
        }
    }
}
```

実行時に文字列から型を探していた処理が、文字列と具象型の対応表に変わりました。

構築（new）とExecuteの呼び出しがどちらも生成コードに現れるため、型・コンストラクタ・メソッドへの静的な参照が生成コードの中で完結します。

利用側はコマンド文字列を渡すだけで、ハンドラーの具象型を一切参照しません。

リフレクションの実験と同じく、コマンド文字列はstatic readonlyフィールド経由で渡し、UnityLinkerが追跡できない入力のままにしています（リスト4.17）。

▼リスト4.17　生成されたディスパッチャーの利用側

```
// コマンド文字列はフィールド経由で渡し、UnityLinker が追跡できない入力にする
private static readonly string[] Commands = { "farewell", "greet" };

private static void RunGeneratedDispatcher()
{
    foreach (var command in Commands)
    {
        var result = CommandDispatcher.Execute(command);
        Debug.Log($"[SourceGeneratorProbe] dispatch command={command}, result={result}"
    );
    }
}
```

これをHighでビルドして実行すると、2つのコマンドの実行に成功することが確認できました。

```
[SourceGeneratorProbe] dispatch command=farewell, result=FarewellCommandHandler.Execute
     succeeded
[SourceGeneratorProbe] dispatch command=greet, result=GreetCommandHandler.Execute
    succeeded
```

文字列そのものはビルド時に追跡できない入力のままですが、文字列から型への対応が実行時の探索ではなく生成コードのswitchになったため、UnityLinkerからは手書きコードと変わらない通常の静的な参照に見えます。

Mono.Cecilによる確認でも、2つのハンドラーの型・コンストラクタ・ExecuteとCommandDispatcher.Executeが保持されていました。

この置き換えの利点は、ストリッピングへの耐性だけではありません。

- 依存がUnityLinkerから見えるため、link.xmlや属性による保持指定の保守が不要になる
- 実行時のリフレクション探索がなくなり、起動コストと実行時失敗の可能性が減る

- 対象の収集がコンパイル時に行われるため、登録漏れや重複をコンパイル時に診断できる

実際のライブラリでも、このような実装を採用しているケースがあります。

シリアライザーでは、MessagePack for C#がIL2CPP環境向けにコード生成を長く提供しており、現在はSource Generatorベースの生成に移行しています。

DIコンテナーでは、VContainerがSourceGeneratorで注入コードを生成して実行時リフレクションを減らす選択肢を提供しており、前節で見た「コンストラクタが消えて解決に失敗する」症状への対応をSource Generatorによって解決しています。

UnityプロジェクトへのSource Generatorの導入手順（netstandard2.0のDLLをRoslyn Analyzerラベル付きで配置する構成）は、Unityマニュアルを参照してください。

参考: Create and use a source generator `https://docs.unity3d.com/6000.3/Documentation/Manual/create-source-generator.html` / MessagePack for C# `https://github.com/MessagePack-CSharp/MessagePack-CSharp` / MemoryPack `https://github.com/Cysharp/MemoryPack` / VContainer `https://github.com/hadashiA/VContainer`

4.4 まとめ

Managed Code Strippingは、Stripping Levelを上げて終わりにできる最適化ではありません。

本章で見てきたとおり、リフレクションや外部コンテンツを介した依存は静的解析から見えず、実行時には必要なコードでもビルドから消えます。

しかも症状はビルドしたアプリでしか現れないため、見えない依存を明示することと、ビルドしたアプリで検証し続けることを、セットで運用する必要があります。

保持を明示する手段は、対象と状況で使い分けます（表4.5）。

表4.5: 対処方法の使い分け

方法	適する状況
[Preserve]	自分で編集できるコードの少数の要素を残す
link.xml	編集できないDLLを含む、決まった対象を残す
Require系属性	フレームワーク側で「実装や派生を残す」契約を宣言する
IUnityLinkerProcessor	ビルドターゲットやコンテンツによって保持対象が変わる
Source Generator	リフレクションによる探索自体を静的な参照に置き換える

あわせて、ストリッピングを効きやすくするための方針を挙げます。

- 問題になるケースをのぞき、[Preserve]やlink.xmlを過剰に利用しない
- リフレクションでの型の探索と生成を減らし、明示登録やSource Generatorによる静的な参照への切り替えを検討する
- コード保持の指定は、できる限り型・メンバー単位まで絞る。アセンブリ全体のpreserve="all"は問題の切り分けに留める
- アプリと別にビルドするコンテンツだけが使う型の保持は、手作業ではなくAddressablesのlink.xmlを利用したりIUnityLinkerProcessorによる生成といった仕組みを活用する
- Stripping Levelを上げたビルドを開発中から動かして検証する

第5章

UnityのLTO設定とAndroid・iOSでの性能比較

キム ミンヒョク Minhyuk Kim

株式会社サイバーエージェント SGEコア技術本部

Unity 6000.5に追加されたAndroid向けのリンク時最適化（LTO）設定は、エンジンコードにしか適用されません。自作のC#から生成されるコードにもLTOを適用するには、別の設定が必要です。両方の設定を試し、ベンチマークと実タイトルで性能、バイナリサイズ、ビルド時間を測りました。

5.1　IL2CPPのコンパイルと翻訳単位

UnityのAndroidビルド(IL2CPP)では、C#をリスト5.1の流れで機械語に変換します。

▼リスト5.1　IL2CPPビルドの処理フロー

```
C# ソース
  ↓ C# コンパイル
IL アセンブリ（.dll）
  ↓ IL2CPP 変換
大量の C++（.cpp）          ←「IL2CPP 生成コード」
  ↓ clang でコンパイル(.cpp ごとに個別最適化）
  ↓ リンク
libil2cpp.so                ← C# コードから生成された最終的なネイティブバイナリ
```

clangはcppを翻訳単位（通称TU）ごとにコンパイルし、オブジェクトファイルとして生成します。TUはコンパイラが個別に読み込み、最適化する単位を指します。

通常のC++ビルドでは、別のTUにある関数本体を参照できないため、短い関数でもインライン化されず、call命令が残ることがあります。IL2CPP生成コードには小さな

関数や安全チェックが多く、この制約の影響を受けやすい構造です。例えばbuf[idx]というC#の1行は、おおよそリスト5.2のC++になります。

▼リスト5.2　IL2CPPが生成する配列アクセスコードの例

```
if (buf == NULL) ThrowNullReferenceException();                // null チェック
if ((uint32_t)idx >= buf->max_length) ThrowIndexOutOfRange(); // 境界チェック
val = buf->m_Items[idx];
```

生成コードの規模が大きくなるほど、TUをまたぐ呼び出しも増え、呼び出しにかかる時間の合計も増えます。

5.2　LTO / ThinLTOとは

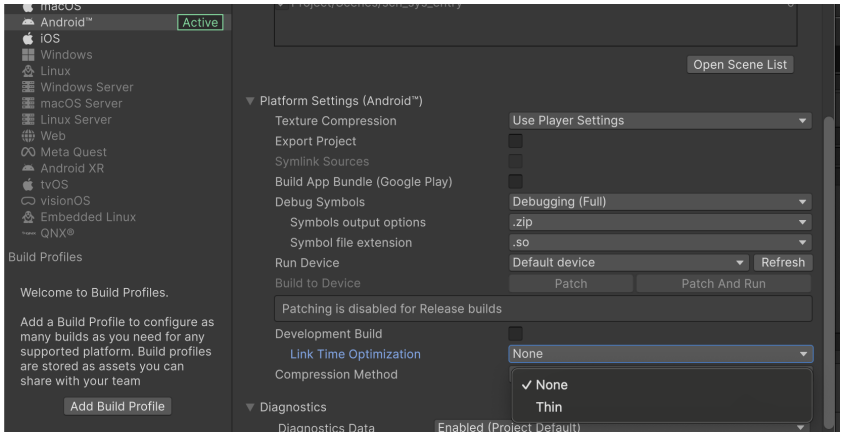
LTO(Link Time Optimization)は、リンク時にTUをまたいだインライン化、定数伝播、不要コード削除を行うclang/LLVMの仕組みです。コンパイル時にはオブジェクトファイルをLLVM bitcode（最適化前の中間表現）として出力します。リンク時にそれらをまとめて最適化し、機械語を生成します。

full LTOは、全ての中間表現を一つにまとめて最適化します。最適化できる範囲は最も広いものの、中間表現の統合と最適化が単一プロセスに集中するため、大規模なビルドでは多くのメモリを使います。今回も実タイトルのビルド中にメモリ使用量が20GBに達し、OSによってプロセスが強制終了されました。

ThinLTOは、コンパイル時に各TUのサマリを作成します。リンク時には必要な関数を他のTUから取り込み（import）、並列に最適化します。full LTOよりもメモリ使用量と処理時間を抑えやすく、大規模ビルドで広く使われています。

5.3　エンジンコードのThinLTO設定

Unity 6000.5で、Link Time Optimizationトグルが追加されました。

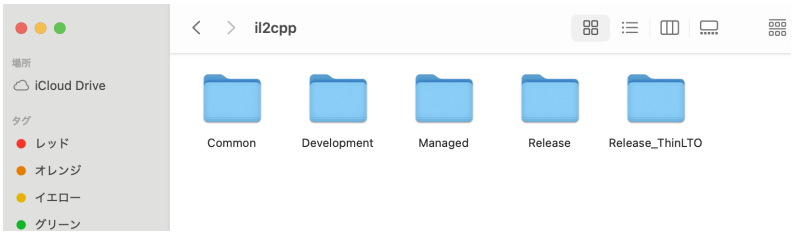


▲図5.1　Build Profiles > Player Settings > Android

このトグルを有効にすると、Unityが提供するThinLTO適用済みのエンジンバイナリが選択されます。対象はUnityエンジンコードで、IL2CPP生成コード（C#由来のコード）は含まれません。エディタのインストールフォルダには、通常版とThinLTO版の事前ビルドされたsoファイルが同梱されています。

▼リスト5.3　Android Playerに同梱されるlibunity.soのバリアント

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/
├── Release/Libs/arm64-v8a/libunity.so          29,032,960 bytes
└── Release_ThinLTO/Libs/arm64-v8a/libunity.so  30,042,536 bytes (+3.5%)
```



▲図5.2　エディタのインストールフォルダ。ReleaseとRelease_ThinLTOが両方とも同梱されており、ビルド時にどちらかが選ばれます

5.3.1　有効条件と、満たさない場合の動作

トグルが有効になる条件は次の3つです。いずれかを満たさない場合、エラーや警告は出ず、LinkTimeOptimizationはNoneとしてビルドされます。

1. IL2CPPスクリプティングバックエンド
2. Strip Engine Codeが無効(未ストリップのprebuiltをそのまま使うため)
3. 非Developmentビルド

Development Buildが有効な場合は、トグル自体が表示されません。適用の有無は、リスト5.4の2点で確認できます。

▼リスト5.4　エンジンコードLTOトグルの適用確認コマンド

```
# ① ビルド入力で ThinLTO が指定されているか
grep -o 'LinkTimeOptimization"[: ]*"[A-Za-z]*"' Library/Bee/Player*-inputdata.json
# ② APK 内の libunity.so が Release_ThinLTO 版と一致するか
unzip -p out.apk lib/arm64-v8a/libunity.so | md5
```

5.4　エンジンコードLTOトグルの効果

同梱されている通常版とThinLTO版のバイナリを比較し、変更が多いサブシステムを中心に実機で性能を測定しました。

5.4.1　同梱バイナリの関数サイズを比較する

Unityは両バリアントのシンボル付きの.soをエディタのインストールフォルダに同梱しています。

▼リスト5.5　Android Playerに同梱されるシンボル付きlibunity.so

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/
├── Release/Symbols/arm64-v8a/libunity.sym.so
└── Release_ThinLTO/Symbols/arm64-v8a/libunity.sym.so
```

`llvm-nm --defined-only --print-size --demangle`で得た関数名を照合し、名前空間とクラス名を使ってサブシステム別に集計しました（表5.1）。

表5.1: エンジンコードLTOトグルによるサブシステム別の関数差分

サブシステム	関数数 (Release → ThinLTO)	サイズが変わった関数数	サイズ変化率	.text差
Particle	1,419 → 1,290	1,043	80.9%	+49.66%
Serialization/Asset	2,890 → 1,387	1,040	75.0%	-16.61%
Mesh/Geometry	1,055 → 765	483	63.1%	+6.79%
Scripting	8,924 → 7,680	4,302	56.1%	+10.27%
Gfx/Shader/Texture	5,062 → 3,935	2,044	52.1%	+24.21%
Physics	2,680 → 2,439	1,229	51.1%	+15.92%
NavMesh	739 → 582	296	51.1%	+11.87%
XR/VFX/Video/その他	3,828 → 2,483	1,230	49.6%	+13.94%
Transform/Scene/Obj	2,027 → 1,630	781	47.9%	+10.95%
Animation/Mecanim	3,226 → 2,428	1,134	46.8%	+13.18%
Box2D	2,169 → 1,562	712	46.4%	-0.43%
JobSystem/Threading	244 → 181	78	43.6%	+3.16%
UI/UIToolkit	883 → 701	293	42.0%	+1.38%
TextCore/Font/ICU	3,500 → 2,101	861	41.9%	-5.73%
(未分類)	29,193 → 21,077	7,511	37.6%	+3.37%
Renderer/Culling	2,789 → 2,199	785	35.7%	+27.58%
C++標準/圧縮/インフラ	13,158 → 9,484	3,071	32.6%	-13.31%
curl/TLS/HTTP	1,809 → 1,558	318	21.0%	+1.17%
Audio/FMOD	2,996 → 2,680	496	18.5%	+2.77%
Android/JNI	1,445 → 1,186	193	16.3%	-5.52%
PhysX	6,000 → 5,997	0	0.0%	-0.05%
合計	96,036 → 73,345	27,900	38.8%	+4.41%

※「サイズが変わった関数数」は関数の本数です。「サイズ変化率」は、ReleaseとThinLTOの両方に存在する関数のうち、サイズが変わった関数の割合を表します。

全体の関数数は96,036から73,345に減りましたが、`.text`のサイズは4.41%増えました。

5.4.2 13本のワークロードによるベンチマーク

表5.1を基にして、1つのAPKに13本のワークロードを入れたベンチマークシーンを作りました。

表5.2: エンジンコードLTOトグルのベンチマークワークロード

ワークロード	対象サブシステム	内容
`particle`	Particle	20,000粒子 + noise/size/colorモジュール、`Simulate`
`gfx`	Gfx/Shader	`MaterialPropertyBlock`に16プロパティ × float/vector + matrixを書いて`Clear`
`cull`	Renderer/Culling	`CalculateFrustumPlanes` + 4,096 AABBの`TestPlanesAABB`
`physbind`	Physicsバインディング	`Rigidbody` / `Collider`プロパティ往復
`anim`	Animation	legacy `Animation` 48リグ × 8ボーン × 10カーブを`Sample()`
`json`	Serialization	`JsonUtility`の往復
`instantiate`	Serialization	`Instantiate` → `DestroyImmediate`(ネイティブのclone/transfer経路)
`xform`	Transform/Scene	階層つきTransformの書き換え
`bind`	Scriptingバインディング	極小のバインディング呼び出しを大量に
`mesh`	Mesh/Geometry	96×96手続きメッシュの再構築
`phys2d`	Box2D	2D剛体の`Simulate`
`text`	TextCore/Font	`TextGenerator.Populate`
`physics`	PhysX	3D剛体の`Simulate`

※ 表5.1のサブシステムのうち、1サンプル100ms程度の安定した合成負荷をC#から作るのが難しいものは、測定対象から外しました。これらについては性能への影響を評価していません。

5.4.3　ワークロード別の測定結果

`libunity.so`だけが異なるAPKをPixel 8a（Tensor G3）で、先ほど紹介したワークロードを測定しました。Release / ThinLTO / ThinLTO / ReleaseのABBA順を3回繰り返し、合計12セッションを実行しています。セッションごとにアプリを再インストールし、間には90秒のクールダウンを挟みました。

値は1回の操作にかかる時間（ns/op）です。操作の単位はワークロードごとに異なるので、絶対値は同じ行の中で比較してください。

表5.3: エンジンコードLTOトグルのワークロード別ベンチマーク結果

ワークロード	Release (ns/op)	ThinLTO (ns/op)	差
`gfx`	990.46	852.49	-13.93%
`cull`	96,520	87,499	-9.35%
`text`	275,043	265,123	-3.61%
`instantiate`	14,620	14,206	-2.83%
`xform`	6,071.8	6,008.1	-1.05%
`phys2d`	222,269	220,327	-0.87%
`mesh`	2,340,210	2,334,143	-0.26%
`json`	15,864.8	15,934.4	+0.44%
`bind`	140.80	141.47	+0.47%
`physbind`	607.28	610.74	+0.57%
`particle`	265,635	267,335	+0.64%
`anim`	28,100	28,563	+1.65%
`physics`	484,641	482,796	-0.38%
[合成: 幾何平均]	21,918	21,380	-2.46%

※ 単位が行ごとに違うため、合成値は比率としてのみ意味を持ちます。

改善が大きかった`gfx`と`cull`は、C#から小さな関数を1サンプルあたり数百万回呼び出す処理です。`text`の`TextGenerator.Populate`も内部でグリフ単位のループを実行します。1回の処理が小さく、呼び出し回数が多い経路で、TUをまたぐインライン化の効果が大きく現れました。

5.4.4 導入時の性能・サイズ・メモリの比較

今回の測定では、バインディング呼び出しの多い一部の経路でCPU時間が短縮する効果がありました。ただし、実タイトルでの改善幅は、フレーム内でこうした呼び出しが占める割合に左右されます。

導入時に払わないといけないコストを表5.4にまとめました。

表5.4: エンジンコードLTOトグル採用時のコスト

区分	実測差
Strip Engine Code OFFによるサイズ増加	APK +5.67MB / インストール +20.3MB
Strip Engine Code OFFによるメモリ増加	RSS +18.0MB / うち回収されにくい領域 +8.0MB
engine ThinLTO本体	APKサイズ +398KB(30,621,595 → 31,019,967)/ メモリは未測定

※ インストールサイズの増分は、APKの増分と、端末で展開された`.so`の増分（arm64の`.so`合計で +14.65MB）の合計です。

Unityは同じトグルについて、5プロジェクトでコールドスタートを10回ずつ測定した結果を公開しています。CPU時間は13.88msから13.36ms（-3.7%）、初期表示までの時間は247.96msから229.90ms（-7.3%）に短縮し、APKサイズは +3.3%、ビルド時間への影響は軽微と報告されています[*1]。

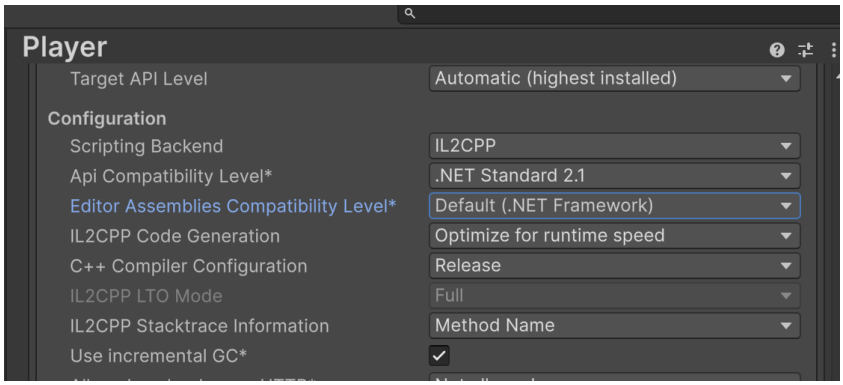
CPUの改善幅は今回の測定と近い水準です。ただし、公開値のAPKサイズ +3.3%はStrip Engine Codeを無効にした構成どうしの比較のため、無効化に伴うインストールサイズ +20.3MB、メモリ +8.0MBの増加は含まれません。

[*1] Unity Androidプラットフォームチーム「Boosting Android performance with LTO optimizations in Unity 6.5」`https://discussions.unity.com/t/boosting-android-performance-with-lto-optimizations-in-unity-6-5/1720008`。以降、本章で「Unityの公開値」と書くのはこの記事の数字です。

5.5 `libil2cpp.so`にLTOをかける方法

C#から変換されたコードを含む`libil2cpp.so`にLTOを適用するには、UnityのMaster構成を使う方法と、clangフラグを直接渡す方法があります。

5.5.1 Master構成とIL2CPP LTO Modeの設定



▲図5.3　IL2CPP LTO ModeとC++ Compiler Configurationの設定画面

Player Settings > ConfigurationのC++ Compiler ConfigurationをMasterにすると、`libil2cpp.so`へのLTOが有効になります。`IL2CPP LTO Mode`は、その際に使うThin / Fullのモードを選ぶ項目です。

Unityの公式記事によると、IL2CPP LTO Modeは6000.5から追加されました。6000.5より前のAndroidでは、LTOの有無についてReleaseとMasterに差はありません。ただ6000.5でAndroidプラットフォームではトグルの選択はできずThin固定で、6000.6からMaster構成でThin / Fullを選択できるようになりました。6000.7時点でも、Release構成のままLTOだけを有効にする項目はありませんでした。

Player SettingsのC++ Compiler Configurationに対応するフラグは、Beeビルドグラフの Il2CppCompilerConfigurationから確認できます。

表5.5: UnityのC++ Compiler Configuration別コンパイラフラグ

バージョン	Release	Master
Android 6000.3 (arm64)	-Os(サイズ優先)、LTOなし	-Os、LTOなし(同一)
Android 6000.5 (arm64)	-Os	-O2(速度とコードサイズのバランスを取る最適化) + ThinLTO
iOS 6000.3 / 6000.5	-O3(コードサイズが増える可能性と引き換えに、より積極的に速度を追う最適化)、LTO OFF	-O3、LTO OFF(同一)

5.5.2 Release構成に-flto=thinを追加する

先述の通り、Master構成に切り替えることが最速でLTOの効果を得られる道ですが、LTOと同時に最適化レベルも-Osから-O2に変わり（表5.5）、デメリットとしてサイズとビルド時間が増えます。Release構成を維持したままThinLTOだけを適用すれば、LTOの効果を分けて測れるうえ、サイズとビルド時間の増加も抑えられます。自作のC#から生成されたlibil2cpp.soにThinLTOを適用するには、リスト5.6のようにclangフラグを直接渡す方法が考えられます。

▼リスト5.6　Release構成でThinLTOを有効化するビルド前処理

```
// 既存の追加引数を退避 → 追記 → 必ず元に戻す
var saved = PlayerSettings.GetAdditionalIl2CppArgs();
try
{
    var extra = "--compiler-flags=-flto=thin --linker-flags=-flto=thin";
    PlayerSettings.SetAdditionalIl2CppArgs(
        string.IsNullOrEmpty(saved) ? extra : saved + " " + extra);
    BuildPipeline.BuildPlayer(options);
}
finally
{
    PlayerSettings.SetAdditionalIl2CppArgs(saved);
}
```

適用できたかは、2段階で確認します（リスト5.7）。

1つ目は、ビルドコマンドにフラグが渡ったかです。Unityのビルドシステム（Bee）は、実行したコンパイルとリンクのコマンドをLibrary/Bee/Player*.dag.jsonに記

録します。このファイルを`-flto=thin`で検索し、IL2CPP生成コードのコンパイルと`libil2cpp.so`のリンクの両方に含まれているかを見ます。

2つ目は、生成されたオブジェクトファイルがLLVM bitcodeになっているかです。LTOを有効にすると、コンパイル結果の`.o`は機械語ではなくbitcodeになります。1つ目で見つけたコンパイルコマンドの出力ファイルを1つ選び、先頭4バイトを表示します。

▼リスト5.7　IL2CPP ThinLTOの適用確認コマンド

```
# 1. コンパイルとリンクのコマンドに -flto=thin があるか
rg -n -- '-flto=thin' Library/Bee/Player*.dag.json
# 2. 生成オブジェクトの先頭4バイト(1. で見つけた .o を指定)
xxd -l 4 Library/Bee/artifacts/Android/.../GeneratedCode.o
```

bitcodeなら`4243 c0de`（`BC..`）、通常の機械語のオブジェクトなら`7f45 4c46`（`.ELF`）が表示されます。フォルダ内の`.o`を適当に選ぶと、LTOの対象外であるサードパーティ製ライブラリのオブジェクトを拾うことがあります。2つ目のコマンドには、必ず1つ目の検索結果に出てきたファイルを指定してください。

5.5.3　ベンチマークの設計

合成コードの生成

このベンチマークでは、通常のコンパイルでは最適化できないTUをまたぐ呼び出しを主な測定対象にします。asmdefの境界とC++のTU境界は同じ概念ではないため、IL2CPP変換後の生成C++とBeeのコンパイルグラフで、呼び出し元と呼び出し先が別TUになることも確認しました。コードジェネレータで生成した内容は次のとおりです。

- asmdefアセンブリ16個 × クラス24個 × メソッド8個 = 3,072メソッド
- 呼び出しチェーンが1周で16のアセンブリ境界を通過、全メソッドが共有アセンブリのヘルパー(`Core.Mix` / `Core.Idx` / 構造体演算)も追加で呼ぶ
- ベンチマークが全クラスを巡回するためmanagedストリッピングに消されない
- メソッドごとに定数を変えてICFによる折り畳みを防止

生成する3,072個のメソッドの例をリスト5.8に示します。

▼リスト5.8　合成ベンチマークで生成するメソッドの例

```
public static int M2(int sel, int x, int[] buf)
{
    x = Core.Mix(x, 655663086);       // 共有アセンブリへの境界越え呼び出し
    x += buf[Core.Idx(x, 7)];         // 境界チェック付き配列アクセス(Idx は & 1023 でマスク)
    return M3(sel, x, buf);           // 小関数チェーン
}
```

2つのワークロード

表5.6: IL2CPP ThinLTOの合成ベンチマークワークロード

ワークロード	何を測るか
direct	クロスアセンブリ静的呼び出しチェーン(LTOが最も効くはずの形)
iface	同種の演算をインターフェイスディスパッチ経由で実行(LTOが呼び出し先を直接化しにくい形)

どちらもlibil2cpp.soの中だけで完結します。各構成で生成されたバイナリは、APKから抽出したlibil2cpp.soのmd5で毎回確認しています。1サンプルが約100msになるよう、実行時に反復回数を調整します。実行ごとに回数が異なるため、1 passあたりの時間に正規化して比較します。

ifaceでは、各アセンブリにIGenOpを実装したクラスを4個ずつ（計64個）生成し、ベンチマークがインターフェイス経由で順に呼び出します（リスト5.9）。メソッド本体ではdirectと同じCore.Mix / Core.Idxを呼びますが、どのクラスのApplyが呼ばれるかは実行時に決まります。

▼リスト5.9　ifaceワークロードで生成するクラスと呼び出しループ

```
sealed class Op0 : IGenOp
{
    int m_State = 20612;
    public int Apply(int x, int[] buf)
    {
        x = Core.Mix(x, 347275849);              // 共有アセンブリへの境界越え呼び出し
        x += buf[Core.Idx(x ^ m_State, 7)];    // directと同種の配列アクセス
        m_State = (m_State + x) | 1;
        return x;
    }
}

// ベンチマーク側：64個の実装をインターフェイス経由で巡回
for (int i = 0; i < ops.Count; i++)
    a += ops[i].Apply(a ^ p, buf);        // 呼び出し先は実行時に決まる
```

5.5.4　CPU実測(Pixel 8a)

表5.7: Pixel 8aにおけるIL2CPP ThinLTOのCPU実測結果

構成	direct (us/pass)	iface (us/pass)
off(Release, -Os)	19.88	0.404
thin(手動ThinLTO)	15.54 (-21.8%)	0.375 (-7.3%)

手動ThinLTOにより、directは21.8%、ifaceは7.3%短縮しました。

directの関数呼び出しと命令数

§5.3.1と同じ呼び出し構造について、LTOの有無で逆アセンブル結果を比較しました。

表5.8: directワークロードの逆アセンブル比較

C000_M2	LTOなし	ThinLTO
命令数	15	11
関数呼び出し(bl)	2	0
スタックフレーム	あり	なし
callee-savedレジスタの退避	x19、x20、x29、x30	なし
メモリアクセス	5回	1回

`Core.Mix`と`Core.Idx`の本体が呼び出し元へ展開され、`bl`命令2個と、スタックやレジスタの退避処理が不要になりました。両関数をまたいだ式もまとめて最適化されます。directの21.8%短縮は、この変化が3,072メソッド分積み重なった結果です。

ifaceの改善が小さい理由

IL2CPPが生成するinterface呼び出しは、概略するとリスト5.10の形です。

▼リスト5.10　IL2CPPが生成するinterface呼び出しコードの概略

```
const VirtualInvokeData& invokeData =
    il2cpp_codegen_get_interface_invoke_data(slot, obj, declaringInterface);
return ((Func)invokeData.methodPtr)(obj, p1, p2, invokeData.method);
```

呼び出し先は実行時に得るinvokeData.methodPtrなので、通常のC++ vtable呼び出しのようには直接化できません。ただし、メソッド本体から別TUのヘルパーを呼ぶ部分はインライン化でき、ifaceでも7.3%短縮しました。

interface / virtual呼び出しに時間がかかっている場合は、sealed、具象型での受け取り、ジェネリクスによる静的ディスパッチも検討できます。ThinLTOで改善しやすいのは、別TUの小さな関数を静的に呼び出す部分です。

5.5.5　サイズとビルド時間

表5.9: IL2CPP ThinLTOのサイズとビルド時間

構成	libil2cpp.so (MB)	同.text	コールドビルド
off	37.36	基準	62s
thin	38.16 (+2.2%)	+2.3%	71s (+15%)

この構成では、インライン展開の影響でlibil2cpp.soが2.2%増えました。LTOは不要コードを削除する場合もあるため、サイズ変化の方向はコードベースと最適化内容に依存します。コールドビルドは62sから71sへ15%増えました。ThinLTOでは、TUをまたいだ最適化と機械語の生成をリンク時に行うため、その分リンクの時間が延びます。

5.6　Master構成で有効になる最適化

Master構成ではLTOと同時に最適化レベルも変わります（表5.5）。CPU時間の短縮がどの設定によるものかを調べるため、それぞれを分けて測定しました。最適化レベル(-Os → -O2)とLTOの影響を個別に調べるため、{-Os, -O2} × {LTOなし, ThinLTO} の2×2を実測しました(Android 6000.5 / arm64)。

表5.10: 最適化レベルとThinLTOのCPU実測比較

direct (us/pass)	LTOなし	ThinLTO
-Os(Release)	19.64(基準)	15.54 (-20.9%)
-O2(Master)	18.47 (-5.9%)	15.54 (-20.9%)

表5.11: 最適化レベルとThinLTOのlibil2cpp.soサイズ比較

libil2cpp.so	LTOなし	ThinLTO
-Os	37.36MB	38.16MB (+2.2%)
-O2	39.14MB	44.14MB (+12.8%)

CPU時間の短縮率は`-O2`単独で5.9%、ThinLTOを含む構成で20.9%でした。このベンチマークでは、Master構成の改善の大半をThinLTOで得られたことになります。Release +手動ThinLTOとMasterのCPU時間は同じ15.54usでしたが、最終的な`libil2cpp.so`のサイズは38.16MBと44.14MBで、Release +手動ThinLTOの方が約13.5%小さくなりました。

表5.12: Release + ThinLTOとMasterのコールドビルド時間

構成	コールドビルド
Release (`-Os`、LTOなし)	62s
Release + ThinLTO	71s (+15%)
Master (`-O2` + ThinLTO)	86s (+39%)

コールドビルドは71sと86sで、ReleaseにThinLTOだけを追加した構成の方が15秒短くなりました。Unityの公開測定でも、Master構成(`libil2cpp.so`側)はCPU -2.3%、初期表示までの時間 -5.7%に対して、`libil2cpp.so`サイズ +18.7%、ビルド時間 +30〜35%でした。CPUの改善幅はUnityの -2.3%に対して、本章のベンチマークでは -20.9%でした。ただし、Unityは実プロジェクトのメインスレッド全体、本章は小さな関数を繰り返し呼ぶ経路だけを測っているため、数字をそのまま比較することはできません。一方、サイズとビルド時間は本章のMasterでも +18.2%、+38.7%で、Unityの公開値と近い結果です。

5.7 実タイトルでの検証

ベンチマークより規模の大きい2つの実タイトルでもThinLTOを試しました。Unityバージョン、アセンブリ数、ソースコード量、バイナリサイズを表5.13に示します。

表5.13: 実タイトルのUnityバージョンとコード規模

項目	タイトルA	タイトルB
Unityバージョン	6000.5.3f1	6000.3.15f1
プレイヤーに乗るアセンブリ数	158	285
コンパイルされる`.cs`	6,969ファイル	16,700ファイル
同 行数	約138万行	約319万行
うち自社コード	1,943ファイル / 約35万行	8,606ファイル / 約154万行
`libil2cpp.so`(非圧縮)	189.0MB	294.3MB

5.7.1 Release + ThinLTOのサイズと適用確認

表5.14: タイトルAのサイズとビルド時間

項目	baseline	ThinLTO	差分
libil2cpp.so(非圧縮)	189.01MB	194.70MB	+5.69MB (+3.0%)
同`.text`	8.54MB	8.98MB	+0.44MB (+5.1%)
APK	165.85MB	166.94MB	+1.08MB (+0.65%)
プレイヤービルド部分	約396s	約433s	約 +9%

`.text` +5.1%はベンチマーク(+2.3%)より大きく、asmdefを細かく分けるほどモジュールをまたぐインライン化が増えるという予想に合う結果でした。189MBのlibil2cpp.soでも、ローカルマシンでThinLTOのリンクを完了できました。

タイトルBでは、`llvm-readelf -S`で`libil2cpp.so`のセクションごとのサイズを比較しました（表5.15）。

表5.15: タイトルBのlibil2cpp.soセクション差分

項目	baseline	ThinLTO	差分
libil2cpp.so(ファイル全体)	294.30MB	304.12MB	+3.34%
`il2cpp`	145.85MB	162.22MB	+11.23%
`.text`	10.69MB	11.14MB	+4.18%
`.rela.dyn`	50.28MB	45.01MB	-10.48%
`.got`	1.76MB	1.4KB	-99.92%
`.gcc_except_table`	3.70MB	3.62MB	-2.32%

`.text`の増加率4.18%は、タイトルA（5.1%）とベンチマーク（2.3%）の間に収まりました。`.got`は1.76MBから1.4KBに減っています。LTOによってモジュールをまたぐ参照が解決され、GOT経由の間接参照が直接参照へ変換されたことを示す結果です。動的再配置エントリを含む`.rela.dyn`の10.5%削減も、この変化に対応します。

5.7.2 実タイトルでのMasterとfull LTO (タイトルA)

§6のMaster構成とfull LTOの比較を、実タイトル規模で試しました。

表5.16: タイトルAのビルド構成比較

タイトルA	baseline	Release + thin	Master	full LTO
libil2cpp.so	189.0MB	194.7MB (+3.0%)	222.2MB (+17.6%)	–
ビルド	453s	465s	537s(9分)	2,903s(48分)後に失敗
リンカ経路のピークRSS	–	–	12.9GB	20GB → `Killed: 9`(OOM)

full LTOは48分のビルド後、リンク中のメモリ使用量が20GBに達し、OSによって強制終了されました。§5.3のベンチマークでも、full LTOはCPU時間がThinLTOと同等のまま、コールドビルドが71sから247sへ約3.5倍になりました。このタイトルではビルドを完了できず、ThinLTOの方が実用的でした。

Masterの`libil2cpp.so`のサイズはbaselineから約33MB増加しましたが、CPU性能

はRelease + thinと同等でした。Release + thinではサイズの増加が約5.7MBに収まるため、このタイトルでもMaster同等の性能をより小さいバイナリで得られました。

5.7.3 実タイトルでのCPUサイクル (タイトルB)

アウトゲームの定常画面で、Pixel 8aの全スレッドのCPUサイクルを合算して測りました。ThinLTOとbaselineを交互に3セッションずつ測定し、各セッションの値は20秒の計測を6回行った平均です。

表5.17: タイトルBのセッション別CPUサイクル

セッション	総サイクル (Gcyc)	UnityMain (Gcyc)
ThinLTO #1	44.88	27.48
ThinLTO #2	45.18	27.36
ThinLTO #3	45.54	27.60
baseline #1	46.06	29.05
baseline #2	45.71	27.89
baseline #3	46.15	28.23
平均(ThinLTO vs baseline)	45.20 vs 45.97 → -1.68%	27.48 vs 28.39 → -3.21%

5.7.4 Unity公開値との比較

Unityが公開したMaster構成（`libil2cpp.so`側）の測定値を、タイトルBの結果と比較します。§6のベンチマーク結果も併記しました。

表5.18: Unity公開値と本章の実測結果の比較

指標	Unity公開(6プロジェクト・Master)	ベンチマーク	タイトルB
CPU	メインスレッド -2.3%	`direct` -21.8%	UnityMain -3.21% / 総サイクル -1.68%
`libil2cpp.so`	+18.7%	+2.2% (Release+thin)/ +18.2%(Master)	+3.0%
ビルド時間	+30〜35%	+15%(Release+thin)/ +38.7%(Master)	+9〜15%

UnityのCPU時間短縮率 -2.3%と、タイトルBのUnityMain -3.21%は、どちらも数%の改善でした。ベンチマークの -21.8%との差は、実タイトルの測定にはLTOの対象外

の処理も含まれることで説明できます。Unityが測定した6プロジェクトでも同様の傾向が見られます。

サイズとビルド時間は構成によって差が出ました。Unityの公開値はMaster構成でそれぞれ +18.7% / +30〜35%です。本章で測ったMaster構成の +18.2%（44.14MB / 37.36MB）/ +38.7%（86s / 62s）とも近い値でした。基準となる`libil2cpp.so`のサイズも37.26MB / 37.36MBと近くなっています。

ReleaseにThinLTOだけを追加した場合は、同等のCPU性能を、サイズ +2〜3%、ビルド時間 +9〜15%で得られました。§6の2×2比較から、Master構成でのサイズとビルド時間の増加は、主に`-O2`への変更によるものと考えられます。

5.7.5 最適化の対象範囲と実タイトルでの改善幅

タイトルBでの改善はメインスレッドに集中しています。UnityMainが -3.21%だったのに対し、非UnityMain全体は +0.8%で、セッション間のドリフト4%の範囲内でした。

バイナリでも、`libil2cpp.so`以外の`.so`はほぼ変化していません。ネイティブプラグイン2本はmd5が完全に一致し、残る1本の`.text`の差は +0.008%で、ビルドの非決定性の範囲でした。今回の`--compiler-flags`による最適化は、これらのプラグインには及んでいません。

UnityMainの -3.2%は、ベンチマークの -22%より小さくなりました。この画面では、表情・まばたき・髪や衣装の揺れ・IKなどがジョブとして実行されます。これらは`libil2cpp.so`の外にあるネイティブバイナリに含まれ、今回のフラグによる最適化の対象外です。

対象外の処理を除いたIL2CPP生成コードだけの改善幅は、UnityMain全体の -3.21%より大きいと考えられます。ただし、正確な値を求めるには、プロファイラで対象内外の処理時間を分離する必要があります。ジョブはスケジューリングによってメインスレッドでも実行されるため、UnityMainの測定値にも対象外の処理が含まれます。

5.8 iOSのThinLTO適用

タイトルBのiOS版でもThinLTOを適用しました。検証用ベンチマークでは効果を確認できましたが、実タイトルではビルド時間の増加に見合う改善は得られませんでした。

5.8.1 iOSでのLTOの構造

表5.19: AndroidとiOSにおけるLTO構造の比較

観点	Android	iOS
il2cppの出力	`libil2cpp.so`	`libGameAssembly.a`
最終リンク	il2cppが呼ぶLLD	Xcodeが`UnityFramework`を作るときのld64
渡す引数	`--compiler-flags=-flto=thin --linker-flags=-flto=thin`	`--compiler-flags=-flto=thin`だけ
ビルド時間が増える場所	Unityのビルドステップ	`Build ipa`(xcodebuild)ステップ

iOSでは、il2cppはアーカイブを作成するだけなので、`--linker-flags`は渡しません。LTOのコード生成は、このbitcodeを含むアーカイブを読み込むXcode側のリンク時に行われます。

生成コードのコンパイルはil2cppが担当するため、フラグは`AdditionalIl2CppArgs`に指定しなければなりません。Xcodeの`LLVM_LTO = YES_THIN`を変更しても、生成コードには適用されません。

エンジンコードは事前ビルド済みの`libiPhone-lib.a`として提供され、bitcodeを持ちません。そのためLTOの対象外となります。AndroidのエンジンコードLTOトグル（§3）のように、LTO適用済みのエンジンへ差し替える設定もありませんでした。

5.8.2 iPhone 13の検証用ベンチマーク

まず、§5.3と同じ検証用ベンチマークをiOSに移植し、生成コードへのThinLTOがiOSでも機能するかを確かめました。`direct`は16個のasmdefに分かれた3,072個の小さなメソッドをまたいで呼ぶ、ThinLTOが最も効果を出しやすい構成です。したがって、ここでの改善率は一般的なタイトルでそのまま得られる値ではなく、条件が合った場合の上限に近い値として扱います。

端末はiPhone 13（`iPhone14,5` / iOS 18.7.8）です。Unity 6000.5.3f1、IL2CPP Release、OptimizeSpeedを使いました。baselineとThinLTOの違いは`--compiler-flags=-flto=thin`の有無だけです。Xcodeは`ReleaseForRunning`、開発署名は`get-task-allow=true`で、UnityのDevelopment Buildは無効にしています。

セッションごとにアンインストール、再インストール、起動を行い、5回のウォームアップ後に30サンプルを記録しました。測定順はbaseline / ThinLTO / ThinLTO / baselineを3ブロック、合計12セッションです。全セッションでサーマル状態はNominalでした。処理回数はセッションごとに自動調整されるため、`direct`と`iface`は1 pass当たり、Physicsは1 step当たりに正規化しています。

表5.20: iPhone 13の検証用ベンチマーク結果

ワークロード	baseline	ThinLTO	差分
`direct`	21,231.31ns/pass	16,184.31ns/pass	-23.77%
`iface`	366.96ns/pass	370.39ns/pass	+0.93%
Physics(対象外の対照)	147.82us/step	147.36us/step	-0.31%(分布は重複)

`direct`は3ブロックすべてで -23.6〜-23.9%となり、baselineとThinLTOの全セッションが完全に分離しました。Androidの -21.8%(§5.4)とも近く、生成コードへのThinLTOがiOSでも実際に機能することを確認できます。一方、`iface`はわずかに悪化し、対象外のPhysicsには差が出ませんでした。ThinLTOの効果は、静的な小関数呼び出しがTUをまたぐ経路に偏っています。同じベンチマークのバイナリサイズを表5.21に示します。

表5.21: iPhone 13検証用ベンチマークのバイナリサイズ

項目	baseline	ThinLTO	差分
UnityFramework	98.76MB	107.52MB	+8.76MB(+8.87%)
`__text`	20.09MB	20.46MB	+0.37MB(+1.83%)
`il2cpp`	22.49MB	30.78MB	+8.30MB(+36.89%)

5.8.3 タイトルBの結果とiOSで採用を見送った理由

次に、タイトルBのiOS版にThinLTOを適用しました。UnityFrameworkのサイズを表5.22、ビルド時間を表5.23に示します。

表5.22: iOS UnityFrameworkのセクション差分

セクション	baseline	ThinLTO	差分
UnityFramework(ファイル全体)	316.53MB	393.47MB	+24.31%
`__TEXT`(セグメント)	261.83MB	338.53MB	+29.29%
`il2cpp`	167.02MB	240.26MB	+43.85%
`__text`	57.99MB	61.70MB	+6.41%
`__gcc_except_tab`	6.85MB	6.63MB	-3.16%
`__unwind_info`	3.44MB	3.38MB	-1.75%
`__got`	18.8KB	18.8KB	±0
ipa(アーカイブ全体)	233.83MB	246.93MB	+5.60%

表5.23: iOSのビルド時間

ステップ	baseline	ThinLTO
`Build ipa`(xcodebuild)	357s(5m57s)	3,094s(51m34s)

UnityFrameworkは +24.31%、ipaは +5.60%増えました。ビルド時間は約9倍、46分の増加でした。増加分の大半はXcode側のリンクと`dsymutil`によるもので、Androidの +9〜15%に比べて大幅に長くなっています。`GCC_OPTIMIZATION_LEVEL`を`-Os`に下げた構成（`-Os` + thin）でも53m51sかかり、短縮できませんでした。

タイトルBの定常画面では、`cpu/wall`の単純平均はThinLTOが +1.08%とわずかに悪化し、サーマル状態が揃った組でも -1.45%の改善にとどまりました。baselineとThinLTOの分布は重なっており、大きなCPU改善を分離できませんでした。メモリ差

の +0.40%もセッション内のばらつき以下です。検証用ベンチマークの -23.77%は、ThinLTOに有利な呼び出しを集めた場合の値であり、実タイトル全体には当てはまりませんでした。

実タイトルで明確な改善を確認できず、コールドビルドやCIで毎回約46分の追加時間が必要になるため、今回の検証ではiOSへの手動ThinLTOを不採用としました。

5.9 自分のタイトルに導入するには

Androidでは、まずRelease構成に`-flto=thin`を追加する方法を勧めます。今回のベンチマークではMasterと同等のCPU性能を、より小さいバイナリと短いビルド時間で得られました。実タイトルの定常画面でも総サイクルが1.68%、UnityMainが3.21%減りました（§7.3）。

エンジンコードLTOトグル（§3）は、Strip Engine Codeの無効化によるサイズとメモリの増加が大きいため、今回の構成では勧めません。

iOSへの手動ThinLTOはビルド時間の増加に見合う改善がなく、採用を見送りました。

採用前に、ReleaseビルドのProfilerキャプチャでPlayerLoop内のスクリプティング時間を調べてください。手動フラグの対象は`libil2cpp.so`内で、レンダースレッド・GPU・エンジン内部・GCは対象外です。別のAOTパイプラインでコンパイルされるジョブも含まれません（§7.5）。DOTS/Jobを多用するタイトルでは、その処理時間も確認します。

例えば33msのフレームのうちスクリプティングが8msなら、その部分を約20%短縮できてもフレーム全体では約1.6msの短縮です。フレーム全体でのスクリプティングの割合が大きく改善の余地があるか、高いフレームレートを維持せねばならない状態でのみ、導入を検討すると良いでしょう。

ビルド時間の増加が開発中の作業に響く場合は、日常の開発ビルドには適用せず、ステージング以降のビルドだけに適用する方法もあります。フラグはビルド前処理で追加するため（リスト5.6）、ビルドの種類ごとに切り替えられます。

設定後は §3.1と §5.2の手順で適用を確かめます。inputdata、バイナリのmd5、コ

ンパイルコマンドの`-flto`、オブジェクトファイルのbitcodeが確認箇所です。LTOの条件を満たさなくてもビルドは成功することがあるため、ビルド結果だけでは適用の有無を判断できません。

第6章

iOSのメモリ上限と仮想アドレス空間

キム ミンヒョク Minhyuk Kim

株式会社サイバーエージェント SGEコア技術本部

ゲームが使える端末のメモリには当然ながら上限があります。シーン切り替え中にアセットが重なったり、一時的な描画バッファが増えたりすると、この上限が問題になります。

ゲーム内でメモリのピークを減らすことに加え、メモリの上限にも調整の余地があります。Xcodeの「Increased Memory Limit」は、対応端末でその上限を引き上げる設定です。別の設定である「Extended Virtual Addressing」は、プロセスが使える仮想アドレス空間を広げます。名前は似ていますが、対処できる領域が異なります。

本章では、2つの設定の使い分けと実機での効果、App Storeでの運用経験を紹介します。数値は筆者の実機計測によるもので、Appleが保証する端末別上限ではありません。

6.1　メモリ上限と仮想アドレス空間は別の制約

アプリがOOM（Out of Memory）になったとき、少なくとも次の2つは分けて考える必要があります。

表6.1: iOSアプリで区別したい2種類のメモリ制約

制約	主に見る値	代表的な現れ方
物理メモリ側	`phys_footprint`	高メモリ使用でシステムに終了される（jetsam）
仮想アドレス空間側	VM Region Summary、Virtual Size	メモリ確保が失敗し、その後にクラッシュする

物理メモリは、実際にRAMへ保持する必要があるページに関係します。iOSでは、アプリが使える物理メモリ量は端末の搭載RAMそのものではなく、その時点のシステム状態やアプリに適用された上限にも左右されます。

Appleの`os_proc_available_memory()`は、現在のアプリがメモリ上限に達するまでに割り当てられる量を返します。Appleの説明では、この戻り値は「現在のメモリ上限からアプリのmemory footprintを引いた値」に対応し、端末の空き物理メモリと同じとは限りません。また、アプリのライフサイクル中にも上限は変わり得るため、値はキャッシュせず参考情報として扱う必要があります[*1]。

仮想アドレス空間は、メモリやファイルをどのアドレスに対応付けるかを管理する範囲です。たとえば、大きなファイルを`mmap`しても、その全ページがただちにRAMへ読み込まれるとは限りません。しかし、その範囲を置くための仮想アドレスは必要です。逆に、物理メモリのfootprintがまだ低く、`os_proc_available_memory()`にも余裕があるのに、新しいアドレス範囲を確保できず`malloc`などが失敗することがあります。

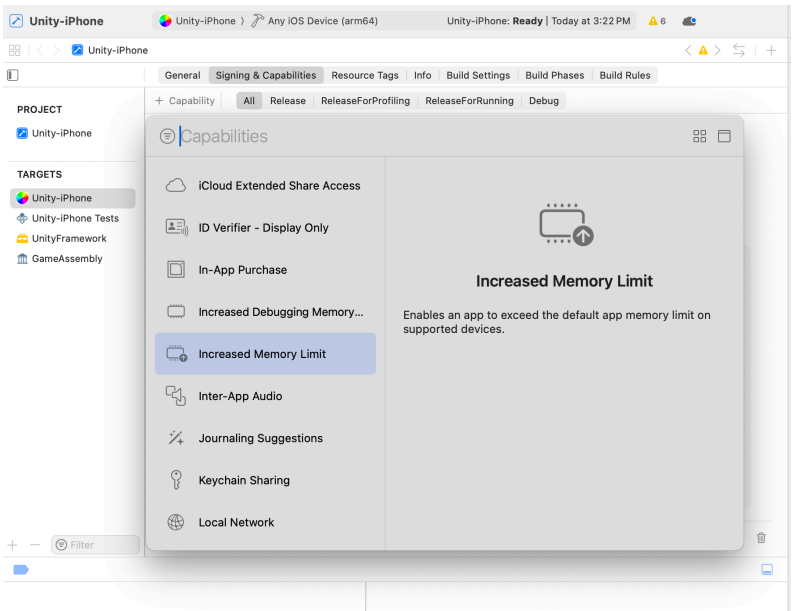
したがって、表6.1の2種類の制約を区別するためには、jetsam reportが残っているのか、通常のクラッシュレポートで割り当て失敗が起点になっているのかを確認する必要があります。`os_proc_available_memory()`が大きな値を返していても、仮想アドレス空間に同じだけ余裕があるとは限りません。

[*1] Apple Developer Documentation, `os_proc_available_memory` `https://developer.apple.com/documentation/os/os_proc_available_memory`

6.2 2つのEntitlementの役割

6.2.1 Increased Memory Limit

Increased Memory Limitのキーは`com.apple.developer.kernel.increased-memory-limit`です。Appleは、対応する端末でデフォルトのアプリメモリ上限を超えることにより、アプリの中核機能が改善し得ることをシステムへ伝えるEntitlementと説明しています[*2]。XcodeのCapability一覧でも、図6.1のように「対応端末で」という条件付きの説明が表示されます。



▲図6.1　XcodeのCapability一覧に表示されるIncreased Memory Limit

つまり、指定すれば全端末で一定量が加算されるわけではありません。Appleのドキュメントにも、増加した上限を利用できるのは一部の端末だけであり、追加メモリを利用できない場合でもアプリが正しく動作するように求める記述があります。

[*2] Apple Developer Documentation, `com.apple.developer.kernel.increased-memory-limit` https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit

Increased Memory Limitの実機計測

表6.2: Increased Memory Limitの有無によるOOM時のメモリ計測結果（iPhone）

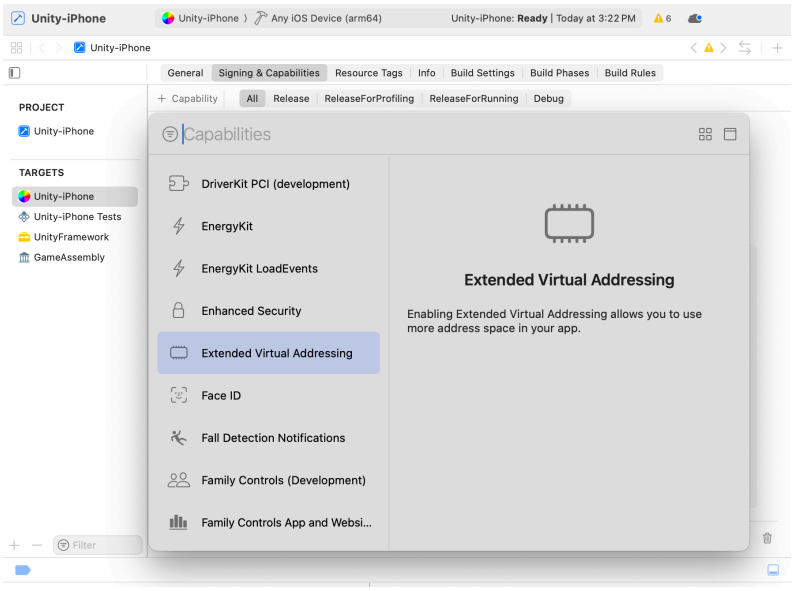
搭載RAM	無効時	有効時
3GB	約1,850MB	約1,850MB
4GB	約2,050MB	約2,290MB
6GB	約3GB	約4GB

4GBと6GBのiPhoneではメモリがそれぞれ増えた一方、3GBのiPhoneでは有効と無効の差がありませんでした。また、iPadではそもそもOOM時のメモリがiPhoneとは異なるため、有効時の結果も異なる可能性が高いです。もし対象ユーザーの端末割合に3GBのiPhoneが多い場合、Increased Memory Limitを付けるだけで救済できると見込むべきではありません。

6.2.2 Extended Virtual Addressing

Extended Virtual Addressingのキーは`com.apple.developer.kernel.extended-virtual-addressing`です。Appleは、より大きなアドレス空間へアクセスできることを示すBoolean値と説明しています。例としてゲームを挙げています[*3]。

[*3] Apple Developer Documentation, Extended Virtual Addressing Entitlement `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.extended-virtual-addressing`



▲図6.2　XcodeのCapability一覧に表示されるExtended Virtual Addressing

この設定はRAMを増やしません。物理メモリ上限に近づいてjetsamされるアプリに対して、Extended Virtual Addressingだけを付けても、同じ量のRAMを使い続けられる保証はありません。効果が期待できるのは、大きなメモリマップ、断片化したアドレス領域、大きな固定予約などにより、物理メモリを使い切る前に仮想アドレス空間が先に窮屈になっている場合です。

有効にすること自体のコストはほぼありません。ページテーブルは実際にマップした範囲にだけ作られるため、上限を広げても増えません[*4]。考えるべきは、効果が出る端末があるかどうかです。Increased Memory Limitを付けた時点でアドレス空間も広がるので、両方を付けても`max_address`が変わらない端末があります。

一方、表6.2の3GB端末のようにIncreased Memory Limitの効果が現れない端末では、Extended Virtual Addressingがアドレス空間を広げる唯一の手段になります。In-

[*4] Apple Open Source, xnu-12377.121.6, `osfmk/arm/pmap/pmap.c`（`pmap_max_64bit_offset`）https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

creased Memory Limitは端末が対応していなければ署名が正しくても無効になりますが、Extended Virtual Addressingにこの条件はありません。

仮想アドレス空間の上限値

Extended Virtual Addressingが広げるのは、footprintの上限とは別のアドレス空間の上限です。この上限は、`pmap_max_64bit_offset()`が搭載メモリに応じて返す表6.3のいずれかの値で決まります[*5]。

表6.3: arm64ユーザプロセスのアドレス空間上限

条件	定数	上限
`max_mem` > 3GB	`ARM64_MAX_OFFSET_DEVICE_LARGE`	17.375GB
`max_mem` 1GB超〜3GB	`ARM64_MAX_OFFSET_DEVICE_SMALL`	13.375GB
`max_mem` 1GB以下	`ARM64_MIN_MAX_ADDRESS`	12.5GB
entitlementsを含むいずれかの手段で拡張	`MACH_VM_JUMBO_ADDRESS`	63GB

`max_mem`はカーネルが認識する物理メモリで、カタログ上のRAM容量より小さくなります。いずれもカーネルのコンパイル時定数なので、OSのバージョンごとに変わる可能性があります。

6.2.3 App Storeでの運用実績

これらのEntitlementsの採用にあたって気になるのが、App Storeの審査に影響を与えないかという点です。筆者が把握する複数の運用中タイトルには、Increased Memory Limit、Extended Virtual Addressingのいずれも、設定を有効にしてApp Storeの審査を通過し、配信している実績があります。

Apple自身も、ゲーム開発者向けの「Bring your high-end game to iPhone 15 Pro」[*6]で、利用可能なメモリを増やすためにIncreased Memory Limitを有効にするよう案内しています。この2つは製品での採用を検討できる公開機能です。個別のアプリ全体の審査

[*5] Apple Open Source, xnu-12377.121.6, `osfmk/arm/pmap/pmap.c`（`pmap_max_64bit_offset`） `https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c`

[*6] Apple Developer Tech Talks, Bring your high-end game to iPhone 15 Pro `https://developer.apple.com/videos/play/tech-talks/111372/`

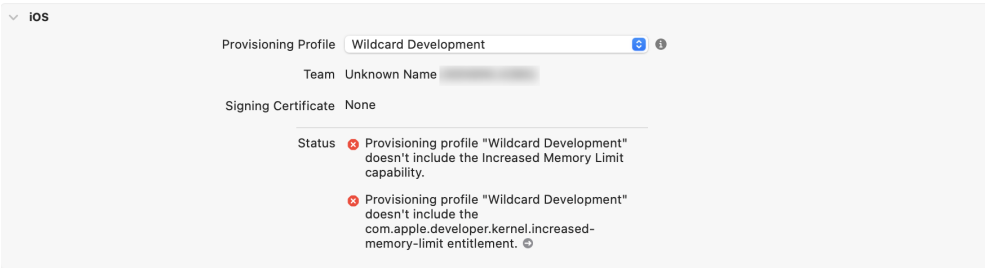
結果を保証するものではありませんが、「メモリ上限に関わる設定だから、審査で説明が求められたり、審査で落ちるのではないか」という心配で候補から外す必要はありません。

6.3 App IDと開発者アカウントを確認する

6.3.1 ワイルドカードのプロファイルは使えない

これら2つのEntitlementは、ワイルドカード形式のApp IDでは有効にできません。`com.example.*`のように複数アプリをまとめて署名している構成の場合は、ゲームのBundle IDと一致するExplicit App IDに切り替える必要があります。

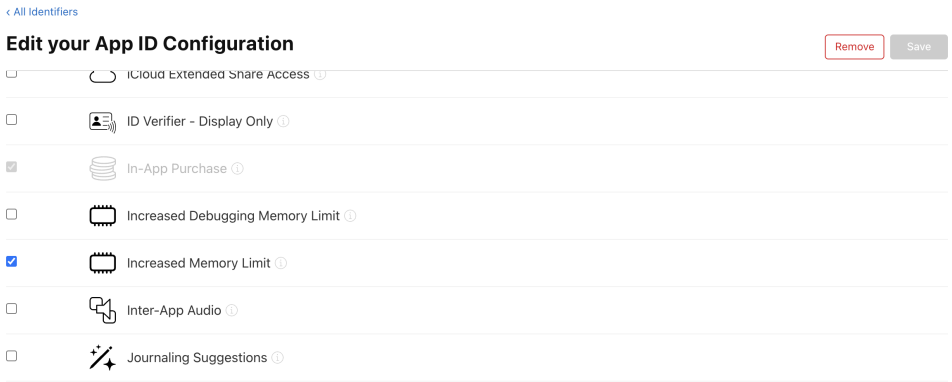
このまま署名しようとすると、XcodeのSigning & Capabilitiesに図6.3のエラーが出ます。Capability名とEntitlementのキーが別々に報告される点に注目してください。プロファイル側がCapabilityを含んでいないことと、キーに対応する許可がないことの両方が指摘されています。



▲図6.3　ワイルドカードのProvisioning Profileで署名しようとしたときのエラー

Apple DeveloperのCertificates, Identifiers & Profilesで対象のApp IDを選び、図6.4のように必要なCapabilityを有効にします。Appleも、App IDに設定したCapabilityがアプリの利用できる機能の許可リストになり、Explicit App IDが必要な機能はワイルドカードでは選べないと説明しています[*7]。

[*7] Apple Developer, Register an App ID `https://developer.apple.com/help/account/identifiers/register-an-app-id/`



▲図6.4　App IDの設定でIncreased Memory Limitを有効にする

手動署名では、Capabilityを変更したあとにProvisioning Profileを再生成し、ビルドで使うものを更新します。自動署名では、正しいTeamとBundle IDで更新されたプロファイルが選ばれることを確認します。古いプロファイルのまま`.entitlements`へキーだけ追加しても、要求する権限と署名側の許可が一致しません。Unity側の設定を調べる前に、まずここを揃えます。

6.3.2　Extended Virtual Addressingには有料メンバーシップが必要

Extended Virtual Addressingは、無料のApple AccountをXcodeのPersonal Teamとして使う構成では有効にできません。AppleのCapability対応表で対象となっているのはApple Developer ProgramとApple Developer Enterprise Programのメンバーシップで、無料のApple Accountは含まれていません[*8]。

一方、Increased Memory LimitはPersonal Teamでも有効にできます。

6.4　Unityのビルド後処理で自動設定する

Entitlementsファイルに入る最小の内容はリスト6.1です。両方を同時に有効にする例です。

[*8] Apple Developer, Supported capabilities (iOS) `https://developer.apple.com/help/account/reference/supported-capabilities-ios/`

▼リスト6.1　メモリ関連Entitlementの最小構成

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
    PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.kernel.increased-memory-limit</key>
    <true/>
    <key>com.apple.developer.kernel.extended-virtual-addressing</key>
    <true/>
</dict>
</plist>
```

XcodeのSigning & Capabilitiesから手で追加することもできますが、書き出しのたびに付け直すことになり、設定の抜けたビルドが混ざりやすくなります。Unityではビルド後処理から、Entitlementsファイルと、それを参照するCODE_SIGN_ENTITLEMENTSをまとめて更新できます。処理が走るのは署名前のビルド時で、アプリの起動後にC#から権限を取得するわけではありません。

6.4.1　Unity 6000.0.11f1以降では専用APIを使う

Unity 6000.0.11f1で、ProjectCapabilityManagerに次のメソッドが追加されました[*9]。

- AddIncreasedMemoryLimit()
- AddExtendedVirtualAddressing()

これらを使えば、EntitlementsへのBoolean値の追加とXcodeプロジェクトへの反映をまとめて行えます。

対応するPBXCapabilityTypeの定義も用意されています。ただし、これはC#のenumではなく、Capabilityの要件を表すクラスです。PBXProject.AddCapability()へこの値を渡すことと、Entitlementsへ必要なキーを追加することは同じではありません。そのため基本的には両方を処理するProjectCapabilityManagerを使います。

以下は、ProjectCapabilityManagerを使いCapabilityを追加するサンプルです。

[*9] Unity 6.0 Scripting API, ProjectCapabilityManager https://docs.unity3d.com/6000.0/Documentation/ScriptReference/iOS.Xcode.ProjectCapabilityManager.html

▼リスト6.2　Unity 6000.0.11f1以降でメモリCapabilityを追加するビルド後処理

```
#if UNITY_IOS
using UnityEditor;
using UnityEditor.Callbacks;
using UnityEditor.iOS.Xcode;

static class IosMemoryCapabilities
{
    [PostProcessBuild]
    static void OnPostprocessBuild(BuildTarget target, string buildPath)
    {
        if (target != BuildTarget.iOS) return;

        string pbxPath = PBXProject.GetPBXProjectPath(buildPath);
        var pbx = new PBXProject();
        pbx.ReadFromFile(pbxPath);
        string mainTarget = pbx.GetUnityMainTargetGuid();

        string entitlements = pbx.GetBuildPropertyForAnyConfig(
            mainTarget, "CODE_SIGN_ENTITLEMENTS");
        if (string.IsNullOrEmpty(entitlements))
            entitlements = "Unity-iPhone.entitlements";

        var capabilities = new ProjectCapabilityManager(
            pbxPath, entitlements, targetGuid: mainTarget);
        capabilities.AddIncreasedMemoryLimit();
        capabilities.AddExtendedVirtualAddressing();
        capabilities.WriteToFile();
    }
}
#endif
```

`UnityEditor.iOS.Xcode`はiOS Build Supportに含まれるため、`#if UNITY_IOS`で囲みます。そうしないと、このモジュールが入っていない環境でEditor全体のコンパイルが通らなくなりますので注意してください。

このサンプルは、全ビルド構成で共通のEntitlementsファイルを使い、そのパスをXcodeプロジェクトからの相対パスで指定する構成を対象にしています。DebugとReleaseでファイルを分けている場合や、パスにビルド変数を含めている場合は、各構成の実際のファイルへ反映する処理に合わせてください。

最後の保存は`capabilities.WriteToFile()`で行います。ほかのビルド後処理でXcodeプロジェクトを編集する場合、`pbx`を保存し直すと、Capability Managerが保存

した変更を上書きする可能性があるため、更新後のファイルを読み直してから変更するよう気をつけてください。

6.4.2 専用APIがないバージョンではplistを直接更新する

専用APIがないEditorで同じ処理を行うには、リスト6.2のProjectCapabilityManagerを生成してから保存するまでの5行を、リスト6.3に置き換えます。

▼リスト6.3 専用Capability APIがないUnityでの代替処理

```
string fullPath = System.IO.Path.Combine(buildPath, entitlements);
System.IO.Directory.CreateDirectory(
    System.IO.Path.GetDirectoryName(fullPath));
var plist = new PlistDocument();
if (System.IO.File.Exists(fullPath))
    plist.ReadFromFile(fullPath);

plist.root.SetBoolean(
    "com.apple.developer.kernel.increased-memory-limit", true);
plist.root.SetBoolean(
    "com.apple.developer.kernel.extended-virtual-addressing", true);
plist.WriteToFile(fullPath);

if (string.IsNullOrEmpty(pbx.FindFileGuidByProjectPath(entitlements)))
    pbx.AddFile(entitlements, entitlements);
pbx.SetBuildProperty(mainTarget, "CODE_SIGN_ENTITLEMENTS", entitlements);
pbx.WriteToFile(pbxPath);
```

既存ファイルを読み込んでから2つのキーを追加することで、Push通知などのEntitlementを消さずに更新します。変更するのは.entitlementsであり、Info.plistではありません。また、ファイルはCODE_SIGN_ENTITLEMENTSから参照させて初めて署名に使われます。

C#コードを追加しただけでアカウント側の権限まで有効になるわけではありません。いずれの方法でもApple Developer側のApp IDやProvisioning Profileは別途設定が必要です。

6.5 適用を2段階で確認する

設定が反映されているかは、署名にキーが入っているかと、実機でカーネルが反映したかの2段階に分けて確認します。

6.5.1 署名に含まれているかを確認する

手作業でもビルド後処理でも、リスト6.1のファイルがプロジェクト内にできただけでは、実機へ入ったアプリに反映された証拠にはなりません。Entitlementは最終的にコード署名へ埋め込まれます。書き出した`.ipa`はzip形式なので、展開して`Payload`内のアプリ本体に対してリスト6.4を実行します。

▼リスト6.4　署名済みアプリに埋め込まれたEntitlementの確認

```
unzip -q YourApp.ipa -d ipa
codesign --display --entitlements :- ipa/Payload/YourApp.app
```

出力されたplistで、リスト6.1の2つのキーが入っているかを確認します。Appleも、このコマンドによる署名後の確認を案内しています[*10]。

6.5.2 実機で上限が変わったかを確認する

リスト6.4でキーが見えても、カーネルが実際に反映したとは限りません。Increased Memory Limitは端末側が対応していない場合、署名が正しくても無効として扱われます。

実機での適用は、カーネルが自プロセスに対して持っている値を直接読んで判断します。それを取得するのが`task_info()`で、iOS SDKに標準で入っているMach APIです（`<mach/mach.h>`）。C#からは呼べないので、`Assets/Plugins/iOS/`に置いた`.mm`から読み出してC#へ返します。

▼リスト6.5　適用状況を読むネイティブプラグイン

```
#import <mach/mach.h>

extern "C" void UPT_ReadLimits(uint64_t *memlimit, uint64_t *ceiling)
{
    task_vm_info_data_t i;
    mach_msg_type_number_t c = TASK_VM_INFO_COUNT;
    if (task_info(mach_task_self(), TASK_VM_INFO, (task_info_t)&i, &c) != KERN_SUCCESS)
     {
        return;
```

[*10] Apple Developer Documentation, Diagnosing Issues with Entitlements `https://developer.apple.com/documentation/bundleresources/diagnosing-issues-with-entitlements`

```
    }

    // footprint上限(Increased Memory Limit)
    *memlimit = i.limit_bytes_remaining + i.phys_footprint;
    // アドレス空間上限: 0xF_C000_0000まで広がっていれば拡張済み
    *ceiling  = i.max_address;
}
```

C#側はDllImport("__Internal")でUPT_ReadLimitsを宣言して呼び出します。

見るべき値は上限です。os_proc_available_memory()が返す残量はfootprintが増えればそのぶん小さくなるので、値だけを見ても上限が上がっていないのか単に使い込んでいるだけなのかを区別できません。

limit_bytes_remainingはそのos_proc_available_memory()とカーネル内で同じ関数から得られる値なので[*11]、現在のfootprintを足せば適用中の上限に戻せます。アドレス空間側のmax_addressも同じ構造体から取れるため、2つのEntitlementを1回の呼び出しで確認できます。max_addressはTASK_VM_INFO_COUNTを渡さないと埋まりません。

ただしmax_addressからExtended Virtual Addressing単独の効果は分離できません。Increased Memory Limitを付けたビルドや、XcodeのEnhanced Securityを有効にしたビルドでも同じ上限になります。確認できるのは「上限が63GBまで広がっているか」までです。

6.6 iOS 26.4で顕在化した仮想アドレス予約

6.6.1 JS VM Gigacage (reserved)は1GBのRAM使用量ではない

iOS 26.4を搭載した3GBのiPhoneでの検証では、クラッシュレポートのVM Region SummaryにJS VM Gigacage (reserved)として約1GBの領域が現れ、Extended Virtual Addressingを有効にするとOOM時のメモリ量の低下を防げました。この領域は本来のGigacageではなく、既存のVM tag名を再利用して表示されたJavaScriptCoreの

[*11] Apple Open Source, xnu-12377.121.6, bsd/kern/kern_memorystatus.c (memorystatus_available_memory_internal) https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/bsd/kern/kern_memorystatus.c

Structure Heap予約です[*12]。

6.6.2 WebKitの修正は予約量を端末RAMに応じて縮小する

JavaScriptCoreの公開バグ315240には、低メモリ端末でStructure Heapの予約が仮想アドレス空間の大きな割合を占めるため、物理メモリ量に応じて予約を縮小する方針が記録されています[*13]。

この問題に対応するWebKitのコミットで、iOS系プラットフォームについて、基本の予約サイズと物理メモリの20%の小さい方を取り、さらに2のべき乗へ切り下げて、Structure Heapの希望予約サイズを決めるようになりました。公開コミットの説明では、3GB端末で512MBになる例が示されています[*14]。

この対応が入ったiOS 26.6.1のiPhone SE（第2世代、RAM 3GB）で計測すると、表示上の予約が約1GBから512MBへ縮小し、26.4で低下していたOOM時のメモリ量が、Extended Virtual Addressingを有効にしなくても解消されていました。

6.7 まとめ

Increased Memory LimitとExtended Virtual Addressingは、有効にしても失うものがほとんどない設定です。ページテーブルは実際にマップした範囲にだけ作られるため、アドレス空間を広げてもコストは増えません。App Storeの審査を通過して配信している実績もあります。Explicit App IDとプロファイルの準備さえできれば、基本的に両方を有効にしておくのがよいと考えています。

ただし、効果は端末に依存します。筆者のiPhoneでの計測では、Increased Memory Limitによる余裕は4GB端末で約240MB、6GB端末で約1GBでしたが、3GB端末では差がありませんでした。iPadでは別途計測が必要です。Extended Virtual Addressingは物理メモリを増やさず、仮想アドレス空間が先に足りなくなる場合にだけ効果があり

[*12] Apple Developer Forums, iPadOS 26.4+ significantly reduced per-app memory limit from 6GB to 3GB on 8GB iPad, breaking memory-intensive apps `https://developer.apple.com/forums/thread/826325`

[*13] WebKit Bugzilla 315240, Scale down StructureAlignedMemoryAllocator reservation in proportion to phys-mem size `https://bugs.webkit.org/show_bug.cgi?id=315240`

[*14] WebKit commit `cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b`, Scale down structure-heap reservation size with physical memory `https://github.com/WebKit/WebKit/commit/cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b`

ます。iOS 26.4で見られたStructure Heap予約の問題はiOS 26.6系で緩和されましたが、古いOSをサポートする場合や、アプリ自身のメモリマップでアドレス空間を多く使う場合には引き続き効果が見込めます。

どちらもメモリ使用量そのものは減らしません。追加の上限がない端末でも動作を続けられるよう、テクスチャや一時RenderTextureのピーク、アセットの常駐期間、ネイティブプラグインの大きなallocation、ファイルマッピングの寿命を減らす作業は別に必要です。2つのEntitlementは、その上で残る余裕を広げるための設定として使います。

Unity®

PERFORMANCE TUNING TIPS 2026

CHAPTER 04

第4部

計測と自動化

CyberAgent Smartphone Games & Entertainment

第1章

Unity で活用する iOS 標準のパフォーマンス計測機能

山崎 雅大 Yamazaki Masahiro

株式会社アプリボット

mao-test-h　X @TEST_H_

1.1　はじめに

Unity には Unity Profiler をはじめとした計測機能が備わっていますが、iOS 向けのアプリであれば、それに加えて OS 標準のパフォーマンス計測機能も活用できます。

Apple が提供するこれらの機能は Unity の Development Build フラグに依存せずに利用できるのが特徴です。例えば、計測処理をまだ組み込めていない段階や、デバッグ機能を無効にしたリリースビルドに近い構成でざっくりと負荷を確認するほか、テスト担当者の端末からトレース結果だけを受け取るといった使い方ができます。 こうして大まかな負荷を把握し、ボトルネックの候補をある程度絞り込んだ後は、Development Build を有効にしたビルドへ切り替え、Unity Profiler や Instruments で詳細に調査するといった使い分けが可能です。

本記事では、iOS が標準で提供する機能の中から以下の 2 つを紹介し、加えて macOS 27 / iOS 27 から追加された新機能についても解説します。

- Metal Performance HUD
- Performance Trace

また、これらの機能を Unity から扱えるようにした OSS 兼サンプルリポジトリとし

て「MetalPerfKit-Unity」[*1] を公開しているので、実装の詳細についてはこちらも併せてご覧ください。

※ 以降の解説では表記を iOS に統一しますが、特に断りがない限り iPadOS も含みます。

1.1.1 検証環境

本記事は以下の環境で動作検証を行っています。

- Unity 6000.0.81f1
- Xcode 26.6 / 27.0 beta
- iOS 26.6 / 27.0 beta
- macOS 27.0 beta

※ 執筆時点では iOS 27、macOS 27、Xcode 27、後述する `StateReporter` API はベータ段階です。本記事は 2026 年 8 月時点の情報であり、正式リリースまでに API やコマンド、設定画面、挙動などが変わる可能性があります。

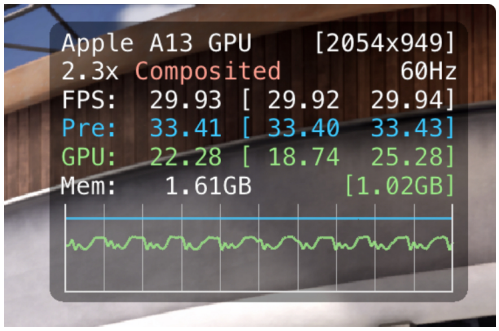
1.2 Metal Performance HUD

Metal Performance HUD は、Metal を使用するアプリケーションのパフォーマンス情報をリアルタイムに画面上へオーバーレイ表示したり、ログとして出力したりできるツールです。 Apple のドキュメント [*2] では、現在の状態を素早く把握し、Xcode や Instruments で詳しくキャプチャする範囲を見つけるための機能として紹介されています。

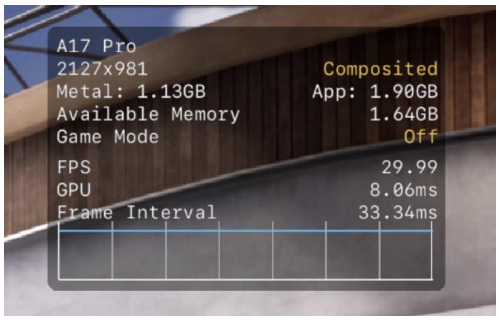
この機能自体は Metal 4 以前 (iOS 16 世代) から存在していましたが、2025 年の WWDC で発表された Metal 4 (iOS 26 世代) に合わせて大きく強化されました。 Metal 4 以前と Metal 4 以降の表示の違いを 図1.1 と 図1.2 に示します。

[*1] MetalPerfKit-Unity `https://github.com/mao-test-h/MetalPerfKit-Unity`

[*2] Monitoring your Metal app’s graphics performance `https://developer.apple.com/documentation/xcode/monitoring-your-metal-apps-graphics-performance`



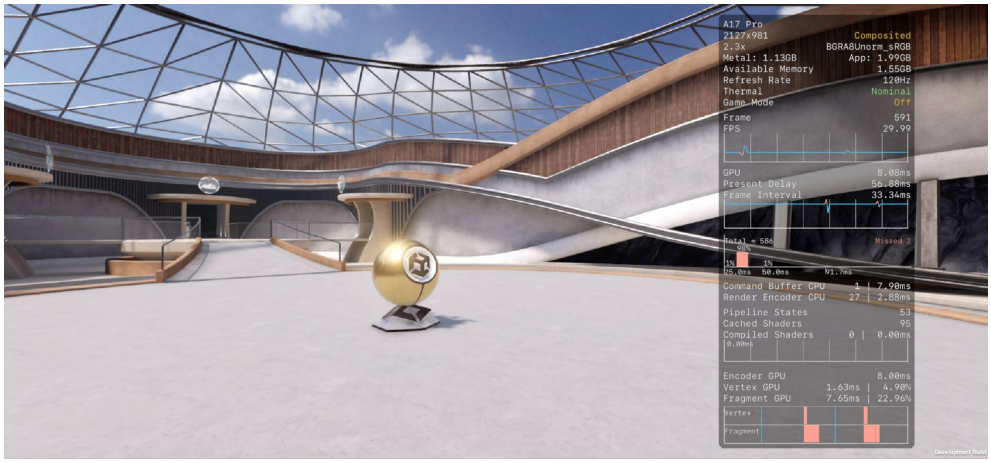
▲図1.1　Metal 4 以前の Metal Performance HUD



▲図1.2　Metal 4 以降の Metal Performance HUD

Metal 4 からの主な強化点は計測項目の拡充のほか、表示位置や不透明度・スケールのカスタマイズ、これらを API から動的に制御できる項目の増加などが挙げられます。

主要なメトリクスを表示した状態の例を 図1.3 に示します。



▲図1.3　主要なメトリクスを表示した状態の例

各メトリクスの詳細については、公式ドキュメント「Understanding the Metal Performance HUD metrics」[*3] をご覧ください。

また、筆者も過去に Metal Performance HUD についての解説記事 [*4] や技術同人誌「UniTips Vol.12」[*5] を公開しています。 基本的な使い方から Unity 向けの活用術などを解説しているので、詳細についてはこちらも参考にしてみてください。

以降のセクションでは、Metal 4 (iOS 26 世代) の内容をベースに解説していきます。

1.2.1　HUD を開くには

HUD を表示する方法はいくつかありますが、ここでは「設定アプリから開く方法」と「API で表示を制御する方法」の 2 つを解説します。

[*3] Understanding the Metal Performance HUD metrics `https://developer.apple.com/documentation/xcode/understanding-metal-performance-hud-metrics`

[*4] 【Unity】モバイルアプリ上で GPU 負荷を計測する `https://qiita.com/mao_/items/3b531d17cacb86036b7f`

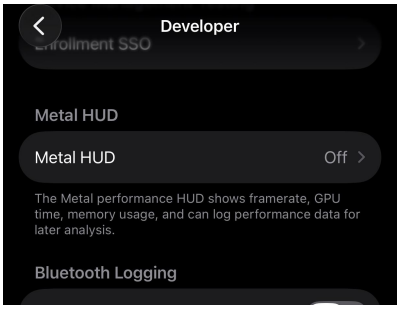
[*5] UniTips Vol.12（「第2章: Metal 4 から強化された Metal Performance HUD の Unity 活用術」にて解説）`https://techbookfest.org/product/8mHpXz8LJCKAsLxENJKwaz?productVariantID=ewK5GtNTjkz4k8heM4BrxU`

設定アプリから開く

iOS デバイスの設定アプリから有効化できます。手順は以下の通りです。

※ 前提として、Developer メニューが表示されている必要があります。また、OS バージョンによっては、メニュー表記が異なる可能性があります。

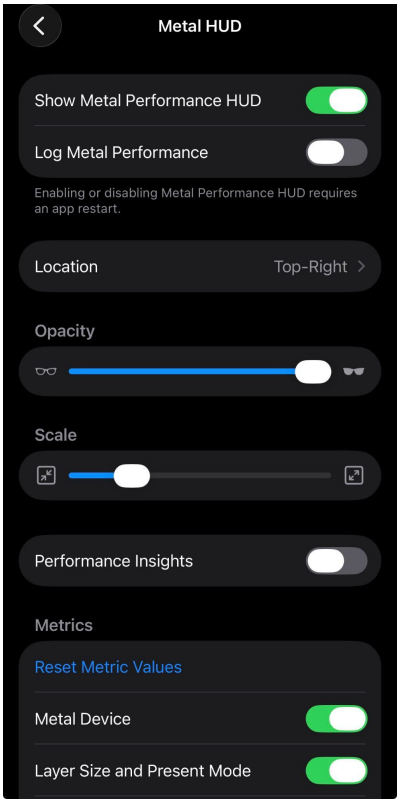
1. iOS の「設定」アプリを開く
2. Developer メニューを開き、その中にある Metal HUD を開く (図1.4を参照)



▲図1.4　Developer メニューから Metal HUD を開く

Metal HUD を開くと 図1.5 の画面が表示されるので、この中の Show Metal Performance HUD を有効化し、対象アプリを再起動すると表示できます。

この画面では、表示する指標の切り替え、不透明度・スケール・表示位置の変更なども行えます。 また、同じセクションにある Log Metal Performance を有効にすると、パフォーマンス情報を一定間隔でログとして出力できます。詳細は後述の「ログ機能について」で解説します。



▲図1.5　Metal HUD の設定画面

API で表示を制御する

アプリの実行中に HUD の表示を動的に制御したい場合は、`CAMetalLayer` クラスの `developerHUDProperties` プロパティに設定値を渡します。リスト1.1 では HUD とログ出力を有効化し、表示位置を左上に指定しています。

▼リスト1.1　developerHUDProperties による HUD の表示制御

```
myMetalLayer.developerHUDProperties = [
    "mode": "default",      // HUD の ON/OFF
    "logging": "default",   // ログ機能の ON/OFF
    "positionX": 0,         // 表示位置（ピクセル単位の X 座標）
    "positionY": 0,         // 表示位置（ピクセル単位の Y 座標）
]
```

HUD を非表示にする場合は mode に disabled を指定します。 従来はこのプロパティで制御できるのは HUD とログ機能の ON/OFF までであり、位置調整などは行えませんでしたが、Metal 4 からは positionX / positionY によるピクセル単位の位置指定が行えるようになりました。

また、この Dictionary には HUD がサポートする環境変数のキーも渡すことができます。 例えば MTL_HUD_OPACITY と MTL_HUD_SCALE を渡すと、不透明度とスケールを 0.0〜1.0 の範囲で調整できます（リスト1.2）。

▼リスト1.2　環境変数キーによる不透明度・スケールの調整

```
myMetalLayer.developerHUDProperties = [
   ...,
   "MTL_HUD_OPACITY": opacity,      // 不透明度 (0.0〜1.0)
   "MTL_HUD_SCALE": scale,          // スケール (0.0〜1.0)
]
```

他にも設定可能な項目が多数あるので、詳細については公式ドキュメント「Customizing the Metal Performance HUD」[*6] をご覧ください。

Unity から呼び出す場合は、描画に使われている CAMetalLayer を iOS ネイティブプラグインで取得し、C# 側から P/Invoke で呼び出す形になります。 CAMetalLayer は UnityFramework 経由で リスト1.3 のように取得できます。

▼リスト1.3　UnityFramework 経由での CAMetalLayer の取得

```
private func getMetalLayer() -> CAMetalLayer? {
   guard let instance = UnityFramework.getInstance(),
         let rootView = instance.appController().rootView,
         let metalLayer = rootView.layer as? CAMetalLayer else {
       return nil
   }
   return metalLayer
}
```

MetalPerfKit-Unity ではこれらの実装を PerformanceHUDSwitcher クラスとして公開しており、C# からは リスト1.4 のように呼び出せます。 なお、ネイティブ API が

[*6] Customizing the Metal Performance HUD https://developer.apple.com/documentation/xcode/customizing-metal-performance-hud

ピクセル単位で位置を指定するのに対し、PerformanceHUDSwitcher では解像度に依存せず扱えるよう座標を 0.0〜1.0 の相対位置で受け取り、内部で描画領域のサイズに応じたピクセル座標へ変換しています。

▼リスト1.4　PerformanceHUDSwitcher の呼び出し例

```
// HUD を画面右上 (x=1.0, y=0.0) に表示する
// (座標は 0.0〜1.0 の相対位置で指定)
PerformanceHUDSwitcher.SetPerformanceHUDVisible(true, 1.0f, 0.0f);
```

完全な実装例については、MetalPerfKit-Unity のソースコードをご覧ください。

1.2.2　ログ機能について

前述の Log Metal Performance や API の logging を有効にすると、アプリの実行中に 1 秒間隔でパフォーマンス情報がログとして出力されます。 ログは metal-HUD: から始まる リスト1.5 のフォーマットで出力され、Xcode のコンソールや macOS の Console.app から確認できます。

▼リスト1.5　metal-HUD ログの出力フォーマット

```
metal-HUD: <first-frame-number-integer>,<graphics-memory-usage-float>,<process-memory-
    usage-float>,<first-frame-present-interval-float>,<first-frame-gpu-time-float>,...<
    last-frame-present-interval-float>,<last-frame-gpu-time-float>
```

先頭から順に「サンプリングを開始したフレーム番号」「グラフィックスメモリ使用量」「プロセス全体のメモリ使用量」が記録され、それ以降は、サンプリングされた各フレームの Frame Present Interval と GPU Time のペアが繰り返し記録されます。 実際の出力例を 図1.6 に示します。



▲図1.6　Metal Performance HUD のログ出力例

カンマ区切りのデータとして集計すれば、フレーム間隔の乱れや GPU Time、メモリ使用量の推移を後から分析できます。 また、これらのログが Unified Logging に残っている範囲であれば、`OSLogStore` API を用いて「過去 N 秒分のログを取得してファイルに保存する」といった実装も可能です。 MetalPerfKit-Unity では `PerformanceLogger` クラスとしてこの機能を提供しています（リスト1.6）。

▼リスト1.6　PerformanceLogger によるログの取得と保存

```
// 過去 60 秒分のログを取得してファイルに保存する
PerformanceLogger.FetchPerformanceLogs(60, FileUtility.GenerateFetchLoggingFilePath());
```

1.3　Performance Trace

Performance Trace は iOS 26 から追加された機能で、Mac に接続することなく、iOS デバイス単体でパフォーマンスに関するデータを収集できます。 収集したトレースファイルは Mac に転送し、Instruments や後述する `metalperftrace` コマンドで分析できます。

デバイス単体で完結するため、長時間のプレイなど Mac に接続した状態では再現しにくい状況を記録できるほか、テスト担当者など他人の端末で収集したトレースファイルを共有してもらい、手元で分析するといった使い方も可能です。

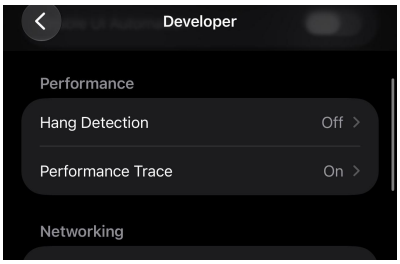
本章では、その中でも WWDC 2026 で機能強化とともに解説された Lookback Collection をメインに紹介します。

1.3.1 使い方

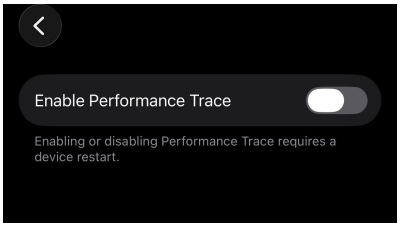
Performance Trace は Developer メニューから有効化する必要があります。 手順は以下の通りです。

1. iOS の「設定」アプリを開く
2. Developer メニューを開き、その中にある Performance Trace を開く
3. Enable Performance Trace を有効化 (有効/無効の切り替えにはデバイスの再起動が必要)
4. 任意の Tracing Mode を選択 (詳しくは後述)

Developer メニュー内の Performance Trace と、その設定画面をそれぞれ 図1.7 と図1.8 に示します。



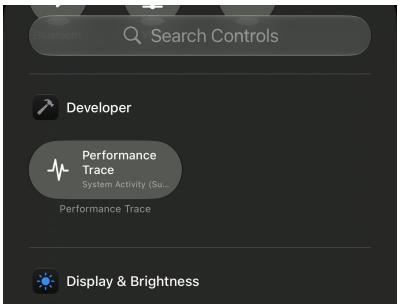
▲図1.7　Developer メニュー内の Performance Trace



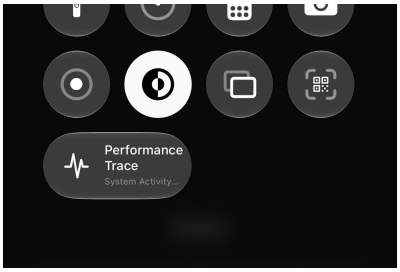
▲図1.8　Enable Performance Trace の設定画面

有効化が完了したら、次にトレースの開始・停止やトレースデータの収集などを行うために、コントロールセンターに Performance Trace を追加します。 こちらはコン

トロールセンターを開き、Developer セクションから追加することができます。追加手順と追加後の状態を 図1.9 と 図1.10 に示します。



▲図1.9　コントロールセンターへの Performance Trace の追加



▲図1.10　Performance Trace コントロールを追加した状態

1.3.2　Tracing Mode について

Tracing Mode には 表1.1 に示す 4 種類 (System Activity は Summary / Detailed の 2 種類) があり、コントロールセンターの Performance Trace コントロールを長押しすることでも切り替えられます。

表1.1: Tracing Mode の種類

モード	概要
Power Profiler	電力消費への影響を記録する
System Activity (Summary / Detailed)	システムの動作状況を記録する

表1.1: Tracing Mode の種類

モード	概要
Lookback Collection	システムが常時記録しているデータを過去に遡って収集する

Power Profiler と System Activity はコントロールセンターの `Performance Trace` コントロールをタップして記録を開始し、再度タップして停止します。 一方、Lookback Collection は記録操作が不要で、タップした時点から過去に遡ってデータを収集します。詳細は後述します。

収集されたトレースファイルは Performance Trace 画面にある `Available Trace Files` に一覧表示され、共有ボタンから AirDrop などで Mac や他の開発者へ転送できます。 共有されたアーカイブを展開すると、Instruments で開ける `.atrc` ファイルが得られます。 なお、トレースファイルはデバイスのストレージ残量が少なくなると自動的に削除されるほか、スワイプ操作で手動削除も可能です。

各モードについての補足は以下の通りです。

Power Profiler

Power Profiler は、アプリによる CPU / GPU / ディスプレイ / ネットワークといった各サブシステムの利用が、デバイスの電力消費に与える影響 (Power Impact) を計測するモードです。 本記事では詳細を割愛しますが、公式ドキュメント「Measuring your app’s power use with Power Profiler」[*7] と WWDC 2025 のセッション「Profile and optimize power usage in your app」[*8] で詳しく解説されています。

System Activity

System Activity は、「Summary」と「Detailed」の 2 種類が用意されているトレースモードです。筆者が確認した範囲では、執筆時点 (2026 年 8 月) において、Apple のドキュメントや WWDC セッションに System Activity の詳しい仕様や、Summary と Detailed の違いに関する説明は見当たりませんでした。 そのため、本記事では各モードの詳細には踏み込まず、実際のトレースで確認できた情報のみを紹介します。

[*7] Measuring your app’s power use with Power Profiler `https://developer.apple.com/documentation/xcode/measuring-your-app-s-power-use-with-power-profiler`

[*8] WWDC 2025 - Profile and optimize power usage in your app `https://developer.apple.com/videos/play/wwdc2025/226/`

参考として、iOS 27 を搭載したデバイスで System Activity (Summary) のトレースを記録し、後述する `metalperftrace` コマンドで分析した結果を リスト1.7 に示します。

▼リスト1.7　System Activity (Summary) トレースの分析結果

```
$ metalperftrace overview system_activity_summary_ios27.atrc
[MetalPerfKit-Unity pid:1974]
   Mem: 799.0 MiB (799.0 Peak, 616.8 Metal, 2577.0 Headroom)
   Total CPU Time: 12.679s (20.47% Sys, 79.53% User)
   Instructions: 43.33B (60.88% P, 39.12% E)
   Cycles: 21.04B (38.97% P, 61.03% E)
   Disk Reads / Writes: 9.13 / 0.08 MiB (Logical Write 0.07)
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics: 3 transitions, last known state:
    URP-HighFidelity
     com.DefaultCompany.MetalPerfKit-Unity.screen: 11 transitions, last known state:
    Title
 UnityView (UnityDefaultViewController) (1290x2796)  Interval 30.037s  Active 30.037s
   44.5 FPS  1202 Frames  1202 Detached  11 Skipped
   Frame Time           avg:  22.47ms  min:  16.67  max: 420.93  stddev:  30.68  p95:
    50.01  p99: 133.36
   CPU Begin-to-Present avg:  17.24ms  min:   1.14  max: 794.63  stddev:  31.76  p95:
    31.28  p99: 111.73
   On-GPU Time          avg:  15.50ms  min:   1.81  max: 424.87  stddev:  37.24  p95:
    51.17  p99: 134.48
   Next Drawable Wait   avg:  14.53ms  min:   0.00  max: 401.43  stddev:  32.11
```

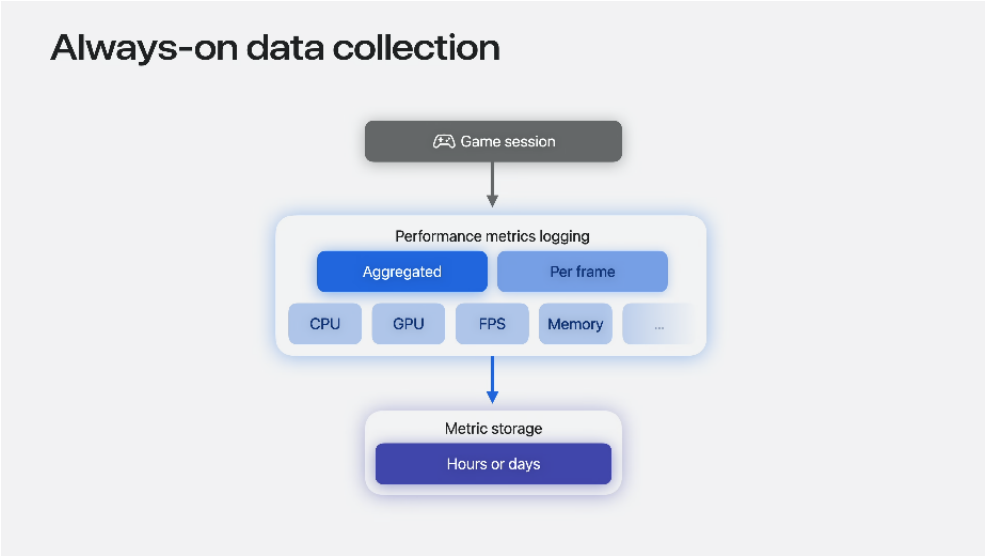
このトレースでは、プロセスごとのメモリ使用量や CPU 時間、ディスク I/O に加えて、`CAMetalLayer` 単位の FPS や Frame Time、CPU Begin-to-Present、On-GPU Time などの統計を確認できました。 なお、`[States]` の項目にはアプリの状態遷移が記録されていますが、こちらは次章で解説する `StateReporter` API によるものなので、詳しくは後述します。

Lookback Collection

Lookback Collection は、システムが常時記録しているパフォーマンスデータを、後から過去に遡って収集するモードです。筆者が確認した限り、Lookback Collection は iOS 26 の時点ですでに `Tracing Mode` の選択肢として存在していましたが、WWDC 2025 や公式ドキュメントでは解説されていませんでした。その後、WWDC 2026 で機能が強化され、その内容を解説するセッション動画「Find and fix performance issues

in your Metal games」[*9] が公開されました。

講演によると、ゲームの実行中は、システムがパフォーマンスメトリクス (CPU、GPU、FPS、メモリなど) を常時記録し、そのデータを数日分保存しています。 Lookback Collection はこの仕組みを利用しているため、テスターによる長時間のプレイセッションであっても、遡ってデータを取得できるのが特徴です。 この仕組みの概要を 図1.11 に示します。



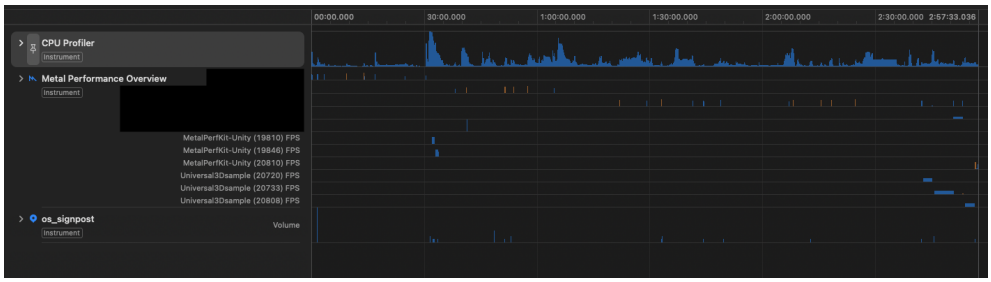
▲図1.11　システムによるパフォーマンスメトリクスの常時記録（WWDC 2026 セッションより）

使い方は、通常通りプレイテストを行った後にコントロールセンターの `Performance Trace` コントロールをタップするだけです。 タップした時点から設定した時間分だけ遡ってトレースの収集・処理が始まり、完了すると通知が届いて `Available Trace Files` に追加されます。

実際に iOS 26 を搭載したデバイスで約 3 時間分の Lookback Collection トレースを収集し、Instruments で開いた例を 図1.12 に示します。 遡った期間中に起動してい

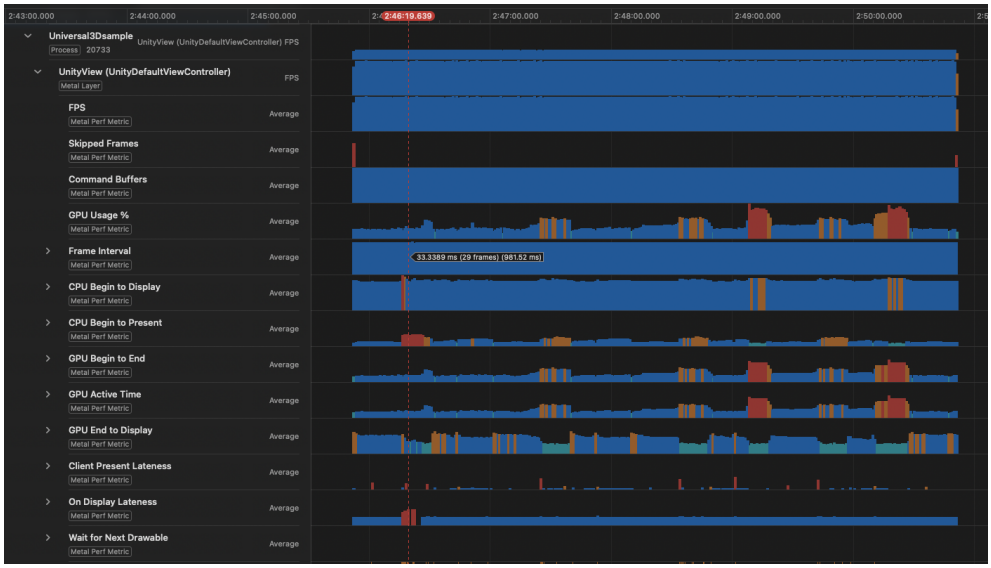
[*9] WWDC 2026 - Find and fix performance issues in your Metal games `https://developer.apple.com/videos/play/wwdc2026/388/`

た複数のアプリ (プロセス) が一覧表示され、それぞれの FPS などの推移を俯瞰できます。



▲図1.12　Lookback Collection のトレースを Instruments で開いた例

さらにプロセスを展開すると、図1.13 のように `CAMetalLayer` 単位で FPS や GPU Usage %、Frame Interval、CPU / GPU 時間といったフレームメトリクスの推移を確認できます。



▲図1.13　CAMetalLayer 単位のフレームメトリクスの推移

※ 手元の iOS 27 環境では、集計中に `Error processing Performance Trace` と表示され、正常に動作しませんでした。こちらの不具合については、iOS / iPadOS 27

の正式リリース時点でも、リリースノートの `Control Center -> Known Issues` に、「Power Profiler や Lookback のトレースを収集すると、トレースが生成されない場合がある」という既知の問題 (184729556) が記載されています。[*10]

1.4 macOS 27 / iOS 27 から追加された新機能

前章で紹介した通り、ゲームの実行中はシステムが Metal パフォーマンスメトリクスを常時記録・保存しています。 WWDC 2026 ではこの仕組みを軸とした新機能として、トレースにアプリ固有の情報を付与する `StateReporter` API と、トレースを CLI から分析できる `metalperftrace` コマンドが発表されました。

1.4.1 StateReporter API

トレースデータでフレームレートが落ちた時刻を特定できても、Metal パフォーマンスメトリクスだけでは、そのときゲームが何をしていたかまでは分かりません。「ゲームのどのシーンを開いているのか」「オブジェクト数はどれくらいか」「グラフィックス設定は何か」といったアプリ固有の情報は、Metal パフォーマンスメトリクスには自動的に含まれないためです。

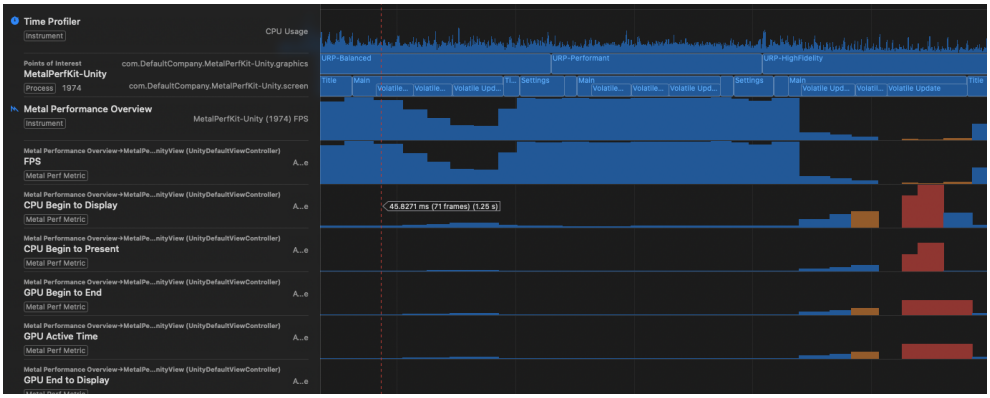
iOS 27 で追加された `StateReporting` フレームワーク [*11] の `StateReporter` API [*12] を用いると、アプリの状態をシステムに報告し、トレースデータにアプリ固有のコンテキストを付与できます。

報告した状態は、`metalperftrace` では状態別の集計に、Instruments では Points of Interest の表示に利用されます。 前章で掲載した System Activity のトレースを Instruments で開いた例を 図1.14 に示します。報告した状態が Points of Interest のトラックとして表示されています。

[*10] iOS & iPadOS 27 Release Notes `https://developer.apple.com/documentation/ios-ipados-release-notes/ios-ipados-27-release-notes`

[*11] Getting started with StateReporting `https://developer.apple.com/documentation/StateReporting/getting-started-with-statereporting`

[*12] StateReporter `https://developer.apple.com/documentation/statereporting/statereporter`



▲図1.14　報告した状態が Points of Interest として表示される例

`StateReporter` は 表1.2 に示す 4 つの要素で構成されます。

表1.2: StateReporter を構成する 4 つの要素

要素	概要	例
Domain	報告対象とする機能領域。逆 DNS 形式の文字列で、ドメインごとに同時にアクティブな状態は 1 つ	`com.mygame.level`
Label	現在の状態を示す短い文字列	`Level 1`
Stable metadata	状態の存続期間中は不変のメタデータ。Label と組み合わせて状態を一意に識別する	`{id: 1001}`
Volatile metadata	状態の中で変化しうる揮発性のメタデータ。更新しても状態遷移は発生しない	`{health: 20}`

ドメインは複数定義でき、例えば「レベル進行」「グラフィックス設定」「ネットワーク状態」をそれぞれ独立したドメインとして報告し、組み合わせて分析できます。

Swift からは リスト1.8 のように利用します。

▼リスト1.8　Swift からの StateReporter の利用例

```
import StateReporting
```

```
// ドメインに対応する Reporter を取得
let reporter = StateReporter.reporter(for: "com.mygame.gameplay")
// 状態遷移を報告
reporter.reportTransition(to: "Playing")
// 状態を終了する場合は nil を渡す
reporter.reportTransition(to: nil)
```

Unity から利用する

StateReporting は Swift / Objective-C 向けの API であるため、Unity から利用するにはネイティブプラグインが必要です。 MetalPerfKit-Unity ではこれをラップした StateReporter クラスをプレビュー機能として公開しており、C# から リスト1.9 のように利用できます。

▼リスト1.9　C# からの StateReporter の利用例

```
using MetalPerfKit;
// ドメインに対応する Reporter を取得
var reporter = StateReporter.ReporterForDomain($"{Application.identifier}.screen");
// 状態遷移を報告（揮発性メタデータ付き）
reporter.ReportTransitionToStateLabel(
    "Main",
    volatileMetadata: new Dictionary<string, StateReporterMetadataValue>
    {
        ["CubeCount"] = cubeCount,
    });
// 状態は維持したまま、揮発性メタデータのみを更新
reporter.ReportVolatileMetadataUpdate(
    new Dictionary<string, StateReporterMetadataValue>
    {
        ["CubeCount"] = cubeCount,
    });
```

また、リポジトリの Assets/_Example/2_MetalPerformanceMetrics には、画面遷移 (Title / Main / Settings) とグラフィックス品質 (URP の 3 段階) をそれぞれ独立したドメインとして報告するサンプルを用意しています。 前章で掲載した System Activity のトレース内の [States] は、このサンプルで記録したものです。

1.4.2　metalperftrace

metalperftrace は macOS 27 から追加された CLI ツールで、トレースファイルの分析のほか、macOS 向けアプリのパフォーマンスメトリクスの収集を行えます。 収

集機能は macOS 上で動作するアプリが対象となるため本記事では扱いませんが、iOS 端末で収集したトレースファイルの分析にはこちらのツールを活用できます。

トレースファイルの分析について

トレースファイルの分析には `overview` サブコマンドを使用します（リスト1.10）。

▼リスト1.10　overview サブコマンドによるトレースファイルの分析

```
# 収集済みのトレースファイルを分析
metalperftrace overview MyGameTrace.atrc
```

出力内容は前章で紹介した通りです。 さらに `--include-state-transitions` を渡すと、`StateReporter` で記録した状態遷移の詳細も確認できます。 リスト1.11 は前述のサンプルアプリのトレースを分析した例で、各グラフィックス品質への変更履歴が、その状態での滞在時間と割合とともに出力されています。

▼リスト1.11　状態遷移の詳細を含めた分析結果

```
$ metalperftrace overview --include-state-transitions system_activity_summary_ios27.
    atrc
[MetalPerfKit-Unity pid:1974]
   ...
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics
       URP-Balanced  (32.75%, 9.904s)
         (2026-08-16T14:22:33.025Z → 2026-08-16T14:22:42.930Z)
       URP-Performant  (29.43%, 8.900s)
         ...
       URP-HighFidelity  (37.82%, 11.437s)
         ...
     com.DefaultCompany.MetalPerfKit-Unity.screen
       Title  (5.07%, 1.535s)
         ...
       Main  (21.09%, 6.379s)
         ...
       Title  (2.22%, 0.671s)
         ...
       Settings  (6.28%, 1.899s)
         ...
```

※ 出力は一部を抜粋・整形して掲載しています。実際には同じ状態も時系列に沿って繰り返し出力され、それぞれに開始・終了のタイムスタンプが付きます。

また、`--aggregate` を渡すと、状態ごとのメトリクス集計が行えます。`--domain com.mygame.graphics` を指定すると特定のドメインに絞り込め、さらに `--state-label "High"` を併用すると特定の状態まで絞り込めます。「グラフィックス設定が High のときだけ平均 FPS が落ちている」といった、アプリの状態とパフォーマンスの相関をコマンド 1 つで確認できます。

また、これらの出力は `--json` を渡すことで JSON 形式でも取得できます。 講演でも「構造化された出力はスクリプトによるリグレッションテストや、AI エージェントによるパフォーマンス問題の自動トリアージに活用できる」と案内されています。Apple の「Metal Developer Tools」ページ [*13] でも、「Hello, agentic debugging」と題して、エージェントと組み合わせたワークフローが紹介されています。

[*13] Metal Developer Tools `https://developer.apple.com/metal/tools/`

第2章

デイリービルドの実機プロファイリングを自動化する

キム ミンヒョク Minhyuk Kim

株式会社サイバーエージェント SGEコア技術本部

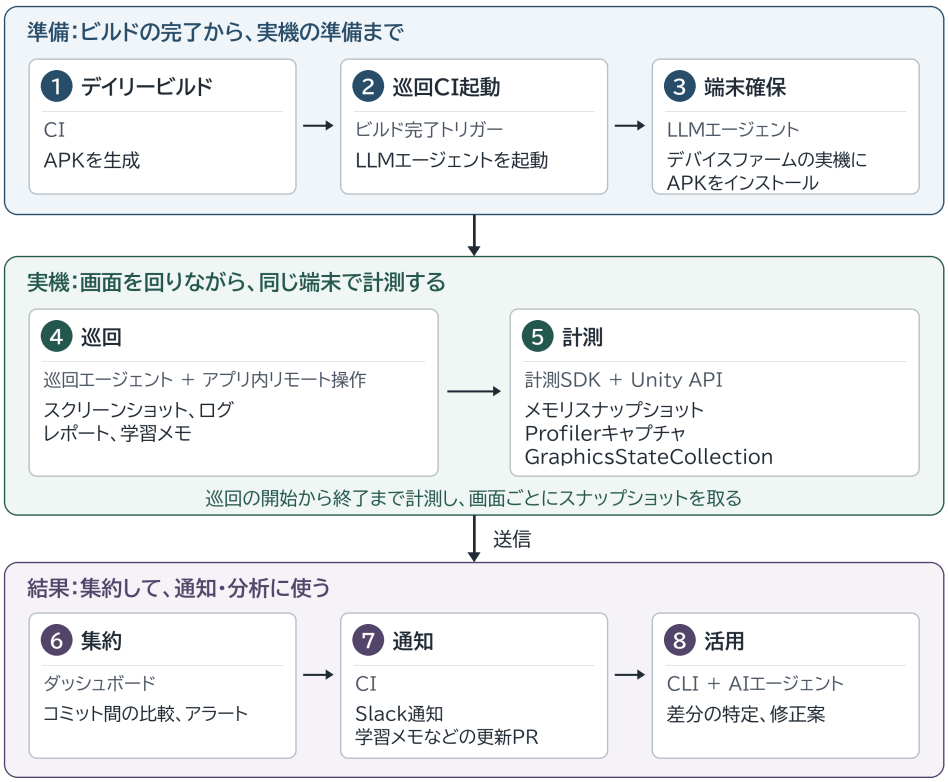
実機のプロファイルは、誰かが必要だと思ったときにしか取られないことがほとんどです。リリース前の追い込みで誰かがProfilerをつないで、スパイクやメモリの増加を見つけて、原因のコミットを探す。この流れでは、いつから悪くなったかを探す範囲が数週間分の差分になりがちです。本章では、デイリービルドのたびに実機で計測を自動で回すための構成を扱います。

構成は3つの部品に分かれます。実機を貸し出すデバイスファーム、アプリに組み込んで外部から操作を受け付けるリモート操作サーバー、計測データをサーバーへ送る計測SDKです。デバイスファームから遠隔で実機を借り、リモート操作でゲームの画面を順に回りながら、計測SDKでメモリスナップショットとProfilerキャプチャを送ります。これをデイリービルドの後続ジョブとしてCIに載せると、ビルドが出るたびにコミット単位の計測データが溜まります。溜まったデータはコマンドラインから取り出せるので、AIエージェントに差分を読ませて修正案まで進める入口にもなります。

本章の内容は、社内プロジェクトで運用しているデイリー巡回のワークフローと、そこに至るまで工夫した点について解説します。執筆時点で対象にしているのはAndroid端末だけで、iOSへの対応は今後予定しています。

2.1　ビルドの後ろに計測ジョブをつなぐ

パイプラインは図2.1の順に進みます。端末の確保、アプリの操作、計測データの送信は、それぞれ別のツールが担当します。どれかを別のツールに置き換えても、この分担はそのまま使えます。



▲図2.1　デイリー計測パイプラインの段階

デイリービルド自体には手を入れません。デイリービルドに別途巡回用のシナリオ組み込みまたは予め設定した計測コードは不要で、こういった計測を念頭においたコードを置かないことにより、内部構成が激しく変化する開発段階でも巡回が壊れることなく、安定性と柔軟性を維持できました。その手法については下で詳しく説明します。

2.2 ビルドの完了をCIで受けて巡回を始める

デイリービルドが完了すると、ビルドの完了トリガーを別のワークフローが購読し、成功したときだけ巡回を起動します。このとき後続ワークフローへ渡すのは、APKをビルドしたrunのIDです。

起動したワークフローは、runのIDからビルドのアーティファクトを特定し、ダウンロードします。そのあと、巡回を担うLLMエージェントを立ち上げます。執筆時点では、モデルにClaude Opus 5.5を使い、推論の深さを決めるeffortはmediumに設定しています。

2.3 Unityの計測はアプリの中から行う

本章で自動化するのは、Unity側の計測です。どのアセットがメモリを占めているか、どのProfilerマーカーが重いかといったUnity内部の内訳を、毎日同じ形で取って比べることを目的にしています。

端末の外側から使える計測手段にも、もちろん有用なものは多くあります。iOSならInstrumentsのTime Profilerでコールスタック単位のボトルネックを追えます。Androidでも、simpleperfやそれを使うAndroid StudioのCPU Profilerで、IL2CPPが生成したネイティブ関数の単位までサンプリングできます。こうした道具を自動化に活用することも十分に考えられますが、今回の範囲では扱いません。

Unityの内訳は、普段はUnityエディタのProfilerやMemory Profilerを実機につないで取ります。ただ、CIの巡回にエディタを挟むと、考えることが一気に増えます。ランナーでプロジェクトを開くたびにLibraryフォルダのインポートを待つことになり、ランナーごとにUnityのライセンスも必要です。さらに、エディタと実機の接続に加えて、エージェントからエディタを操作するUnity MCPのような経路も用意しなければならず、構成が複雑になります。

そのため、CI上ではeditorは使わず、ランタイムのAPIのみを使用し、アプリケーション内で計測を完結させることにしました。

2.4　端末はデバイスファームから借りる

デバイスファームに使うのはブラウザから実機を操作できるSTFという基盤です[*1]。ただし、本章で使うのは画面を通したデバイスファームの操作ではなく、APIを通じた端末の確保のみで、端末を予約するとリモートのadbエンドポイントが返り、CIのランナーからはUSB接続と同じように扱えます。予約からAPKのインストールまでをスクリプト1本にまとめ、CIからは1ステップで呼びます。

巡回だけが目的ならAndroid 10以上の空いている端末が1台あれば十分ですが、計測を日ごとに比較したいなら、端末が毎日変わると比較になりません。計測用には機種を固定し、空きがなければその日はスキップする運用にしたほうが、比較できないデータを溜めずに済みます。

2.5　アプリ内リモート操作で画面を回る

アプリの操作には、アプリに組み込んだリモート操作サーバーを使います。アプリが起動すると端末の中だけで命令を待ち受け、CIのランナーからはadb経由でつないで命令を送ります。アプリが動いているかの確認、シーン上のオブジェクト構成の取得、スクリーンショットの撮影、ログの取得といった決まった命令のほかに、JavaScriptで書いた任意の処理をアプリの中で実行できます。

このJavaScriptはJint[*2]で解釈され、C#の型に直接触れます。UIのボタンを名前で見つけて押すことも、ナビゲーションの現在ノードを読むことも、計測SDKを開始することも、すべて同じ経路で行えます。実機の上でC#のコードをその場で書いて試せる環境に近いもので、計測SDKのAPIもビルドし直さずに外から呼べます。閾値や対象画面を変えるたびにAPKを作り直す必要がありません。

計測を毎日比較するには、毎日同じ画面を同じ順序で通る必要があります。固定座標のタップは端末の解像度やレイアウトが変わると押す位置がずれるので、ランタイムのUIオブジェクトを名前で取得し、UIフレームワークのイベントを直接発火させます。現在どの画面にいるかは、ナビゲーションの状態を読んで判定します。プロジェクト側に画面遷移グラフの定義があるなら、それをそのまま巡回計画の入力にできます。

[*1] DeviceFarmer/stf（OpenSTFの後継リポジトリ） `https://github.com/DeviceFarmer/stf`
[*2] sebastienros/jint, JavaScript Interpreter for .NET `https://github.com/sebastienros/jint`

2.6 スナップショットとキャプチャの2種類を上げる

計測データの送信には、筆者の所属組織で開発している計測基盤を使います。アプリに組み込んだSDKが計測データをサーバーへ送り、サーバー側で蓄積して、ダッシュボードで日ごとの比較や悪化したときのアラートを出す仕組みです。

SDKが送れるのは、UnityのメモリスナップショットやProfilerキャプチャだけではありません。CPU時間の内訳やドローコール数などの描画負荷、コード中の任意の区間の処理時間、アプリの起動時間、一定間隔で取ったメモリ使用量の推移なども送れるので、巡回に組み込めば同じように定点観測の対象にできます。本章では、そのうち表2.1の2種類に絞って説明します。

表2.1: デイリー巡回で送る計測データ

種類	取るタイミング	前提	送る量の考え方
メモリスナップショット	各画面に到達して安定したあと	Development Build	画面ごとに1枚、すべて送る
Profilerキャプチャ	巡回の開始から終了まで1本	Profilerが有効なビルド	閾値を超えたフレームだけ送る

2.6.1 スナップショットは画面ごとに取る

メモリスナップショットはUnityのMemory Profilerと同じ形式で、Unityの仕様上Development Buildでしか取れません[*3]。デイリービルドの巡回用APKはDevelopment Buildにしておく必要があり、リリース設定のビルドを計測に流用することはできません。

スナップショットは、巡回が画面に到達しロードが落ち着いたところで1枚取ります。名前には画面名とコミットハッシュを入れ、あとで同じ画面の別の日のデータを並べられるようにします。

[*3] Unity Manual, Memory Profiler package `https://docs.unity3d.com/Packages/com.unity.memoryprofiler@1.1/manual/index.html`

2.6.2　キャプチャは巡回ごとに1本取り、端末上で絞り込む

Profilerキャプチャは、Unityが書き出す生のProfilerログをSDKが端末上で読み進め、条件に合うフレームだけを別ファイルへ書き出してアップロードするものです。全フレームをそのまま残さないのは、Profilerが長時間の記録に向いていないからです。エディタのProfilerウィンドウが保持できるのは最大2,000フレームで、60fpsなら30秒あまりです。古いフレームから捨てられるので、数十分の巡回をエディタにつないで記録しても、残るのは最後の30秒あまりです。都度ファイルとして書き出せばフレーム数の上限はなくなりますが、今度はファイルが巡回1回分で数十GBに膨らみます。CIのアーティファクトとして残すにも、毎日溜めるにも大きすぎます。

キャプチャは巡回の最初に開始し、最後に停止とアップロードをまとめて行います。開始時に決めるのは、どのフレームを残すかの閾値で、フレーム時間とGC割り当て量の2つで指定します。停止は、生ログの読み残しを消化してから完了します。絞り込みは計測中も並行して進んでいるので、Deep Profilingのように書き出しが速い場合でも読み残しはほとんど溜まりません。それでも、停止が終わる前に端末を返却しないよう、停止は巡回の締め切りから余裕をもって開始します。

2.6.3　区切りだけを読み、GCは時刻で突き合わせる

生ログは、スレッドごとのバッファをブロック単位で順に書き足したものです。どのブロックがどのフレームのものかを知るには、フレームの区切りを見つける必要があります。手がかりになるのはProfilerの書き出し方です。Profilerはフレームの区切りを書く直前と直後にスレッドのバッファを切り替えるので、区切りは十数バイトだけの単独のブロックとして届きます。そこで絞り込みのスレッドは、この長さ以下のブロックだけを開いて区切りかどうかを確かめます。それより長いブロックは中身を読まず、そのスレッドの現在のフレームに属するものとして扱います。メインスレッドで次のフレームの区切りが届くとフレームの長さが決まるので、そこで残すかどうかを判定し、残すフレームのブロックだけを元の順序のまま書き出します。サンプラー名やスレッド名のように、どのフレームにも属さない定義のブロックはすべて書き出すので、出力はUnityのProfilerウィンドウでそのまま開けます。

GC割り当て量は、生ログからは取れません。生ログのフレーム統計にもGCの欄はありますが、Unity 2022.3から6000.5まで、確認したどのキャプチャでも0でした。そ

こで計測中は、メインスレッドで毎フレーム、ProfilerRecorderで`GC Allocated In Frame`カウンタを読みます。ProfilerRecorderの値には時刻が付かないので、そのフレームの中で`ProfilerUnsafeUtility.Timestamp`を読んで値に添えておきます。この時刻は、生ログのフレームの区切りに記録される時刻と同じ時計です。絞り込みのスレッドは、あるフレームの開始から次のフレームの開始までの間に打たれた値を、そのフレームのGC割り当て量として突き合わせます。あるフレームの値が手に入るのは次のフレームの中なので、値が届くまで数フレーム分は判定を待ちます。

端末の上で絞り込みを行うと、それ自体が計測を重くしないかが気になるかもしれません。絞り込みは専用のスレッドで行い、メインスレッドで行うのはデータの増え方の確認とGCカウンタの読み取りだけです。前述のとおり、ほとんどのブロックは中身を読まないので、絞り込みにかかる処理はごく一部のデータに限られます。絞り込みのスレッドと、Profilerのデータを書き出すDispatcherスレッドは、スレッド名で見分けて出力から除いています。また、絞り込みの処理はDeep Profilingの対象から外しています。対象に含めると、絞り込みのスレッドのメソッド呼び出しの一つひとつがサンプルになり、ちょうど読んでいる生ログに書き込まれてしまうからです。ただし、GCカウンタはプロセス全体の値なので、絞り込みのスレッドが割り当てる1フレームあたり数KBも含まれます。後述する閾値と比べれば十分に小さい量ですが、Profilerを有効にすること自体の負荷も含めて、計測による上乗せがまったく無いとは言えません。

2.6.4 閾値は見たい問題とデータ量で決める

筆者の巡回では、現在300ms・2MBを閾値にしています。30fpsのタイトルなら通常のおよそ10倍のフレーム時間で、プレイ中の細かな引っかかりではなく、画面遷移やロードで起きる大きな停止が昨日より増えていないかを見るための設定です。この設定なら、巡回全体のうち残るのはごく一部のフレームだけで、毎日送り続けられる量に収まっています。閾値を下げれば細かな引っかかりも拾えますが、残るフレームが増えてファイルが大きくなり、後述する出力の上限にも早く届きます。対象タイトルのフレームレートや、計測で解決したい問題に合わせて、データ量を見ながら閾値を調整して運用してください。

2.6.5 長い巡回を端末の容量に収める

巡回が長くなると、生ログの置き場所も問題になります。SDKは生ログの書き出し先を一定の大きさごとに切り替えて連番のファイルに分け、絞り込みのスレッドは読

み終えたファイルから削除します。これで、端末のストレージ使用量は数ファイル分で頭打ちになります。絞り込みが遅れて未処理のファイルが溜まりすぎたときは、記録を止めます。ファイルを切り替えると、Unityは新しいファイルの先頭に、それまでに登場したサンプラー名やオブジェクト名の定義をすべて書き直します。出力は1本のファイルにつなぐので、2つ目以降の定義はほとんどが重複です。そこで、直前に書いたものとバイト単位で同じ定義は出力から省いています。出力ファイルには1.5GBの上限を設け、次に残すフレームで上限を超えるところで記録を止めます。アップロードできるのは2GBまでなので、その手前で止めて、上限までに残したフレームは確実に送れるようにしています。

キャプチャには、各フレームのスクリーンショットも埋め込んでいます。Profilerの数値だけでは、スパイクが起きたときにどの画面で何が表示されていたかがわかりにくいためです。画面は非同期のGPUリードバックで取るので、画像が手に入るのは数フレーム後になります。そこで、最後に手に入った画像を毎フレームの終わりに埋め込み直し、どのフレームを残しても画像が付くようにしています。Unity 6.6からは、Profilerの標準機能でもスクリーンショットを連続して記録できるようになり、既定でスクリーンショットを撮る機能が有効になっています。ただし撮影は15フレームに1回固定で、筆者のAndroid端末で記録された画像は256×112ピクセルしかなく、UIの文字が読めませんでした。また、AsyncGPUReadbackが有効ではない端末では選択肢がなく、静かにスクリーンショットが無効になりました。300ms・2MBで絞り込んだある日のキャプチャでは、残った71フレームのうち、標準のスクリーンショットが付いていたのは6フレームだけで、残すフレームが飛び飛びになる絞り込みとは相性が悪いので、SDK側の撮影を使い続けています。

2.6.6　メタデータは全データに付ける

キャプチャのアップロード時に添えるメタデータのほかに、SDKには全計測データに共通の情報を付ける仕組みがあります。ビルド時に埋め込んだコミットハッシュ、ビルドのrun ID、端末モデルを初期化直後に登録しておけば、スナップショットにもキャプチャにも同じ情報が付き、操作側から毎回渡す必要も無くなります。計測結果をコミット順に並べられるのは、この登録があるからです。

2.7　シェーダーのウォームアップデータも巡回で集める

巡回は毎日ゲームの主な画面をひととおり通るので、シェーダーのウォームアップに使うデータを集める場にもなります。GraphicsStateCollectionは、実行中に使われたシェーダーバリアントと描画状態の組み合わせを記録しておき、次回以降に事前にウォームアップできるようにするアセットです。

収集には計測SDKは使わず、リモート操作からUnityのAPIを直接呼びます。巡回の開始時に記録を始め、いつもどおり画面を回り、終了時に記録を止めて端末上のアプリの保存領域にファイルとして書き出します。書き出したファイルはadbでCIのランナーへ取り出します。エディタやPlayer接続は使わないので、計測と同じくCIの中だけで完結します。ただし、IL2CPPのビルドではコードストリッピングによって使われていない型のメンバーが削られ、リモート操作から呼べなくなることがあります。巡回用のビルドでは、link.xmlでGraphicsStateCollectionの型を残しておく必要があります。

取り出したファイルを、そのままプロジェクトに入れることはしません。巡回がエラーで止まらずに終わり、ファイルの保存まで済んだ日だけ、同じプラットフォームと品質設定の既存アセットに足し合わせ、ウォームアップ時に見つからないと警告されたバリアントを取り除いたうえで、アセット更新のPRを自動で作り、そのまま自動でマージします。毎日の巡回で通った画面の記録が、こうして少しずつ積み上がっていきます。

とはいえ、この方法だけで必要なバリアントをすべて網羅できるわけではありません。記録されるのは巡回が実際に通った画面と、そのとき描画されたものだけです。巡回で通らない画面や、特定の条件でしか出ないエフェクトなどは記録から漏れます。こうした取りこぼしや、ウォームアップしたのにスタッターが残るといった落とし穴については、第1部6章を参照してください。本書を頭から順に読んでここまで来られた方は、すでに目を通されているかもしれません。

2.8　悪化の検出から修正PRまでをエージェントに任せる

毎日データが溜まっても、ダッシュボードを毎朝開く人がいなければ溜まっただけで終わります。この計測基盤にはコマンドラインからデータを取得するCLIもあり、本章のパイプラインでは、悪化の確認から修正PRの作成までを、このCLIを使うエージェ

ントに任せています。人が毎回依頼を書くのではなく、計測の後ろに続く工程として毎日回します。

巡回の計測が終わると、巡回とは別の比較用のエージェントが立ち上がります。このエージェントはCLIで今日と前日の計測データを取得し、同じ画面のスナップショットやキャプチャに残ったスパイクを比べて、悪化した箇所が無いかを確認します。悪化が見つかった場合は、どの画面のどの値がどれだけ変わったか、比較に使ったデータ、その間に入ったコミットの範囲をまとめて、バックログにイシューとして登録します。悪化が無ければ何も登録しません。

登録されたイシューは、さらに別の、修正を担当するエージェントが拾います。イシューの内容を確認し、コミット履歴から原因を絞り込んだうえで修正を加え、PRを作ります。人が関わるのは、そのPRのレビューからです。

2.8.1 修正の確認も同じ仕組みで実機で行う

修正の確認は、デイリーの巡回とは分けて行います。デイリーの巡回は決まった画面を決まった順に回るもので、イシューごとに確かめたい画面や条件に合わせるには向いていないためです。そこで、バックログ上でイシューが修正完了（確認待ち）の状態になったことをきっかけに、修正確認用のエージェントが動きます。このエージェントは、デバイスファームの端末、アプリ内リモート操作、計測SDKというデイリーの巡回と同じ仕組みを使って、修正を取り込んだビルドを実機で動かし、イシューに残した比較元のデータと突き合わせます。悪化が解消したことを実機で確認できて初めて、チケットをQAに渡します。修正が効いたかを確かめるために、開発者が実機を触る必要はありません。

この確認の仕組みは、パフォーマンスの悪化に限らず、日頃の不具合修正にも使えます。前述のとおり、リモート操作では、実質リフレクション同様で直接触れる処理はその場で実行できるので、エージェントはゲームの中で起きることをほぼすべて操作し、読み取れます。イシューに再現手順と期待する結果が書かれていれば、その手順を実機上でたどり、ゲーム内部の状態まで含めて修正できているかを確かめられます。

2.9 毎日回し続けるための運用

2.9.1 巡回エージェントと計測手順の役割分担

本章の巡回は、コーディングエージェントに画面遷移グラフとリモート操作の使い方を渡して巡回させています。到達できない画面を回避し、新しい画面が増えたら計画を組み直せる柔軟さがある反面、1回の巡回に90ターン前後、費用にして5ドル前後が掛かります。計測の開始と停止のように毎回同じでなければならない部分は、エージェントの判断に委ねず、再利用可能な手順として固定し、エージェントにはそれを所定のタイミングで実行させるだけにしています。巡回の学習内容も、次回の入力としてリポジトリにPRで残しています。

2.9.2 結果の受け渡し

巡回の結果はレポート、スクリーンショット、ログ、機械可読な結果ファイルとして残し、CIが結果ファイルを読んでSlackへ通知します。計測そのものの結果はダッシュボードにあるので、通知にはアップロードの成否とダッシュボードへのリンクがあれば足ります。比較で悪化が見つかった日は、登録したイシューへのリンクを付けて同じチャンネルに通知します。逆に、アップロードが無い日は、計測されていないことも通知に含めます。計測が知らないうちに止まっている状態が最も見つけにくいためです。

2.10 今後の課題

スナップショットとキャプチャを同じ巡回の中で取っているため、2つのデータが互いに混ざります。スナップショットの取得中は数秒間フレームが止まるので、同時に走っているキャプチャにはスパイクとして記録されます。逆に、キャプチャ中はProfilerのバッファのメモリがスナップショットに含まれるため、データを読むときには注意が必要です。データを正確に見たい場合は、スナップショット用とキャプチャ用で巡回を分けることも選択肢ですが、端末を借りる時間がおよそ倍になるため、頻度を落とすなどの折り合いが必要です。

Profilerキャプチャの粒度にも課題があります。デイリー計測ではDeep Profilingを切っているので、ゲーム側のスクリプトはMonoBehaviourの`Update`のような単位でしか見えません。それより細かく見たい箇所には、ProfilerMarkerを手で追加するか、

Deep Profilingを有効にすることが必要になります。そのため、指定したアセンブリのメソッドに、ビルド時の後処理でProfilerMarkerを自動で挿入する方法などを模索しています。対象を絞った軽いDeep Profilingのように使えるので、データ量と負荷を抑えたまま、内訳を細かく取れるようになると考えています。

2.11 まとめ

デイリービルドの後ろに、デバイスファームで端末を借り、アプリ内リモート操作でゲームを動かし、計測SDKへメモリスナップショットとフィルタ済みのProfilerキャプチャを送るジョブをつなぎました。コミットハッシュをメタデータに添えることで計測結果はコミット単位の履歴になり、CLIで取り出せば、悪化の検出からイシューの登録、修正PRの作成までをAIエージェントに任せられます。

本章の構成は大がかりに見えるかもしれませんが、すべてを一度にそろえる必要はありません。これから計測の自動化を計画されているなら、まずはビルドマシンに実機を1台USBでつなぎ、adb越しにリフレクションで動くインタプリタからメモリスナップショットを撮るだけでも、立派なメモリの定点観測になるかもしれません。本章が、皆さんの計測自動化の第一歩の助けになれば幸いです。

第3章

チューニングバイブルをAIコーディングエージェントのスキルにする

中村光寿 Teruhisa Nakamura

株式会社アプリボット

3.1　はじめに

本書のようなチューニングナレッジには、ひとつの構造的な弱点があります。読んだ人にしか効かないことです。各章には実測に裏打ちされた知見が詰まっていますが、それがプロジェクトのコードレビューで実際に指摘されるかどうかは、レビュアーが該当の章を読んでいて、レビューの瞬間に思い出せるかにかかっています。チーム全員が全章を暗記することは期待できませんし、運用フェーズで人が入れ替われば知見は薄れていきます。一方で、コードを書く現場にはAIコーディングエージェント（Claude Code、OpenAI Codex、GitHub Copilot など）が常駐するようになりました。レビュー依頼やCI に組み込めば、すべての差分に同じ基準を当てられます。足りないのは、Unity のパフォーマンスに関する体系的な評価基準だけです。そこで本章では、チューニングバイブルの知見を「AIエージェントが実行できる評価項目」に変換し、特定のツールに依存しない汎用スキルとしてパッケージングする方法を紹介します。筆者が実際に、前作で公開済みの知見に基づく77ルールへ、本書（2026年版）の原稿群から抽出した86ルールを統合し、計163の評価ルールとしてスキル化した事例をもとに、変換の設計指針と運用上の注意点をまとめます。

3.2 Agent Skills という共通フォーマット

3.2.1 SKILL.md の仕様

AIエージェントに専門知識を持たせる方法として、2025年末に Agent Skills[*1] というオープンな仕様が公開されました。スキルの実体は SKILL.md という Markdown ファイルを中心としたディレクトリで、次のような構成をとります。

▼リスト3.1　スキルディレクトリの構成

```
unity-perf-review/
├── SKILL.md          # メタデータ + 実行手順(必須)
├── references/       # ルール集などの参照ドキュメント
├── scripts/          # 実行可能スクリプト(任意)
└── assets/           # テンプレート等(任意)
```

SKILL.md の先頭にはフロントマターと呼ばれるメタデータを書きます。必須フィールドは name と description の2つだけです。

▼リスト3.2　SKILL.md のフロントマター（description は誌面向けに短縮し、metadata 等の任意フィールドは省略）

```
---
name: unity-perf-review
description: >
  Unity プロジェクトのコード・アセット・設定をパフォーマンス観点で
  レビューする。C# スクリプト、シェーダー、.meta、ProjectSettings を
  163 のルールと照合し、Critical / Warning / Info の指摘を出力する。
  Unity のパフォーマンスレビューや最適化調査を依頼されたときに使う。
license: MIT
---
```

重要なのは description の書き方です。エージェントはユーザーの依頼内容と description を照合してスキルを自動発動するため、「何をするか」だけでなく「いつ使うか」まで書いておくと発動精度が上がります。

3.2.2 主要エージェントの対応状況

この仕様の価値は、複数のエージェントが同じフォーマットを読めることにあります。2026年8月時点の対応状況を表3.1に示します。

[*1] Agent Skills 仕様 https://agentskills.io/specification

表3.1: 主要コーディングエージェントのスキル対応（2026年8月時点）

エージェント	対応	配置場所
Claude Code	SKILL.md ネイティブ対応[*2]	claude/skills/ または ~/.claude/skills/
OpenAI Codex	SKILL.md 対応	agents/skills/ または ~/.agents/skills/
GitHub Copilot	SKILL.md 対応[*3]	github/skills/
Gemini CLI	SKILL.md 対応[*4]	gemini/skills/ または ~/.gemini/skills/

Codex / Copilot / Gemini CLI は共通の .agents/skills/ 配置にも対応しています。また Copilot はプロジェクトの .claude/skills/ も読むため、.claude/skills/ への配置1つで Claude Code と Copilot の両方に届きます（Copilot の個人用配置は ~/.copilot/skills/ または ~/.agents/skills/）。対応状況・配置場所は時点依存のため、導入時に各エージェントの公式ドキュメントで一次情報を確認してください。一度チューニングナレッジを SKILL.md 形式に落とし込めば、どのエージェントを使うメンバーにも同じレビュー基準（ルールと手順）を配布できます。Agent Skills が共通化するのはパッケージ形式と手順であり、モデルやツール権限の差で実行結果までは揃いません。それでも「レビュー基準はリポジトリに、実行はエージェントに」という分業が成立します。

3.3 知見を「評価項目」に変換する

スキル化の本体は、SKILL.md を書くことではなく、章の知見をエージェントが機械的に適用できる粒度まで分解することです。筆者は次のフォーマットに統一しました。

▼リスト3.3 ルールフォーマット（ParticleSystem の章からの抽出例。grep_pattern の除外条件と対処の詳細は誌面向けに短縮）

```
### US11 ParticleSystem.Simulateへの大きな秒数指定
- レベル: Warning(Noiseモジュール有効時はCritical)
- 対象: コードレビュー / アセット設定
- 検出パターン: Simulateの第1引数に大きな秒数を渡している。
  特にNoiseModule有効なParticleSystemとの併用
- grep_pattern: \.Simulate\(
- 理由: Simulateは指定時間まで一括シミュレーションする。
  実測で20秒指定によりNoiseJobが約2000回コールされ
  約50msのスパイクが発生
- 対処: Noise有効パーティクルではSimulateの利用を控える
```

このフォーマットには4つの設計判断が入っています。

3.3.1 検出の機械化可能性を明示する

各ルールには grep_pattern として、C# コードや Unity の YAML（.meta、Prefab、ProjectSettings）に対する機械実行可能な検索仕様を持たせます。多くは1本の正規表現ですが、複数行の突き合わせや「あるべき記述の不在」を検出するものも含みます。エージェントはこの仕様で一次検出し、ヒット箇所だけを精読して判断します。これにより大規模リポジトリでも現実的なコストでレビューできます。正規表現で書けない項目を無理に書かないことも同じくらい重要です。たとえば「Addressables 未登録の共有アセットが複数バンドルから参照されている」は grep では取れません。そうしたルールには (手動確認) と明記し、Addressables Analyze などの確認手段を対処欄に書きます。検出可能性を偽ると、エージェントは誤検出を量産するか、検出できないルールを黙ってスキップするようになります。

3.3.2 実測値を理由欄に残す

「理由: パフォーマンスが悪化するため」というルールは説得力を持ちません。原稿にある実測値をそのまま理由欄に引用します。

- 未圧縮テクスチャ: 2048x2048 で Compressed 2.7MB / Uncompressed 16.0MB（約6倍差）
- 1体1グラフの手動 Playable: 600体で 8.396ms 対 全体1グラフ 4.500ms（1.87倍差）
- Simulate(20f) と Noise の併用: NoiseJob 約2000コール・約50ms のスパイク

数値があると、エージェントの指摘がそのまま「なぜ直すべきか」の説明になります。レビューを受けた開発者が原稿の当該章に立ち返る導線にもなります。なお、いずれも原稿記載の特定環境での実測値です。ルールの理由欄には代表値を引用し、計測条件の詳細は出典章を参照する運用にしています。

3.3.3 レベルと上書きで運用に耐えさせる

ルールには Critical / Warning / Info の3レベル（Critical=劣化の確実な原因で、適用条件の成立を確認できたら修正必須、Warning=条件次第で問題化する潜在リスク、Info=任意の改善推奨）を付け、さらにプロジェクト側で上書きできる仕組み（rules-overrides）を用意します。たとえば「LINQ 使用は Critical」でも、初期化時（init コンテキスト）は Warning に緩和、Editor ツール（tooling）では Skip、のような調整です。チューニ

ングルールは本質的にコンテキスト依存です。上書き機構がないと「うちのプロジェクトでは合わない指摘が多い」となってスキルごと無効化されます。基準は共通、適用はプロジェクト固有、という2層構造が長持ちのコツです。

3.3.4 静的に検出できない知見は診断ナレッジに分ける

原稿の知見には、コードを読んでも分からないものが多くあります。「CPU 処理を削ったのにフレーム時間が変わらず Semaphore.WaitForSignal などの待機マーカーに時間が移った場合は、レンダースレッド側がクリティカルパスにある可能性を疑い、Timeline ビューで待ち先を確認する」のような、プロファイラの読み方に関する知見です。待機マーカー単独では律速要因を断定できない、という限界も知見の一部として残します。これらは評価ルールとは別ファイルに「症状 → 原因 → 対処」の表として分離しました（表3.2）。筆者のリポジトリではプロファイル解析用のエージェント（Claude Code 向け）が診断時にだけ読み込みます。本章で配布するスキルには参照資料として同梱しており、静的レビューで判断できない疑いを「プロファイル計測推奨」として報告する際の根拠になります。

表3.2: 診断ナレッジの例

症状	原因	対処
CPUチャートが谷とスパイクの櫛状	間引きの位相ずらし忘れ	index % interval で位相分散
MeshSkinning.CalcMatrices が大きい	ボーンTransform階層の実体化	Optimize Game Objects 適用
iOSのみ、エラーと Shader.CreateGPUProgram がなく GpuProgramMetal.GetCachedPipeline だけ発生	シェーダーのAssetBundle重複パッキングを疑う	重複インスタンスを確認し専用バンドル化
iOSのみ、ウォームアップ完了後もSubPass内シェーダーでPSOコンパイル	SubPass（Native Render Pass）内シェーダーのウォームアップ失敗	SubPass見直しまたはロード中のプリレンダリング

3.4 スキルとして組み立てる

3.4.1 段階的読み込みとチェックリスト方式

163ルールをすべて SKILL.md 本文に書くと、エージェントのコンテキストを圧迫します。Agent Skills の設計思想は段階的読み込み（progressive disclosure）です。

1. エージェントは常時、フロントマターの name と description だけを見ている
2. スキルが発動したら SKILL.md 本文（実行フロー）を読む
3. 必要になった参照ドキュメントだけを references/ から読む

初期に何を読むか（パス等を含むか）はエージェント実装に依存しますが、「本文と参照資料は発動まで読まれない」という原則は共通です。ここで注意したいのが網羅性です。素朴には「対象ファイル種別に応じてカテゴリ別ルールファイルを丸ごと読み込む」構成が考えられますが、この方式ではエージェントがコードより先に読むルールが最大で2万トークンを超える量（文字数からの概算。トークン数はモデル依存です）に達します。数十のルールを「全部覚えて適用する」ことをモデルの注意力に依存させると、一部のルールが黙って読み飛ばされます。そこで、全ルールから「ID / レベル / 適用条件 / grep_pattern / ルール名 / 詳細ファイル名」を1ルール1行に圧縮したチェックリストを自動生成し、レビューを2段階に分けました。

1. **一次スキャン**: 同梱スクリプト scan.py がチェックリスト全行の grep_pattern を対象ファイルに機械実行し、ヒット一覧（ファイル:行）と Coverage を出力する（python3 が無い環境ではエージェントが全行の grep を直接実行するフォールバック）
2. **二次精査**: ヒットしたルールについてだけ、詳細ファイルから検出条件・理由・対処を読み込んで誤検出を除外する

grep で表現できない一部のルールはチェックリスト上で MANUAL と明記し、対象種別に該当するときだけ精読します。「あるべき記述の不在」が指摘条件になるルールには ABSENT を付し、grep がヒットしなかった場合に詳細を確認します（ヒットの有無と指摘条件の対応を逆転させないための明示です）。一次スキャンをスクリプトにしたのは、LLM に数十本の grep を1本ずつ実行させると読み飛ばしが起きうるうえ、正規表現の方言差（PCRE の先読みは macOS 標準 grep で動かない等）で結果が環境依存になるためです。これで一次スキャンの網羅性はモデルの記憶力から切り離されます。指摘の正しさ（二次精査での誤検出除外）は引き続きエージェントの判断です。事前に読むのはチェックリストだけ（コード用約16KB・アセット用約8KB。トークン数はモデル依存ですが、合計で1万トークン前後の概算）になり、詳細はヒットした分しか読みません。チェックリストはルール本体から自動生成するため、ルールを追加しても手動での同期作業は発生しません。

3.4.2　環境に応じたルール除外

ルールにはUnityバージョンや構成に依存するものがあります。本スキルに収録したRenderGraph 関連ルール（URP 向け）は Unity 6 以降が対象で、LTO 系は IL2CPP 専用、iOS 固有の PSO 問題は Android のみのタイトルには無関係です。該当しないルールの照合はノイズと手間にしかなりません。そこで環境依存ルールには適用条件タグ（unity>=6000.3、urp、il2cpp、ios など）を持たせ、プロジェクトの設定ファイルから生成した環境プロファイルと突き合わせて「明確に条件を満たさないルールだけ」を除外します。タグの突き合わせは一次スキャンスクリプトが決定的に処理し、検出できなかった項目（unknown）は安全側（照合対象のまま）に倒し、スキップしたルール ID はレポートに列挙して監査可能にします。生成手順は導入手順の節で説明します。

3.4.3　出力にルールIDを強制する

出力フォーマットでは、各指摘へのルールID付与を必須にします。

▼リスト3.4　レビュー出力の例

```
### Critical
- Assets/Scripts/EffectPlayer.cs:42 -
  Noise有効パーティクルへのSimulate(20f)呼び出し [US11]
  - 根拠: NoiseJobが約2000回コールされ約50msのスパイク(実測)
  - 推奨: Noiseを外すか秒数を最小化する
```

IDがあると、指摘の根拠をルールファイルまで遡って検証でき、「エージェントがそれっぽく言っているだけ」との区別がつきます。誤検出だった場合も、どのルールの検出パターンを直せばよいかが明確になります。

3.5　配布と各エージェントへの配置

スキルの公開は、ディレクトリごと共有するだけで成立します。GitHub リポジトリでも zip でも構いません。利用者は表3.1の配置場所にコピーすれば、それぞれのエージェントがスキルを発見します。運用上のポイントは、ルールの単一ソースを保つことです。筆者のリポジトリでは、ルール本体は rules/ ディレクトリだけに置き、配布用パッケージはスクリプトで組み立てています（利用者側の導入手順は次節）。

▼リスト3.5　配布パッケージの組み立て（メンテナー向け）

```
./package-skill.sh
cp -r dist/unity-perf-review /path/to/repo/.claude/skills/   # Claude Code
cp -r dist/unity-perf-review /path/to/repo/.agents/skills/   # Codex / Copilot / Gemini
     CLI
cp -r dist/unity-perf-review /path/to/repo/.github/skills/   # Copilot 固有配置(上2行いず
    れかで届くため任意)
```

エージェントごとにルールを二重管理し始めると、更新のたびに基準がズレていきます。「ソースは1か所、配置はスクリプト」を最初に決めておくことを勧めます。

3.6　自分のプロジェクトへ導入する

ここまでの仕組みを読者のプロジェクトで動かす手順です。python3 が導入済みなら、コピーと初期生成は10分程度が目安です。リポジトリに追加されるのはスキルディレクトリと環境プロファイル（tuning-env.md）だけで、Unity の Assets / ProjectSettings には変更を加えません。

3.6.1　スキルの取得と配置

スキル一式（SKILL.md、ルール163件を同梱した references/、環境検出・一次スキャン・チェックリスト再生成の3スクリプト、導入手順の README）は本書のリポジトリの skills/unity-perf-review/ で公開しています[*5]。取得して、使っているエージェントのスキルディレクトリへコピーするだけです。なおスキルは「エージェントへの指示と実行可能スクリプト」を含む配布物です。信頼できない配布元のスキルは SKILL.md と scripts/ の中身を確認してから導入し、バージョンを固定することを勧めます。

▼リスト3.6　スキルの取得と配置

```
git clone https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026
    .git
cd UnityPerformanceTuningTips2026/skills

# 使っているエージェントに応じていずれかへコピー
cp -r unity-perf-review /path/to/your-repo/.claude/skills/   # Claude Code
cp -r unity-perf-review /path/to/your-repo/.agents/skills/   # Codex / Copilot / Gemini
     CLI(共通配置)
cp -r unity-perf-review /path/to/your-repo/.github/skills/   # Copilot 固有配置(上2行いず
    れかで届くため任意)
```

[*5] https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026/tree/main/skills/unity-perf-review

リポジトリ直下に配置すればチーム全員に同じレビュー基準が配られます。個人で試すだけならホームディレクトリ側（~/.claude/skills/ など）でも動きます。

3.6.2 配置後の状態

Claude Code の場合、配置と次項の環境プロファイル生成が済むとリポジトリは次の状態になります（リスト3.7）。

▼リスト3.7 配置後のリポジトリ構成

```
your-repo/
|-- .claude/
|   \`-- skills/
|       \`-- unity-perf-review/
|           |-- SKILL.md                      # name/description + 実行フロー
|           |-- README.md                     # 人間向けの導入手順
|           |-- references/
|           |   |-- checklist-code.md         # 一次スキャン用（コード系）
|           |   |-- checklist-asset.md        # 一次スキャン用（アセット/設定系）
|           |   |-- checklist-profile.md      # プロファイル解析用(静的レビューでは未使用の
    付属資料)
|           |   |-- rules-csharp.md など計13ファイル  # 詳細ルール（計163）
|           |   |-- diagnosis-knowledge.md    # 症状->原因->対処の診断集
|           |   \`-- rules-overrides.example.md
|           \`-- scripts/
|               |-- detect-env.py             # 環境プロファイル生成
|               |-- scan.py                   # 一次スキャンの機械実行
|               \`-- gen-checklists.py         # ルール修正時のチェックリスト再生成
|-- tuning-env.md                             # 環境プロファイル（次項で生成）
\`-- (Unityプロジェクト本体は無変更)
```

3枚のチェックリストは対象別（コード / アセット / プロファイル）で、1つのルールが複数枚に載ることがあるため、行数の合計はルール数163と一致しません。配置した時点でエージェントに載るのは、原則としてフロントマターの name と description（合計で数百トークン。前述のとおり実装によってはパス等が加わります）だけです。レビューを依頼すると、前述の段階的読み込みに従って SKILL.md 本文 → 対象に応じたチェックリスト → ヒットしたルールの詳細、の順に必要な分だけ読み込まれます。通常は163ルール全文（約125KB、概算で4万トークン強）を一括投入せずに済みます。rules-overrides.example.md はそのままでは無効です。プロジェクト固有の調整を始めるときに rules-overrides.md へリネームして編集します（後述）。

3.6.3　環境プロファイルの生成

前述のとおり163ルールには環境依存のものが含まれます。プロジェクトに該当しないルールを除外判断させるため、環境プロファイルを生成します。

▼リスト3.8　環境プロファイルの生成（clone したリポジトリの skills/ から実行）

```
python3 unity-perf-review/scripts/detect-env.py \
    /path/to/your-repo /path/to/your-repo/tuning-env.md
```

配置後に実行する場合は、パスを配置先（.claude/skills/ や .agents/skills/ など、使っているエージェントのスキルディレクトリ）に読み替えてください。Unity プロジェクトがリポジトリ直下ではなくサブディレクトリにあるモノレポ構成でも、リポジトリルートを渡せば設定ファイルを再帰的に探索して検出します。スクリプトの実行には python3 が必要です（無い環境では tuning-env.md を手動作成するか、この手順を飛ばして全ルール照合で運用できます。一次スキャンの scan.py も python3 を使いますが、無い場合はエージェントが grep を直接実行するフォールバックで動きます）。スクリプトは ProjectVersion.txt、Packages/manifest.json、GraphicsSettings.asset、ProjectSettings.asset から Unity バージョン・レンダーパイプライン・Scripting Backend・Addressables の有無を推定します。あくまで設定ファイルからの推定値であり、確定情報は tuning-env.md への手動記入が正です。検出できなかった項目（プラットフォーム別に設定が分かれる Scripting Backend の混在時など）は unknown となり、安全側（照合対象のまま）で扱われます。出力された tuning-env.md を開いて内容を確認し、特に出荷プラットフォームと SideFX VAT の使用有無は手動で補正し、render_pipeline も想定どおりの値かを確認してください。

3.6.4　プロジェクト固有の調整（任意）

意図的にルールから外れている箇所がある場合は、rules-overrides.md にレベルの上書きを記録します。初期化時の LINQ を許容する、Editor ツールではログルールを無効化する、といった調整です。環境プロファイルが「その環境では原理的に適用不能」なルールの除外を担い、overrides が「プロジェクト都合の調整」を担う、という役割分担です。ルールそのものを追加・修正したい場合は references/ の rules-*.md を編集し、python3 scripts/gen-checklists.py references/ でチェックリストを再生成します。チェックリストは自動生成物のため直接編集しません。配布物だけでこの追加・再生成のサイクルが完結します。

3.6.5　実行する

エージェントに依頼するだけです。通常は依頼文と description の一致で自動発動します（発動はモデル判断のため、確実に使わせたい場合は依頼文にスキル名 unity-perf-review を明記するか、各エージェントの明示呼び出しを使います）。

▼リスト3.9　レビューの依頼例

```
この差分をパフォーマンス観点でレビューして
Assets/Scripts/Battle/ 配下をチューニングバイブルのルールでチェックして
```

出力の最後に付く Coverage セクションで、照合したルール数・環境スキップしたルール数（ID付き）・grep ヒット数が確認できます。これらの件数は一次スキャンスクリプトの出力の転記で、エージェントの記憶には依存しません（二次精査での採用・誤検出除外の件数のみエージェントが集計します）。除外されすぎだと感じたら tuning-env.md を、指摘が多すぎると感じたら rules-overrides.md を調整してください。

3.7　運用で気をつけること

実際にプロジェクトへ導入して分かった注意点を挙げます。**初回はリポジトリ全体ではなく、差分か特定ディレクトリから始めます。** 一次スキャン自体はプロジェクト全体でも短時間で終わりますが、運用歴のあるコードベースではヒットするルールが数十件規模になり、二次精査（誤検出の除外）の負荷がそこに集中します。狭い範囲で精査し、恒常的に許容するものを overrides に落としてから対象を広げるほうが、指摘がノイズに埋もれずに定着します。**静的解析は意図した設定も指摘します。** たとえば意図的に非圧縮にしているテクスチャも AS02（テクスチャ圧縮未設定）でヒットします。検出結果を鵜呑みに一括修正せず、1件ずつ意図を確認してから直すこと。恒常的に許容するものは overrides に理由付きで記録することで、次回以降のノイズを減らせます。overrides の調整単位はルール全体またはコンテキスト（in-game=ホットパス / init=初期化 / tooling=Editor）で、特定アセットだけの例外は備考欄にパスや理由を書いてエージェントの判断材料にします（専用の適用範囲指定は現バージョンでは未対応です）。**実測値には計測環境が付いています。** ルールに引用した数値は特定の Unity バージョン・端末での実測です。「Pixel 8a での実測」を最新ハイエンド端末の判断に外挿はできません。ルール自体に対象バージョンを書き、Unity やレンダーパイプラインの更新をルール見直しのトリガーにします。**最後に判断するのは人です。** 本書の別章の言葉を借りれば「規約を決める・見直すのは人、守らせるのは機械」です。AIエー

ジェントは「守らせる機械」の新しい実装であって、規約=評価項目の妥当性に責任を持つのは引き続き人間です。エージェントの指摘に納得できなかったら、それはルールの検出パターンか理由欄を改善するシグナルだと捉えてください。

3.8 おわりに

チューニングバイブルの知見をルール化してみて実感したのは、「実測値と対処がセットで書かれた原稿は、ほぼそのまま機械可読な評価項目になる」ということです。逆に言えば、根拠の数値がない経験則はルール化の段階で弱さが露呈します。これは執筆側へのフィードバックでもあります。計測環境と数値を明記した知見は、書籍の読者だけでなく、AIエージェントを通じてすべてのコードレビューに届きます。本章で紹介したスキルとルール集は、単一の正解というより出発点です。各プロジェクトの overrides が育ち、新しい章の知見がルールとして追加されていく、生きた規約集になることを期待しています。

[*2] Claude Code の Skills ドキュメント `https://code.claude.com/docs/en/skills`

[*3] GitHub Docs "About agent skills" `https://docs.github.com/en/copilot/concepts/agents/about-agent-skills`

[*4] Gemini CLI の Agent Skills ドキュメント `https://geminicli.com/docs/cli/using-agent-skills/`

おわりに

Unity 6世代で道具は大きく変わった一方で、向き合っている問題そのものは前作の頃とほとんど変わっていません。新しい機能は、以前からの問題に対する新しい選択肢です。選択肢が増えたぶん、どれを選ぶかの判断が問われます。

どこを見て、何と比べ、どこで打ち切るか。AIコーディングエージェントが開発の一部を担うようになった今、この判断の基準を持つことの重要性はむしろ増していると感じています。エージェントに計測や修正を任せられる範囲は今後も広がっていくはずですが、何をもって良しとするかを決めるのは、依然として私たち自身です。品質を保証できたと言える線は、プロジェクトごとに異なり、運用を続けるなかで見直されていきます。その線を自分たちで引くことが、チューニングの第一歩です。本書が、その手がかりになれば幸いです。

Unityも端末も、これからも変わり続けます。本書の内容も、いずれ見直しが必要になる時が来るでしょう。その時には、新しい環境で得た知見を再び持ち寄り、前作と本書に続く一冊としてお届けできればと思います。

最後まで読んで頂き、ありがとうございました。

著者紹介

本書に携わった著者を紹介します。
なお、各著者の所属・肩書・プロフィールは執筆時点の情報です。

キム ミンヒョク（Minhyuk Kim）
株式会社サイバーエージェント SGEコア技術本部 / クライアントエンジニア

本書の企画・取りまとめのほか、序章、第3部の導入「2026年のアセット・設定・ビルド動向」などの執筆を担当。
複数のスマートフォンゲーム開発会社でリードエンジニアやエンジニアリングマネージャーを務めたのち、2023年にSGEコア技術本部（コアテク）に中途入社。社内プロジェクトの品質担保を担い、リリースタイトルのパフォーマンスチューニングを手がけてきました。現在は横断組織の立場から、個別の対応にとどまらず、全社のアプリケーション品質を底上げする仕組みづくりに取り組んでいます。

Unite Tokyo 2019「今すぐ現場で覚えておきたい最適化技法」
CEDEC2024「モバイルゲームの品質を保証せよ！ パフォーマンスモニタリングのススメ」
Unite Seoul 2025「効率的な社内Unityプロジェクト品質管理戦略と最適化事例」

清原 隆行（Takayuki Kiyohara）/ X：@nezumimusume
株式会社サイバーエージェント SGEコア技術本部

第1部の導入「2026年のUnityグラフィックス動向」と「Unity 6 Render Graph タイルメモリ最適化」の執筆を担当。

大阪のゲームデベロッパーで13年間勤務し、主にコンソールゲーム機で国内、海外

のゲーム開発に携わったのち、ゲームクリエイター育成の専門学校に転職し６年間ほど後進の育成に努めた。その講師時代に「HLSLシェーダーの魔導書」を執筆。現在はサイバーエージェント ゲーム・エンターテイメント事業部のSGEコア技術本部（コアテク）にて、Unityのグラフィックスエンジニアとして従事。

CEDEC+KYUSHU 2022「独自ゲームエンジンを利用した、グラフィックエンジニアの育成」
CEDEC+KYUSHU 2023「ゲームエンジン全盛の時代に、あえて新卒研修でソフトウェアレンダラーの作り方を教えてみた」
CEDEC2024「統合スタイライズドレンダリングシステムの開発について紹介 ～共通利用ができる基盤を目指して～」
CEDEC2025「Unity Graphics Test Frameworkを使ったビジュアルリグレッションテスト」
CEDEC2026「Anime 4Kをモバイルで使ってみた」

チャン ユービン（Yubing Zhang）/ X: @vzmc12
株式会社サイバーエージェント SGEコア技術本部

「RenderGraphのUnsafePassを使うべき場面」「URPのFinalBlitPassを省略する方法」の執筆を担当。

前職でUnityクライアントエンジニアとして5年間ゲーム開発を経験。2022年にサイバーエージェントへ中途入社し、SGEコア技術本部に所属。現在はグラフィックスエンジニアとして、「NovaShader」「Sirius」などのグラフィックス基盤開発や、各プロジェクトの技術支援を担当。

CEDEC2024「統合スタイライズドレンダリングシステムの開発について紹介 ～共通利用ができる基盤を目指して～」

永留 遼平（Ryouhei Nagatome）/ X：@rn49rn49 / GitHub：@rngtm
株式会社Colorful Palette / グラフィックスエンジニア

「モバイル向けシェーダー最適化：超越関数の計測と軽量化」の執筆を担当。

2022年に株式会社Colorful Paletteに中途入社後、モバイル向けのゲームタイトルにてグラフィックスエンジニアとして従事。

瀬戸 徳（Noboru Seto）/ GitHub：@ruccho
株式会社サイバーエージェント SGEコア技術本部 / クライアントエンジニア

第2部の導入「2026年のスクリプティング・アニメーション動向」と「Renderer Shader User Value を活用したグラフィックス最適化」の執筆を担当。
2023年に新卒入社後、SGEコア技術本部（コアテク）にて共通ライブラリ等の開発に従事。アセット配信、セキュリティ、ビルドパイプライン、プロファイリング、ネイティブなどの領域を扱う。画面録画ライブラリ「Instant Replay for Unity」を OSS として開発中。

天野 択郎（Takuro Amano）
株式会社サイバーエージェント SGEコア技術本部

「GraphicsStateCollectionによるシェーダーウォームアップ」の執筆を担当。
2026年にSGEコア技術本部（コアテク）へ中途入社。プロジェクトサポートチームに所属し、ゲーム子会社を横断して運用中タイトルの起動時間・メモリ・描画まわりのパフォーマンスチューニングを支援しています。実機計測の自動化と、計測ハーネスの整備に取り組んでいます。

斉藤 俊介（Shunsuke Saito）
株式会社Colorful Palette

「Unity 3D キャラクターアニメーションのチューニング」の執筆を担当。

2021年に株式会社サイバーエージェントに入社し、同年株式会社Colorful Paletteへ配属。運用・開発中のタイトルにて3Dアニメーション制御やシネマティクシステムの実装、チューニングなどを主に担当しています。

CEDEC2024 「「プロジェクトセカイ カラフルステージ！ feat. 初音ミク」3周年アップデートのUI全リニューアル！ デザイナーとエンジニアの相互学習とUI開発フローの効率化について」

塩塚 勇気（Yuki Shiotsuka）
株式会社QualiArts / テクニカルアーティスト

「ResetControllerStateでAnimatorプーリングを安全に行う」の執筆を担当。
株式会社QualiArtsにて、テクニカルアーティストとしてキャラクターアニメーション制御、物理シミュレーション、パイプライン整備に従事。

CEDEC2024「UnityのHumanoidと共存するキャラクタジョイント制御と制作ワークフロー」

畳 悠樹（Yuuki Tatami）/ X：@tatamin0207
株式会社サイバーエージェント SGEコア技術本部 / グラフィックスエンジニア

「VATによるボーンアニメーション負荷軽減」の執筆を担当。
コンシューマゲーム開発会社で約6年間の開発経験を経て、2023年にSGEコア技術本部（コアテク）に中途入社。事業部横断のグラフィックス基盤「SIRIUS」の開発を担当し、Houdini・AI関連の推進・啓蒙も行っています。また、社外向け勉強会であるCA.unityの運営責任者を務めています。

CEDEC2024「統合スタイライズドレンダリングシステムの開発について紹介 ～共通利用ができる基盤を目指して～」
CEDEC2026「グラフィックス基盤によるVAT活用 ― "使える技術"として組織に届ける」

加藤 有也（Yuya Kato）
株式会社Colorful Palette / グラフィックスエンジニア

「ParticleSystemで長い時間Simulateをすると巨大なスパイクが発生する」の執筆を担当。
株式会社Colorful Paletteにて3Dグラフィック領域を担当。キャラクターの演出制御から空間表現、描画品質の改善まで、3Dに関わる実装を横断的に手がけています。

稲田 健人（Taketo Inada）
株式会社サイバーエージェント SGEコア技術本部 / クライアントエンジニア

「UniTaskとキャンセル処理のライフタイム管理」「AssetBundleのライフタイム管理」の執筆を担当。
複数のスマートフォンゲーム開発会社で10年以上クライアント開発に携わり、リードエンジニアやマネージャーとして大規模タイトルの開発・運用を経験したのち、2026年にSGEコア技術本部（コアテク）へ中途入社。プロジェクトサポートチームに所属し、開発・運用中タイトルの品質・パフォーマンス改善を支援しています。また、計測基盤の開発や、開発体験の向上にも取り組んでいます。

財津 大睦（Hiromu Zaitsu）/ X：@RomGameclient
株式会社サムザップ / グラフィックスエンジニア

「UIシステムの使い分け－ IMGUI / uGUI / UI Toolkit」「運用フェーズのアセット肥大化とProject Auditorによる調査」の執筆を担当。
株式会社サムザップにて、グラフィックスエンジニアとして描画関連の実装やパフォーマンス改善に取り組んでいます。ゲームの運用開発と新規開発を経て、現在は開発推進室に所属し、「Sumzap Engineering Blog」にて記事の執筆も行っています。

佐久間 尚輝（Naoki Sakuma）
株式会社サイバーエージェント SGEコア技術本部 / グラフィックスエンジニア

「端末に応じた描画設定の切り替え」の執筆を担当。
複数のスマートフォンゲーム開発会社で、クライアントエンジニア、グラフィックスエンジニアとしてゲーム開発に携わり、2025年にSGEコア技術本部（コアテク）へ中途入社。現在は、事業部横断で複数のグラフィックス機能の開発を担当するほか、子会社の開発支援も行っています。

向井 祐一郎（Yuichiro Mukai）/ X：@yucchiy_ / GitHub：@yucchiy
株式会社アプリボット 技術統括本部

「Unity Managed Code Stripping 実践ガイド」の執筆を担当。
株式会社アプリボットの技術統括本部にて、プロジェクトサポートに従事。サイバーエージェントの「Developer Experts」（`https://www.cyberagent.co.jp/techinfo/info/detail/id=31305`）にUnity領域で認定されています。個人では、ブログ（`https://blog.yucchiy.com`）でUnityのニュースを毎週まとめる「Unity Weekly」を連載しているほか、社内の有志と技術書典で「UniTips」を執筆、UnityをCLIから操作

するツール「UniCli」を開発しています。

山崎 雅大（Masahiro Yamazaki）/ X：@TEST_H_ / GitHub：@mao-test-h
株式会社アプリボット 技術統括本部 / クライアントエンジニア

「Unity で活用する iOS 標準のパフォーマンス計測機能」の執筆を担当。
複数のスマートフォンゲーム開発会社でクライアントエンジニアとしてゲーム開発に携わったのち、ライブ配信サービスでクライアントエンジニア、モバイルエンジニアとして経験を積み、2025年にアプリボットへ中途入社。基盤チームとして、非ゲーム領域やモバイルネイティブ領域のサポートなどに従事しています。また、個人ではQiita（`https://qiita.com/mao_`）にUnity × iOS向けの記事などを投稿しています。

中村 光寿（Teruhisa Nakamura）/ X: @megalo_23
株式会社アプリボット 技術統括本部 / ゲームクリエイター

「チューニングバイブルをAIコーディングエージェントのスキルにする」の執筆を担当。
リードエンジニア、ディレクターなどを経て、プロジェクトの立ち上げや企画、最適化まで幅広く従事。チューニングの相談を受ける機会が多かったため、今回のスキル化でAIに任せ、面白いゲーム作りに集中したいと目論んでいる。

免責事項

本書に記載された内容は情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について当社はいかなる責任も負いません。

Unity パフォーマンスチューニング ティップス 2026

2026年11月2日 初版第1刷 発行

著　者　　株式会社 サイバーエージェント ゲーム・エンターテイメント事業部
デザイン　株式会社 サイバーエージェント SGE統括本部 PRデザイン室
発行所　　株式会社 サイバーエージェント

(C) 2026 CyberAgent, Inc.



CyberAgent